COMP2211: Introduction to Artificial Intelligence

WU Jinjun

November 14, 2026

Contents

1 Advanced Python **4**
1.1 Motivation . . . 4
1.2 More Advanced Python Programming Elements . . . 4
1.3 Object-Oriented Programming . . . 6
1.4 Importing Libraries and Data . . . 9
1.5 NumPy . . . 11

2 Naive Bayes Classifier **22**
2.1 Machine Learning Basics . . . 22
2.2 Bayes' Rule . . . 24
2.3 Naive Bayes . . . 27

3 K-Nearest Neighbors Classifier **36**
3.1 Motivation: T-shirt Size . . . 36
3.2 The KNN Model . . . 37
3.3 Example: T-shirt Size . . . 38
3.4 Scikit-Learn Implementation . . . 39
3.5 Standardization . . . 40
3.6 Distance Functions . . . 42
3.7 Selecting K . . . 44
3.8 Model Evaluation . . . 45
3.9 KNN Analysis . . . 49
3.10 Practice: Purchase Prediction . . . 50

4 K-Means Clustering **53**
4.1 Motivation: Customer Segmentation . . . 53
4.2 Clustering . . . 54
4.3 K-Means . . . 54
4.4 Analysis of K-Means . . . 58
4.5 Leader Clustering . . . 61
4.6 Practice: Medicine Clustering . . . 62

5 Neurons **65**
5.1 Classification . . . 65
5.2 ANN and Neuron . . . 65

6 Multi-Layer Perceptron **75**
6.1 From Neuron to MLP . . . 75
6.2 MLP Architecture . . . 76
6.3 Gradient Descent . . . 78
6.4 Example: Learning XOR . . . 80
6.5 MLP from Scratch . . . 82
6.6 Example: Handwritten Digits . . . 85
6.7 Batch Size . . . 92
6.8 MLP Analysis . . . 94
6.9 Practice . . . 99

7 Image Processing **101**
7.1 Introduction . . . 101
7.2 Image Preprocessing . . . 101
7.3 Digital Images . . . 102
7.4 OpenCV Basics . . . 105
7.5 Operation Types . . . 111
7.6 Convolution . . . 116
7.7 Practice: Worked Example . . . 121

8 Convolutional Neural Networks **122**
8.1 Introduction . . . 122
8.2 What is a CNN? . . . 122
8.3 Layers of a CNN . . . 123
8.4 CNN Training Process . . . 129
8.5 Dropout . . . 129
8.6 Example: Handwritten Digits . . . 130
8.7 Typical Architecture . . . 135
8.8 Practice . . . 136

9 PyTorch **138**
9.1 PyTorch and CUDA . . . 138
9.2 Tensors . . . 139
9.3 Example: Handwritten Digits . . . 143
9.4 Automatic Differentiation . . . 151

10 Minimax and Alpha-Beta Pruning **152**
10.1 Games and Tic-Tac-Toe . . . 152
10.2 Minimax . . . 154
10.3 Alpha-Beta Pruning . . . 162

11 AI Ethics **167**
11.1 AI Ethics . . . 167
11.2 AI Principles . . . 169
11.3 Areas . . . 171
11.4 AI Fairness . . . 171
11.5 Moral Machines . . . 174

12 Reinforcement Learning **176**
12.1 Introduction . . . 176
12.2 Basic Concepts . . . 178

12.3 Problem Formulation 181
12.4 Markov Decision Processes 186

1 Advanced Python

1.1 Motivation

1.1.1 Why More Python?

You have taken COMP1021/COMP1029P, so you can program in Python. But the Python you learned there (mostly Turtle graphics) is not enough to write AI programs — it only covers the basics with a brief introduction to classes and small programs.

This chapter is a **crash course** on more advanced Python programming elements and other essentials for writing AI programs.

Goal of This Chapter

Bridge the gap between "introductory Python" and "the Python you actually need for AI":
- **advanced language features**: comprehensions, `zip`, variable arguments, object-oriented programming;
- **importing libraries and data**;
- the **NumPy** library for numeric computing.

1.1.2 Python Versions

- As of January 1, 2020, Python has officially dropped support for **Python 2**.
- For this course, all code uses **Python 3.12.12**. The latest version is Python 3.14.2 (released on 5 December 2025).
- You can check your Python version at the command line after activating your environment:

```
import sys
print(sys.version)
```

1.2 More Advanced Python Programming Elements

1.2.1 Data Types and the type() Function

Like most languages, Python has a number of basic types including **integers**, **floats**, **booleans**, **strings**, and containers including **lists**, **dictionaries**, **sets**, and **tuples**.

The `type()` function returns the class type of the argument (object) passed as parameter, and is mostly used for **debugging** purposes.

Syntax: `type(<object>)`

```
x = 3
print(type(x))          # <class 'int'>
y = 3.14
print(type(y))          # <class 'float'>
z = True
print(type(z))          # <class 'bool'>
s = "COMP2211"
print(type(s))          # <class 'str'>
l = [1, 2, 3]
print(type(l))          # <class 'list'>
d = {'cat': 'cute', 'dog': 'furry'}
print(type(d))          # <class 'dict'>
st = {1, 2, 3}
print(type(st))         # <class 'set'>
t = (1, 2, 3)
print(type(t))          # <class 'tuple'>
```

1.2.2 List Comprehensions

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Syntax:

`<new-list> = [ <expression> for <element> in <list> if <condition> ]`

Parameters:

- `expression`: operation we perform on each value of the original list;
- `element`: a temporary variable storing each item of the original list;
- `list`: the name of the list we want to go through;
- `condition`: if it evaluates to True, the processed element is added to the new list;
- `new-list`: the new list that stores the results.

Note: the `if` clause is optional, and additional `for` clauses may be added.

```
# Without a comprehension
nums = [0, 1, 2, 3, 4]
squares = []
for x in nums:
    squares.append(x ** 2)
print(squares)          # [0, 1, 4, 9, 16]

# With a list comprehension
nums = [0, 1, 2, 3, 4]
squares = [x ** 2 for x in nums]
print(squares)          # [0, 1, 4, 9, 16]

# With an optional condition
nums = [0, 1, 2, 3, 4]
even_squares = [x ** 2 for x in nums if x % 2 == 0]
print(even_squares)     # [0, 4, 16]
```

1.2.3 Dictionary Comprehensions

These are similar to list comprehensions, but construct dictionaries:

```
nums = [0, 1, 2, 3, 4]
even_num_to_square = {x: x ** 2 for x in nums if x % 2 == 0}
print(even_num_to_square)   # {0: 0, 2: 4, 4: 16}
```

1.2.4 Set Comprehensions

Like lists and dictionaries, we can easily construct sets:

```
from math import sqrt
nums = {int(sqrt(x)) for x in range(30)}
print(nums)             # {0, 1, 2, 3, 4, 5}
```

1.2.5 Lists vs. Tuples

A tuple is in many ways similar to a list; one of the most important differences is that **tuples can be used as keys in dictionaries and as elements of sets, while lists cannot**.

```
# A dictionary with tuple keys
d = {(x, x + 1): x for x in range(10)}
t = (5, 6)              # a tuple
print(type(t))          # <class 'tuple'>
```

```
print(d[t])              # 5
print(d[(1, 2)])         # 1
```

1.2.6 Parallel Iteration: zip()

The `zip()` method takes iterables or containers and returns a single iterator object, having mapped values from all the containers. If the passed containers have different lengths, the one with the fewest items decides the length of the new iterator.

Syntax: `zip(<iterator1>, <iterator2>, <iterator3>, ...)`

```
fruits = ['apple', 'peach', 'banana', 'guava', 'papaya']
colors = ['red', 'pink', 'yellow', 'green', 'orange']
for name, color in zip(fruits, colors):
    print(name, 'is', color)

# apple is red
# peach is pink
# banana is yellow
# guava is green
# papaya is orange
```

1.2.7 Functions with Arbitrary Arguments

- If you do not know how many arguments will be passed, add a single asterisk `*` before the parameter name. The function then receives a **tuple** of arguments.
- If you do not know how many keyword arguments will be passed, add two asterisks `**` before the parameter name. The function then receives a **dictionary** of arguments.

```
def print_months(*months):
    print("The last month is " + months[-1])

print_months("January", "February", "March", "April", "May", "June",
             "July", "August", "September", "October",
             "November", "December")
# The last month is December

def print_month(**month):
    print("The month is " + month["name"] + ", it is the " +
          month["order"] + "th month, it has " +
          month["days"] + " days.")

print_month(name="September", order="9", days="30")
# The month is September, it is the 9th month, it has 30 days.
```

1.3 Object-Oriented Programming

1.3.1 Classes and Objects

Object-Oriented Programming (OOP) is a way of structuring a program by bundling related **properties** (data) and **behaviors** (functionality) into individual objects. It is introduced through the use of `class` and objects.

Class and Object Basics

- A **class** is a user-defined data type from which objects are created (keyword `class`). It bundles data (instance variables) and functionality (methods).
- **Instance variables** belong to an object; they are public by default and accessed with the dot (`.`) operator.

- **Methods** have an extra first parameter `self`; we do not give a value for it when calling the method — Python provides it.
- `__init__` is similar to a **constructor** in Java/C++, used to initialize the instance variables of objects.

Defining a class:

```
class <class-name>:
    def __init__(self, <arguments>):    # constructor
        self.<instance-variable> = <value>
        ...
    def <method-name>(self, <arguments>):
        <statements>
        ...
```

Creating an object, calling a method, modifying an instance variable:

```
<object> = <class-name>(<arguments>)        # create an object
<object>.<method-name>(<arguments>)         # call a method
<object>.<instance-variable> = <value>      # modify a variable
```

```
import math

class Circle:
    def __init__(self, radius):     # constructor with parameter
   radius
        self.radius = radius        # instance variable radius

    def area(self):
        return math.pi * self.radius ** 2

    def circumference(self):
        return 2 * math.pi * self.radius

circle_object = Circle(10)          # create an object
circle_object.radius = 100          # modify its radius
print("Area:", circle_object.area())
print("Circumference:", circle_object.circumference())
# Area: 31415.926535897932
# Circumference: 628.3185307179587
```

1.3.2 Public, Protected, and Private Members

All members in a Python class are **public** by default, i.e., any member can be accessed from outside the class. To restrict access, we can make them protected or private.

- **Protected:** accessible within the class and available to its sub-classes. By convention, a **single underscore** `_` is added as a prefix.
- **Private:** accessible within the class only. By convention, a **double underscore** `__` is added as a prefix.

"Private" Is Only a Convention

"By convention" means a responsible programmer refrains from accessing or modifying members prefixed with `_` or `__` from outside the class. But if they do, it still works. :(

```
class Pokemon:
    def __init__(self, name='Pikachu', Ndex='0025', types='Electric'):
        self.__name = name          # private instance variables
        self.__Ndex = Ndex
        self.__types = types

    def print(self):
        print('--- Print Pokemon ---')
```

```
        print('Name: ' + self.__name)
        print('Ndex: ' + self.__Ndex)
        print('Type: ' + self.__types)

vulpix = Pokemon('Vulpix', '0037', 'Fire')
print('Name: ' + vulpix.__name)     # Error
vulpix.__name = 'AlolanVulpix'      # OK, but does not work
vulpix.print()                      # OK
```

To retrieve and modify instance variable values, we need to define **accessors** and **mutators**.

1.3.3 Accessors and Mutators

- **Accessor** (getter): returns the value of a protected/private instance variable.
- **Mutator** (setter): modifies the value of a protected/private instance variable.

```
class Pokemon:
    def __init__(self, name='Pikachu', Ndex='0025', types='Electric'):
        self.__name = name
        self.__Ndex = Ndex
        self.__types = types

    def get_name(self): return self.__name         # accessor
    def get_Ndex(self): return self.__Ndex         # accessor
    def get_types(self): return self.__types       # accessor

    def set_name(self, name): self.__name = name   # mutator
    def set_Ndex(self, Ndex): self.__Ndex = Ndex   # mutator
    def set_types(self, types): self.__types = types  # mutator

vulpix = Pokemon('Vulpix', '0037', 'Fire')
print('Name: ' + vulpix.get_name())
print('Ndex: ' + vulpix.get_Ndex())
print('Type: ' + vulpix.get_types())
vulpix.set_name('AlolanVulpix')
vulpix.set_types('Ice')
vulpix.print()
```

1.3.4 Class Variables

A **class variable** is a variable defined within a class and outside any method. It is **shared by all instances** of the class, so changing its value is reflected in all instances. Class variables are accessed using the **class name**.

```
class Pokemon:
    num_pokemon = 0                     # class variable

    def __init__(self, name='Pikachu', Ndex='0025', types='Electric'):
        self.__name = name
        self.__Ndex = Ndex
        self.__types = types
        Pokemon.num_pokemon += 1        # shared across all instances

vulpix = Pokemon('Vulpix', '0037', 'Fire')
togepi = Pokemon('Togepi', '0175', 'Fairy')
print("Total number of pokemons:", Pokemon.num_pokemon)   # 2
```

Aspect	Class Variable	Instance Variable
Definition	Defined within the class but outside any method.	Defined within methods, typically the constructor.
Scope	A change affects all instances.	A change affects only that instance.
Initialization	Inside or outside the class definition.	Typically in the constructor.
Access	Via the class name, then the variable name.	Via the instance name, then the variable name.
Usage	Data shared by all instances (constants, defaults).	Data unique to each object (object properties).

1.3.5 Inheritance

Inheritance is a mechanism that allows us to define a new class based on existing classes. All the attributes and methods of existing classes are inherited by the new class. The existing classes are called super/parent/base classes, and the new class is called the sub/child/derived class.

Syntax:

```
class <subclass>(<superclass>):
    def __init__(self, <arguments>):          # constructor
        super().__init__(<attributes>)        # invoke super class init
        self.<instance-variable> = <value>
        ...
    def <method-name>(self, <arguments>):
        <statements>
        ...

# EvolPokemon inherits from Pokemon
class EvolPokemon(Pokemon):
    def __init__(self, name='Pikachu', Ndex='0025',
                 types='Electric', evol='Raichu'):
        super().__init__(name, Ndex, types)     # super class init
        self.__evol = evol                      # new instance
   variable

    def get_evol(self): return self.__evol      # accessor
    def set_evol(self, evol): self.__evol = evol  # mutator

    def print(self):                            # method overriding
        super().print()                         # super class print
        print('Evolve into: ' + self.__evol)

jigglypuff = EvolPokemon('Jigglypuff', '0039', 'Normal', 'Wigglytuff')
jigglypuff.print()
# --- Print Pokemon ---
# Name: Jigglypuff
# Ndex: 0039
# Type: Normal
# Evolve into: Wigglytuff
```

1.4 Importing Libraries and Data

1.4.1 Modules, Packages, and Libraries

- **Module:** a bunch of related code (functions, classes, variables) saved in a file with the extension `.py`.
- **Package:** basically a directory of a collection of modules.
- **Library:** a collection of packages.

1.4.2 Imports

The `import` keyword is the most common way to access external modules. Modules are files (`.py`) that contain functions, classes, and variables you can reference without writing them yourself. This is how you get access to machine learning libraries like `numpy`, `keras`, `sklearn`, `tensorflow`, `pandas`, `pytorch`, and `matplotlib`.

`import` is similar to `#include <something>` in C++. However, you usually only need a few functions from each module, in which case we use the `from` keyword.

Syntax:

```
from [module] import [function/class/variable]
from [package].[module] import [function/class/variable]
```

```
# Import only the sqrt function from the math module
from math import sqrt
x = 100
print(sqrt(x))          # 10

# The "import all" syntax also works ...
from math import *
print(sqrt(x))          # 10
```

Avoid `import *`

We should generally avoid importing all functions, since we do not know whether a function (e.g., `sqrt`) comes from the intended library. Using "import all" for multiple libraries may cause clashes or ambiguity, decreasing readability and making errors harder to spot. For example, after `from math import *` and `from cmath import *`, calling `sqrt` actually uses `cmath.sqrt`, a different implementation from `math.sqrt`.

One of the most frequently used machine learning libraries is `sklearn`. A simplified layout:

```
sklearn/
    __init__.py
    discriminant_analysis.py
    cluster/
        KMeans.py
        SpectralClustering.py
        affinity_propagation.py
        ...
    linear_model/
        bayes.py
        logistic.py
    neural_network/
        multilayer_perceptron.py
```

Each "." refers to accessing something inside a package:

- `import sklearn.cluster` imports all modules inside the package.
- `import sklearn.cluster.KMeans` imports only the `KMeans` module.
- Writing `sklearn.cluster.KMeans` every time is long, so we usually import the module directly:

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=2, random_state=600)

# Use "as" to create an alias (a nickname)
from sklearn.cluster import KMeans as km
kmeans = km(n_clusters=2, random_state=600)
```

1.4.3 Importing and Exporting Data

Your data can come in many forms. Here we deal with data in **CSV** (comma-separated-values) form, since that is fairly common. In Python, most data manipulation is done with the **pandas** library.

To import data from Google Drive, we first "mount" the directory (it asks you to visit a link to allow Colab to access your drive):

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
df = pd.read_csv('/content/drive/My Drive/training_data.csv',
                 index_col=False)
```

The `index_col` argument controls which column is used as the index:

- `index_col=False`: none of the current columns is used as the index; instead, a new column is added on the left with numbers `0, 1, 2, ...` (like Excel's row indices).
- `index_col=0` or `index_col="Name"`: the given column (by position or by name) is used as the column index.

```
import pandas as pd
import numpy as np
# Create a DataFrame with two columns
output = pd.DataFrame({'CustomerId': np.array([1, 2, 3]),
                       'CanRepayLoan': np.array([0, 0, 1])})
# Convert to CSV and export to the root of My Drive
output.to_csv('/content/drive/My Drive/loans.csv', index=False)
```

1.5 NumPy

1.5.1 What Is NumPy?

NumPy

NumPy is short for Numerical Python and is the core library for numeric and scientific computing in Python. It provides a **high-performance multidimensional array object** and tools for working with these arrays.

1.5.2 Arrays

A NumPy array is a grid of values, all of the **same type**, and is indexed by a tuple of non-negative integers: a rank-k array A with $\text{shape}(A) = (d_1, d_2, \ldots, d_k)$ has entries $A[i_1, i_2, \ldots, i_k]$ where $0 \leq i_m < d_m$ for each dimension m.

- The number of dimensions k is the **rank** of the array.
- The **shape** $\text{shape}(A) = (d_1, d_2, \ldots, d_k)$ gives the size along each dimension, so the total number of elements is $\prod_{m=1}^{k} d_m$.

We can initialize NumPy arrays from nested Python lists and access elements using square brackets.

```
import numpy as np

a = np.array([1, 2, 3])          # create a rank 1 array
print(type(a))                   # <class 'numpy.ndarray'>
print(a.shape)                   # (3,)
print(a[0], a[1], a[2])          # 1 2 3
```

```
a[0] = 5                        # change an element
print(a)                        # [5 2 3]

b = np.array([[1, 2, 3], [4, 5, 6]])    # create a rank 2 array
print(b.shape)                  # (2, 3)
print(b[0, 0], b[0, 1], b[1, 0])        # 1 2 4
```

NumPy also provides many functions to create arrays:

```
import numpy as np

a = np.zeros((2, 2))            # array of all zeros
b = np.ones((1, 2))             # array of all ones
c = np.full((2, 2), 7)          # constant array
d = np.eye(2)                   # 2x2 identity matrix
e = np.random.random((2, 2))    # array of random values
```

1.5.3 Array Indexing and Slicing

Expression	Description
`a[i]`	Select the element at index `i` (counting from 0).
`a[-i]`	Select the `i`-th element from the end (last is `-1`).
`a[i:j]`	Select elements with index from `i` to `j-1`.
`a[:]` or `a[0:]`	Select all elements in the given axis.
`a[:i]`	Select index 0 up to `i-1`.
`a[i:]`	Select index `i` up to the last element.
`a[i:j:n]`	Select index `i` through `j` (exclusive) with step `n`.
`a[::-1]`	Select all elements in reverse order.

Since arrays may be multidimensional, you must specify a slice for each dimension:

```
import numpy as np
# A rank 2 array with shape (3, 4)
a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])

# Subarray of the first 2 rows and columns 1 and 2
b = a[:2, 1:3]                  # b has shape (2, 2)

# A slice is a view into the same data
print(a[0, 1])                  # 2
b[0, 0] = 77                    # same data as a[0, 1]
print(a[0, 1])                  # 77
```

The basic slice syntax is `i:j:k`, where `i` is the start, `j` is the stop, and `k` is the step (`k != 0`). For $k > 0$ it selects the indices $i,\ i+k,\ i+2k,\ \ldots$ that stay strictly below j:

$$a[i{:}j{:}k] = \big(a_i,\ a_{i+k},\ a_{i+2k},\ \ldots\big), \qquad \text{all indices} < j, \quad k > 0.$$

A negative index p is interpreted as $n + p$ (where n is the length along that axis), and a negative k steps towards smaller indices.

```
import numpy as np
a = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

print(a[1:7:2])     # [1 3 5]
print(a[-2:10])     # [8 9]
print(a[-3:3:-1])   # [7 6 5 4]

print(a[1:100:2])   # [1 3 5 7 9]
print(a[4:3:2])     # []
print(a[100:200:2]) # []
print(a[::-2])      # [9 7 5 3 1]
```

```
print(a[0:-1:-2])    # []
print(a[0::-2])      # [0]
```

Mixing integer indexing with slice indexing yields an array of **lower rank**, while using only slices keeps the same rank:

```
row_r1 = a[1, :]         # rank 1 view of the second row
row_r2 = a[1:2, :]       # rank 2 view of the second row
print(row_r1, row_r1.shape)     # [5 6 7 8] (4,)
print(row_r2, row_r2.shape)     # [[5 6 7 8]] (1, 4)

col_r1 = a[:, 1]
col_r2 = a[:, 1:2]
print(col_r1, col_r1.shape)     # [2 6 10] (3,)
print(col_r2, col_r2.shape)     # [[2] [6] [10]] (3, 1)
```

Expression	Shape
`arr[:2, 1:]`	$(2, 2)$
`arr[2]`	$(3,)$
`arr[2, :]`	$(3,)$
`arr[2:, :]`	$(1, 3)$
`arr[:, :2]`	$(3, 2)$
`arr[1, :2]`	$(2,)$
`arr[1:2, :2]`	$(1, 2)$

1.5.4 Integer Array Indexing

Integer array indexing allows you to construct arbitrary arrays using the data from another array:

```
import numpy as np
a = np.array([[1, 2], [3, 4], [5, 6]])

print(a[[0, 1, 2], [0, 1, 0]])      # [1 4 5]
# Equivalent to:
print(np.array([a[0, 0], a[1, 1], a[2, 0]]))    # [1 4 5]

# You can reuse the same element from the source array
print(a[[0, 0], [1, 1]])            # [2 2]
print(np.array([a[0, 1], a[0, 1]])) # [2 2]
```

A useful trick is selecting or mutating one element from each row of a matrix:

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
b = np.array([0, 2, 0, 1])      # an array of indices

print(a[np.arange(4), b])       # [ 1  6  7 11]  select
a[np.arange(4), b] += 10        # mutate one element per row
print(a)
# [[11  2  3]
#  [ 4  5 16]
#  [17  8  9]
#  [10 21 12]]
```

1.5.5 Boolean Array Indexing

Boolean array indexing lets you pick out arbitrary elements of an array, frequently those satisfying some condition:

```
import numpy as np
a = np.array([[1, 2], [3, 4], [5, 6]])

bool_idx = (a > 2)              # element-wise comparison
print(bool_idx)
# [[False False]
#  [ True  True]
#  [ True  True]]

print(a[bool_idx])              # [3 4 5 6]
print(a[a > 2])                 # [3 4 5 6]  (same, one line)
```

1.5.6 Data Types

Every NumPy array is a grid of elements of the **same type**. NumPy tries to guess a datatype when you create an array, but array-constructing functions usually also accept an optional argument to specify it explicitly.

```
import numpy as np

x = np.array([1, 2])                    # let NumPy choose
print(x.dtype)                          # int64
x = np.array([1.0, 2.0])
print(x.dtype)                          # float64
x = np.array([1, 2], dtype=np.int64)    # force a datatype
print(x.dtype)                          # int64
print(type(x))                          # <class 'numpy.ndarray'>
```

Note the difference between a Python type and a NumPy type: for `x = np.array([1, 2], dtype=np.int64)`, `x.dtype` is `int64`, while `type(x)` is `numpy.ndarray`.

Category	Data Type	Description
Numeric	`np.int8 / 16 / 32 / 64`	8/16/32/64-bit signed integer
	`np.uint8 / 16 / 32 / 64`	8/16/32/64-bit unsigned integer
Floating Point	`np.float16 / 32 / 64 / 128`	16/32/64/128-bit floating point
Complex	`np.complex64 / 128 / 256`	64/128/256-bit complex number
Boolean	`np.bool_`	Boolean type (True/False)
String	`np.str_`, `np.bytes_`	variable-length string, byte string
Object	`np.object_`	general-purpose type for arbitrary objects
Datetime	`np.datetime64`, `np.timedelta64`	dates/times, time differences
Void	`np.void`	flexible type

1.5.7 Array Math

Basic mathematical functions operate **element-wise** on arrays and are available both as operator overloads and as functions in the NumPy module. For two arrays $\mathbf{x}, \mathbf{y}$ of the same shape, every operation acts entry by entry:

$$(\mathbf{x}+\mathbf{y})_i = x_i + y_i, \qquad (\mathbf{x}-\mathbf{y})_i = x_i - y_i, \qquad (\mathbf{x}\odot\mathbf{y})_i = x_i\, y_i,$$
$$(\mathbf{x}/\mathbf{y})_i = x_i/y_i, \qquad \left(\sqrt{\mathbf{x}}\right)_i = \sqrt{x_i}\,.$$

```
import numpy as np
x = np.array([[1, 2], [3, 4]], dtype=np.float64)
y = np.array([[5, 6], [7, 8]], dtype=np.float64)

print(x + y)            # elementwise sum
print(np.add(x, y))
print(x - y)            # elementwise difference
```

```
print(np.subtract(x, y))
print(x * y)             # elementwise product
print(np.multiply(x, y))
print(x / y)             # elementwise division
print(np.divide(x, y))
print(np.sqrt(x))        # elementwise square root
```

1.5.8 Vectors, Matrices, and Tensors

- A **vector** is a 1-dimensional array of numbers, $\mathbf{v} = (v_1, v_2, \ldots, v_n) \in \mathbb{R}^n$.
- A **matrix** is a 2-dimensional array of numbers, $A \in \mathbb{R}^{m \times n}$, whose entry at row i and column j is a_{ij}.
- A **tensor** is a multi-dimensional array, $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_k}$.

They are useful in expressing numerical information and various operations, so they are important in statistics, machine learning, and computer science.

1.5.9 Dot Product and Matrix Multiplication

- The **dot product** of two vectors $\mathbf{v}, \mathbf{w} \in \mathbb{R}^n$ is the sum of the products of their corresponding entries; its output is a **scalar**:

$$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{n} v_i \, w_i.$$

- **Matrix multiplication** is a matrix version of the dot product. For $X \in \mathbb{R}^{m \times n}$ and $Y \in \mathbb{R}^{n \times p}$, the product $XY \in \mathbb{R}^{m \times p}$ has entries

$$(XY)_{ij} = \sum_{k=1}^{n} x_{ik} \, y_{kj},$$

  i.e., the dot product of row i of X with column j of Y.

Matrix multiplication can be performed using the `dot` function/method, the `matmul` method, or the `@` operator (Python 3.5+). For N-dimensional arrays ($N > 2$) the results of `dot` and `matmul` differ slightly, so **prefer `matmul` or `@`**.

```
import numpy as np
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])
v = np.array([9, 10])
w = np.array([11, 12])

# Dot product of vectors -> 219
print(v.dot(w))
print(np.dot(v, w))
# Matrix / vector product -> [29 67]
print(x.dot(v))
print(np.dot(x, v))
# Matrix / matrix product -> [[19 22] [43 50]]
print(x.dot(y))
print(np.dot(x, y))
```

`matmul` is only available as a function in the NumPy module; the `@` operator is equivalent to it:

```
print(np.matmul(v, w))  # 219
print(v @ w)
print(np.matmul(x, v))  # [29 67]
print(x @ v)
```

```
print(np.matmul(x, y))  # [[19 22] [43 50]]
print(x @ y)
```

1.5.10 Aggregation Functions

One of the most useful NumPy functions is `sum`; many others follow similar rules for operating on axes. For an array $X \in \mathbb{R}^{m \times n}$ with entries x_{ij}, reducing all axes gives a scalar, whereas reducing a single axis gives a vector: with `axis=0` the rows are collapsed (one value per column), and with `axis=1` the columns are collapsed (one value per row):

$$\sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij}, \qquad \Big(\sum_{i=1}^{m} x_{ij}\Big)_{j=1}^{n}, \qquad \Big(\sum_{j=1}^{n} x_{ij}\Big)_{i=1}^{m}.$$

The mean is the sum divided by the number of elements,

$$\bar{X} = \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij}, \qquad \bar{X}_j = \frac{1}{m}\sum_{i=1}^{m} x_{ij}, \qquad \bar{X}_i = \frac{1}{n}\sum_{j=1}^{n} x_{ij},$$

and max and min aggregate along axes in the same way.

```
import numpy as np
x = np.array([[1, 2], [3, 4]])

print(np.sum(x))            # sum of all elements -> 10
print(np.sum(x, axis=0))    # sum of each column -> [4 6]
print(np.sum(x, axis=1))    # sum of each row -> [3 7]

print(np.mean(x))           # mean of all -> 2.5
print(np.mean(x, axis=0))   # mean of each column -> [2. 3.]
print(np.mean(x, axis=1))   # mean of each row -> [1.5 2.5]
print(np.max(x))            # 4
print(np.max(x, axis=0))    # [3 4]
print(np.max(x, axis=1))    # [2 4]
print(np.min(x))            # 1
print(np.min(x, axis=0))    # [1 2]
print(np.min(x, axis=1))    # [1 3]
```

1.5.11 Transpose

To transpose a matrix, simply use the `T` attribute of an array object, the `transpose()` function, or the `transpose()` method. For a matrix $A \in \mathbb{R}^{m \times n}$ the transpose $A^\top \in \mathbb{R}^{n \times m}$ swaps rows and columns,

$$(A^\top)_{ij} = a_{ji}.$$

```
import numpy as np
x = np.array([[1, 2], [3, 4]])
print(x)                # [[1 2] [3 4]]
print(x.T)              # [[1 3] [2 4]]
print(np.transpose(x))  # [[1 3] [2 4]]
print(x.transpose())    # [[1 3] [2 4]]
```

Transpose also works on multi-dimensional arrays. The `axes` parameter permutes the axes according to the values specified:

```
import numpy as np
# A 3D array: 2 layers, 3 rows, 4 columns, values 0 to 23
arr = np.arange(24).reshape(2, 3, 4)
# Take the current last axis (2) as axis 0, the first (0) as
```

```
# axis 1, and the second (1) as axis 2: pass [2, 0, 1]
arr_T = np.transpose(arr, [2, 0, 1])
print(arr.transpose([2, 0, 1]))     # same output
# Element-wise mapping: arr[i, j, k] = arr_T[k, i, j]
```

In general, a permutation $\sigma = (\sigma_0, \sigma_1, \ldots, \sigma_{N-1})$ makes the m-th axis of the result correspond to axis σ_m of the input. For the example above, $\sigma = (2, 0, 1)$, so each element moves as

$$A'_{kij} = A_{ijk}.$$

Taking the transpose of a rank 1 (1D) array does **nothing**:

```
import numpy as np
v = np.array([1, 2, 3])
print(v)                # [1 2 3]
print(v.T)              # [1 2 3]
print(np.transpose(v))  # [1 2 3]
```

1.5.12 Reshaping

Reshaping means changing the **shape** of an array — adding or removing dimensions, or changing the number of elements in each dimension.

Syntax: `numpy.reshape(arr, newshape)`

The new shape must be compatible with the original: if `arr` has shape $(a_0, a_1, \ldots, a_{n-1})$ and `newshape` is $(b_0, b_1, \ldots, b_{m-1})$, then $a_0 a_1 \cdots a_{n-1} = b_0 b_1 \cdots b_{m-1}$. One dimension may be `-1`, in which case its value is inferred.

```
import numpy as np
arr1 = np.arange(12)
arr2 = arr1.reshape(4, 3)       # 4 rows, 3 columns
# arr2 = arr1.reshape(4, -1)    # same effect
print(arr2)
# [[ 1  2  3]
#  [ 4  5  6]
#  [ 7  8  9]
#  [10 11 12]]
print("Shape of arr2:", arr2.shape)     # (4, 3)
```

Question: how to create the earlier 3D array (2 layers, 3 rows, 4 columns, values 0 to 23) in one line of code? **Answer:** `np.arange(24).reshape(2, 3, 4)`.

1.5.13 Increasing the Dimensions of an Array

`np.newaxis` and `np.expand_dims()` allow us to increase the dimensions of an array by adding new axes. Note that `numpy.newaxis` is an alias for `None`. Placing `np.newaxis` inside `[]` adds a new dimension of size 1 at that position.

Syntax: `numpy.expand_dims(arr, axis)`, where `axis` is the position where the new axis (or axes) is placed.

```
import numpy as np
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)                     # [1 2 3 4 5]
print(arr1[np.newaxis])         # [[1 2 3 4 5]]  add a row
print(arr1[None])               # [[1 2 3 4 5]]  np.newaxis == None

arr4 = np.array([[1, 2, 3], [4, 5, 6]])
# Add a new dimension of size 1 (a column)
print(arr4[:, :, np.newaxis].shape)     # (2, 3, 1)
# Add a new dimension of size 1 (a row)
```

```
print(arr4[:, np.newaxis, :].shape)     # (2, 1, 3)

print(np.expand_dims(arr4, 2).shape)    # (2, 3, 1)
print(np.expand_dims(arr4, 1).shape)    # (2, 1, 3)
```

1.5.14 View vs. Copy

- **View:** a reference to the original data — changes made through a view affect the original data.
- **Copy:** a new object with the same data — modifications to a copy do not affect the original. Useful for preserving the original data.

```
import numpy as np
import copy

# View
arr = np.array([1, 2, 3])
view_arr = arr[1:3]          # creates a view
view_arr[0] = 10             # modifies arr
print(arr)                   # [ 1 10  3]

# Copy
original = np.array([1, 2, 3])
copy_list = copy.copy(original)     # creates a copy
copy_list[0] = 10            # does not affect original
print(original)              # [1 2 3]
```

Operations that produce a **view**:

- `arr[1:3]` (slicing), `arr.reshape(...)`, `arr.transpose()`;
- `arr[:, :]` (subsetting), `arr.view()`, `arr[np.newaxis]`, `np.expand_dims(arr, axis)`.

Operations that produce a **copy**:

- `np.copy(arr)`, `arr.copy()`, `arr.astype(new_dtype)`, `np.array(arr)`;
- `arr.tolist()`, `arr.flatten()`, `arr[np.arange(n)]` (indexing with an array).

1.5.15 Broadcasting

Broadcasting is a powerful mechanism that allows NumPy to work with arrays of **different shapes** when performing arithmetic operations. Frequently we have a smaller array and a larger array, and we want to use the smaller one multiple times on the larger one. Broadcasting solves arithmetic between arrays of differing shapes by replicating the smaller array along the last mismatched dimension.

Motivation. We want to add the vector `v` to each row of the matrix `x`. An explicit loop works but could be slow when `x` is very large:

```
import numpy as np
x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
v = np.array([1, 0, 1])

y = np.empty_like(x)         # same shape as x
# Add v to each row with an explicit loop
for i in range(4):
    y[i, :] = x[i, :] + v
```

We could stack copies of `v` into a matrix `vv` and add element-wise (`vv = np.tile(v, (4, 1))`). Broadcasting performs the same computation without actually creating those copies:

```
x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
```

```
v = np.array([1, 0, 1])
y = x + v       # add v to each row using broadcasting
# x has shape (4, 3) and v has shape (3,); this works as if v
# had shape (4, 3), where each row was a copy of v
```

Broadcasting Rules

1. If the arrays do not have the same rank, **prepend the shape of the lower-rank array with 1s** until both shapes have the same length.
2. The two arrays are **compatible** in a dimension if they have the same size in that dimension, or if one of them has size 1 there.
3. The arrays can be broadcast together if they are compatible in all dimensions.
4. After broadcasting, each array behaves as if it had the shape equal to the element-wise maximum of the two input shapes.
5. In any dimension where one array had size 1 and the other > 1, the first behaves as if it were copied along that dimension.

Equivalently, aligning the two shapes from the right (and padding the shorter one with leading 1s), the result has shape

$$\text{shape}(A * B) = \big(\max(a_1, b_1),\ \max(a_2, b_2),\ \ldots,\ \max(a_k, b_k)\big),$$

where a_m and b_m are the sizes of the two arrays along the m-th aligned dimension.

Example. Given `A = np.array([1, 2, 3])` and `B = np.array([2])`:

- Same rank (both 1)? Yes.
- Compatible in all dimensions? Yes — `B` has size 1.
- So `A * B` works and gives `[2 4 6]`; `B` behaves as if `[2, 2, 2]`.

Given `A = np.array([1, 2, 3])` and `B = np.array([[4, 4, 4], [3, 3, 3]])`:

- `A` has rank 1 and `B` has rank 2, so prepend 1s to `A`: shape `(1, 3)`.
- Compatible in all dimensions? Yes.
- `A * B` gives `[[4 8 12], [3 6 9]]`; `A` behaves as if `[[1, 2, 3], [1, 2, 3]]`.

Array	Shape	Note
A	(1 (prepended), 3)	A has size 1
B	(2, 3)	same size in the last dimension
A * B	(2, 3)	result

Practice. For each pair, state whether they are compatible; if so, give the resulting shape of `A * B`.

Pair	A	B	Result
1	5×4	scalar (shape `1`)	Yes: 5×4
2	5×4	4	Yes: 5×4
3	$15 \times 3 \times 5$	$15 \times 1 \times 5$	Yes: $15 \times 3 \times 5$
4	$15 \times 3 \times 5$	3×5	Yes: $15 \times 3 \times 5$
5	$15 \times 3 \times 5$	3×1	Yes: $15 \times 3 \times 5$
6	$16 \times 6 \times 7$	16×6	No

Broadcasting in practice. Two illustrations:

Example 1: Centering an array (grade book). Suppose 4 students each took 3 exams, stored in a 4×3 array. Centering subtracts the per-column mean so that every column has mean 0:

$$x'_{ij} = x_{ij} - \bar{x}_j, \qquad \bar{x}_j = \frac{1}{4}\sum_{i=1}^{4} x_{ij}.$$

```
import numpy as np
scores = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
score_mean = scores.mean(0)     # 0 means the first dimension
print(score_mean)               # [5.5 6.5 7.5]

# Center the scores by subtracting the mean (broadcasting)
scores_centered = scores - score_mean
print(scores_centered.mean(0))  # [0. 0. 0.]
```

Example 2: Pairwise distances. Given `x` with shape (M, D) and `y` with shape (N, D), we want the Euclidean distance between each pair of rows (so $M \times N$ values):

$$d_{ij} = \|\mathbf{x}_i - \mathbf{y}_j\|_2 = \sqrt{\sum_{k=1}^{D} \left(x_{ik} - y_{jk}\right)^2}, \qquad i = 1, \ldots, M, \quad j = 1, \ldots, N.$$

The two arrays cannot be broadcast directly, so we reshape them:

```
import numpy as np
# x is a shape-(3, 3) array, y is a shape-(2, 3) array
reshaped_x = x.reshape(3, 1, 3)
reshaped_y = y.reshape(1, 2, 3)
diffs = reshaped_x - reshaped_y          # shape (3, 2, 3)
# Via broadcasting, diffs[i, j] stores x[i] - y[j]
dists = np.sqrt(np.sum(diffs ** 2, axis=2))     # shape (3, 2)
```

Try It Yourself

Can you compute the pairwise distances without `reshape`, using `np.newaxis` instead? And using `np.expand_dims`?

1.5.16 NumPy Arrays vs. Python Lists

NumPy arrays consume less memory than Python lists, are faster, and are more convenient to use.

- A NumPy array contains a **single pointer to one contiguous block of data**.
- A Python list contains a pointer to a **block of pointers**, each of which points to a full Python object.

Measuring actual memory, a list of 1000 elements occupies about **37116 bytes**, while a NumPy array of 1000 elements occupies only about **8104 bytes**.

Timing element-wise multiplication of two size-10^6 sequences: a Python list comprehension takes about `0.130 s`, while the NumPy array operation takes about `0.006 s`.

Finally, arithmetic differs: `[1, 2, 3] + 4` raises a `TypeError`, whereas `np.array([1, 2, 3]) + 4` gives `[5 6 7]`.

1.5.17 Practice Problems

Practice Problems (NumPy)

1. How to create array `A` of size 15, with all zeros?
2. How to find the memory size of array `A`?
3. How to create array `B` with values ranging from 20 to 60?
4. How to create array `C` as the reversed array of `B`?
5. How to create a 4×4 array `D` with values from 0 to 15 (top to bottom, left to right)?
6. How to find the dimensions of array `E = [[3, 4, 5], [6, 7, 8]]`?

7. How to find the indices of non-zero elements from `F = [0, 3, 0, 0, 4, 0]`?
8. How to create a $3 \times 3 \times 3$ array `G` with random values?
9. How to find the maximum value in `H = [1, 13, 0, 56, 71, 22]`?
10. How to find the minimum value in `H`?
11. How to find the mean of `H`?
12. How to find the standard deviation of `H`?
13. How to find the median of `H`?
14. How to transpose array `D`?
15. How to append `[4, 5, 6]` to `I = [1, 2, 3]`?
16. How to add, subtract, multiply, and divide two arrays `J = [1, 2, 3]` and `K = [4, 5, 6]` element-wise?
17. How to find the total sum of the elements of `I`?
18. How to find the natural log of `I`?
19. How to build `L = [8, 8, 8, 8, 8]` using the `full` or `repeat` function?
20. How to sort `M = [2, 5, 7, 3, 6]`?
21. How to find the index of the maximum value in `M`?
22. How to find the index of the minimum value in `M`?
23. How to find the indices that would sort `M`?
24. How to find the inverse of `N = [[6, 1, 1], [4, -2, 5], [2, 8, 7]]`?
25. How to find the absolute value of `N`?
26. How to extract the third column of `O = [[11, 22, 33], [44, 55, 66], [77, 88, 99]]`?
27. How to extract the sub-array of the odd rows and even columns of `P = [[3, 6, 9, 12], [15, 18, 21, 24], [27, 30, 33, 36], [39, 42, 45, 48], [51, 54, 57, 60]]`?

Answers

```
1.  A = np.zeros(15)
2.  print(A.size * A.itemsize)
3.  B = np.arange(20, 61)
4.  C = B[::-1]
5.  D = np.arange(16).reshape(4, 4)
6.  E = np.array([[3, 4, 5], [6, 7, 8]]); print(E.shape)
7.  F = np.array([0, 3, 0, 0, 4, 0]); print(F.nonzero())
8.  G = np.random.random((3, 3, 3))
9.  H = np.array([1, 13, 0, 56, 71, 22]); print(H.max())
10. print(H.min())
11. print(H.mean())
12. print(H.std())
13. print(np.median(H))
14. print(np.transpose(D))
15. I = np.array([1, 2, 3]); I = np.append(I, [4, 5, 6])
16. J = np.array([1, 2, 3]); K = np.array([4, 5, 6])
    print(J + K); print(J - K); print(J * K); print(J / K)
17. print(np.sum(I))
18. print(np.log(I))
19. L = np.full(5, 8)      # or L = np.repeat(8, 5)
20. M = np.array([2, 5, 7, 3, 6]); print(np.sort(M))
21. print(M.argmax())
22. print(M.argmin())
23. print(M.argsort())
24. N = np.array([[6, 1, 1], [4, -2, 5], [2, 8, 7]])
    print(np.linalg.inv(N))
25. print(np.abs(N))
26. O = np.array([[11, 22, 33], [44, 55, 66], [77, 88, 99]])
    print(O[:, 2])
27. P = np.array([[3, 6, 9, 12], [15, 18, 21, 24], [27, 30, 33,
```

```
    36],
                 [39, 42, 45, 48], [51, 54, 57, 60]])
     print(P[::2, 1::2])
```

2 Naive Bayes Classifier

2.1 Machine Learning Basics

2.1.1 Definition

Definition of Machine Learning

Machine Learning is the science and engineering of getting computers to **act without being explicitly programmed**.

There are three major types of machine learning:

1. **Supervised learning**
2. **Unsupervised learning**
3. **Reinforcement learning** (not covered in this course; self-study only)

2.1.2 Supervised Learning

In supervised learning, we form a **training dataset** consisting of examples with inputs and the correct outputs.

- The correct outputs are called **labels**, so the training dataset is called a **labelled dataset**.
- The labelled dataset is used to train the computer/machine to yield the desired output, i.e., the machine learns the relationship/mapping between the inputs and the outputs.

Supervised learning can be further divided into two types:

1. **Classification**: classify the data into categories (also called classes, represented using different labels). For example, yes or no; positive, negative or neutral.
2. **Regression**: give a real or continuous value as output. For example, salary based on work experience, weight based on height.

Example: The Idea of Supervised Learning

Suppose we want to train our machine to do maths. Instead of explicitly programming it, we pass a set of input data and the corresponding answers to the machine, and ask it to learn/figure out the relationship between the data and the answers:

1.	2	4	5 = 3	5.	6	2	2 = 10
2.	5	2	8 = 2	6.	3	1	1 = 2
3.	2	2	1 = 3	7.	5	3	4 = 11
4.	4	2	2 = 6	8.	1	8	1 = 7

Question: For this example, what is the training dataset? What is the label? Can you figure out the relationship between the data and the labels?

Answer:

- The **training dataset** is the set of 8 examples; each example has three inputs (e.g., $2, 4, 5$) and one correct output.
- The **label** of each example is the number after the "=" sign (e.g., 3 for the first example).
- A hidden relationship that fits all examples is:

$$\text{output} = (\text{first} \times \text{second}) - \text{third},$$

e.g., $2 \times 4 - 5 = 3$, $5 \times 2 - 8 = 2$, $6 \times 2 - 2 = 10$, $1 \times 8 - 1 = 7$.

Real-life applications of supervised learning.

- **Image classification** — e.g., Facebook can recognize your friend in a picture from an album of tagged photos.
- **Visual recognition** — e.g., the ability of a machine to identify objects, places, people, actions, and images.
- **Signature recognition** — e.g., automatic signature recognition for automating bank cheque processing.
- **Spam detection** — e.g., Google can detect and filter out spam and phishing emails with about 99.9% accuracy.
- **Weather forecasting** — e.g., the HK Observatory incorporates machine learning to strengthen its monitoring and forecasting techniques.

2.1.3 Unsupervised Learning

In unsupervised learning:

- We use an **unlabelled dataset** (i.e., the correct outputs are not given) and train the machine to learn on itself.
- The computer/machine tries to **find a pattern** in the unlabelled data and respond.

Unsupervised learning can be further divided into two groups:

1. **Clustering**: group data into clusters that are similar to each other within a cluster, and dissimilar to objects belonging to another cluster. For example, finding out which customers made similar product purchases.
2. **Association**: a rule-based machine learning approach to discover the probability of the co-occurrence of items in a collection. For example, finding out which products were purchased together.

Example: The Idea of Unsupervised Learning

Once again, suppose we want to train our machine to do maths. This time we pass a set of input data, but not the corresponding answers, and ask it to figure out the patterns:

1.	2	4	5 =?	5.	6	2	2 =?
2.	5	2	8 =?	6.	3	1	1 =?
3.	2	2	1 =?	7.	5	3	4 =?
4.	4	2	2 =?	8.	1	8	1 =?

Question: Is this a suitable task for the machine to learn the patterns?

Answer: No. Without any answers (labels), the machine has no signal telling it what kind of pattern to look for — the bare input triples do not reveal an arithmetic rule, nor do they form meaningful clusters. This task is inherently a supervised one; removing the labels makes it ill-posed.

Real-life applications of unsupervised learning.

- **Semantic clustering**:
  - Semantically similar words share a similar context.
  - Grouping images/videos based on similarities.
  - Grouping audios based on acoustic similarity.
- **Identifying accident-prone areas**: used to identify accident-prone areas and introduce safety measures based on the intensity of those accidents.

2.1.4 Comparison between Supervised and Unsupervised Learning

	Supervised Learning	**Unsupervised Learning**
Training data	Labeled data	Unlabeled data
Labeling	Supervised learning needs a domain expert to label data	
Categories	Classification and regression	Clustering and association
Goal	Calculate outcomes	Discover underlying patterns
Training time	Shorter	Longer
Accuracy	High	Low
Optimal strategy	Depend on the data	Depend on the data and learning algorithm
Algorithms	Naive Bayes, K-Nearest Neighbour, Decision Tree, Linear Regression, Logistic Regression, Support Vector Machine	K-Means Clustering, Hierarchical Clustering, Apriori Algorithm

2.1.5 Example: Disease 'Z'

Suppose we have a dataset consisting of symptoms and the results showing whether patients are suffering from disease 'Z' or not:

Data	**Blood Pressure**	**Fever**	**Diabetes**	**Vomit**	**Suffering from disease Z**
1	High	High	Yes	No	No
2	High	High	Yes	Yes	No
3	Low	High	Yes	No	Yes
4	Normal	Mild	Yes	No	Yes
5	Normal	No fever	No	No	Yes
6	Normal	No fever	No	Yes	No
7	Low	No fever	No	Yes	Yes
8	High	Mild	Yes	No	No
9	High	No fever	No	No	Yes
10	Normal	Mild	No	No	Yes
11	High	Mild	No	Yes	Yes
12	Low	Mild	Yes	Yes	Yes
13	Low	High	No	No	Yes
14	Normal	Mild	Yes	Yes	No

Question: Can we estimate whether a person suffers from disease 'Z' based on the symptoms (Blood Pressure = High, Fever = No, Diabetes = Yes, Vomit = Yes) from the above table?

Answer: Yes — use **Bayes' rule**.

2.2 Bayes' Rule

2.2.1 Conditional Probability

Conditional Probability

Conditional probability, denoted by $P(B \mid A)$, means: what is the probability that an event B occurs, **given that** event A has already occurred? For events A and B with $P(A) > 0$,

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)}.$$

Rearranging gives the **product (multiplication) rule**,

$$P(A \cap B) = P(B \mid A)\, P(A) = P(A \mid B)\, P(B).$$

Two events are **independent** iff $P(A \cap B) = P(A)\, P(B)$, i.e. $P(B \mid A) = P(B)$.

Example: Modes of Transport

The table below shows how undergraduate and postgraduate students travel:

Transport	**Car**	**Foot**	**Bicycle**	**Total**
Undergraduate Students	200	330	120	650
Postgraduate Students	250	320	80	650
Total	450	650	200	1300

Question 1: What is the probability $P(\text{Car} \mid \text{Undergraduate Students})$?
Answer: We condition on the row of undergraduate students (total = 650):

$$P(\text{Car} \mid \text{Undergraduate Students}) = \frac{200}{650} \approx 0.31$$

Question 2: What is the probability $P(\text{Bicycle} \mid \text{Postgraduate Students})$?
Answer: Conditioning on the postgraduate row (total = 650):

$$P(\text{Bicycle} \mid \text{Postgraduate Students}) = \frac{80}{650} \approx 0.12$$

2.2.2 Bayes' Rule

Bayes' rule is named after the 18th-century British mathematician **Thomas Bayes**. It is the most important rule in data science: it is the mathematical rule that describes **how to update a belief, given some evidence**.

Bayes' Rule

For a belief (class) B and evidence E with $P(E) > 0$,

$$P(B \mid E) = P(B) \times \frac{P(E \mid B)}{P(E)} = \frac{P(B)\,P(E \mid B)}{P(E)}.$$

Equivalently, since $P(E)$ does not depend on the belief, the posterior is **proportional** to prior $\times$ likelihood:

$$P(B \mid E) \;\propto\; P(B)\,P(E \mid B).$$

where

- E: **Evidence** (also called feature)
- B: **Belief** (also called class)
- $P(B \mid E)$: **Posterior probability** — how often the belief happens given that the evidence happens
- $P(B)$: **Prior probability** — how likely the belief is on its own
- $P(E \mid B)$: **Likelihood** — how often the evidence happens given that the belief happens
- $P(E)$: **Marginal probability** — how likely the evidence is on its own (computed by the law of total probability)

Example: Smoke and Fire

Let E: Smoke, B: Fire.

- $P(B)$: how often there is fire
- $P(E)$: how often there is smoke
- $P(B \mid E)$: how often there is fire when we can see smoke
- $P(E \mid B)$: how often there is smoke when there is fire

So Bayes' rule is a formula telling us $P(\text{Fire} \mid \text{Smoke})$ when we know $P(\text{Smoke} \mid \text{Fire})$.

Given: Dangerous fires are rare (1%): $P(B) = P(\text{Fire}) = 0.01$. Smoke is fairly

common (10%) due to BBQ: $P(E) = P(\text{Smoke}) = 0.1$. And 90% of dangerous fires make smoke: $P(E \mid B) = P(\text{Smoke} \mid \text{Fire}) = 0.9$.

Solution: The probability of a dangerous fire when there is smoke is

$$P(B \mid E) = P(\text{Fire} \mid \text{Smoke}) = \frac{P(\text{Fire})\,P(\text{Smoke} \mid \text{Fire})}{P(\text{Smoke})} = \frac{(0.01)(0.9)}{0.1} = 0.09$$

2.2.3 Probability Tree View

We can represent the relationship between E and B as a tree. Starting from the root, we first branch on the belief, then on the evidence:

- With probability $P(B)$ we reach belief B; from there the evidence E occurs with probability $P(E \mid B)$, and NOT E occurs with probability $P(\text{NOT } E \mid B)$.
- With probability $P(\text{NOT } B)$ we reach NOT B; from there the evidence E occurs with probability $P(E \mid \text{NOT } B)$, and NOT E occurs with probability $P(\text{NOT } E \mid \text{NOT } B)$.

Adding up the two branches on which the evidence E occurs gives the **law of total probability**:

$$P(E) = P(E \mid B)\,P(B) + P(E \mid \text{NOT } B)\,P(\text{NOT } B)$$

It is okay if we know $P(E \mid B)$, $P(B)$, $P(E \mid \text{NOT } B)$, $P(\text{NOT } B)$. So we have

$$P(B \mid E) = \frac{P(B)\,P(E \mid B)}{P(E \mid B)\,P(B) + P(E \mid \text{NOT } B)\,P(\text{NOT } B)} = \frac{P(B)\,P(E \mid B)}{P(E)}$$

Example: Virus Detection

A doctor gives a patient a test for a particular virus.

- It is believed that 0.1% of the population has this virus.
- In 99% of the cases in which the virus is present, the test is positive.
- In 95% of the cases in which the virus is not present, the test is negative.

Question: If the test turns out to be positive, what is the chance that the virus is present?

Solution (Bayes' rule). Let V be the part of the population carrying the virus, and H the remaining part. Then

$$P(V) = 0.001, \qquad P(H) = 1 - 0.001 = 0.999$$

$$P(+ \mid V) = 0.99, \qquad P(- \mid H) = 0.95$$

We want $P(V \mid +) =?$ According to Bayes' rule,

$$P(V \mid +) = \frac{P(V)\,P(+ \mid V)}{P(+)}$$

What is $P(+)$? In fact, by the law of total probability,

$$\begin{aligned} P(+) &= P(+ \mid V)\,P(V) + P(+ \mid H)\,P(H) \\ &= (0.99)(0.001) + (1 - 0.95)(0.999) \\ &= (0.99)(0.001) + (0.05)(0.999) \\ &= 0.05094 \end{aligned}$$

So,

$$P(V \mid +) = \frac{(0.001)(0.99)}{0.05094} \approx 0.02$$

Even with a positive test result, the patient only has about a 2% chance of carrying the virus!

A Positive Test Does Not Mean the Disease Is Likely

The virus-detection result ($\approx 2\%$) feels counter-intuitive because the test is "99% accurate". The key is the **base rate**: the virus is extremely rare ($P(V) = 0.001$), so the false positives from the huge healthy population (0.05×0.999) vastly outnumber the true positives (0.99×0.001). Never ignore the prior $P(B)$ when interpreting Bayes' rule — confusing $P(E \mid B)$ with $P(B \mid E)$ is the classic base-rate fallacy.

2.2.4 Multiple Prior Beliefs

Suppose there are n possible beliefs (classes) $B_1, B_2, \ldots, B_n$. The probability tree now has n branches at the top level: branch B_i is taken with probability $P(B_i)$, and from it the evidence E occurs with probability $P(E \mid B_i)$ (and NOT E with $P(\text{NOT } E \mid B_i)$).

General Form of Bayes' Rule

$$\begin{aligned} P(B_i \mid E) &= \frac{P(B_i)\, P(E \mid B_i)}{P(E \mid B_1)\, P(B_1) + P(E \mid B_2)\, P(B_2) + \cdots + P(E \mid B_n)\, P(B_n)} \\ &= \frac{P(B_i)\, P(E \mid B_i)}{\sum_{j=1}^{n} P(E \mid B_j)\, P(B_j)} \end{aligned}$$

Each belief B_i is also called a **class**. For instance, V (population carrying the virus) and H (the population that does not carry the virus) are two different classes in the virus detection example.

2.2.5 Multiple Evidence

Say we have d pieces of evidence $E = \{e_1, e_2, e_3, \ldots, e_d\}$. Then

$$\begin{aligned} P(B_i \mid E) &= \frac{P(B_i)\, P(E \mid B_i)}{P(E \mid B_1)\, P(B_1) + P(E \mid B_2)\, P(B_2) + \cdots + P(E \mid B_n)\, P(B_n)} \\ &= \frac{P(B_i)\, P\big((e_1, e_2, e_3, \ldots, e_d) \mid B_i\big)}{\sum_{j=1}^{n} P\big((e_1, e_2, e_3, \ldots, e_d) \mid B_j\big)\, P(B_j)} \end{aligned}$$

The difficulty is that the joint likelihood $P\big((e_1, e_2, \ldots, e_d) \mid B_i\big)$ is hard to estimate from data directly — this is exactly where the naive assumption of Naive Bayes comes in.

2.3 Naive Bayes

2.3.1 Naive Assumptions

The Bayes' rule is referred to as **Naive Bayes** when we assume that each piece of evidence **conditionally independently** and **equally** contributes to the probability of a sample belonging to a certain belief. In other words, the presence of one particular piece of evidence in a belief does not affect the presence of another one.

The Two Naive Assumptions

Let the input be a feature vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ and let $c \in \mathcal{C}$ denote a class.

- **Conditional independence**: given the class c, the features are assumed to be mutually independent, i.e. for all $i \neq k$,

$$P(x_i, x_k \mid c) = P(x_i \mid c)\, P(x_k \mid c).$$

  For example, given a certain outcome (e.g., going for a walk), the temperature being "Hot" has nothing to do with the humidity being "High", and the outlook being "Rainy" does not affect the wind.
- **Equal contribution**: each piece of evidence contributes to the outcome; knowing only temperature and humidity alone cannot predict the outcome accurately.

The conditional-independence assumption makes the **joint likelihood factorize** into a product of one-feature likelihoods:

$$P(\mathbf{x} \mid c) = P(x_1, x_2, \ldots, x_d \mid c) = \prod_{j=1}^{d} P(x_j \mid c).$$

The Assumptions Are (Almost) Never True

The assumptions made by Naive Bayes are **not** generally correct in real-world situations. In fact, the independence assumption is never correct — but the classifier often **works well in practice** despite it. Do not claim the features are truly independent; claim the model is useful anyway.

2.3.2 The Naive Bayes Formula

With each piece of evidence making an independent and equal contribution given the belief, Bayes' rule can be transformed to the following. Write the input as a feature vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ and let $\mathcal{C} = \{c_1, c_2, \ldots, c_n\}$ be the set of classes. Combining Bayes' rule with the conditional-independence factorization gives

$$P(c \mid \mathbf{x}) = \frac{P(c)\, P(\mathbf{x} \mid c)}{P(\mathbf{x})} = \frac{P(c) \prod_{j=1}^{d} P(x_j \mid c)}{\sum_{c' \in \mathcal{C}} P(c') \prod_{j=1}^{d} P(x_j \mid c')}.$$

Equivalently, in the belief/evidence notation (B_i for the class, $E = \{e_1, e_2, \ldots, e_d\}$ for the evidence):

$$\begin{aligned} P(B_i \mid E) &= \frac{P(B_i)\, P\big((e_1, e_2, e_3, \ldots, e_d) \mid B_i\big)}{\sum_{j=1}^{n} P\big((e_1, e_2, e_3, \ldots, e_d) \mid B_j\big)\, P(B_j)} \\ &= \frac{P(B_i)\, P(e_1 \mid B_i)\, P(e_2 \mid B_i)\, P(e_3 \mid B_i) \cdots P(e_d \mid B_i)}{\sum_{j=1}^{n} P(e_1 \mid B_j)\, P(e_2 \mid B_j)\, P(e_3 \mid B_j) \cdots P(e_d \mid B_j)\, P(B_j)} \end{aligned}$$

Question: Given a test data sample with evidence E, we want to know which belief (class) it belongs to, so we calculate $P(B_1 \mid E), P(B_2 \mid E), \ldots, P(B_n \mid E)$ following the Naive Bayes formula above. However, do we need to strictly follow the formula and compute the actual probability values?

Answer: No. If we are only interested in knowing which belief has the highest probability value rather than the actual probability value, we can further simplify the

equation by **removing the denominator**, because the denominator remains constant for a given input.

Naive Bayes Classification Rule

The predicted class is the **maximum a posteriori** (MAP) estimate, obtained by maximizing the posterior, or equivalently the unnormalized posterior:

$$\hat{y} = \arg\max_{c \in \mathcal{C}} P(c \mid \mathbf{x}) = \arg\max_{c \in \mathcal{C}} \; P(c) \prod_{j=1}^{d} P(x_j \mid c).$$

In the belief/evidence notation the same rule reads

$$B_{NB} = \arg\max_{B_i} \; P(B_i) \Big(P(e_1 \mid B_i)\, P(e_2 \mid B_i)\, P(e_3 \mid B_i) \cdots P(e_d \mid B_i) \Big).$$

When the classes are equally likely (uniform prior), $P(c)$ is constant and the MAP rule reduces to the **maximum-likelihood** (MLE) rule

$$\hat{y} = \arg\max_{c \in \mathcal{C}} \prod_{j=1}^{d} P(x_j \mid c).$$

The class with the highest score wins. (If actual probabilities are needed afterwards, normalize the scores so that they sum to 1.)

Now consider a feature e_j; suppose e_j has m possible values: $e_j \in \{v_1, v_2, \ldots, v_m\}$. Given a training set $\mathcal{D} = \{(\mathbf{x}^{(t)}, y^{(t)})\}_{t=1}^{N}$, the likelihood is estimated from the training data by **counting**: let

$$n_{ijk} = \#\{\, t : y^{(t)} = B_i,\; e_j^{(t)} = v_k \,\}, \qquad n_i = \#\{\, t : y^{(t)} = B_i \,\}.$$

Then the maximum-likelihood estimate is the relative frequency

$$\hat{P}(e_j = v_k \mid B_i) = \frac{n_{ijk}}{n_i} = \frac{P(e_j = v_k, B_i)}{P(B_i)} = \frac{\text{Count of } (e_j = v_k, B_i)}{\text{Count of } B_i}.$$

Question: What if there is no observation of $(e_j = v_k, B_i)$, i.e., Count of $(e_j = v_k, B_i) = 0$? This leads to the zero frequency problem below.

2.3.3 Laplace Smoothing

Zero Frequency Problem

If the categorical feature has a category in the test data set which was not observed in the training data set, then the model will assign a 0 probability to the whole product and will be unable to make a prediction. This is known as **Zero Frequency**. A single zero term wipes out the entire posterior, because everything is multiplied together.

Solution: α-Laplacian (add-α) Smoothing

Idea: pretend you have seen every value of the feature given every belief α more times than you actually did:

$$\hat{P}(e_j = v_k \mid B_i) = \frac{n_{ijk} + \alpha}{n_i + m\,\alpha} = \frac{\text{Count of } (e_j = v_k, B_i) + \alpha}{\text{Count of } B_i + m \cdot \alpha}, \qquad \alpha > 0,$$

where m is the number of possible values of e_j, n_{ijk} is the count of training examples with class B_i and feature value v_k, and n_i is the count of class B_i.

- $\alpha = 1$ is called **1-Laplace smoothing**: pretend that you have seen every value 1 more time than you actually did.

2.3.4 Example: Disease 'Z'

Example: Disease 'Z' Prediction (Discrete Features)

Using the disease-'Z' dataset of Section 2.1.5, for a patient with symptoms (Blood Pressure, Fever, Diabetes, Vomit) = (High, No, Yes, Yes), what is the probability of suffering from disease 'Z'?

Step 1: Count the occurrences. From the 14 records (class counts: Yes = 9, No = 5):

Blood Pressure counts		
	Yes	**No**
High	2	3
Normal	3	2
Low	4	0

Fever counts		
	Yes	**No**
High	2	2
Mild	4	2
No fever	3	1

Diabetes counts		
	Yes	**No**
Yes	3	4
No	6	1

Vomit counts		
	Yes	**No**
Yes	3	3
No	6	2

Disease 'Z'	
Yes	**No**
9	5

Step 2: Convert counts to relative frequencies (likelihoods and priors). Each table cell is a co-occurrence count $n_{ijk} = \#\{t : y^{(t)} = c,\ e_j^{(t)} = v_k\}$, and each likelihood cell instantiates the relative-frequency formula

$$\hat{P}(e_j = v_k \mid c) = \frac{n_{ijk}}{n_c}, \qquad \hat{P}(c) = \frac{n_c}{N},$$

where n_c is the class count and $N = 14$. Dividing each count by the corresponding class count (9 for Yes, 5 for No), and the class counts by 14:

$$P(\text{Yes}) = \tfrac{9}{14}, \qquad P(\text{No}) = \tfrac{5}{14}$$

$$\begin{aligned}
P(\text{BP=High} \mid \text{Yes}) &= \tfrac{2}{9} & P(\text{BP=High} \mid \text{No}) &= \tfrac{3}{5} \\
P(\text{Fever=No} \mid \text{Yes}) &= \tfrac{3}{9} & P(\text{Fever=No} \mid \text{No}) &= \tfrac{1}{5} \\
P(\text{Diab=Yes} \mid \text{Yes}) &= \tfrac{3}{9} & P(\text{Diab=Yes} \mid \text{No}) &= \tfrac{4}{5} \\
P(\text{Vomit=Yes} \mid \text{Yes}) &= \tfrac{3}{9} & P(\text{Vomit=Yes} \mid \text{No}) &= \tfrac{3}{5}
\end{aligned}$$

Step 3: Apply the Naive Bayes classification rule.

$$B_{NB} = \arg\max_{B_i}\ P(B_i)\left(P(e_1 \mid B_i)\, P(e_2 \mid B_i)\, P(e_3 \mid B_i) \cdots P(e_d \mid B_i)\right)$$

We compare the two scores (priors times products of likelihoods):

$$\begin{aligned}
&P(\text{DiseaseZ=Yes}) \cdot \big(P(\text{BP=High} \mid \text{Yes}) \cdot P(\text{Fever=No} \mid \text{Yes}) \\
&\qquad \cdot P(\text{Diabetes=Yes} \mid \text{Yes}) \cdot P(\text{Vomit=Yes} \mid \text{Yes})\big) \\
&= \frac{9}{14} \times \frac{2}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{3}{9} = 0.005291 \\
&P(\text{DiseaseZ=No}) \cdot \big(P(\text{BP=High} \mid \text{No}) \cdot P(\text{Fever=No} \mid \text{No}) \\
&\qquad \cdot P(\text{Diabetes=Yes} \mid \text{No}) \cdot P(\text{Vomit=Yes} \mid \text{No})\big) \\
&= \frac{5}{14} \times \frac{3}{5} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5} = 0.020571
\end{aligned}$$

Step 4: Decide and (optionally) normalize. Since $0.020571 > 0.005291$, the result is: **the patient is NOT suffering from disease 'Z'**.
After normalization, the posterior probabilities are

$$\frac{0.020571}{0.020571 + 0.005291} = 0.7954 \text{ (No)}, \qquad \frac{0.005291}{0.020571 + 0.005291} = 0.2046 \text{ (Yes)}$$

Scores vs. Probabilities

The argmax scores (0.005291, 0.020571) are unnormalized posteriors — they are enough for **classification** (just pick the larger one). If you need real probabilities, divide each score by the sum of all scores. Note also the potential zero-frequency trap in this very dataset: $P(\text{BP=Low} \mid \text{No}) = \frac{0}{5} = 0$, which would zero-out the "No" score for any future patient with Low blood pressure unless smoothing is applied.

2.3.5 Scikit-Learn Implementation

```
import numpy as np
# CategoricalNB is for doing Naive Bayes classifier for categorical
# features
from sklearn.naive_bayes import CategoricalNB

# Forming training data
# Features: Blood Pressure (0=High,1=Normal,2=Low), Fever
# (0=High,1=Mild,2=No Fever),
#           Diabetes (0=Yes,1=No), Vomit (0=Yes,1=No)
training = np.array([[0, 0, 0, 1], [0, 0, 0, 0], [2, 0, 0, 1], [1, 1,
   0, 1], [1, 2, 1, 1],
                     [1, 2, 1, 0], [2, 2, 1, 0], [0, 1, 0, 1], [0, 2,
   1, 1], [1, 1, 1, 1],
                     [0, 1, 1, 0], [2, 1, 0, 0], [2, 0, 1, 1], [1, 1,
   0, 0]])

# Forming the label set
# Labels: 0=Yes, 1=No
outcome = np.array([1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1])

# This is the new data to be evaluated using the trained AI (model)
new_sample = np.array([[0, 2, 0, 0]])

# Train Naive Bayes classifier according to training, outcome
clf = CategoricalNB(alpha=1.0e-10).fit(training, outcome)

# Perform classification on new_sample, and get the probability
# estimates
pred_class = clf.predict(new_sample)
prob = clf.predict_proba(new_sample)

# Print the results
print(pred_class, prob)
```

Notes on the Scikit-Learn Code

- `CategoricalNB` is the Naive Bayes classifier for **categorical** features; every category must be encoded as an integer $(0, 1, 2, \dots)$.
- The hyperparameter `alpha` is exactly the α of α-Laplace smoothing. Here `alpha=1.0e-10` ≈ 0, i.e., smoothing is (almost) switched off so that the results match the hand computation.
- `clf.predict(...)` returns the predicted class; `clf.predict_proba(...)` returns the normalized posterior probability of each class.

- The new sample $[0, 2, 0, 0]$ encodes (BP = High, Fever = No fever, Diabetes = Yes, Vomit = Yes), matching the worked example.

2.3.6 Gaussian Naive Bayes

In the last example, the symptoms Blood Pressure and Fever take discrete values. What if they are continuous? If Blood Pressure and Fever were continuous, the calculation would have been different. We assume the likelihood follows a **Gaussian (normal) distribution**:

Gaussian (Normal) Probability Density

The Gaussian (normal) density with mean μ and standard deviation $\sigma > 0$ is

$$f(x) = \frac{1}{\sqrt{2\pi}\,\sigma}\,\exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

The likelihood of feature x_j under class c is then this density with class- and feature-specific parameters:

$$P(x_j \mid c) = f(x_j; \mu_{jc}, \sigma_{jc}) = \frac{1}{\sqrt{2\pi}\,\sigma_{jc}}\,\exp\!\left(-\frac{(x_j-\mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

For a class c with n_c training examples, the parameters are the maximum-likelihood estimates (sample mean and variance)

$$\mu_{jc} = \frac{1}{n_c}\sum_{t:\,y^{(t)}=c} x_j^{(t)}, \qquad \sigma_{jc}^2 = \frac{1}{n_c}\sum_{t:\,y^{(t)}=c}\left(x_j^{(t)}-\mu_{jc}\right)^2.$$

We need to calculate the mean and standard deviation of Blood Pressure and Fever for each belief (i.e., suffering from disease 'Z', and not suffering from disease 'Z'). Suppose:

Blood Pressure	**Mean μ**	**Std. dev. σ**
Suffering from disease 'Z' = Yes	73	6.2
Suffering from disease 'Z' = No	75	7.9
Fever	**Mean μ**	**Std. dev. σ**
Suffering from disease 'Z' = Yes	79	10.2
Suffering from disease 'Z' = No	86	9.7

Question: Any potential problem of this approach? (Think: are BP and Fever really independent given the class? Are they really Gaussian? Also see the density-vs-probability note below.)

Example: Disease 'Z' Prediction (Continuous BP and Fever)

For a patient with Blood Pressure = 66, Fever = 97, Diabetes = Yes, Vomit = Yes:

Step 1: Gaussian likelihoods for class Yes.

$$P(\text{BP=66} \mid \text{DiseaseZ=Yes}) = \frac{1}{\sqrt{2\pi}\,(6.2)}\,\exp\!\left(-\frac{(66-73)^2}{2\,(6.2)^2}\right) = 0.0340187$$

$$P(\text{Fever=97} \mid \text{DiseaseZ=Yes}) = \frac{1}{\sqrt{2\pi}\,(10.2)}\,\exp\!\left(-\frac{(97-79)^2}{2\,(10.2)^2}\right) = 0.00824276$$

Step 2: Posterior score for class Yes (Diabetes and Vomit stay discrete, using

$\frac{3}{9}$ and $\frac{3}{9}$ from before):

$$P(\text{DiseaseZ=Yes}) \cdot \big(P(\text{BP=66} \mid \text{Yes}) \cdot P(\text{Fever=97} \mid \text{Yes}) \cdot P(\text{Diabetes=Yes} \mid \text{Yes}) \cdot P(\text{Vomit=Yes} \mid \text{Yes})\big)$$
$$= \frac{9}{14} \times (0.0340187) \times (0.00824276) \times \frac{3}{9} \times \frac{3}{9} = 0.00002$$

Step 3: Gaussian likelihoods for class No.

$$P(\text{BP=66} \mid \text{DiseaseZ=No}) = \frac{1}{\sqrt{2\pi}\,(7.9)} \exp\left(-\frac{(66-75)^2}{2\,(7.9)^2}\right) = 0.0263909$$

$$P(\text{Fever=97} \mid \text{DiseaseZ=No}) = \frac{1}{\sqrt{2\pi}\,(9.7)} \exp\left(-\frac{(97-86)^2}{2\,(9.7)^2}\right) = 0.0216215$$

Step 4: Posterior score for class No (discrete likelihoods $\frac{4}{5}$ and $\frac{3}{5}$):

$$P(\text{DiseaseZ=No}) \cdot \big(P(\text{BP=66} \mid \text{No}) \cdot P(\text{Fever=97} \mid \text{No}) \cdot P(\text{Diabetes=Yes} \mid \text{No}) \cdot P(\text{Vomit=Yes} \mid \text{No})\big)$$
$$= \frac{5}{14} \times (0.0263909) \times (0.0216215) \times \frac{4}{5} \times \frac{3}{5} = 0.0000978$$

Step 5: Decide and normalize. Since $0.0000978 > 0.00002$, the result is: **the patient is NOT suffering from disease 'Z'**.
After normalization:

$$\frac{0.0000978}{0.0000978 + 0.00002} = 0.8302 \text{ (No)}, \qquad \frac{0.00002}{0.0000978 + 0.00002} = 0.1698 \text{ (Yes)}$$

Density Value ≠ Probability

The probability of a continuous random variable taking an exact value is 0. Strictly speaking, the numbers 0.0340187, 0.00824276, etc. above are **density values** $f(x)$, not probabilities. For the sake of simplicity we use the notation $P(e_i \mid B_i)$ loosely here; substituting $x = e_i$ into $f(x)$ works for classification because the density ratios approximate the probability ratios of small intervals around e_i. For details, refer to the supplementary notes: Why can $P(e_i \mid B_i)$ be found by substituting $x = e_i$ into $f(x)$?

2.3.7 Underflow Prevention

Recall the Naive Bayes classifier:

$$B_{NB} = \arg\max_{B_i} \; P(B_i) \left(P(e_1 \mid B_i)\, P(e_2 \mid B_i)\, P(e_3 \mid B_i) \cdots P(e_d \mid B_i)\right)$$

Multiplying lots of probabilities (each between 0 and 1 by definition) can result in **floating-point underflow** — the product becomes smaller than the smallest representable floating-point number.

Log-Space Trick

Since $\log(xy) = \log(x) + \log(y)$ and $P(\mathbf{x} \mid c) = \prod_{j=1}^{d} P(x_j \mid c)$, the likelihood of one sample equals

$$\log P(\mathbf{x} \mid c) = \sum_{j=1}^{d} \log P(x_j \mid c).$$

It is therefore better to perform all computations by **summing logs of probabilities** rather than multiplying probabilities. Define the log-score

$$\text{score}(c) = \log P(c) + \sum_{j=1}^{d} \log P(x_j \mid c) = \log\left(P(c) \prod_{j=1}^{d} P(x_j \mid c) \right).$$

Because log is monotonically increasing, arg max is preserved, so the belief with the highest log-score is still the most probable:

$$B_{NB} = \arg\max_{B_i} \left(\log P(B_i) + \sum_{j=1}^{d} \log P(e_j \mid B_i) \right).$$

2.3.8 Applications

- **Real-time prediction**: the Naive Bayes algorithm is fast and always ready to learn, hence it is best suited for real-time predictions.
- **Multi-class prediction**: the probability of multiple classes of any target variable can be predicted using a Naive Bayes algorithm.
- **Text classification**: where Naive Bayes is mostly used is spam filtering in emails (Naive Bayes is widely used for text classification). Due to its better performance with multi-class problems and its independence rule, the Naive Bayes algorithm performs better, or has a higher success rate, in text classification. Therefore it is also used in sentiment analysis and spam filtering.
- **Recommendation systems**: a Naive Bayes classifier together with collaborative filtering builds a recommendation system that uses machine learning and data mining techniques to filter unseen information and predict whether a user would like a given resource or not.

2.3.9 Pros and Cons

Advantages

- It is very easy to implement and obtains very good results on large datasets.
- It has **computational efficiency**: the assumption that all features are independent makes a Naive Bayes algorithm very fast compared to complicated algorithms. In some cases, speed is preferred over higher accuracy.
- It works well on large-size datasets.
- It can also be used to predict multiple classes.
- It can also be used in Natural Language Processing for text classification.

Disadvantages

- The strong assumption that the features are independent is hardly true in real-life applications.
- If the dataset size is small, it will decrease the precision value.
- When the goal is predicting probability instead of classification, the method provides very **biased** results.
- It has trouble with the **zero frequency problem** (unless smoothing is applied).

2.3.10 Practice: Petting the Animal

Practice Problem: Petting Animals

Training dataset:

Data	Animals	Size	Color	Can we pet them?
1	Dog	Medium	Black	Yes
2	Dog	Big	White	No
3	Rat	Small	White	Yes
4	Cow	Big	White	Yes
5	Cow	Small	Brown	No
6	Cow	Big	Black	Yes
7	Rat	Big	Brown	No
8	Dog	Small	Brown	Yes
9	Dog	Medium	Brown	Yes
10	Cow	Medium	White	No
11	Dog	Small	Black	Yes
12	Rat	Medium	Black	No
13	Rat	Small	Brown	No
14	Cow	Big	White	Yes

Question: For the animal (Cow, Medium, Black), can we pet it?

Solution. Class counts: Yes = 8, No = 6 (out of 14). Given the Naive Bayes formula

$$P(B_i \mid E) \propto P(B_i)\left(P(e_1 \mid B_i)\,P(e_2 \mid B_i)\,P(e_3 \mid B_i)\right)$$

we have, with $(\text{Animals}, \text{Size}, \text{Color}) = (\text{Cow}, \text{Medium}, \text{Black})$:

$$P(\text{Pet=Yes}) \cdot \big(P(\text{Cow} \mid \text{Yes}) \cdot P(\text{Medium} \mid \text{Yes}) \cdot P(\text{Black} \mid \text{Yes})\big)$$
$$= \frac{8}{14} \times \frac{3}{8} \times \frac{2}{8} \times \frac{3}{8} = 0.02$$
$$P(\text{Pet=No}) \cdot \big(P(\text{Cow} \mid \text{No}) \cdot P(\text{Medium} \mid \text{No}) \cdot P(\text{Black} \mid \text{No})\big)$$
$$= \frac{6}{14} \times \frac{2}{6} \times \frac{2}{6} \times \frac{1}{6} = 0.0079$$

Result: We **can** pet it, since $0.02 > 0.0079$.
After normalization:

$$\frac{0.02}{0.02 + 0.0079} = 0.7168 \text{ (Yes)}, \qquad \frac{0.0079}{0.02 + 0.0079} = 0.2832 \text{ (No)}$$

The corresponding Scikit-Learn implementation:

```
import numpy as np
# CategoricalNB is for doing Naive Bayes classifier for categorical
# features
from sklearn.naive_bayes import CategoricalNB

# Forming training data
# Features: Animals (0=Dog,1=Rat,2=Cow), Size
# (0=Big,1=Medium,2=Small),
#           Color (0=Black, 1=White, 2=Brown)
training = np.array([[0, 1, 0], [0, 0, 1], [1, 2, 1], [2, 0, 1], [2,
   2, 2],
                     [2, 0, 0], [1, 0, 2], [0, 2, 2], [0, 1, 2], [2,
   1, 1],
                     [0, 2, 0], [1, 1, 0], [1, 2, 2], [2, 0, 1]])

# Forming the label set
# Labels: 0=Yes, 1=No
outcome = np.array([0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0])
```

```
# This is the new data to be evaluated using the trained AI (model)
new_sample = np.array([[2, 1, 0]])

# Train Naive Bayes classifier according to training, outcome
clf = CategoricalNB(alpha=1.0e-10).fit(training, outcome)

# Perform classification on new_sample and get the probability
# estimates
pred_class = clf.predict(new_sample)
prob = clf.predict_proba(new_sample)

# Print the results
print(pred_class, prob)
```

The new sample $[2, 1, 0]$ encodes (Animals = Cow, Size = Medium, Color = Black), and the classifier predicts class 0 (Yes — we can pet it), consistent with the hand computation.

3 K-Nearest Neighbors Classifier

3.1 Motivation: T-shirt Size

Motivating Problem

Suppose we have the height, weight and T-shirt size of some customers as follows:

Height (in cm)	158	158	158	160	160	163	163	160	163
Weight (in kg)	58	59	63	59	60	60	61	64	64
T-shirt Size	M	M	M	M	M	M	M	L	L

Height (in cm)	165	165	165	168	168	168	170	170	170
Weight (in kg)	61	62	65	62	63	66	63	64	68
T-shirt Size	L	L	L	L	L	L	L	L	L

Can we predict the T-shirt size of a new customer given only the height and weight information we have?

Questions to think about:

- Can we solve it using the Naive Bayes Classifier? How?
- What is the problem of the Naive Bayes model in this task?

Why Not Naive Bayes Here?

Height and weight are continuous numeric attributes, and they are clearly **not conditionally independent** (taller people tend to be heavier). Naive Bayes assumes conditional independence between features given the class, which is violated here and hurts its effectiveness. Formally, for features $\mathbf{x} = (x_1, \ldots, x_n)$ and class y it assumes

$$P(x_1, \ldots, x_n \mid y) = \prod_{j=1}^{n} P(x_j \mid y),$$

whereas here $P(\text{height}, \text{weight} \mid y) \neq P(\text{height} \mid y)\, P(\text{weight} \mid y)$. KNN makes no such assumptions, so it fits this task naturally.

3.2 The KNN Model

3.2.1 What is KNN?

K-Nearest Neighbor (KNN)

- K-Nearest Neighbor (aka KNN) is one of the most used machine learning algorithms due to its **simplicity**.
- KNN is a **lazy learning**, **non-parametric** algorithm, as it does not make any assumptions on the data being studied (e.g., the distribution of the data).
- It uses data with several classes to predict the classification of the new sample point. In other words, it captures information of all training data and classifies the new data based on **similarity**.

Why "Lazy" Learning?

KNN is called lazy because it does **no real training**: it simply stores/memorizes all the training data. All the computation (distance calculation, sorting, voting) is deferred until a test sample needs to be classified. There is no explicit model-building phase.

3.2.2 Parametric Model vs. Non-parametric Model

	Parametric Model	**Non-parametric Model**
Assumption	Relies on specific assumptions about the underlying distribution of the population being studied. Typically assumes that the data follows a known probability distribution, and estimates the parameters of this distribution using the available data.	Does not rely on specific assumptions about the underlying distribution of the population being studied.
Basic idea	There is a set of fixed parameters that are used to determine a probability model that is used in Machine Learning as well.	No need to make any assumption of parameters for the given population or the population we are studying. There is no fixed set of parameters available.
Example	Naive Bayes: classifies data points based on Bayes' theorem and assuming conditional independence between features.	KNN: classifies data points based on the k nearest neighbours.

3.2.3 Steps of the KNN Algorithm

The Six Steps of KNN

1. Prepare training data and test data.
2. Select a value K.
3. Determine which distance function is to be used.
4. Compute the distance of the new data to its n training samples.
5. Sort the distances obtained and take the K-nearest data samples.
6. Assign the test sample to the class based on the **majority vote** of its K nearest neighbors.

Which Steps are "Training" and Which are "Testing"?

Only **Step 1** (preparing/storing the training data) can be regarded as "training" — and that is exactly why KNN is called lazy: the training phase does essentially nothing but memorize the data. **Steps 2–6** happen at testing time: choosing K, computing distances, sorting and voting are all performed when a new test sample arrives.

3.2.4 The KNN Prediction Rule

Formally, let the training set be $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, where each example is a feature vector $\mathbf{x}_i \in \mathbb{R}^n$ with class label y_i. Given a query point $\mathbf{x} \in \mathbb{R}^n$, its k **nearest neighbours** are the training points closest to it under the chosen distance d:

$$N_k(\mathbf{x}) = \{\, i : d(\mathbf{x}_i, \mathbf{x}) \text{ is among the } k \text{ smallest over } \mathcal{D} \,\}.$$

Classification. Let c range over the class labels. Each neighbour casts one vote for its own class, and the prediction is the **majority vote**

$$\hat{y} = \arg\max_{c} \sum_{i \in N_k(\mathbf{x})} \mathbb{1}[\, y_i = c \,].$$

Regression. The prediction is simply the **average** of the neighbours' target values:

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i.$$

Because the prediction depends only on which Voronoi cell the query falls into, the resulting decision boundary is piecewise linear (a union of Voronoi-cell edges).

3.3 Example: T-shirt Size

3.3.1 Step 1–2: Prepare Data and Select K

Training data: the 18 customers given in the motivating problem (height, weight, T-shirt size).

Testing data: a new customer with

$$\text{Height} = 161 \text{ cm}, \qquad \text{Weight} = 61 \text{ kg}.$$

Let's pick $K = 5$.

3.3.2 Step 3: Choose the Distance

Let $\mathbf{x}_i \in \mathbb{R}^n$ be a training data point with n attributes,

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{in})^\top,$$

and let $\mathbf{x} \in \mathbb{R}^n$ be a testing data point (the query), which also has n attributes,

$$\mathbf{x} = (x_1, x_2, \ldots, x_n)^\top.$$

Assume **Euclidean distance** is used; in vector form this is the ℓ_2 norm of the difference,

$$d_2(\mathbf{x}_i, \mathbf{x}) = \|\mathbf{x}_i - \mathbf{x}\|_2 = \sqrt{\sum_{j=1}^{n} (x_{ij} - x_j)^2}.$$

3.3.3 Step 4: Compute Distances

For the testing point $\mathbf{x} = (161, 61)^\top$, e.g., for the first training sample $\mathbf{x}_1 = (158, 58)^\top$:

$$\|\mathbf{x}_1 - \mathbf{x}\|_2 = \sqrt{(158-161)^2 + (58-61)^2} = \sqrt{9+9} = \sqrt{18} \approx 4.24.$$

Repeating for all 18 training samples:

Height (cm)	158	158	158	160	160	163	163	160	163
Weight (kg)	58	59	63	59	60	60	61	64	64
Size	M	M	M	M	M	M	M	L	L
Distance	4.24	3.61	3.61	2.24	1.41	2.24	2.00	3.16	3.61

Height (cm)	165	165	165	168	168	168	170	170	170
Weight (kg)	61	62	65	62	63	66	63	64	68
Size	L	L	L	L	L	L	L	L	L
Distance	4.00	4.12	5.66	7.07	7.28	8.60	9.22	9.49	11.40

3.3.4 Step 5: Take the K Nearest

Height (cm)	160	163	160	163	160	158	158	163	165
Weight (kg)	60	61	59	60	64	59	63	64	61
Size	M	M	M	M	L	M	M	L	L
Distance	1.41	2.00	2.24	2.24	3.16	3.61	3.61	3.61	4.00

Height (cm)	165	158	165	168	168	168	170	170	170
Weight (kg)	62	58	65	62	63	66	63	64	68
Size	L	M	L	L	L	L	L	L	L
Distance	4.12	4.24	5.66	7.07	7.28	8.60	9.22	9.49	11.40

The neighbour set $N_k(\mathbf{x})$ is obtained by keeping the $k = 5$ entries with the smallest distances; here

$$\{d_2(\mathbf{x}_i, \mathbf{x}) : i \in N_5(\mathbf{x})\} = \{1.41,\ 2.00,\ 2.24,\ 2.24,\ 3.16\}.$$

3.3.5 Step 6: Majority Vote

Height (cm)	160	163	160	163	160
Weight (kg)	60	61	59	60	64
Size	M	M	M	M	L
Distance	1.41	2.00	2.24	2.24	3.16

Among the 5 customers, 4 of them had "Medium T-shirt Sizes" and 1 had "Large T-shirt Size". Writing $c \in \{\mathrm{M}, \mathrm{L}\}$ for the candidate labels, the vote counts are

$$\sum_{i \in N_k(\mathbf{x})} \mathbb{1}[y_i = \mathrm{M}] = 4, \qquad \sum_{i \in N_k(\mathbf{x})} \mathbb{1}[y_i = \mathrm{L}] = 1,$$

so the majority-vote rule $\hat{y} = \arg\max_c \sum_{i \in N_k(\mathbf{x})} \mathbb{1}[y_i = c]$ yields $\hat{y} = \mathrm{M}$.

Conclusion of the Worked Example

We classify the test data as "Medium T-shirt", i.e., the best guess for the customer with height = 161 cm and weight = 61 kg is **Medium T-shirt Size**.

3.4 Scikit-Learn Implementation

```
# Import LabelEncoder function
from sklearn.preprocessing import LabelEncoder
# Import KNeighborsClassifier function
from sklearn.neighbors import KNeighborsClassifier
import numpy as np                                        # Import NumPy

# Assign features and label variables
height = np.array ([158,158,158,160,160,163,163,160,163,165,
                    165,165,168,168,168,170,170,170])
weight =
   np.array ([58,59,63,59,60,60,61,64,64,61,62,65,62,63,66,63,64,68])
size = np.array(['M', 'M', 'M', 'M', 'M', 'M', 'M', 'L', 'L',
                 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L'])

# Create LabelEncoder that used to convert string labels to numbers
encoder = LabelEncoder()
# Convert string labels into numbers
label = encoder.fit_transform(size)
# Combine height and weight into single list of tuples
features = list(zip(height, weight))
# Create a KNN classifier and set K=5
model = KNeighborsClassifier(n_neighbors=5)
# Train the model using the training sets
model.fit(features, label)
# Predict output
predicted = model.predict([[161, 61]])   # height = 161, weight = 61
# Convert number to string label
predicted = encoder.inverse_transform(predicted)
print(predicted)
```

Output: `['M']` — the model predicts Medium, matching our manual computation.

3.5 Standardization

Why Standardize?

When the training data are measured in **different units**, it is important to standardize the variables (i.e., the attributes) before calculating distance. For example, if one attribute is height in cm and the other is weight in kg, then height will influence the distance calculation more (its numeric range is larger). To make the attributes comparable, we apply z-score standardization: each raw attribute value x is replaced by

$$z = \frac{x - \mu}{\sigma},$$

where μ is the mean and σ is the standard deviation of that attribute over the training data. For a set of values $\{x_1, \ldots, x_N\}$ these statistics are

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}.$$

An alternative rescaling is min–max normalization,

$$x' = \frac{x - \min_i x_i}{\max_i x_i - \min_i x_i},$$

which maps the attribute into $[0, 1]$, though it is more sensitive to outliers than z-score standardization.

3.5.1 Effect of Standardization

For the T-shirt data:

$$\text{Mean of height} = 164, \quad \text{Std of height} = 4.33,$$
$$\text{Mean of weight} = 62.33, \quad \text{Std of weight} = 2.63.$$

After standardization with $z = (x - \mu)/\sigma$, the **5th closest value changed**, as height was dominating earlier before standardization. Testing data: Height = 161 cm and weight = 61 kg.

Height (cm)	160	163	160	163	160	158	158	163	165
Weight (kg)	60	61	59	60	64	59	63	64	61
Size	M	M	M	M	L	M	M	L	L
Distance	1.41	2.00	2.24	2.24	3.16	3.61	3.61	3.61	4.00
Distance (std.)	0.44	0.46	0.60	0.79	1.16	1.03	1.03	1.23	0.92

Height (cm)	165	158	165	168	168	168	170	170	170
Weight (kg)	62	58	65	62	63	66	63	64	68
Size	L	M	L	L	L	L	L	L	L
Distance	4.12	4.24	5.66	7.07	7.28	8.60	9.22	9.49	11.40
Distance (std.)	1.00	1.33	1.78	1.66	1.79	2.49	2.22	2.37	3.37

Sorting by the standardized distances: $0.44, 0.46, 0.60, 0.79, 0.92, 1.00, \ldots$ — the sample $(165, 61)$ with standardized distance 0.92 now enters the top-5, replacing the sample with raw distance 3.16 (standardized 1.16). The 5 nearest are still 4 M's and 1 L, so the classification remains **M**.

3.5.2 Implementation with Standardization

```
# Import LabelEncoder function
from sklearn.preprocessing import LabelEncoder
# Import KNeighborsClassifier function
from sklearn.neighbors import KNeighborsClassifier
import numpy as np                                    # Import NumPy

# Assign features and label variables
height = np.array ([158,158,158,160,160,163,163,160,163,165,
                    165,165,168,168,168,170,170,170])
# ddof=1 is sample std
h_mean = np.mean(height); h_std = np.std(height, ddof=1)
height = (height - h_mean) / h_std
weight =
   np.array ([58,59,63,59,60,60,61,64,64,61,62,65,62,63,66,63,64,68])
# ddof=1 is sample std
w_mean = np.mean(weight); w_std = np.std(weight, ddof=1)
weight = (weight - w_mean) / w_std
size = np.array(['M', 'M', 'M', 'M', 'M', 'M', 'M', 'L', 'L',
                 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L'])

# Create LabelEncoder that used to convert string labels to numbers
encoder = LabelEncoder()
# Convert string labels into numbers
label = encoder.fit_transform(size)
# Combine height and weight into single list of tuples
features = list(zip(height, weight))
# Create a KNN classifier and set K=5
model = KNeighborsClassifier(n_neighbors=5)
# Train the model using the training sets
model.fit(features, label)
# Predict output (the test point must be standardized with the SAME
```

```
# mean/std)
# (161, 61)
predicted = model.predict([[(161 - h_mean) / h_std, (61 - w_mean) /
   w_std]])
# Convert number to string label
predicted = encoder.inverse_transform(predicted)
print(predicted)
```

Standardize the Test Point with Training Statistics

The test sample must be standardized using the **mean and standard deviation of the training data** (not its own statistics, and not left unstandardized). Also note `np.std(..., ddof=1)` uses the sample standard deviation (divisor $n-1$).

3.6 Distance Functions

There are a lot of different distance functions available. Some common ones are:

- Euclidean distance (the one we used earlier)
- Manhattan distance (aka City block distance)
- Cosine distance
- Hamming distance

3.6.1 Euclidean Distance

For two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the Euclidean distance is the ℓ_2 norm of their difference:

$$d_2(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}.$$

3.6.2 Minkowski Distance

The Minkowski distance generalizes both the Manhattan ($p = 1$) and Euclidean ($p = 2$) distances:

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}, \qquad p \geq 1.$$

As $p \to \infty$ it approaches the Chebyshev (ℓ_∞) distance $\max_i |x_i - y_i|$.

3.6.3 Manhattan Distance

Manhattan Distance (City-block Distance)

The Manhattan distance is the sum of the absolute differences between points across all the dimensions, i.e., the ℓ_1 norm:

$$d_1(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_1 = \sum_{i=1}^{n} |x_i - y_i|,$$

where $|\cdot|$ denotes absolute value.

Manhattan Distance: Example

Training data $\mathbf{x} = (160, 60)^\top$; Testing data $\mathbf{y} = (163, 61)^\top$.

$$d_1(\mathbf{x}, \mathbf{y}) = |160 - 163| + |60 - 61| = 3 + 1 = 4.$$

Question: When is Manhattan distance preferred over Euclidean distance? (Manhattan is often preferred for high-dimensional data or when movement is restricted to axis-aligned directions, e.g., city blocks / grid paths.)

3.6.4 Cosine Distance

Cosine Distance

Based on the cosine of the angle θ between the two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

$$\cos\theta = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \, \sqrt{\sum_{i=1}^n y_i^2}} \in [-1, 1].$$

The cosine distance is then defined as

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \cos\theta.$$

Cosine Distance: Example

Training data $\mathbf{x} = (160, 60)^\top$; Testing data $\mathbf{y} = (163, 61)^\top$.

$$\cos\theta = \frac{(160 \times 163) + (60 \times 61)}{\sqrt{160^2 + 60^2} \, \sqrt{163^2 + 61^2}} = 0.99999977,$$

hence

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - 0.99999977 = 2.26 \times 10^{-7}.$$

Interpreting Cosine Distance

Using Cosine distance, we get values between 0 and 2, where:
- 0 means the training data and test data are 100% similar to each other,
- 2 means they are absolutely different,
- values between 0 and 2 mean varying degrees of dissimilarity.

Cosine distance measures orientation (angle), not magnitude — useful when only the direction of the vector matters.

3.6.5 Hamming Distance

Hamming Distance

Hamming distance is a way for comparing two binary data strings. While comparing two binary strings $\mathbf{u}, \mathbf{v} \in \{0, 1\}^n$ of equal length, the Hamming distance is the **number of bit positions in which the two bits are different**:

$$d_H(\mathbf{u}, \mathbf{v}) = \sum_{i=1}^{n} \mathbb{1}[\, u_i \neq v_i \,].$$

Hamming Distance: Example

Suppose there are two strings `1101 1001` and `1001 1101`. Comparing bit by bit:

$$\begin{array}{cccccccc} 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ & +1 & & & & +1 & & \end{array}$$

The Hamming distance between the two strings is 2.

When to Use Hamming Distance

Hamming distance is for **categorical data**. You will encounter bit strings when you one-hot encode categorical columns of data. More details will be given in a later topic.

3.7 Selecting K

3.7.1 K and Outliers

Sensitivity to Outliers

A **low K-value is sensitive to outliers**, and a **higher K-value is more resilient to outliers** as it considers more voters to decide the prediction.

3.7.2 Good Value of K

Choosing a Good K and Breaking Ties

To break a tie:
- Decrease K by 1 until we have broken the tie.
- Put more weight on the nearest points than on the farther points.

K should not be too small or too large.
Rule of thumb: Set

$$K = \sqrt{N},$$

where N is the number of training examples in the dataset. (An explanation will be given in a supplementary note.)

Weighted KNN. To reduce the influence of far-away neighbours (and to break ties), we may weight each neighbour $i \in N_k(\mathbf{x})$ by a decreasing function of its distance $d_i = \|\mathbf{x}_i - \mathbf{x}\|_2$, for example

$$w_i = \frac{1}{d_i} \qquad \text{or} \qquad w_i = \frac{1}{d_i^2}.$$

The classification rule then becomes a weighted majority vote

$$\hat{y} = \arg\max_c \sum_{i \in N_k(\mathbf{x})} w_i \mathbb{1}[\, y_i = c \,],$$

and weighted KNN regression predicts the weighted average

$$\hat{y} = \frac{\sum_{i \in N_k(\mathbf{x})} w_i \, y_i}{\sum_{i \in N_k(\mathbf{x})} w_i}.$$

Setting all weights to $w_i = 1$ recovers the unweighted rules from the previous subsection.

3.7.3 Using Cross-validation to Estimate K

Suppose we select K using the d-fold cross validation algorithm.

Input of the d-fold cross-validation algorithm:

- A KNN model
- A list of candidate K values: $[K_1, K_2, \ldots, K_q]$
- A training dataset consisting of N data samples
- The number of folds: d

Output: the best K value among $[K_1, K_2, \ldots, K_q]$.

Steps of d-fold Cross-validation

1. Split the training data into d groups, or folds, of approximately equal size.
2. Hold the first group. This is called the **validation set**.
3. Train your classifier on the remaining data.
4. For each value of K:
   - Classify each data in the validation set, using its K-nearest neighbors in the training set.
   - Record the error.
5. Repeat steps 2–4 for all d choices of the validation set.
6. For each choice of K, find the **average error** across validation sets. Choose the value of K with the lowest error.

Questions:

- How many errors are computed during the cross-validation process? (For each fold, one error per candidate K; with d folds and q candidate values of K, a total of $d \times q$ errors are computed, which are then averaged per K.)
- How to evaluate the model, i.e., measure the error? (This leads us to the next section — model evaluation.)

3.8 Model Evaluation

Model evaluation for classification problems:

- Confusion Matrix
- Precision, Recall, and F1 Score
- Matthew's Correlation Coefficient (MCC)

3.8.1 2×2 Confusion Matrix for Binary Classification

The Four Entries of a Confusion Matrix

- **True Positive (TP):** the number of predictions where the classifier correctly predicts the positive class as positive.
- **True Negative (TN):** the number of predictions where the classifier correctly predicts the negative class as negative.
- **False Positive (FP):** the number of predictions where the classifier incorrectly predicts the negative class as positive.
- **False Negative (FN):** the number of predictions where the classifier incorrectly predicts the positive class as negative.

From the confusion matrix:

$$\text{Accuracy} = \frac{TP + TN}{FP + FN + TP + TN} = 1 - \text{Error}$$

$$\text{Error} = \frac{FP + FN}{FP + FN + TP + TN} = 1 - \text{Accuracy}$$

3.8.2 Precision and Recall

Precision and Recall

Precision: what fraction of samples predicted as a positive class were actually positive:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: what fraction of positive samples were correctly predicted as positive by the classifier:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Interpretation:

- **Precision** tells us how much we can trust the model when it predicts an individual sample as Positive.
- **Recall** measures the ability of the model to find all the Positive samples in the dataset.

Precision–Recall Trade-off

The model may have high precision while low recall, or high recall while low precision. Imagine that we simply classify all data as positive:

- Recall $= \frac{TP}{TP + FN} = 1$ (every positive is found, $FN = 0$);
- Precision $= \frac{TP}{TP + FP}$ becomes low, because all negative samples become false positives.

So a high recall alone does **not** mean a good classifier.

3.8.3 F1-score

F1-score

F1-score combines precision and recall into a single measure. Mathematically, it is the **harmonic mean** of precision and recall:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

In a perfect world, we want a model that has a precision of 1 and a recall of 1. That means an F1-score of 1, i.e., 100% accuracy — which is often not the case for a machine learning model.

3.8.4 MCC

Matthew's Correlation Coefficient

Matthew's Correlation Coefficient (aka MCC) is a **best single-value classification metric** which helps to summarize the confusion matrix (or an error matrix):

$$\text{MCC} = \frac{TN \times TP - FN \times FP}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

If the prediction returns good rates for all 4 of these entities (i.e., TP, TN, FP, FN), it is said to be a reliable measure producing high scores.
To suit most correlation coefficients, MCC also ranges between $+1$ and -1:

- $+1$ is the best agreement between the predicted and actual values.
- 0 is no agreement — meaning that the prediction is random according to the actuals.
- -1 indicates total disagreement.

Example of MCC

Confusion matrix with entries:

$$TP = 90, \qquad FP = 4, \qquad TN = 1, \qquad FN = 5.$$

Substituting these values into the formula:

$$\text{MCC} = \frac{1 \times 90 - 5 \times 4}{\sqrt{(90+4)(90+5)(1+4)(1+5)}} = \frac{90 - 20}{\sqrt{94 \times 95 \times 5 \times 6}} \approx 0.14.$$

0.14 means the classifier is **very close to a random guess classifier**. Hence, MCC helps us to identify the ineffectiveness of the classifier in classifying especially the negative class samples (note $TN = 1$ out of $1 + 4 = 5$ actual negatives) — something that a high accuracy ($\frac{90+1}{100} = 0.91$) alone would hide.

3.8.5 Multi-class Confusion Matrix

Unlike binary classification, there are **no positive or negative classes**. We compute TP, TN, FP, FN per class (one-vs-rest).

Consider the following 3×3 confusion matrix for three classes (Apple, Orange, Mango), reconstructed from the lecture example:

		Predicted		
		Apple	Orange	Mango
Actual	Apple	7	1	3
Actual	Orange	8	2	2
Actual	Mango	9	3	1

Per-class Metrics: Apple

To find TP, TN, FP and FN for class **Apple**:
- **TP**: both actual and predicted values are apple. So $TP = 7$.
- **TN**: both actual and predicted values are non-apple. So $TN = 1 + 2 + 2 + 3 = 8$.
- **FP**: the predicted value is apple, but the actual value is non-apple. So $FP = 8 + 9 = 17$.
- **FN**: the predicted value is non-apple, but the actual value is apple. So $FN = 1 + 3 = 4$.

Therefore:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{7}{7 + 17} = 0.29$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{7}{7 + 4} = 0.64$$

$$\text{F1-score} = \frac{2TP}{2TP + FP + FN} = \frac{2(7)}{2(7) + 17 + 4} = \frac{14}{35} = 0.4$$

Similarly, we can calculate the measures for the other classes:

Class	**Precision**	**Recall**	**F1-score**
Apple	0.29	0.64	0.40
Orange	0.33	0.17	0.22
Mango	0.17	0.08	0.11

Question: How to combine the F1-score of each class to have a single measure for the whole model?

- Micro F1
- Macro F1
- Weighted F1

3.8.6 Micro F1

Micro F1 (Micro-averaged F1-score)

- It is calculated by considering the **total TP, total FP and total FN** of the model.
- It does not consider each class individually; it calculates the metrics **globally**.

Micro F1: Worked Example

For our example:

$$\text{Total } TP = 7 + 2 + 1 = 10$$
$$\text{Total } FP = (8 + 9) + (1 + 3) + (3 + 2) = 17 + 4 + 5 = 26$$
$$\text{Total } FN = (1 + 3) + (8 + 2) + (9 + 3) = 4 + 10 + 12 = 26$$
$$\text{Precision} = \frac{10}{10 + 26} = 0.28, \qquad \text{Recall} = \frac{10}{10 + 26} = 0.28$$
$$\text{Micro F1} = \frac{2TP}{2TP + FP + FN} = \frac{2(10)}{2(10) + 26 + 26} = 0.28$$

Now you can see that since we are calculating the metrics globally, all the measures become equal:

$$\text{Precision} = \text{Recall} = \text{Micro F1} = \text{Accuracy}.$$

3.8.7 Macro F1

Macro F1 (Macro-averaged F1-score)

It calculates the metrics for each class **individually** and then takes the **unweighted mean** of the measures.

Macro F1: Worked Example

According to the data in the table shown earlier:

$$\text{Apple's F1-score} = 0.40, \quad \text{Orange's F1-score} = 0.22, \quad \text{Mango's F1-score} = 0.11$$

Hence,

$$\text{Macro F1} = \frac{0.40 + 0.22 + 0.11}{3} = 0.24.$$

3.8.8 Weighted F1

Weighted F1 (Weighted-averaged F1-score)

Unlike Macro F1, it takes a **weighted mean** of the measures. The weight for each class is the **total number of samples** of that class.

Weighted F1: Worked Example

Since we had 11 Apples, 12 Oranges, and 13 Mangoes:

$$\text{Weighted F1} = \frac{(0.4 \times 11) + (0.22 \times 12) + (0.11 \times 13)}{11 + 12 + 13} = 0.24.$$

3.8.9 Regression Errors

There are many ways to measure errors for regression. Here are a few examples, where $(a_1, a_2, \ldots, a_m)$ are the actual labels and $(p_1, p_2, \ldots, p_m)$ are the predicted labels.

Regression Error Metrics

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{\sum_{i=1}^{m} |a_i - p_i|}{m}$$

Mean Square Error (MSE):

$$\text{MSE} = \frac{\sum_{i=1}^{m} (a_i - p_i)^2}{m}$$

Mean Absolute Percentage Error (MAPE):

$$\text{MAPE} = \frac{\sum_{i=1}^{m} \frac{|a_i - p_i|}{a_i}}{m}$$

3.9 KNN Analysis

3.9.1 Pros and Cons

Pros of KNN

- Easy to understand.
- No assumptions about data.
- Can be applied to both **classification and regression**. (For KNN regression, the resulting value can be calculated by taking the average of the output values of the top K nearest neighbors, i.e., $\hat{y} = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i$.)
- Works easily on multi-class problems.

Cons of KNN

- Memory intensive / computationally expensive (must store all training data and compute distances to every training sample at prediction time).
- Sensitive to the scale of data (hence standardization is important).
- Struggles when there is a high number of attributes (curse of dimensionality).
- Does not work well with categorical features, since it is difficult to find the distance between dimensions with categorical features.

3.9.2 Curse of Dimensionality

In the cons above, the "curse of dimensionality" can be made precise. For a fixed number of samples N in n dimensions, the average squared Euclidean distance to the nearest

neighbour grows (roughly) with n, while the relative spread of distances shrinks:

$$\frac{d_{\max}(\mathbf{x}) - d_{\min}(\mathbf{x})}{d_{\min}(\mathbf{x})} \to 0 \quad \text{as } n \to \infty,$$

where $d_{\min}(\mathbf{x})$ and $d_{\max}(\mathbf{x})$ are the smallest and largest distances from $\mathbf{x}$ to the training points. In other words, in high dimensions all points become almost equidistant, so the notion of "nearest neighbour" loses meaning and KNN degrades.

3.9.3 Speed Up KNN

Ways to Speed Up KNN

- Reduce the dimension of training data (e.g., using Principal Component Analysis).
- Use good data structures to store the data, e.g., **KD-tree**.
- Parallelize the distance computations.

3.9.4 KNN Applications

- Handwritten character classification
- Fast content-based image retrieval
- Intrusion detection
- Fault detection for semiconductor manufacturing processes
- ...

3.10 Practice: Purchase Prediction

Practice Problem

Suppose we have the age and salary of some customers, as well as whether they purchased a certain item or not:

Age	19	35	26	27	19	27	27	32	25	35	26
Salary (k)	19	20	43	57	76	58	84	150	33	65	80
Purchased	No	No	No	No	No	No	No	Yes	No	No	No

Age	26	20	32	18	29	47	45	46	48	45	47
Salary (k)	52	86	18	82	80	25	26	28	29	22	49
Purchased	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes

Can we predict whether a new customer will purchase the item given only the age and salary of the customer?

3.10.1 Step 1–2: Prepare Data and Select K

Training data: the 22 customers above. **Testing data:**

$$\text{Age} = 20, \qquad \text{Salary} = 38\text{k}.$$

Let's pick $K = 5$.

3.10.2 Step 3: Choose the Distance

Let $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{in})^\top$ be a training point and $\mathbf{x} = (x_1, x_2, \ldots, x_n)^\top$ the query. Using Euclidean distance:

$$d_2(\mathbf{x}_i, \mathbf{x}) = \|\mathbf{x}_i - \mathbf{x}\|_2 = \sqrt{\sum_{j=1}^{n} (x_{ij} - x_j)^2}.$$

3.10.3 Step 4: Compute Distances

For the test point $\mathbf{x} = (20, 38)^\top$, e.g., for the first training sample $\mathbf{x}_1 = (19, 19)^\top$:

$$\|\mathbf{x}_1 - \mathbf{x}\|_2 = \sqrt{(19-20)^2 + (19-38)^2} = \sqrt{1 + 361} = \sqrt{362} \approx 19.03.$$

Age	19	35	26	27	19	27	27	32	25	35	26
Salary (k)	19	20	43	57	76	58	84	150	33	65	80
Purchased	No	No	No	No	No	No	No	Yes	No	No	No
Distance	19.03	23.43	7.81	20.25	38.01	21.19	46.53	112.64	7.07	30.89	42.44

Age	26	20	32	18	29	47	45	46	48	45	47
Salary (k)	52	86	18	82	80	25	26	28	29	22	49
Purchased	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes
Distance	15.23	48.00	23.32	44.05	42.95	29.97	27.73	27.86	29.41	29.68	29.15

3.10.4 Step 5: Take the 5 Nearest

Age	25	26	26	19	27	27	32	35	45	46	47
Salary (k)	33	43	52	19	57	58	18	20	26	28	49
Purchased	No	No	No	No	No	No	No	No	Yes	Yes	Yes
Distance	7.07	7.81	15.23	19.03	20.25	21.19	23.32	23.43	27.73	27.86	29.15

Age	48	45	47	35	19	26	29	18	27	20	32
Salary (k)	29	22	25	65	76	80	80	82	84	86	150
Purchased	Yes	Yes	No	No	No	No	No	No	No	No	Yes
Distance	29.41	29.68	29.97	30.89	38.01	42.43	42.95	44.05	46.53	48.00	112.64

3.10.5 Step 6: Majority Vote

Age	25	26	26	19	27
Salary (k)	33	43	52	19	57
Purchased	No	No	No	No	No
Distance	7.07	7.81	15.23	19.03	20.25

Among the 5 customers, **5 of them did not purchase the item**. With labels $c \in \{\text{Yes}, \text{No}\}$, the vote counts are

$$\sum_{i \in N_k(\mathbf{x})} \mathbb{1}[y_i = \text{No}] = 5, \qquad \sum_{i \in N_k(\mathbf{x})} \mathbb{1}[y_i = \text{Yes}] = 0,$$

so the majority-vote rule gives $\hat{y} = \text{No}$.

Conclusion of the Practice Problem

We classify the test data as "No", i.e., the best guess for the customer with age = 20 and salary = 38k is that they **will not purchase the item**.

3.10.6 Scikit-Learn Implementation

```
# Import LabelEncoder function
from sklearn.preprocessing import LabelEncoder
# Import KNeighborsClassifier function
from sklearn.neighbors import KNeighborsClassifier
import numpy as np                                    # Import NumPy

# Assign features and label variables
age = np.array([19, 35, 26, 27, 19, 27, 27, 32, 25, 35, 26,
                26, 20, 32, 18, 29, 47, 45, 46, 48, 45, 47])
salary = np.array([19, 20, 43, 57, 76, 58, 84, 150, 33, 65, 80,
                   52, 86, 18, 82, 80, 25, 26, 28, 29, 22, 49])
purchased =
   np.array(['No','No','No','No','No','No','No','Yes','No','No','No',

   'No','No','No','No','No','Yes','Yes','Yes','Yes','Yes','Yes'])

# Create LabelEncoder that used to convert string labels to numbers
encoder = LabelEncoder()
# Convert string labels into numbers
label = encoder.fit_transform(purchased)
# Combine age and salary into single list of tuples
features = list(zip(age, salary))
# Create a KNN classifier and set K=5
model = KNeighborsClassifier(n_neighbors=5)
# Train the model using the training sets
model.fit(features, label)
# Predict output
predicted = model.predict([[20, 38]])   # age = 20, salary = 38
# Convert number to string label
predicted = encoder.inverse_transform(predicted)
print(predicted)
```

Output: `['No']`.

3.10.7 Effect of Standardization

$$\begin{aligned} \text{Mean of age} = 31.86, \quad & \text{Std of age} = 10.18, \\ \text{Mean of salary} = 53.73, \quad & \text{Std of salary} = 32.46. \end{aligned}$$

After standardization using $z = \dfrac{x - \mu}{\sigma}$, the **5 closest values remain unchanged**. Testing data: Age = 20 and Salary = 38k.

Age	25	26	26	19	27	27	32	35	45	46	47
Salary (k)	33	43	52	19	57	58	18	20	26	28	49
Purchased	No	No	No	No	No	No	No	No	Yes	Yes	Yes
Distance	7.07	7.81	15.23	19.03	20.25	21.19	23.32	23.43	27.73	27.86	29.15
Dist. (std.)	0.51	0.61	0.73	0.59	0.90	0.92	1.33	1.57	2.48	2.57	2.67

Age	48	45	47	35	19	26	29	18	27	20	32
Salary (k)	29	22	25	65	76	80	80	82	84	86	150
Purchased	Yes	Yes	No	No	No	No	No	No	No	No	Yes
Distance	29.41	29.68	29.97	30.89	38.01	42.43	42.95	44.05	46.53	48.00	112.64
Dist. (std.)	2.76	2.50	2.68	1.69	1.17	1.42	1.57	1.37	1.58	1.48	3.65

The five smallest standardized distances are $0.51, 0.59, 0.61, 0.73, 0.90$ — the same five neighbors as before, so the prediction stays **No**.

3.10.8 Scikit-Learn Implementation with Standardization

```
# Import LabelEncoder function
from sklearn.preprocessing import LabelEncoder
# Import KNeighborsClassifier function
from sklearn.neighbors import KNeighborsClassifier
import numpy as np                                    # Import NumPy

# Assign features and label variables
age = np.array([19, 35, 26, 27, 19, 27, 27, 32, 25, 35, 26,
                26, 20, 32, 18, 29, 47, 45, 46, 48, 45, 47])
a_mean = np.mean(age)
# ddof=1 is to make the divisor n-1, i.e., sample std
a_std = np.std(age, ddof=1)
age = (age - a_mean) / a_std
salary = np.array([19, 20, 43, 57, 76, 58, 84, 150, 33, 65, 80,
                   52, 86, 18, 82, 80, 25, 26, 28, 29, 22, 49])
s_mean = np.mean(salary)
# ddof=1 is to make the divisor n-1, i.e., sample std
s_std = np.std(salary, ddof=1)
salary = (salary - s_mean) / s_std
purchased =
   np.array(['No','No','No','No','No','No','No','Yes','No','No','No',

   'No','No','No','No','No','Yes','Yes','Yes','Yes','Yes','Yes'])

# Create LabelEncoder that used to convert string labels to numbers
encoder = LabelEncoder()
# Convert string labels into numbers
label = encoder.fit_transform(purchased)
# Combine age and salary into single list of tuples
features = list(zip(age, salary))
# Create a KNN classifier and set K=5
model = KNeighborsClassifier(n_neighbors=5)
# Train the model using the training sets
model.fit(features, label)
# Predict output
# age=20, salary=38
predicted = model.predict([[(20 - a_mean) / a_std, (38 - s_mean) /
   s_std]])
# Convert number to string label
predicted = encoder.inverse_transform(predicted)
print(predicted)
```

Output: `['No']` — consistent with the manual computation.

4 K-Means Clustering

4.1 Motivation: Customer Segmentation

Mall Customers Dataset

Given the following simplified Mall customers dataset with three attributes: **age**, **income** (in thousands of dollars), and **expense score** (1–100, where 1 means low spending and 100 means high spending).

Person	1	2	3	4	5	6	7	8	9	10
Age	19	67	35	60	65	49	70	70	57	68
Income (×1000)	15	19	24	30	38	42	46	49	54	59
Expense score (1–100)	39	14	35	4	35	52	56	55	51	55

Person	11	12	13	14	15	16	17	18	19	20
Age	23	65	27	47	57	43	56	40	37	34
Income (×1000)	62	63	67	71	75	78	79	87	97	103
Expense score (1–100)	41	52	56	9	5	17	35	13	32	23

Question: Could we cluster the customers based on their characteristics to determine the targeted advertisement and make the marketing budget more efficient?

4.2 Clustering

Definition of Clustering

Clustering is grouping things into "natural" categories when no class label is available.

4.2.1 Applications

- Labeling a large set of data samples can be **costly**.
- Clustering can be used for finding features that will be useful later for categorization.

4.2.2 Use Cases

- **T-shirt sizing:** Group people of similar sizes together to make "small", "medium", and "large" T-shirts.
  - Tailor-made for each person: too expensive.
  - One-size-fits-all: does not fit all.
- **Document organization:** Given a collection of text documents, organize them according to their content similarities to produce a topic hierarchy.
- **Marketing:** Segment customers by their similarity to perform targeted marketing.
- **Image segmentation:** Partition a digital image into multiple image segments, also known as image regions.

4.3 K-Means

K-Means Clustering

- K-Means clustering is an **unsupervised learning** algorithm. There is no labeled data for this clustering, unlike supervised learning.
- It performs the division of data into **non-overlapping clusters** that share similarities and are dissimilar to the data belonging to another cluster.
- The term 'K' is a number telling the system how many clusters you need to create. For example, $K = 3$ refers to three clusters.

Supervised vs. Unsupervised

Do not confuse K-Means with K-Nearest Neighbors (KNN). KNN is a supervised classifier that needs labeled training data; K-Means is an unsupervised clustering algorithm that works on unlabeled data. The "K" in the two algorithms also means different things: number of neighbors vs. number of clusters.

4.3.1 Notation

Let the set of data points be

$$D = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}, \qquad n = |D|,$$

where each data point is a d-dimensional real vector

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{id})^\top \in \mathbb{R}^d,$$

and d is the number of dimensions (the number of features/attributes). Here the scalars $x_{i1}, \ldots, x_{id}$ are the coordinates of $\mathbf{x}_i$, and $(\cdot)^\top$ denotes the transpose.

To perform K-Means clustering, we must first specify the desired number of clusters K. The algorithm partitions the data into K disjoint clusters indexed by

$$C_1, C_2, \ldots, C_K \subseteq \{1, 2, \ldots, n\}, \qquad C_k \cap C_{k'} = \varnothing \;\; (k \neq k'), \qquad \bigcup_{k=1}^{K} C_k = \{1, 2, \ldots, n\}.$$

The assignment of point i to cluster k is recorded by an indicator

$$r_{ik} = \mathbb{1}[\, i \in C_k \,] \in \{0, 1\}, \qquad \sum_{k=1}^{K} r_{ik} = 1 \;\text{ for every } i,$$

so that $C_k = \{i : r_{ik} = 1\}$. Each cluster has a centre, called the **centroid**, denoted by a vector

$$\boldsymbol{\mu}_k = (\mu_{k1}, \mu_{k2}, \ldots, \mu_{kd})^\top \in \mathbb{R}^d, \qquad k = 1, \ldots, K.$$

4.3.2 Algorithm

K-Means Clustering Algorithm

K-Means seeks the cluster memberships r_{ik} and the centroids $\boldsymbol{\mu}_k$ that minimise the **within-cluster sum of squares** (also called the inertia, or sum of squared error):

$$J = \sum_{i=1}^{n} \sum_{k=1}^{K} r_{ik} \big\| \mathbf{x}_i - \boldsymbol{\mu}_k \big\|^2.$$

Given K, Lloyd's algorithm minimises J by alternately performing the following two steps:

1. **Initialisation.** Choose K data points (seeds) as the initial centroids $\boldsymbol{\mu}_1^{(0)}, \ldots, \boldsymbol{\mu}_K^{(0)}$ (e.g., chosen at random).
2. **Assignment step.** Assign each data point to the centroid closest to it (in squared Euclidean distance):

$$r_{ik} = \begin{cases} 1, & k = \arg\min_j \big\| \mathbf{x}_i - \boldsymbol{\mu}_j \big\|^2, \\ 0, & \text{otherwise.} \end{cases}$$

3. **Update step.** Re-compute each centroid as the mean of the points assigned to it:

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i = \frac{\sum_{i=1}^{n} r_{ik}\, \mathbf{x}_i}{\sum_{i=1}^{n} r_{ik}}, \qquad C_k = \{i : r_{ik} = 1\}.$$

4. **Iteration.** If a convergence criterion is **NOT** met, repeat steps 2–3.

Question: What steps are training? (Steps 2–3 are the iterative training loop.)

4.3.3 Example: Mall Customers

Example: K-Means on the Mall Customers Dataset ($K = 3$)

Perform K-Means clustering on the simplified Mall customers dataset with attributes age, income, and expense score (the 20-person table given above).

Step 1: Randomly pick 3 data points as initial centroids.

- 1st centroid: $(70, 46, 56)$
- 2nd centroid: $(27, 67, 56)$
- 3rd centroid: $(37, 97, 32)$

Step 2: Find the distances between each data point with the 3 centroids. For two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ the Euclidean distance is

$$d(\mathbf{a}, \mathbf{b}) = \big\|\mathbf{a} - \mathbf{b}\big\| = \sqrt{\sum_{t=1}^{d} (a_t - b_t)^2},$$

and we compute $\mathrm{DC}_j = \|\mathbf{x}_i - \boldsymbol{\mu}_j\|$ for every point i and centroid j. (Since $u \mapsto u^2$ is increasing on $u \geq 0$, the assignment step may equivalently minimise the squared distance $\|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$.)

Person	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
DC1	62	50	46	55	23	22	0	3	16	13	52	18	48	58	60	57	42	67	65	75
DC2	55	75	49	72	52	34	48	47	33	42	16	38	0	51	60	44	38	49	40	49
DC3	84	86	73	76	65	60	65	63	51	54	39	48	40	36	40	25	26	22	0	11

where DC1, DC2, DC3 are the distances between the data points and the 1st, 2nd, and 3rd centroids, respectively. For example, for Person 1 at $(19, 15, 39)$:

$$\mathrm{DC1} = \sqrt{(19-70)^2 + (15-46)^2 + (39-56)^2} = \sqrt{2601 + 961 + 289} = \sqrt{3851} \approx 62$$

Step 3: Assign each data point to the closest centroid, i.e., set $r_{ik} = 1$ for $k = \arg\min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$ and $r_{ik} = 0$ for all other k.

Person	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Cluster	2	1	1	1	1	1	1	1	1	1	2	1	2	3	3	3	3	3	3	3

Step 4: Re-compute the centroids using the current cluster memberships, i.e., apply the update step $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$ to each attribute separately.

New 1st centroid (persons 2, 3, 4, 5, 6, 7, 8, 9, 10, 12):

$$x_1 = (67 + 35 + 60 + 65 + 49 + 70 + 70 + 57 + 68 + 65)/10 = 60.6$$
$$x_2 = (19 + 24 + 30 + 38 + 42 + 46 + 49 + 54 + 59 + 63)/10 = 42.4$$
$$x_3 = (14 + 35 + 4 + 35 + 52 + 56 + 55 + 51 + 55 + 52)/10 = 40.9$$

New 2nd centroid (persons 1, 11, 13):

$$x_1 = (19 + 23 + 27)/3 = 23$$
$$x_2 = (15 + 62 + 67)/3 = 48$$
$$x_3 = (39 + 41 + 56)/3 = 45.33$$

New 3rd centroid (persons 14, 15, 16, 17, 18, 19, 20):

$$x_1 = (47 + 57 + 43 + 56 + 40 + 37 + 34)/7 = 44.86$$
$$x_2 = (71 + 75 + 78 + 79 + 87 + 97 + 103)/7 = 84.29$$
$$x_3 = (9 + 5 + 17 + 35 + 13 + 32 + 23)/7 = 19.14$$

Iterations at a glance:

Round	1st centroid	2nd centroid	3rd centroid
1st round	$(70, 46, 56)$	$(27, 67, 56)$	$(37, 97, 32)$
2nd round	$(60.6, 42.4, 40.9)$	$(23, 48, 45.3)$	$(44.9, 84.3, 19.1)$
3rd round	$(63.4, 44.4, 41.6)$	$(26, 42, 42.8)$	$(44.9, 84.3, 19.1)$
4th round	$(63.4, 44.4, 41.6)$	$(26, 42, 42.8)$	$(44.9, 84.3, 19.1)$

The 3rd and 4th rounds produce identical centroids, so the algorithm has **converged**.
Why it converges. Let $R^{(t)}$ denote the assignment matrix at round t (entries $r_{ik}^{(t)}$) and $\boldsymbol{\mu}^{(t)} = (\boldsymbol{\mu}_1^{(t)}, \ldots, \boldsymbol{\mu}_K^{(t)})$ the centroids. Each iteration never increases the objective:

$$J\big(R^{(t+1)}, \boldsymbol{\mu}^{(t+1)}\big) \leq J\big(R^{(t)}, \boldsymbol{\mu}^{(t)}\big),$$

because the assignment step minimises J over R for fixed $\boldsymbol{\mu}$, and the update step minimises J over $\boldsymbol{\mu}$ for fixed R. Since $J \geq 0$ is bounded below and takes finitely many distinct values, Lloyd's algorithm terminates at a local minimum of J after finitely many rounds.
Conclusion:

- **Cluster 1:** The average age is 63 years old, the average annual income is \$44K, and the average expense score is 42 of 100.
- **Cluster 2:** The average age is 26 years old, the average annual income is \$42K, and the average expense score is 43 of 100.
- **Cluster 3:** The average age is 45 years old, the average annual income is \$84K, and the average expense score is 19 of 100.

4.3.4 Implementation using Scikit-Learn

```
# Import the required libraries
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Unlabled training data
data = np.array([[19, 15, 39], [67, 19, 14], [35, 24, 35], [60, 30,
   4], [65, 38, 35],
[49, 42, 52], [70, 46, 56], [70, 49, 55], [57, 54, 51], [68, 59, 55],
[23, 62, 41], [65, 63, 52], [27, 67, 56], [47, 71, 9], [57, 75, 5],
[43, 78, 17], [56, 79, 35], [40, 87, 13], [37, 97, 32], [34, 103,
   23]])

# Initial centroids
init_centroids = np.array([[70, 46, 56], [27, 67, 56], [37, 97, 32]])

# Create a KMeans object by specifying
# - Number of clusters (n_clusters) = 3, initial centroids (init) =
# init_centroids
# - Number of time the k-means algorithm will be run with different
#   centroid seeds (n_init) = 1
# - Maximum number of iterations of the k-means algorithm for a single
#   run (max_iter) = 4
kmeans = KMeans(n_clusters=3, init=init_centroids, n_init=1,
   max_iter=4)

kmeans.fit(data)          # Compute k-means clustering
# Predict the closest cluster each sample in data belongs to
labels = kmeans.predict(data)
centroids = kmeans.cluster_centers_     # Get resulting centroids

# Figure width = 10 inches, height = 10 inches
fig = plt.figure(figsize=(10, 10))
# Defining 3D axes so that we can plot 3D data
```

```
ax = fig.add_subplot(projection='3d')

# Get boolean arrays representing entries with labels = 0, 1, and 2
a = np.array(labels == 0); b = np.array(labels == 1); c =
   np.array(labels == 2)

# Plot centroids with color = black, size = 50 units, transparency =
# 20%, and put label
ax.scatter(centroids[:,0], centroids[:,1], centroids[:,2],
           c="black", s=50, alpha=0.8, label="Centroids")
# Plot data in the different clusters (1st in red, 2nd in green, 3rd
# blue)
ax.scatter(data[a,0], data[a,1], data[a,2], c="red", s=40, label="1st
   Cluster")
ax.scatter(data[b,0], data[b,1], data[b,2], c="green", s=40,
   label="2nd Cluster")
ax.scatter(data[c,0], data[c,1], data[c,2], c="blue", s=40,
   label="3rd Cluster")
ax.legend()                              # Show legend
ax.set_xlabel("Age")                     # Put x-axis label "Age"
ax.set_ylabel("Income (K)")              # Put y-axis label "Income (K)"
# Put z-axis label "Expense Score (1-100)"
ax.set_zlabel("Expense Score (1-100)")
# Put figure title
ax.set_title("Customer Segmentation - K-Means Clustering")
```

4.4 Analysis of K-Means

4.4.1 Stopping Criterion

When Does K-Means Stop?

- **No/minimum re-assignments** of data points to different clusters, i.e., $r_{ik}^{(t+1)} = r_{ik}^{(t)}$ for all i, k; or
- **No/minimum change of centroids**, i.e., $\left\|\boldsymbol{\mu}_k^{(t+1)} - \boldsymbol{\mu}_k^{(t)}\right\| < \epsilon$ for all k, for some small tolerance $\epsilon > 0$; or
- **Minimum decrease in the Sum of Squared Error (SSE)** between successive iterations:

$$\text{SSE} = J = \sum_{k=1}^{K} \sum_{i \in C_k} \left\|\mathbf{x}_i - \boldsymbol{\mu}_k\right\|^2 = \sum_{k=1}^{K} \sum_{i \in C_k} \text{dist}(\mathbf{x}_i, \boldsymbol{\mu}_k)^2$$

where
  - $C_k = \{i : r_{ik} = 1\}$ is the index set of the kth cluster,
  - $\boldsymbol{\mu}_k$ is the centroid of cluster C_k,
  - $\text{dist}(\mathbf{x}_i, \boldsymbol{\mu}_k) = \|\mathbf{x}_i - \boldsymbol{\mu}_k\|$ is the (Euclidean) distance between data point $\mathbf{x}_i$ and centroid $\boldsymbol{\mu}_k$.

Because every iteration only decreases J, the sequence $J^{(t)}$ is non-increasing and bounded below by 0; the loop stops once this decrease (or the change in assignments/centroids) falls below a small threshold.

4.4.2 Common Questions

1. **How to choose K?**
   Both methods evaluate a range of K by means of the objective $J(K)$:

- **Elbow method:** plot the objective $J(K)$ against K and pick the "elbow" — the K beyond which increasing K no longer substantially reduces J, i.e., the marginal gain $J(K-1) - J(K)$ flattens out.
- **Silhouette method:** for a point i in cluster C_k, let $a(i)$ be the mean distance to the other points of its own cluster (intra-cluster distance) and $b(i)$ the mean distance to the points of the nearest other cluster (inter-cluster distance):

$$a(i) = \frac{1}{|C_k| - 1} \sum_{\substack{j \in C_k \\ j \neq i}} \big\|\mathbf{x}_i - \mathbf{x}_j\big\|, \qquad b(i) = \min_{k' \neq k} \frac{1}{|C_{k'}|} \sum_{j \in C_{k'}} \big\|\mathbf{x}_i - \mathbf{x}_j\big\|.$$

  The **silhouette coefficient** of point i is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1], \qquad \bar{s} = \frac{1}{n} \sum_{i=1}^{n} s(i),$$

  and one chooses the K that maximises the average silhouette $\bar{s}$.

These methods will be discussed in more detail in advanced-level machine learning courses.

2. **What distance metric should be used for K-Means?**
It depends on your data. Normally, K-Means uses **Euclidean distance**

$$d(\mathbf{a}, \mathbf{b}) = \big\|\mathbf{a} - \mathbf{b}\big\| = \sqrt{\sum_{t=1}^{d} (a_t - b_t)^2},$$

for which the arithmetic mean $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$ is exactly the point minimising the sum of squared distances to the cluster members.

3. **Do we need to standardize the data before doing K-Means clustering?**
Yes. Due to the fact that clustering algorithms, including K-Means, use distance-based measurements to determine the similarity between data points, it is recommended that every attribute be standardized to a **mean of zero** and a **standard deviation of one** (a z-score),

$$x'_{it} = \frac{x_{it} - \bar{x}_t}{\sigma_t}, \qquad \bar{x}_t = \frac{1}{n} \sum_{i=1}^{n} x_{it}, \qquad \sigma_t = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \big(x_{it} - \bar{x}_t\big)^2},$$

because every dataset has features with different measurement units, such as age or income.

Standardize Before Clustering

Features with large numeric ranges (e.g., income in the tens of thousands) will dominate the Euclidean distance over features with small ranges (e.g., expense score 1–100). Standardizing each feature to mean 0 and standard deviation 1 prevents any single feature from dominating the distance computation.

4.4.3 Clustering Quality

High Quality Clustering

A high quality clustering:

- **Maximises inter-cluster distance (isolation)** — i.e., the distance between clusters, for example the minimum centroid separation

$$\text{isolation} = \min_{k \neq k'} \big\|\boldsymbol{\mu}_k - \boldsymbol{\mu}_{k'}\big\|.$$

- **Minimises intra-cluster distance (compactness)** — i.e., the scatter within a cluster, which is precisely what the objective measures:

$$\text{compactness} = J = \sum_{k=1}^{K} \sum_{i \in C_k} \left\| \mathbf{x}_i - \boldsymbol{\mu}_k \right\|^2.$$

The quality of a clustering result depends on the algorithm, the distance function, and the application.

4.4.4 Weaknesses

Weakness 1: Sensitive to Outliers

K-Means is **sensitive to outliers**:
- Outliers are data points that are very far away from other data points.
- Outliers could be errors in the data recording or some special data points with very different values.
- Outliers can drag centroids away and lead to undesirable clustering.

How to deal with outliers?
- Remove some data points in the clustering process that are much further away from the centroids than other data points. To be safe, we may want to monitor these possible outliers over a few iterations and then decide to remove them.
- Perform **random sampling**. Since in sampling we only choose a small subset of the data points, the chance of selecting an outlier is very small.
- Assign the rest of the data points to the clusters by distance or similarity comparison, or classification.

Weakness 2: Sensitive to Initial Seeds

The algorithm is **sensitive to the initial seeds** $\boldsymbol{\mu}_1^{(0)}, \ldots, \boldsymbol{\mu}_K^{(0)}$. Because Lloyd's algorithm only guarantees convergence to a local minimum of J, not-so-good seeds can produce not-so-good clustering results, while different (better) seeds can produce good results. In practice, K-Means is often run multiple times with different random seeds (e.g., via `n_init` in scikit-learn) and the run with the smallest J is kept.
A better initialisation is **k-means++**: choose the first centroid uniformly at random, then repeatedly choose the next seed $\mathbf{x}_i$ with probability proportional to its squared distance to the nearest centroid chosen so far,

$$P(\mathbf{x}_i) = \frac{\min_{k'} \left\| \mathbf{x}_i - \boldsymbol{\mu}_{k'}^{(0)} \right\|^2}{\sum_{j=1}^{n} \min_{k'} \left\| \mathbf{x}_j - \boldsymbol{\mu}_{k'}^{(0)} \right\|^2},$$

which spreads the initial centroids out and usually yields a better (smaller) J.

Weakness 3: Only Works for Hyper-Spherical Clusters

The K-Means algorithm is **not suitable for discovering clusters that are not hyper-ellipsoids (or hyper-spheres)**. If the true clusters have elongated, curved, or irregular shapes, K-Means will fail to recover them.

4.4.5 Pros and Cons

- **Pros:**
  - Easy to understand and implement.
  - It is efficient, given K and the number of iterations is small.
- **Cons:**

– It is only applicable if the **mean is defined**. For categorical data, **K-mode** is used instead, i.e., the centroid is represented by the most frequent values, coordinate-wise

$$\mu_{kt} = \underset{i \in C_k}{\text{mode}}\ x_{it}, \qquad t = 1, \ldots, d.$$

– The user needs to specify K.
– It is sensitive to outliers.

Summary

- Despite its weaknesses, K-Means is still the **most popular** clustering algorithm due to its simplicity and efficiency — and other clustering algorithms have their own lists of weaknesses.
- There is no clear evidence that any other clustering algorithm performs better in general, although they may be more suitable for some specific types of data or applications.
- Comparing different clustering algorithms is a difficult task — no one knows the correct clusters.

4.4.6 Remark: PCA

PCA Before K-Means

Principal Component Analysis (PCA) is one of the easiest, most intuitive, and most frequently used methods for **dimensionality reduction**, projecting data onto its orthogonal feature subspace. Given centred data, PCA finds an orthonormal matrix $W \in \mathbb{R}^{d \times d'}$ $(d' \leq d)$ and maps each point to a lower-dimensional vector

$$\mathbf{z}_i = W^\top (\mathbf{x}_i - \bar{\mathbf{x}}), \qquad \bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{x}_i.$$

- The main idea of PCA is to reduce the dimensionality of a data set consisting of many variables correlated with each other, while retaining the variation present in the dataset to the maximum extent.
- It is a common practice to **apply PCA before K-Means clustering** is performed.
- It is believed that it improves the clustering results in practice (noise reduction).

4.5 Leader Clustering

Leader Clustering

Leader Clustering is a K-means-like clustering method that receives two parameters: an integer K and a real-valued threshold T. Let the current leaders be $\boldsymbol{\ell}_1, \ldots, \boldsymbol{\ell}_m$ (initially $m = K$).

1. Start by selecting K instances (called **leaders**) and assign each training instance $\mathbf{x}$ to the cluster of its nearest leader, i.e., $\arg\min_{1 \leq k \leq m} \|\mathbf{x} - \boldsymbol{\ell}_k\|$.
2. Scan the instances: an instance $\mathbf{x}$ becomes a **new leader** (a new cluster is created) if its distance to every current leader exceeds the threshold T,

$$\min_{1 \leq k \leq m} \left\|\mathbf{x} - \boldsymbol{\ell}_k\right\| > T;$$

   otherwise it joins the cluster of its nearest leader.

Unlike K-Means, the number of clusters m can **grow during training**.

Exam-Style Practice: Leader Clustering vs. K-Means (Midterm 2024S Problem 6)

(a) Which method handles outliers better? Leader Clustering. An outlier far from all leaders simply becomes a new leader: a new cluster is generated for it **without influencing the other clusters**. In K-Means, an outlier drags its cluster's centroid and distorts the clustering.
(b) Does Leader Clustering mitigate K-Means' drawback of having to specify K? Yes — the number of clusters may increase during training, so we no longer need to know K in advance. But we now have to specify the threshold T instead: the clustering result is **sensitive to** T (a small T creates many clusters; a large T creates few).

4.6 Practice: Medicine Clustering

Practice Problem: Group Medicines with K-Means ($K = 2$)

Given 4 types of medicines, each with two attributes (weight index and pH index):

Medicine	A	B	C	D
Weight Index	1	2	4	5
pH Index	1	1	3	4

Group these medicines into $K = 2$ groups using K-Means clustering. Use $(1, 1)$ and $(2, 1)$ as the initial centroids (i.e., seeds) $\boldsymbol{\mu}^{(0)}$ and perform K-Means clustering, alternating the assignment step $r_{ik} = 1$ for $k = \arg\min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$ and the update step $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$, until there are no re-assignments of data points to different clusters.

Solution

Step 1: Use the 2 given data points as initial centroids.
- 1st centroid: $(1, 1)$
- 2nd centroid: $(2, 1)$

Step 2: Find the distances between each data point with the 2 centroids (Euclidean distance).
For example, for medicine C at $(4, 3)$:

$$\text{DC1} = \sqrt{(4-1)^2 + (3-1)^2} = \sqrt{9+4} = \sqrt{13} \approx 3.6$$
$$\text{DC2} = \sqrt{(4-2)^2 + (3-1)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.8$$

Medicine	A	B	C	D
DC1	0	1	3.6	5
DC2	1	0	2.8	4.2

where DC1, DC2 are the distances between the data points and the 1st and 2nd centroids, respectively.
Step 3: Assign each data point to the closest centroid.

Medicine	A	B	C	D
DC1	0	1	3.6	5
DC2	1	0	2.8	4.2
Cluster	1	2	2	2

Step 4: Re-compute the centroids using the current cluster memberships.

New 1st centroid (cluster 1 = {A}):

$$x_1 = 1, \qquad x_2 = 1$$

New 2nd centroid (cluster 2 = {B, C, D}):

$$x_1 = (2 + 4 + 5)/3 = 3.67, \qquad x_2 = (1 + 3 + 4)/3 = 2.67$$

Step 5: Find the distance between each data point with the 2 centroids.
1st centroid: $(1, 1)$; 2nd centroid: $(3.67, 2.67)$.

Medicine	A	B	C	D
DC1	0	1	3.6	5
DC2	3.1	2.4	0.5	1.9

For example, for medicine A: DC2 $= \sqrt{(1 - 3.67)^2 + (1 - 2.67)^2} = \sqrt{7.13 + 2.79} \approx 3.1$.

Step 6: Assign each data point to the closest centroid.

Medicine	A	B	C	D
DC1	0	1	3.6	5
DC2	3.1	2.4	0.5	1.9
Cluster	1	1	2	2

Medicine B is re-assigned from cluster 2 to cluster 1.

Step 7: Re-compute the centroids using the current cluster memberships.
New 1st centroid (cluster 1 = {A, B}):

$$x_1 = (1 + 2)/2 = 1.5, \qquad x_2 = (1 + 1)/2 = 1$$

New 2nd centroid (cluster 2 = {C, D}):

$$x_1 = (4 + 5)/2 = 4.5, \qquad x_2 = (3 + 4)/2 = 3.5$$

Step 8: Find the distance between each data point with the 2 centroids.
1st centroid: $(1.5, 1)$; 2nd centroid: $(4.5, 3.5)$.

Medicine	A	B	C	D
DC1	0.5	0.5	3.2	4.6
DC2	4.3	3.5	0.7	0.7

Step 9: Assign each data point to the closest centroid.

Medicine	A	B	C	D
DC1	0.5	0.5	3.2	4.6
DC2	4.3	3.5	0.7	0.7
Cluster	1	1	2	2

The assignments are the same as in Step 6 — no re-assignments occur, so the stopping criterion is met.

Step 10: Re-compute the centroids (they stay the same).
New 1st centroid:

$$x_1 = (1 + 2)/2 = 1.5, \qquad x_2 = (1 + 1)/2 = 1$$

New 2nd centroid:

$$x_1 = (4 + 5)/2 = 4.5, \qquad x_2 = (3 + 4)/2 = 3.5$$

Iterations at a glance:

Round	1st centroid	2nd centroid
1st round	$(1, 1)$	$(3.67, 2.67)$
2nd round	$(1.5, 1)$	$(4.5, 3.5)$
3rd round	$(1.5, 1)$	$(4.5, 3.5)$

(The initial seeds were $(1, 1)$ and $(2, 1)$; after round 1 the centroids are $(1, 1)$ and $(3.67, 2.67)$.)
Conclusion:
- **Cluster 1** (medicines A, B): The average weight index is 1.5, and the average pH index is 1.
- **Cluster 2** (medicines C, D): The average weight index is 4.5, and the average pH index is 3.5.

4.6.1 Implementation using Scikit-Learn

```
# Import the required libraries
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Unlabled training data
data = np.array([[1, 1], [2, 1], [4, 3], [5, 4]])

# Initial centroids
init_centroids = np.array([[1, 1], [2, 1]])

# Create a KMeans object by specifying
# - Number of clusters (n_clusters) = 2, initial centroids (init) =
# init_centroids
# - Number of time the k-means algorithm will be run with different
#   centroid seeds (n_init) = 1
# - Maximum number of iterations of the k-means algorithm for a single
#   run (max_iter) = 3
kmeans = KMeans(n_clusters=2, init=init_centroids, n_init=1,
   max_iter=3)

kmeans.fit(data)          # Compute k-means clustering
# Predict the closest cluster each sample in data belongs to
labels = kmeans.predict(data)
centroids = kmeans.cluster_centers_     # Get resulting centroids

# Defining 2D axes so that we can plot 2D data
fig, ax = plt.subplots()

# Get boolean arrays representing entries with labels = 0 and 1
a = np.array(labels == 0); b = np.array(labels == 1)

# Plot centroids with color = black, size = 50 units, transparency =
# 20%, and put label
ax.scatter(centroids[:,0], centroids[:,1], c="black", s=50,
   alpha=0.8, label="Centroids")
# Plot data in the different clusters (1st in red, 2nd in green) with
# transparency = 50%
ax.scatter(data[a,0], data[a,1], c="red", s=40, alpha=0.5, label="1st
   Cluster")
ax.scatter(data[b,0], data[b,1], c="green", s=40, alpha=0.5,
   label="2nd Cluster")
ax.legend()                       # Show legend
ax.set_xlabel("Weight Index")   # Put x-axis label "Weight Index"
ax.set_ylabel("pH Index")       # Put y-axis label "pH Index"
ax.set_title("Medicine - K-Means Clustering")  # Put figure title
```

5 Neurons

5.1 Classification

5.1.1 Definition

Classification

Classification is the process of predicting the **class** of given data. Classes are also called **labels** or **categories**.

5.1.2 Classification Examples

- **Spam detection in email service:** 2 classes — spam and not spam.
- **Handwritten digit recognition:** 10 classes — $0, 1, 2, \ldots, 9$.
- **Image classification:** 3 classes — cat, dog, none of them.

5.1.3 Motivation

- Image recognition and classification is a **subdomain of computer vision**. It is an algorithm that looks at an image and assigns it a label from a collection of predefined labels or categories (e.g., a dog, a cat).
- Image classification and recognition are **vital components in robotics**, such as autonomous vehicles or domestic robots.
- It is also important in **security systems** such as face recognition, **image search engines** such as Google or Bing image search, and **medical imaging** such as cancer detection.

5.1.4 Running Example: MNIST

We will build an **Artificial Neural Network** to recognize/classify handwritten digits. This is the running application that motivates everything in this chapter and the next.

5.2 ANN and Neuron

5.2.1 Artificial Neural Network

Artificial Neural Network (ANN)

- Artificial neural networks (ANN) are **one of the most powerful artificial intelligence and machine learning algorithms**.
- An ANN is considered a **universal function approximator** that transforms inputs into outputs.
- As the name suggests, it **draws inspiration from neurons in our brain** and the way they are connected.

5.2.2 Biological vs. Artificial Neuron

An artificial neuron mirrors the structure of a biological neuron:

Biological Neuron	Artificial Neuron
Dendrites	Inputs
Cell Nucleus (computation unit)	Node
Axon	Output
Synapse	Weight
Neuron	Node = a linear function & an activation function

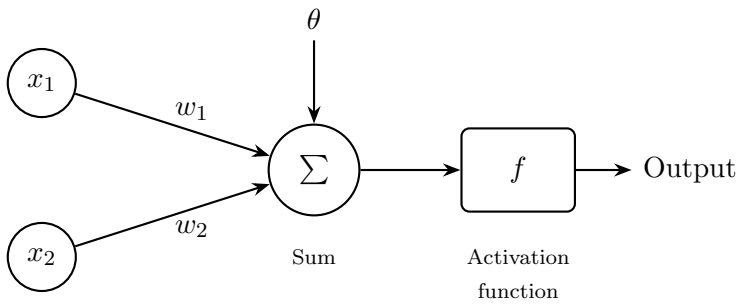
5.2.3 Artificial Neuron

- An **artificial neuron** is a simple **biological neuron model** in an artificial neural network.
- It performs certain calculations to detect input data capabilities.
- One of the most simplest models of neuron is called the **perceptron** (or **threshold logic unit**).

Formally, an artificial neuron takes an input vector $\mathbf{x} = (x_1, x_2, \ldots, x_n)^\top \in \mathbb{R}^n$, a weight vector $\mathbf{w} = (w_1, w_2, \ldots, w_n)^\top \in \mathbb{R}^n$ and a bias $b \in \mathbb{R}$. It first computes the **weighted sum** (the linear part) and then applies an **activation function** f:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b, \qquad a = f(z).$$

In the course's notation the bias is written θ (as in the figure below and in the worked examples), so here $b \equiv \theta$.



Neuron = Linear Function + Activation Function

Neuron = a linear function & an activation function

- **Linear function:** a linear transformation that forms the **weighted sum of the inputs and the bias**,

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b.$$

- **Activation function:** a (usually nonlinear) function $f : \mathbb{R} \to \mathbb{R}$ applied to the weighted sum, deciding **whether a perceptron/neuron should be activated or not**; the output of the neuron is $a = f(z)$.
- **Note:** The primary role of the activation function is to **transform the summed weighted input** from the node into an **output value** to be fed to the next hidden layer or as output.

Common activation functions with their ranges:

$$\begin{aligned}
\text{linear:} \quad & f(z) = z, && \text{range } \mathbb{R}, \\
\text{sigmoid :} \quad & f(z) = \sigma(z) = \frac{1}{1+e^{-z}}, && \text{range } (0,1), \\
\tanh : \quad & f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, && \text{range } (-1,1), \\
\text{ReLU :} \quad & f(z) = \max(0, z), && \text{range } [0,\infty), \\
\text{step:} \quad & f(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}, && \text{range } \{0,1\}.
\end{aligned}$$

5.2.4 Example: Neuron Output

Example: Computing the Output of a Single Neuron

A neuron with two inputs computes

$$a = f(\mathbf{w} \cdot \mathbf{x} + \theta) = f(w_1 x_1 + w_2 x_2 + \theta)$$

Suppose $x_1 = 3$, $x_2 = 5$, $w_1 = 0.2$, $w_2 = 0.3$, $\theta = 0.3$, and the activation function is the sigmoid $\sigma(z) = \dfrac{1}{1+e^{-z}}$.

Solution. First compute the weighted sum (linear part), then apply the activation function:

$$a = \sigma\big(0.2 \cdot 3 + 0.3 \cdot 5 + 0.3\big) = \sigma(2.4) = \frac{1}{1+e^{-2.4}} = 0.91683$$

The neuron outputs 0.91683 (a value between 0 and 1, close to 1 because the weighted sum 2.4 is positive and fairly large).

5.2.5 Sigmoid

Sigmoid/Logistic Activation Function

The sigmoid (logistic) function and its derivative are

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma : \mathbb{R} \to (0,1),$$

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) = \frac{e^{-z}}{\big(1+e^{-z}\big)^2}.$$

- Takes **any real value** as input and outputs values in the range **0 to 1**.
- The **larger** the input, the closer the output value will be to 1; the **smaller** the input, the closer the output will be to 0.
- One of the **most widely used** activation functions.
- Commonly used for models where we have to **predict the probability as an output**.
- **Differentiable** with a **smooth gradient**.
- The limitations will be discussed later after we learn MLP.

5.2.6 Binary Step

Binary Step (Heaviside) Activation Function

The binary step function (Heaviside step) and its derivative are

$$f(z) = H(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad H'(z) = 0 \quad (z \neq 0).$$

- When using the binary step activation function, a neuron is called a **Perceptron** (or **threshold logic unit**).
- **Cannot provide multi-value outputs**, thus cannot be used for multi-class problems.
- The **gradient of the step function is zero**, which causes a hindrance in the **backpropagation** process in MLP (covered later).
- Suitable to **represent some logical operations** in a neuron.

Sigmoid vs. Binary Step — Gradient Matters

The sigmoid is differentiable with a smooth gradient, which is exactly what gradient-based training (backpropagation in MLPs) needs. The binary step function has **zero gradient almost everywhere**, so it **cannot** be used together with backpropagation. Also remember: the binary step only outputs 0 or 1, so it cannot handle multi-class problems on its own.

5.2.7 Perceptron: AND Gate

We will work on the problem of the **AND** logical operation. The truth table of logical AND is:

x_1	x_2	**Output**
0	0	0
0	1	0
1	0	0
1	1	1

Setup:

- The weights and bias are randomly generated, say $w_1 = 0.1$, $w_2 = 0.5$, $\theta = -0.8$.
- The learning rate is $\eta = 0.2$.
- The activation function is the binary step function:

$$f(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

5.2.8 Perceptron Learning Rules

Notation:

$\mathbf{x} = (x_1, x_2)^\top$ are the inputs	Δw_i is the change of the weight w_i
$\mathbf{w} = (w_1, w_2)^\top$ are the weights	$\Delta\theta$ is the change of the bias
θ (also written b) is the bias	η is the learning rate
O (also written $\hat{y}$) is the predicted output	T (also written y) is the target (label)

Perceptron Learning Rules

The perceptron predicts $\hat{y} = f(\mathbf{w} \cdot \mathbf{x} + b)$ and updates its parameters only when this prediction differs from the target y:

1. If the output is **correct** (i.e., $\hat{y} = y$), the weights and the bias are **not updated**.
2. If the output is **incorrect** (i.e., $\hat{y} \neq y$), the weights and the bias are **updated** so that the new prediction moves closer to y:

$$\Delta w_i = \eta\,(y - \hat{y})\,x_i, \qquad \Delta b = \eta\,(y - \hat{y}),$$

$$w_i \leftarrow w_i + \Delta w_i = w_i + \eta\,(y - \hat{y})\,x_i, \qquad b \leftarrow b + \Delta b = b + \eta\,(y - \hat{y}),$$

for $i \in \{1, 2\}$. In the course's notation the target is written T, the output O and the bias θ (so $y \equiv T$, $\hat{y} \equiv O$, $b \equiv \theta$), and the same rule reads

$$\Delta w_i = \eta\,(T - O)\,x_i, \qquad \Delta\theta = \eta\,(T - O), \qquad w_i \leftarrow w_i + \Delta w_i, \quad \theta \leftarrow \theta + \Delta\theta.$$

Error and training objective. The quantity $e = y - \hat{y}$ (i.e. $T - O$) is the **error** of the current example. Over a training set $\{(\mathbf{x}^{(j)}, y^{(j)})\}_{j=1}^{N}$ the two most common error/loss measures are the **0/1 error** and the **squared error**:

$$E_{0/1} = \frac{1}{N} \sum_{j=1}^{N} \mathbb{1}\left[\hat{y}^{(j)} \neq y^{(j)}\right], \qquad E_{\text{sq}} = \frac{1}{2} \sum_{j=1}^{N} \left(y^{(j)} - \hat{y}^{(j)}\right)^2.$$

The training objective is to minimise such an error; the perceptron learning rule above is a mistake-driven procedure that, for **linearly separable** data, drives $E_{0/1}$ to 0 in a finite number of updates.

When Do Updates Happen?

Updates happen **only on misclassified examples**. Note the factor $(T - O)$: since both T and O are in $\{0, 1\}$, the error term $(T - O)$ is $+1$ (when $T = 1, O = 0$), -1 (when $T = 0, O = 1$), or 0 (correct, no update). Consequently, $\Delta w_i = 0$ whenever the input $x_i = 0$ — an input that did not contribute to the weighted sum does not get its weight changed.

5.2.9 Example: AND Training

AND Gate — Full Perceptron Learning Trace ($\eta = 0.2$)

Initial weights: $w_1 = 0.1$, $w_2 = 0.5$, $\theta = -0.8$.

x_1	x_2	T	O	Δw_1	w_1	Δw_2	w_2	$\Delta\theta$	θ
Initial									
-	-	-	-	-	0.1	-	0.5	-	-0.8
Round 1									
0	0	0	0	0	0.1	0	0.5	0	-0.8
0	1	0	0	0	0.1	0	0.5	0	-0.8
1	0	0	0	0	0.1	0	0.5	0	-0.8
1	1	1	0	0.2	0.3	0.2	0.7	0.2	-0.6
Round 2									
0	0	0	0	0	0.3	0	0.7	0	-0.6
0	1	0	1	0	0.3	-0.2	0.5	-0.2	-0.8
1	0	0	0	0	0.3	0	0.5	0	-0.8
1	1	1	1	0	0.3	0	0.5	0	-0.8
Round 3									
0	0	0	0	0	0.3	0	0.5	0	-0.8
0	1	0	0	0	0.3	0	0.5	0	-0.8
1	0	0	0	0	0.3	0	0.5	0	-0.8
1	1	1	1	0	0.3	0	0.5	0	-0.8

Step-by-step solution. For each example, compute the weighted sum $s = w_1x_1 + w_2x_2 + \theta$ and apply the binary step: $O = 1$ if $s \geq 0$, else $O = 0$.
Round 1 ($w_1 = 0.1$, $w_2 = 0.5$, $\theta = -0.8$):

- **Step 1:** $(x_1, x_2) = (0, 0)$, $T = 0$. $s = 0.1(0) + 0.5(0) - 0.8 = -0.8 < 0 \Rightarrow O = 0$. Correct ($T = O$): no update.
- **Step 2:** $(0, 1)$, $T = 0$. $s = 0.1(0) + 0.5(1) - 0.8 = -0.3 < 0 \Rightarrow O = 0$. Correct: no update.
- **Step 3:** $(1, 0)$, $T = 0$. $s = 0.1(1) + 0.5(0) - 0.8 = -0.7 < 0 \Rightarrow O = 0$. Correct: no update.
- **Step 4:** $(1, 1)$, $T = 1$. $s = 0.1(1) + 0.5(1) - 0.8 = -0.2 < 0 \Rightarrow O = 0$. **Incorrect** ($T \neq O$), so update:

$$\Delta w_1 = \eta(T - O)x_1 = 0.2 \times 1 \times 1 = 0.2 \;\Rightarrow\; w_1 = 0.1 + 0.2 = 0.3$$

$$\Delta w_2 = \eta(T - O)x_2 = 0.2 \times 1 \times 1 = 0.2 \;\Rightarrow\; w_2 = 0.5 + 0.2 = 0.7$$

$$\Delta\theta = \eta(T - O) = 0.2 \times 1 = 0.2 \;\Rightarrow\; \theta = -0.8 + 0.2 = -0.6$$

Round 2 ($w_1 = 0.3$, $w_2 = 0.7$, $\theta = -0.6$):

- **Step 1:** $(0, 0)$, $T = 0$. $s = -0.6 < 0 \Rightarrow O = 0$. Correct: no update.
- **Step 2:** $(0, 1)$, $T = 0$. $s = 0.3(0) + 0.7(1) - 0.6 = 0.1 \geq 0 \Rightarrow O = 1$. **Incorrect**, so update:

$$\Delta w_1 = 0.2 \times (0 - 1) \times 0 = 0 \;\Rightarrow\; w_1 = 0.3$$

$$\Delta w_2 = 0.2 \times (0 - 1) \times 1 = -0.2 \;\Rightarrow\; w_2 = 0.7 - 0.2 = 0.5$$

$$\Delta\theta = 0.2 \times (0 - 1) = -0.2 \;\Rightarrow\; \theta = -0.6 - 0.2 = -0.8$$

- **Step 3:** $(1, 0)$, $T = 0$. $s = 0.3(1) + 0.5(0) - 0.8 = -0.5 < 0 \Rightarrow O = 0$. Correct: no update.
- **Step 4:** $(1, 1)$, $T = 1$. $s = 0.3(1) + 0.5(1) - 0.8 = 0 \geq 0 \Rightarrow O = 1$. Correct: no update.

Round 3 ($w_1 = 0.3$, $w_2 = 0.5$, $\theta = -0.8$):

- **Step 1:** $(0, 0)$, $T = 0$. $s = -0.8 < 0 \Rightarrow O = 0$. Correct.
- **Step 2:** $(0, 1)$, $T = 0$. $s = 0.5(1) - 0.8 = -0.3 < 0 \Rightarrow O = 0$. Correct.
- **Step 3:** $(1, 0)$, $T = 0$. $s = 0.3(1) - 0.8 = -0.5 < 0 \Rightarrow O = 0$. Correct.
- **Step 4:** $(1, 1)$, $T = 1$. $s = 0.3(1) + 0.5(1) - 0.8 = 0 \geq 0 \Rightarrow O = 1$. Correct.

All four examples are classified correctly in Round 3, so training stops. **Final weights and bias:** $w_1 = 0.3$, $w_2 = 0.5$, $\theta = -0.8$.

Edge Case: Weighted Sum Exactly Equal to Zero

The binary step is defined as $f(x) = 1$ when $x \geq 0$. In Round 2 Step 4 and Round 3 Step 4, the weighted sum is exactly 0, which counts as "activated" ($O = 1$). Do not use $x > 0$ by mistake — that would flip these outputs and break the solution.

5.2.10 From Scratch

```
import math  # Import math module

class Perceptron:
    def __init__(self):
        """ Perceptron initialization """
        self.w = [0.1, 0.5]       # Weights
        self.theta = -0.8         # Bias
        self.learningRate = 0.2   # Eta

    def response(self, x):
        """ Perceptron output """
        # Calculate weighted sum
        y = x[0] * self.w[0] + x[1] * self.w[1] + self.theta
        # If weighted sum >= 0, return 1. Otherwise return 0
        if y >= 0:
            return 1
        else:
            return 0

    def updateWeights(self, x, iterError):  # iterError: T - O
        """ Weights update """
        # wi = wi + eta * (T-O) * xi
        self.w[0] += self.learningRate * iterError * x[0]
        self.w[1] += self.learningRate * iterError * x[1]

    def updateBias(self, iterError):
        """ Bias update """
        # theta = theta + eta * (T-O)
        self.theta += self.learningRate * iterError

    def train(self, data):
        """ Training """
        learned = True  # Should perform training
        round = 0       # Initialize round to 0
        while learned:             # While learned
            totalError = 0.0       # Initialize
            for x in data:         # For each data
                r = self.response(x)           # Calculate output
```

```
                if x[2] != r:                   # If different
                    roundError = x[2] - r       # Error
                    # Update weights
                    self.updateWeights(x, roundError)
                    self.updateBias(roundError)         # Update bias
                    # Update total error
                    totalError += abs(roundError)
            round += 1
            if math.isclose(totalError, 0) or round >= 100:
                print("Total number of rounds (epochs): ", round)
                print("Final weights: ", self.w)
                print("Final bias: ", self.theta)
                learned = False

""" Main function """
perceptron = Perceptron()  # Create Perceptron object
trainset = [[0,0,0], [0,1,0], [1,0,0], [1,1,1]]  # Define training set
perceptron.train(trainset)  # Perform training
```

Each training example is a triple `[x1, x2, T]`; `x[2]` is the target label. Training stops when a full pass over the data makes zero errors (`math.isclose(totalError, 0)`) or when the safety cap of 100 rounds is reached.

5.2.11 Scikit-Learn

```
import numpy as np  # Import NumPy
# Import Perceptron class from Scikit-Learn
from sklearn.linear_model import Perceptron

inputs = np.array([[0,0], [0,1], [1,0], [1,1]])  # Inputs
outputs = np.array([0, 0, 0, 1])                 # Expected outputs

# Create and fit a perceptron model
# Set learning rate (eta0)
model = Perceptron(eta0=0.2)
model.fit(inputs, outputs)

# Use the trained model to predict the outputs
predicted_outputs = model.predict([[0,0], [1,0], [1,1], [0,1]])
print(predicted_outputs)  # Print the predicated outputs
print(model.coef_)        # Print the final weights
print(model.intercept_)   # Print the bias
```

After fitting, `model.coef_` stores the learned weights, while `model.intercept_` stores the bias. The learning rate is set through the parameter `eta0`.

5.2.12 Stopping Rules

Training can be stopped using any of the following criteria:

1. **Use maximum training time.** The training may go a bit beyond the specified time limit in order to complete the current cycle.
2. **The maximum number of training cycles allowed.** If the maximum number of cycles is exceeded, then training stops.
3. **Use minimum accuracy change.** Training will continue until the specified accuracy change is attained.

5.2.13 Learning and Epoch

Learning and Epoch

- **Learning** is the process of **updating weights** in the perceptron.
- **Epoch** refers to **one cycle through the full training dataset**.

In the AND example, we set weights $w_1 = 0.1$, $w_2 = 0.5$ initially, but they cause some errors. Then, we update the weight values (ending with $w_1 = 0.3$, $w_2 = 0.5$, $\theta = -0.8$) and predict all instances correctly. The whole process takes only a few rounds/epochs — in the trace above, weight updates happen in Rounds 1 and 2, and Round 3 passes through the full dataset with zero errors, confirming convergence.

Observation. According to the results of the perceptron learning procedure, $w_1 = 0.3$, $w_2 = 0.5$, $\theta = -0.8$. The relationship between the inputs $\mathbf{x} = (x_1, x_2)^\top$ and the output $\hat{y}$ is

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w} \cdot \mathbf{x} + \theta = 0.3x_1 + 0.5x_2 - 0.8 < 0 \\ 1 & \text{otherwise} \end{cases} \iff \begin{cases} 0 & \text{if } 3x_1 + 5x_2 < 8 \\ 1 & \text{otherwise} \end{cases}$$

5.2.14 Decision Boundary

A perceptron with weight vector $\mathbf{w}$ and bias b (here $b \equiv \theta$) partitions the input space $\mathbb{R}^n$ by the **hyperplane**

$$\mathbf{w} \cdot \mathbf{x} + b = 0,$$

predicting

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 0, \\ 0 & \text{if } \mathbf{w} \cdot \mathbf{x} + b < 0. \end{cases}$$

For the AND gate this hyperplane is $0.3x_1 + 0.5x_2 - 0.8 = 0$, i.e. $3x_1 + 5x_2 = 8$. The straight line defined by $3x_1 + 5x_2 = 8$ is called the **Decision Boundary** of the perceptron.

Decision Boundary

In a binary classification problem, a **decision boundary** or **decision surface** is a **hypersurface that partitions the underlying vector space into two sets**, one for each class. The classifier will classify all the points on **one side** of the decision boundary as belonging to one class and all those on the **other side** as belonging to the other class.

5.2.15 Can a Perceptron Learn XOR?

Question: Perceptron Learning for the XOR Gate

Can we apply the same perceptron learning procedure for the XOR gate, which has the truth table below? If so, show all the steps. If not, explain why.

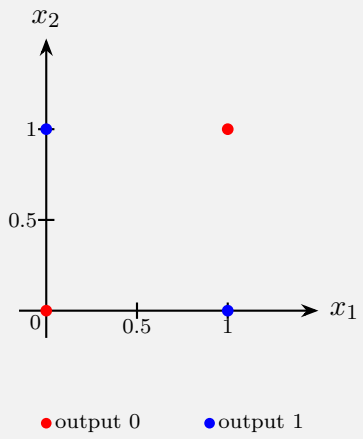
x_1	x_2	**Output**
0	0	0
0	1	1
1	0	1
1	1	0

Answer: **No**, a perceptron cannot implement XOR. A binary dataset $\{(\mathbf{x}^{(j)}, y^{(j)})\}_{j=1}^{N}$ with $y^{(j)} \in \{0, 1\}$ is **linearly separable** if and only if there exist $\mathbf{w}$ and b such that

$$y^{(j)} = 1 \iff \mathbf{w} \cdot \mathbf{x}^{(j)} + b \geq 0 \quad \text{for all } j,$$

equivalently $\big(2y^{(j)} - 1\big)\big(\mathbf{w} \cdot \mathbf{x}^{(j)} + b\big) > 0$ for all j. The XOR labels are **not linearly**

separable: the points $(0,0)$, $(1,1)$ have output 0 while $(0,1)$, $(1,0)$ have output 1, and no straight line $\mathbf{w}\cdot\mathbf{x}+b=0$ separates these two sets, so no $(\mathbf{w},b)$ satisfies the condition above. To solve the problem, we need a **multi-layer perceptron (MLP)**.



Points with output 0 are red, and points with output 1 are blue: the two classes occupy opposite corners, so no straight line separates them.

Single Perceptron = Linear Separator Only

A single perceptron can only learn **linearly separable** functions because its decision boundary is the single hyperplane $\mathbf{w}\cdot\mathbf{x}+\theta=0$. The Boolean gates AND ($w_1=0.3$, $w_2=0.5$, $\theta=-0.8$), OR ($w_1=0.5$, $w_2=0.7$, $\theta=-0.2$) and NOT ($w=-1$, $\theta=0.5$, with a single input) are all linearly separable, so a single perceptron can realise them. XOR is **not** linearly separable, so no choice of w_1, w_2, θ can classify all four points correctly — the learning procedure would never converge. Stacking perceptrons into multiple layers (MLP) overcomes this limitation.

5.2.16 Practice: OR Gate

Practice: Apply the Perceptron Learning Procedure to the OR Gate

Please apply the perceptron learning procedure for the OR gate. The truth table of logical OR is:

x_1	x_2	**Output**
0	0	0
0	1	1
1	0	1
1	1	1

Assume the weights and bias are randomly generated, say $w_1=0.1$, $w_2=0.5$, $\theta=-0.8$. Also, we set learning rate $\eta=0.2$. The activation function is the binary step

$$f(z)=\begin{cases}0 & \text{if } z<0\\ 1 & \text{if } z\geq 0\end{cases}$$

Solution. Full trace ($\eta=0.2$):

x_1	x_2	T	O	Δw_1	w_1	Δw_2	w_2	$\Delta\theta$	θ
Initial									
-	-	-	-	-	0.1	-	0.5	-	-0.8
Round 1									
0	0	0	0	0	0.1	0	0.5	0	-0.8
0	1	1	0	0	0.1	0.2	0.7	0.2	-0.6
1	0	1	0	0.2	0.3	0	0.7	0.2	-0.4
1	1	1	1	0	0.3	0	0.7	0	-0.4
Round 2									
0	0	0	0	0	0.3	0	0.7	0	-0.4
0	1	1	1	0	0.3	0	0.7	0	-0.4
1	0	1	0	0.2	0.5	0	0.7	0.2	-0.2
1	1	1	1	0	0.5	0	0.7	0	-0.2
Round 3									
0	0	0	0	0	0.5	0	0.7	0	-0.2
0	1	1	1	0	0.5	0	0.7	0	-0.2
1	0	1	1	0	0.5	0	0.7	0	-0.2
1	1	1	1	0	0.5	0	0.7	0	-0.2

Step-by-step details (only the steps with updates are expanded; all other steps are already correct, so $\Delta w_1 = \Delta w_2 = \Delta\theta = 0$):

Round 1 ($w_1 = 0.1$, $w_2 = 0.5$, $\theta = -0.8$):

- **Step 1:** $(0,0)$, $T = 0$. $s = -0.8 < 0 \Rightarrow O = 0$. Correct: no update.
- **Step 2:** $(0,1)$, $T = 1$. $s = 0.1(0) + 0.5(1) - 0.8 = -0.3 < 0 \Rightarrow O = 0$. **Incorrect**:

$$\Delta w_1 = 0.2 \times 1 \times 0 = 0 \;\Rightarrow\; w_1 = 0.1, \qquad \Delta w_2 = 0.2 \times 1 \times 1 = 0.2 \;\Rightarrow\; w_2 = 0.7,$$
$$\Delta\theta = 0.2 \times 1 = 0.2 \;\Rightarrow\; \theta = -0.6$$

- **Step 3:** $(1,0)$, $T = 1$. $s = 0.1(1) + 0.7(0) - 0.6 = -0.5 < 0 \Rightarrow O = 0$. **Incorrect**:

$$\Delta w_1 = 0.2 \times 1 \times 1 = 0.2 \;\Rightarrow\; w_1 = 0.3, \qquad \Delta w_2 = 0.2 \times 1 \times 0 = 0 \;\Rightarrow\; w_2 = 0.7,$$
$$\Delta\theta = 0.2 \times 1 = 0.2 \;\Rightarrow\; \theta = -0.4$$

- **Step 4:** $(1,1)$, $T = 1$. $s = 0.3 + 0.7 - 0.4 = 0.6 \geq 0 \Rightarrow O = 1$. Correct: no update.

Round 2 ($w_1 = 0.3$, $w_2 = 0.7$, $\theta = -0.4$):

- **Step 1:** $(0,0)$, $T = 0$. $s = -0.4 < 0 \Rightarrow O = 0$. Correct.
- **Step 2:** $(0,1)$, $T = 1$. $s = 0.7 - 0.4 = 0.3 \geq 0 \Rightarrow O = 1$. Correct.
- **Step 3:** $(1,0)$, $T = 1$. $s = 0.3 - 0.4 = -0.1 < 0 \Rightarrow O = 0$. **Incorrect**:

$$\Delta w_1 = 0.2 \times 1 \times 1 = 0.2 \;\Rightarrow\; w_1 = 0.5, \qquad \Delta w_2 = 0, \qquad \Delta\theta = 0.2 \;\Rightarrow\; \theta = -0.2$$

- **Step 4:** $(1,1)$, $T = 1$. $s = 0.5 + 0.7 - 0.2 = 1.0 \geq 0 \Rightarrow O = 1$. Correct.

Round 3 ($w_1 = 0.5$, $w_2 = 0.7$, $\theta = -0.2$):

- **Step 1:** $(0,0)$, $T = 0$. $s = -0.2 < 0 \Rightarrow O = 0$. Correct.
- **Step 2:** $(0,1)$, $T = 1$. $s = 0.7 - 0.2 = 0.5 \geq 0 \Rightarrow O = 1$. Correct.
- **Step 3:** $(1,0)$, $T = 1$. $s = 0.5 - 0.2 = 0.3 \geq 0 \Rightarrow O = 1$. Correct.
- **Step 4:** $(1,1)$, $T = 1$. $s = 0.5 + 0.7 - 0.2 = 1.0 \geq 0 \Rightarrow O = 1$. Correct.

The training **converges in 3 epochs** if the initial weights are $w_1 = 0.1$, $w_2 = 0.5$, the initial bias is $\theta = -0.8$ and the learning rate is $\eta = 0.2$. The **final weights and bias** are:

$$w_1 = 0.5, \qquad w_2 = 0.7, \qquad \theta = -0.2$$

(Equivalently, the learned decision boundary is $0.5x_1 + 0.7x_2 - 0.2 = 0$, i.e., $5x_1 + 7x_2 = 2$.)

Summary

- **Classification** predicts the class (label/category) of given data; examples include spam detection (2 classes), handwritten digit recognition (10 classes) and image classification. Image classification is a key subdomain of computer vision, vital in robotics, security, image search and medical imaging.

- An **ANN** is a universal function approximator inspired by biological neurons: dendrites → inputs, nucleus → node, axon → output, synapse → weight.
- **Neuron = linear function (weighted sum + bias) + activation function:** $z = \mathbf{w} \cdot \mathbf{x} + b = \sum_i w_i x_i + b$ and $a = f(z)$. Example: with $x_1 = 3$, $x_2 = 5$, $w_1 = 0.2$, $w_2 = 0.3$, $\theta = 0.3$ and sigmoid activation, $a = \sigma(2.4) = \frac{1}{1+e^{-2.4}} = 0.91683$.
- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$, derivative $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$, differentiable with smooth gradient, used for probability outputs. **Binary step:** $H(z) = 0$ if $z < 0$ and 1 if $z \geq 0$, derivative $H'(z) = 0$; defines the **perceptron** (threshold logic unit); zero gradient ⇒ incompatible with backpropagation; binary outputs only.
- **Perceptron learning rule:** if $\hat{y} = y$ (i.e. $T = O$), no update; otherwise $w_i \leftarrow w_i + \eta\,(y - \hat{y})\,x_i$ and $b \leftarrow b + \eta\,(y - \hat{y})$, i.e. in course notation $\Delta w_i = \eta(T - O)x_i$, $\Delta\theta = \eta(T - O)$, $w_i \leftarrow w_i + \Delta w_i$, $\theta \leftarrow \theta + \Delta\theta$. The objective is to minimise a loss such as the 0/1 error $E_{0/1} = \frac{1}{N}\sum_j \mathbb{1}[\hat{y}^{(j)} \neq y^{(j)}]$ or the squared error $E_{\text{sq}} = \frac{1}{2}\sum_j (y^{(j)} - \hat{y}^{(j)})^2$.
- Worked traces: **AND** converges to $w_1 = 0.3$, $w_2 = 0.5$, $\theta = -0.8$; **OR** converges in 3 epochs to $w_1 = 0.5$, $w_2 = 0.7$, $\theta = -0.2$ (with $w_1 = 0.1$, $w_2 = 0.5$, $\theta = -0.8$, $\eta = 0.2$ initially).
- **Learning** = updating weights; an **epoch** = one full cycle through the training dataset. **Stopping rules:** maximum training time, maximum number of cycles, minimum accuracy change.
- The **decision boundary** is the hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$ (e.g., $3x_1 + 5x_2 = 8$ for AND), which partitions the vector space into two half-spaces, one per class: $\hat{y} = 1$ iff $\mathbf{w} \cdot \mathbf{x} + b \geq 0$. A single perceptron can only learn **linearly separable** problems (AND, OR, NOT; separability holds iff $\exists\, \mathbf{w}, b$ with $\big(2y^{(j)} - 1\big)\big(\mathbf{w} \cdot \mathbf{x}^{(j)} + b\big) > 0$ for all j) — it **cannot learn XOR**; that requires a **multi-layer perceptron**.
- Implementations: a perceptron can be coded from scratch (weights, bias, response, update, training loop with zero-error stop) or via scikit-learn (`Perceptron(eta0=0.2)`, `fit`, `predict`, `coef_`, `intercept_`).

6 Multi-Layer Perceptron

6.1 From Neuron to MLP

6.1.1 Single-Neuron Limits

Recall that the **artificial neuron** is a simple biological neuron model used in an artificial neural network. A single neuron (perceptron) has a couple of important limitations:

1. It can only represent a **limited set of functions**.
2. It can only distinguish (by the value of its output) the sets of inputs that are **linearly separable** in the inputs.

The XOR Problem

One of the simplest examples of non-separable sets is the logical function **XOR**. No single straight line can separate the inputs $(0, 0), (1, 1)$ (output 0) from $(0, 1), (1, 0)$ (output 1), so a single perceptron cannot learn XOR.

6.1.2 Connecting Perceptrons

How do we remedy these limitations? The output of one perceptron can be connected to the input of other perceptron(s). This makes it possible to extend the computational

possibilities of a single perceptron.

Multi-Layer Perceptron

Connecting perceptrons so that the output of one feeds the input of others gives the **Multi-Layer Perceptron (MLP)**, which can represent non-linearly-separable functions such as XOR.

6.2 MLP Architecture

6.2.1 Architecture

A **multi-layer perceptron (MLP) neural network** is a type of **feed-forward neural network**. ("Feed-forward" means the nodes in this network do not form a cycle.) It consists of three types of layers:

- **Input layer** (also called layer i),
- **Hidden layer** (also called layer j),
- **Output layer** (also called layer k).

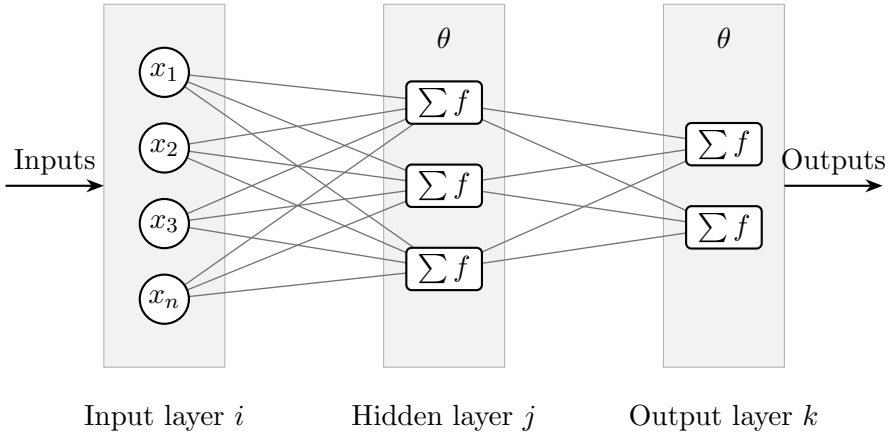
Notation used in the network diagram:

- $x_1, x_2, \ldots, x_n$ are the **inputs**.
- $\sum$ is a **summation** (weighted sum of incoming signals).
- w_{ab} is the **weight** connecting node a to node b.
- f is an **activation function**.
- θ is a **bias**.

Formally, an MLP with L weight layers maps an input vector $\mathbf{a}^{(0)} = \mathbf{x} \in \mathbb{R}^{n_0}$ to an output $\mathbf{a}^{(L)} \in \mathbb{R}^{n_L}$ by composing affine maps with a nonlinear activation f applied element-wise:

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = f\big(\mathbf{z}^{(l)}\big), \qquad l = 1, 2, \ldots, L,$$

where $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix with entries $w_{ij}^{(l)}$, $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector, and $\mathbf{z}^{(l)}$ is the pre-activation of layer l. The scalar bias θ in the diagram plays the role of an entry of $\mathbf{b}^{(l)}$. A single neuron is the special case $L = 1$ with f the (step) activation.



6.2.2 Practical Questions

How to initialize the weights and biases?

Initialize them to some **small random values**.

How to perform training?

1. Let the network calculate the output with the given inputs (**forward propagation**).
2. Calculate the **error/loss function** (i.e., the difference between the calculated outputs and the target outputs).
3. Update the weights and biases between the hidden and output layer (**backward propagation**).
4. Update the weights and biases between the input and hidden layer (**backward propagation**).
5. Go back to step 1.

Formally, training seeks the parameter set $\theta = \{W^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$ that minimises the empirical loss on the training set $\{(\mathbf{x}^{(j)}, \mathbf{y}^{(j)})\}_{j=1}^{N}$:

$$J(\theta) = \frac{1}{N} \sum_{j=1}^{N} L\big(\mathbf{y}^{(j)}, \hat{\mathbf{y}}^{(j)}\big), \qquad \hat{\mathbf{y}}^{(j)} = \mathrm{MLP}\big(\mathbf{x}^{(j)}; \theta\big).$$

The per-sample loss L is chosen by the task: the squared error for regression,

$$L_{\mathrm{MSE}}(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|^2,$$

and the cross-entropy for classification,

$$L_{\mathrm{CE}}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{c} y_c \log \hat{y}_c.$$

Thus the total mean squared error is $L = \frac{1}{N}\sum_j \|\mathbf{y}^{(j)} - \hat{\mathbf{y}}^{(j)}\|^2$. Steps 1–4 above are the numerical realisation of this objective: step 1 evaluates $\hat{\mathbf{y}}^{(j)}$ (**forward propagation**), step 2 evaluates L, and steps 3–4 apply the chain rule to obtain $\nabla_\theta J$ and descend it (**backward propagation**).

When to stop training?

- After a **fixed number of iterations** through the loop.
- Once the **training error falls below some threshold**.
- Stop at a **minimum of the error on the validation set**.

Training Cost

Training can be very slow in networks with multiple hidden layers!

6.2.3 Weight and Bias Update Rules

Assume the activation function is the **sigmoid function** $\sigma(x) = \dfrac{1}{1+e^{-x}}$, and the error/loss function is

$$E = \frac{1}{2} \sum_{\text{all } k} (O_k - T_k)^2.$$

The formulas for updating weights and biases are derived by **minimizing the loss function using gradient descent**.

With one hidden layer, write the layer-wise forward pass as

$$\mathbf{z}^{(j)} = W^{(j)}\mathbf{x} + \mathbf{b}^{(j)}, \quad \mathbf{a}^{(j)} = \sigma\big(\mathbf{z}^{(j)}\big), \quad \mathbf{z}^{(k)} = W^{(k)}\mathbf{a}^{(j)} + \mathbf{b}^{(k)}, \quad \mathbf{o} = \mathbf{a}^{(k)} = \sigma\big(\mathbf{z}^{(k)}\big).$$

The **error term** of a unit is the derivative of the loss with respect to its pre-activation. Applying the chain rule to the output layer gives

$$\delta_k \equiv \frac{\partial E}{\partial z_k^{(k)}} = \frac{\partial E}{\partial O_k} \cdot \frac{\partial O_k}{\partial z_k^{(k)}} = (O_k - T_k)\, \sigma'\Big(z_k^{(k)}\Big),$$

and, because E depends on the hidden pre-activation $z_j^{(j)}$ only through the output layer,

$$\delta_j \equiv \frac{\partial E}{\partial z_j^{(j)}} = \sigma'\Big(z_j^{(j)}\Big) \sum_{k \in K} \frac{\partial E}{\partial z_k^{(k)}} \cdot \frac{\partial z_k^{(k)}}{\partial a_j^{(j)}} = \sigma'\Big(z_j^{(j)}\Big) \sum_{k \in K} \delta_k \, w_{jk}.$$

Using the sigmoid derivative $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) = O(1 - O)$ recovers exactly the factors $O_k(1 - O_k)$ and $O_j(1 - O_j)$ below. The remaining partial derivatives (with O_i the input to layer j) are

$$\frac{\partial E}{\partial w_{jk}} = \delta_k \, O_j, \quad \frac{\partial E}{\partial \theta_k} = \delta_k, \qquad \frac{\partial E}{\partial w_{ij}} = \delta_j \, O_i, \quad \frac{\partial E}{\partial \theta_j} = \delta_j,$$

so the updates are the generic gradient-descent rule applied entry-wise.

In vector form, with $\odot$ the element-wise product and $\top$ the transpose, the error term of layer l propagates backwards as

$$\delta^{(l)} = \big(W^{(l+1)}\big)^{\top} \delta^{(l+1)} \odot f'\big(\mathbf{z}^{(l)}\big),$$

and the gradients of the loss with respect to that layer's parameters are

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \big(\mathbf{a}^{(l-1)}\big)^{\top}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}.$$

Output layer (layer k):

$$\delta_k = (O_k - T_k)\, O_k(1 - O_k)$$
$$w_{jk} \leftarrow w_{jk} - \eta \, \delta_k O_j, \qquad \theta_k \leftarrow \theta_k - \eta \, \delta_k$$

Hidden layer (layer j):

$$\delta_j = O_j(1 - O_j) \sum_{k \in K} \delta_k \, w_{jk}$$
$$w_{ij} \leftarrow w_{ij} - \eta \, \delta_j O_i, \qquad \theta_j \leftarrow \theta_j - \eta \, \delta_j$$

Meaning of the Symbols

- O_i is the input in layer i; θ_j is the bias of node in layer j.
- O_j is the output of node in layer j; θ_k is the bias of node in layer k.
- O_k is the output of node in layer k; T_k is the target output of node in layer k.
- w_{ij} is the weight connecting node in layer i to node in layer j.
- w_{jk} is the weight connecting node in layer j to node in layer k.
- $\delta_k \equiv \partial E / \partial z_k^{(k)}$ is the **error term** of a unit in layer k, used to update w_{jk} and θ_k.
- $\delta_j \equiv \partial E / \partial z_j^{(j)}$ is the **error term** of a unit in layer j, used to update w_{ij} and θ_j.
- η is the **learning rate**.

6.3 Gradient Descent

6.3.1 Intuitive Idea of Gradient Descent

Consider an error/loss function plotted against a weight. It has **many local minima** (points where the gradient is 0, i.e., the slope is horizontal).

- If we choose an initial weight (say point #1) at a certain learning rate, and the slope at that point is negative, then by moving towards the negative of the slope at that point, we converge to the minimum near that starting region (say point #4).
- But if we start at a different initial weight, we may descend into a completely different

local minimum.

Local Minima

Gradient descent only guarantees convergence to a **local** minimum, not the global minimum. The result depends on the **initial weights** and the **learning rate**.

6.3.2 Formal Update Rule

To minimize the function $E = \frac{1}{2}\sum_{\text{all } k}(O_k - T_k)^2$, we can follow the **negative of the gradient** (slope in 2D), and thus go in the direction of **steepest descent**. This is **gradient descent**.

Formally, for a parameter α (α can be a weight or a bias), if we start at a point α_0 and move a positive distance η in the direction of the negative gradient, then our new and improved α_1 will look like this:

$$\alpha_1 = \alpha_0 - \eta \nabla E(\alpha_0)$$

More generally, we can write a formula for tuning α_t into α_{t+1} (the subscript t in α_t represents the iteration step):

$$\alpha_{t+1} = \alpha_t - \eta \nabla E(\alpha_t)$$

Starting from an initial guess α_0, we keep improving little by little until we find a local minimum α_T, i.e., $\nabla E(\alpha_T) = 0$.

Equivalently, if we collect all the weights and biases into a single parameter vector θ, the same rule—updating each parameter opposite to the gradient—is written compactly as

$$\theta \leftarrow \theta - \eta \nabla_\theta L,$$

where $\eta > 0$ is the learning rate and $\nabla_\theta L$ is the gradient of the loss with respect to every parameter simultaneously.

Implementation

This process may take thousands of iterations, so we typically implement gradient descent with a computer.

6.3.3 Why Not Closed Form?

Question: Why is gradient descent used rather than directly finding a closed-form mathematical solution?

Answers:

- For most **non-linear regression problems**, there is **no closed-form solution**.
- Even for those with a closed-form solution, gradient descent is **computationally cheaper (faster)** to find the solution.

6.3.4 Sigmoid Review

For multi-layer perceptrons, the **Sigmoid function** is used as an activation function for neurons since it is **continuous and differentiable** (i.e., it can be used to find the weight updating rules easily).

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative of the Sigmoid

$$\frac{d\,\sigma(x)}{d\,x} = z \cdot (1 - z), \qquad \text{where } z = \sigma(x).$$

This simple form (expressed using the output itself) is exactly why the update rules contain the factors $O_k(1 - O_k)$ and $O_j(1 - O_j)$.

6.3.5 Learning Steps

1. **Forward propagation:** Run the network forward with your input data to get the network output.
2. **Backward propagation:** For each output node, compute
$$\delta_k = (O_k - T_k)\, O_k(1 - O_k).$$
3. **Backward propagation:** For each hidden node, compute
$$\delta_j = O_j(1 - O_j) \sum_{k \in K} \delta_k\, w_{jk}.$$
4. **Backward propagation:** Update the weights and biases as follows:
$$w_{jk} \leftarrow w_{jk} - \eta\, \delta_k O_j, \qquad \theta_k \leftarrow \theta_k - \eta\, \delta_k,$$
$$w_{ij} \leftarrow w_{ij} - \eta\, \delta_j O_i, \qquad \theta_j \leftarrow \theta_j - \eta\, \delta_j,$$
where η denotes the learning rate; typically, η is a value between 0 and 1.

6.4 Example: Learning XOR

XOR Logical Operation

Suppose that we will work on the problem of the XOR logical operation. The truth table of logical XOR is:

x_1	x_2	T
0	0	0
0	1	1
1	0	1
1	1	0

Assume the weights are randomly generated, say $w_1 = -0.65$, $w_2 = 0.64$, $w_3 = 1.11$, $w_4 = 0.84$, $w_5 = 0.86$, and $w_6 = -1.38$. Also, assume the biases are randomly generated, say $\theta_1 = 0$, $\theta_2 = 0$, and $\theta_3 = 0$. Finally, assume $\eta = 0.5$. The activation function is $f(x) = \dfrac{1}{1 + e^{-x}}$.

The network has: two inputs x_1, x_2; two hidden nodes j_1, j_2 with w_1, w_2 feeding j_1 and w_3, w_4 feeding j_2; one output node k fed by w_5 (from j_1) and w_6 (from j_2).

6.4.1 Round 1, Step 1: Forward

Inputs: $x_1 = 0$, $x_2 = 0$; Actual Output: $T = 0$; $\eta = 0.5$.

Weights: $w_1 = -0.65$, $w_2 = 0.64$, $w_3 = 1.11$, $w_4 = 0.84$, $w_5 = 0.86$, $w_6 = -1.38$.
Biases: $\theta_1 = 0$, $\theta_2 = 0$, $\theta_3 = 0$.

Calculations:

$$\textstyle\sum_1 = x_1 \cdot w_1 + x_2 \cdot w_2 = 0 \cdot (-0.65) + 0 \cdot (0.64) = 0$$

$$\text{Output } (O_{j1}): \; f(\textstyle\sum_1 + \theta_1) = f(0 + 0) = 0.5$$

$$\sum_2 = x_1 \cdot w_3 + x_2 \cdot w_4 = 0 \cdot 1.11 + 0 \cdot 0.84 = 0$$
$$\text{Output } (O_{j2}): \ f(\sum_2 + \theta_2) = f(0 + 0) = 0.5$$
$$\sum_3 = O_{j1} \cdot w_5 + O_{j2} \cdot w_6 = 0.5 \cdot (0.86) + 0.5 \cdot (-1.38) = -0.26$$
$$\text{Output } (O_k): \ f(\sum_3 + \theta_3) = f(-0.26 + 0) = 0.435364$$

6.4.2 Round 1, Step 1: Backward

$$\delta_k = (O_k - T_k)\, O_k(1 - O_k) = (0.435364 - 0)(0.435364)(1 - 0.435364) = 0.107022$$

Update hidden-to-output weights and output bias:

$$\text{New } w_5 = \text{Old } w_5 - \eta\, \delta_k O_{j1} = 0.86 - (0.5)(0.107022)(0.5) = 0.833245$$
$$\text{New } w_6 = \text{Old } w_6 - \eta\, \delta_k O_{j2} = -1.38 - (0.5)(0.107022)(0.5) = -1.406756$$
$$\text{New } \theta_3 = \text{Old } \theta_3 - \eta\, \delta_k = 0 - (0.5)(0.107022) = -0.053511$$

Hidden node deltas:

$$\delta_{j1} = O_{j1}(1 - O_{j1}) \sum_{k \in K} \delta_k w_{jk} = 0.5(1 - 0.5)(0.107022)(0.86) = 0.023010$$

$$\delta_{j2} = O_{j2}(1 - O_{j2}) \sum_{k \in K} \delta_k w_{jk} = 0.5(1 - 0.5)(0.107022)(-1.38) = -0.036923$$

Update input-to-hidden weights and hidden biases:

$$\text{New } w_1 = \text{Old } w_1 - \eta\, \delta_{j1} x_1 = -0.65 - 0.5(0.023010)(0) = -0.65$$
$$\text{New } w_2 = \text{Old } w_2 - \eta\, \delta_{j1} x_2 = 0.64 - 0.5(0.023010)(0) = 0.64$$
$$\text{New } w_3 = \text{Old } w_3 - \eta\, \delta_{j2} x_1 = 1.11 - 0.5(-0.036923)(0) = 1.11$$
$$\text{New } w_4 = \text{Old } w_4 - \eta\, \delta_{j2} x_2 = 0.84 - 0.5(-0.036923)(0) = 0.84$$
$$\text{New } \theta_1 = \text{Old } \theta_1 - \eta\, \delta_{j1} = 0 - (0.5)(0.023010) = -0.011505$$
$$\text{New } \theta_2 = \text{Old } \theta_2 - \eta\, \delta_{j2} = 0 - (0.5)(-0.036923) = 0.018462$$

Zero Inputs Freeze Weights

Since $x_1 = x_2 = 0$ in this step, the input-to-hidden weights w_1, w_2, w_3, w_4 do not change (the update term $\eta\, \delta_j x_i$ is zero). Only the biases of the hidden nodes change. The weights connected to the hidden layer (w_5, w_6) do change because the hidden outputs O_{j1}, O_{j2} are non-zero.

6.4.3 Round 1, Step 2: Forward

Inputs: $x_1 = 0$, $x_2 = 1$; Actual Output: $T = 1$; $\eta = 0.5$.

Weights: $w_1 = -0.65$, $w_2 = 0.64$, $w_3 = 1.11$, $w_4 = 0.84$, $w_5 = 0.833245$, $w_6 = -1.406756$.
Biases: $\theta_1 = -0.011505$, $\theta_2 = 0.018462$, $\theta_3 = -0.053511$.

Calculations:

$$\sum_1 = x_1 \cdot w_1 + x_2 \cdot w_2 = 0 \cdot (-0.65) + 1 \cdot (0.64) = 0.64$$
$$\text{Output } (O_{j1}): \ f(\sum_1 + \theta_1) = f(0.64 + (-0.011505)) = 0.652148$$
$$\sum_2 = x_1 \cdot w_3 + x_2 \cdot w_4 = 0 \cdot 1.11 + 1 \cdot 0.84 = 0.84$$
$$\text{Output } (O_{j2}): \ f(\sum_2 + \theta_2) = f(0.84 + 0.018462) = 0.702339$$
$$\sum_3 = O_{j1} \cdot w_5 + O_{j2} \cdot w_6 = 0.652148 \cdot 0.833245 + 0.702339 \cdot (-1.406756) = -0.444621$$
$$\text{Output } (O_k): \ f(\sum_3 + \theta_3) = f(-0.444621 - 0.053511) = 0.377980$$

6.4.4 Round 1, Step 2: Backward

$$\delta_k = (O_k - T_k)\, O_k(1 - O_k) = (0.377980 - 1)(0.377980)(1 - 0.377980) = -0.146244$$

Update hidden-to-output weights and output bias:

$$\text{New } w_5 = \text{Old } w_5 - \eta\, \delta_k O_{j1} = 0.833245 - (0.5)(-0.146244)(0.652148) = 0.880931$$

$$\text{New } w_6 = \text{Old } w_6 - \eta\, \delta_k O_{j2} = -1.406756 - (0.5)(-0.146244)(0.702339) = -1.355400$$

$$\text{New } \theta_3 = \text{Old } \theta_3 - \eta\, \delta_k = -0.053511 - (0.5)(-0.146244) = 0.019611$$

Hidden node deltas:

$$\delta_{j1} = O_{j1}(1-O_{j1}) \sum_{k \in K} \delta_k w_{jk} = 0.652148\,(1-0.652148)(-0.146244)(0.833245) = -0.027463$$

$$\delta_{j2} = O_{j2}(1-O_{j2}) \sum_{k \in K} \delta_k w_{jk} = 0.702339\,(1-0.702339)(-0.146244)(-1.406756) = 0.043010$$

Update input-to-hidden weights and hidden biases:

$$\text{New } w_1 = \text{Old } w_1 - \eta\, \delta_{j1} x_1 = -0.65 - 0.5(-0.027463)(0) = -0.65$$

$$\text{New } w_2 = \text{Old } w_2 - \eta\, \delta_{j1} x_2 = 0.64 - 0.5(-0.027463)(1) = 0.653732$$

$$\text{New } w_3 = \text{Old } w_3 - \eta\, \delta_{j2} x_1 = 1.11 - 0.5(0.043010)(0) = 1.11$$

$$\text{New } w_4 = \text{Old } w_4 - \eta\, \delta_{j2} x_2 = 0.84 - 0.5(0.043010)(1) = 0.818495$$

$$\text{New } \theta_1 = \text{Old } \theta_1 - \eta\, \delta_{j1} = -0.011505 - (0.5)(-0.027463) = 0.002227$$

$$\text{New } \theta_2 = \text{Old } \theta_2 - \eta\, \delta_{j2} = 0.018462 - (0.5)(0.043010) = -0.003043$$

6.4.5 Round 10000, Step 4: Converged

After many iterations ($\eta = 0.5$), the parameters converge to:

$$\begin{aligned}
&\text{New } w_5 = 9.533777, \quad \text{New } w_6 = -9.290053,\\
&\text{New } w_1 = -6.251654, \quad \text{New } w_2 = 6.229623,\\
&\text{New } w_3 = -5.717901, \quad \text{New } w_4 = 5.967069,\\
&\text{New } \theta_3 = 4.417774, \quad \text{New } \theta_1 = -3.406953, \quad \text{New } \theta_2 = 2.851217.
\end{aligned}$$

Round 10001, Forward Propagation (verification):

Input	Target	Network Output
$(x_1 = 0,\, x_2 = 0)$	0	$y = 0.016973$
$(x_1 = 0,\, x_2 = 1)$	1	$y = 0.984139$
$(x_1 = 1,\, x_2 = 0)$	1	$y = 0.980513$
$(x_1 = 1,\, x_2 = 1)$	0	$y = 0.015179$

XOR Learned

After 10000 rounds of training, the MLP outputs are very close to the XOR targets $(0, 1, 1, 0)$ — something a single perceptron can never achieve.

6.5 MLP from Scratch

6.5.1 Initialization

```
import numpy as np # Import NumPy

class MultiLayerPerceptron :
```

```
    def __init__(self):
        """ Multi-layer perceptron initialization """
        self.wij = np.array([ # Weights between input and hidden layer
            [-0.65, 0.64],    # w1, w2
            [1.11, 0.84]      # w3, w4
        ])
        # Weights between hidden and output layer
        self.wjk = np.array([
            [0.86],           # w5
            [-1.38]           # w6
        ])
        self.tj = np.array([  # Biases of nodes in the hidden layer
            [0.0],            # Theta 1
            [0.0]             # Theta 2
        ])
        # Bias in the output layer, Theta 3
        self.tk = np.array([[0.0]])
        self.learning_rate = 0.5    # Eta
        self.max_round = 10000      # Number of rounds
```

6.5.2 Sigmoid and Forward Propagation

```
    def sigmoid(self, z, sig_cal=False):
        """ Sigmoid function and the calculation of z * (1-z) """
        # If sig_cal is True, return sigmoid
        if sig_cal: return 1 / (1 + np.exp(-z))
        # If sig_cal is False, return z * (1-z)
        return z * (1 - z)

    def forward(self, x, predict=False):
        """ Forward propagation """
        # Get the training example as a column vector. Shape (2,1)
        sample = x.reshape(len(x), 1)
        # Compute the hidden node outputs. Shape (2,1)
        yj = self.sigmoid(self.wij.dot(sample) + self.tj,
sig_cal=True)
        # Compute the output of node in the output layer. Shape (1,1)
        yk = self.sigmoid(self.wjk.transpose().dot(yj) + self.tk,
                          sig_cal=True)
        # If predict is True, return the output of node in the layer
        # node
        if predict: return yk
        # Return (data sample, hidden node outputs, predicted output)
        return (sample, yj, yk)
```

6.5.3 Backpropagation

```
    def backpropagation(self, values, Tk):
        Oi = values[0] # Input sample
        Oj = values[1] # Hidden node outputs
        Ok = values[2] # Predicted output
        """ back propagation """
        # deltak = (Ok-Tk)Ok(1-Ok). Shape (1,1)
        deltaK = np.multiply((Ok - Tk), self.sigmoid(Ok))
        # deltaj = Oj(1-Oj)(deltak)(Wjk). Shape (2,1)
        deltaJ = np.multiply(self.sigmoid(Oj), deltaK[0][0] *
self.wjk)
        # wjk = wjk - eta(deltak)(Oj). Shape (2,1)
```

```
        self.wjk -= self.learning_rate * deltaK[0][0] * Oj
        # thetak = thetak - eta(deltak). Shape (1,1)
        self.tk -= self.learning_rate * deltaK
        # wij = wij - eta(deltaj)(Oi). Shape (2,2)
        s = self.learning_rate * deltaJ.dot(Oi.T)
        # Alternative: s = self.learning_rate * deltaJ * Oi.T
        self.wij -= s
        # thetaj = thetaj - eta(deltaj). Shape (2,1)
        self.tj -= self.learning_rate * deltaJ
```

6.5.4 Training Loop

```
    def train(self, X, T):
        """ Training """
        # Train max_round number of rounds
        for i in range(self.max_round):
            for j in range(m):          # Use all the samples
                print(f'Iteration: {i+1} and {j+1}')
                # Forward propagation
                values = self.forward(X[j])
                self.backpropagation(values, T[j]) # Back propagation

    def print(self):
        print(f'wij: {self.wij}')
        print(f'wjk: {self.wjk}')
        print(f'tj: {self.tj}')
        print(f'tk: {self.tk}')

m = 4 # Number of training samples
```

6.5.5 Data, Training, and Testing

```
X = np.array([ # Input data
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
T = np.array([ # Target values
    [0],
    [1],
    [1],
    [0]
])

mlp = MultiLayerPerceptron() # Create an object
mlp.train(X, T)              # Call train function
mlp.print()                  # Print all the parameter values

for k in range(m):           # Testing
    Ok = mlp.forward(X[k], True)
    print(f'y{k}: {Ok}')
```

These Formulas Are Specific to Sigmoid + Squared Error

The update formulas

$$\delta_k = (O_k - T_k)O_k(1 - O_k), \qquad \delta_j = O_j(1 - O_j)\sum_{k \in K} \delta_k w_{jk}$$

only apply for the error/loss function $E = \frac{1}{2}\sum_{\text{all } k}(O_k - T_k)^2$ and the Sigmoid activation function. If a different error/loss function or activation function is used in an MLP, we need to **differentiate the error/loss function again** (i.e., re-derive the update rules, similar to the derivations in the supplementary notes).

6.6 Example: Handwritten Digits

We will build an MLP artificial neural network to recognize/classify handwritten digits.

6.6.1 Terminologies

Key Terms

- **Training data:** The data our model learns from. Sometimes split into training and validation data.
- **Testing data:** The data kept secret from the model until after it has been trained. Testing data is used to evaluate our model.
- **Loss function:** A function used to (1) do differentiation to update parameters during training; (2) monitor each epoch to decide the convergence during training; (3) quantify how accurate a model's predictions were. The only objective of the neural network is optimizing/minimizing the loss function.
- **Optimization algorithm:** Controls exactly how the parameters are adjusted during training, e.g., gradient descent.

6.6.2 Dataset: MNIST

We use the **Modified National Institute of Standards and Technology (MNIST)** dataset. This dataset contains two sets of samples:

- **Training data:** 60000 images of 28 pixel × 28 pixel handwritten digits from 0 to 9.
- **Testing data:** 10000 images of 28 pixel × 28 pixel.

6.6.3 Procedures

1. Import the required libraries and define a global variable
2. Load the data
3. Explore the data
4. Build the model
5. Compile the model
6. Train the model
7. Evaluate the model accuracy
8. Save the model
9. Use the model
10. Plot the confusion matrix

6.6.4 Step 1: Import the Required Libraries and Define a Global Variable

```
import numpy as np                          # Import numpy library
import matplotlib.pyplot as plt             # Import matplot library
import seaborn as sn                        # Import seaborn library
import pandas as pd                         # Import pandas library
import math                                 # Import math library
import datetime                             # Import datetime library
from keras.datasets import mnist            # Import MNIST dataset
from keras.models import Sequential         # Import Sequential class
from keras.layers import Dense, Flatten     # Import Dense, Flatten class
from keras import regularizers              # Import regularizers
from tensorflow.keras.optimizers import Adam  # Import Adam optimizer
# Import sparse categorical crossentropy loss
from keras.metrics import  sparse_categorical_crossentropy
from keras.callbacks import TensorBoard     # Import TensorBoard class
from keras.models import load_model         # Import load_model method
# Import plot_model method
from tensorflow.keras.utils import plot_model
# Import confusion_matrix method
from tensorflow.math import confusion_matrix
from tensorflow.keras import activations # Import activations module

epochs = 1 # Number of epochs to train the model
```

6.6.5 Step 2: Load the Data

```
# x_train is a NumPy array of grayscale image data
# y_train is a NumPy array of digit labels (in range 0-9)
# x_test is a NumPy array of grayscale image data
# y_test is a NumPy array of digit labels (in range 0-9)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Print the data shape
print('x_train:', x_train.shape)
print('y_train:', y_train.shape)
print('x_test:', x_test.shape)
print('y_test:', y_test.shape)
```

Output:

```
x_train: (60000, 28, 28)
y_train: (60000,)
x_test: (10000, 28, 28)
y_test: (10000,)
```

6.6.6 Step 3: Explore the Data

```
# Show the pixel values (from 0 - 255) of the first image
pd.DataFrame(x_train[0])

# Show the image in binary form
plt.imshow(x_train[0], cmap=plt.cm.binary)
plt.show()
```

Display a grid of 25 sample images:

```
numbers_to_display = 25 # Display 25 images
# Compute number of images per row
num_cells = math.ceil(math.sqrt(numbers_to_display))
```

```
plt.figure(figsize=(10, 10)) # 10x10 inches
# Show all the images
for i in range(numbers_to_display):
    # number of rows, number of columns
    plt.subplot(num_cells, num_cells, i + 1)
    plt.xticks([])  # Remove all xticks
    plt.yticks([])  # Remove all yticks
    plt.grid(False) # No grid lines
    # Display data as a binary image
    plt.imshow(x_train[i], cmap=plt.cm.binary)
    # Show training image labels
    plt.xlabel(y_train[i])
plt.show() # Show the figure
```

6.6.7 Step 4: Build the Model

Instead of building the model from scratch, we will use the software library **Keras**.

Layers:

- **Layer 1:** Flatten layer that will flatten a 2D image into 1D.
- **Layer 2:** Hidden Dense layer 1 with 128 neurons and ReLU activation.
- **Layer 3:** Hidden Dense layer 2 with 128 neurons and ReLU activation.
- **Layer 4:** Output Dense layer with 10 Softmax outputs. This layer represents the guess, i.e., the 0th output represents the probability that the input digit is 0, the 1st output represents the probability that the input digit is 1, etc.

ReLU Activation Function

ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- A derivable function that allows for backpropagation while simultaneously making it **computationally efficient**.
- The neurons will only be deactivated if the output of the linear transformation is less than 0.
- **Accelerates the convergence of gradient descent** due to its linear property.
- **Limitation:** the Dying ReLU problem.

The Dying ReLU Problem

- The negative side of the graph makes the gradient value **zero**.
- During the backpropagation process, the weights and biases for some neurons are **not updated**.
- This creates **dead neurons** which never get activated.
- All the negative input values become zero immediately, which **decreases the model's ability** to fit or train from the data properly.

Softmax Activation Function The output of the sigmoid function is in the range of 0 to 1, which can be thought of as a probability. But what about **multi-class classification**?

Softmax Activation Function

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j \in \text{output layer}} e^{z_j}}$$

- A combination of multiple sigmoids.
- Calculates the **relative probabilities**. Similar to the sigmoid/logistic activation function, the Softmax function returns the probability of each class.
- Most commonly used as an activation function for the **output layer** of the neural network in the case of **multi-class classification**.

```
model = Sequential() # Create a Sequential object

# Input layer
# Add a flatten layer to convert the image data to a single column
model.add(Flatten(input_shape=x_train.shape[1:]))

# Hidden layer 1
# Add a dense layer (fully-connected layer) and use ReLU activation
# function.
# This layer uses L2 loss, computed as l2 * reduce_sum(square(x)),
# where l2 is 0.002
model.add(Dense(units=128, activation='relu',
                kernel_regularizer=regularizers.l2(0.002)))

# Hidden layer 2
# Add a dense layer (fully-connected layer) and use ReLU activation
# function.
model.add(Dense(units=128, activation=activations.relu,
                kernel_regularizer=regularizers.l2(0.002)))

# Output layer
# Add a dense layer (fully-connected layer) and use softmax
# activation.
model.add(Dense(units=10, activation='softmax'))

# We apply kernel_regularizer to penalize the weights which are very
# large
# causing the network to overfit; after applying kernel_regularizer
# the
# weights will become smaller.



model.summary() # Print model summary
# Plot the model
plot_model(model, show_shapes=True, show_layer_names=True)

Model: "sequential"
 _________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 128)               100480
_________________________________________________________________
dense_1 (Dense)              (None, 128)               16512
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1290
=================================================================
Total params: 118,282
Trainable params: 118,282
Non-trainable params: 0
_________________________________________________________________
```

Question: How to calculate #param in a dense layer?

Counting Parameters in a Dense Layer

In a dense layer,

$$\begin{aligned}\#\text{param} &= \#\text{weights} + \#\text{biases} \\ &= \#\text{prev. nodes} \cdot \#\text{curr. nodes} + \#\text{curr. nodes} \\ &= (\#\text{prev. nodes} + 1) \cdot \#\text{curr. nodes}\end{aligned}$$

For example:

- dense layer 1: $(784 + 1) \times 128 = 100480$;
- dense layer 2: $(128 + 1) \times 128 = 16512$;
- output layer: $(128 + 1) \times 10 = 1290$.

6.6.8 Step 5: Compile the Model

```
# Create an Adam optimizer by creating an object
# Set learning rate to 0.001
# Note: Optimizers are Classes or methods used to change the
# attributes
# of your machine/deep learning model such as weights and learning
# rate
# in order to reduce the losses.
adam_optimizer = Adam(learning_rate=0.001)

# Compile the model, i.e., configures the model for training
# Use crossentropy loss function since there are two or more label
# classes.
# Use adam algorithm (a stochastic gradient descent method)
# Use accuracy as metric, i.e., report on accuracy
model.compile(
    optimizer=adam_optimizer,
    loss= sparse_categorical_crossentropy ,
    metrics=['accuracy']
)
```

Categorical / Multi-class Cross-entropy Loss

$$\text{CCE} = -\sum_{i \in C} y_i \cdot \log(p_i),$$

where y represents the actual labels, usually a **one-hot vector**; p represents predictions, usually the output of **Softmax**.

- Mostly used to measure the **distance between two distributions**.
- Measures how close the predictions (model distribution) are to the actual labels (data distribution).

Optimisers beyond plain SGD. Plain SGD uses only the current gradient $g_t = \nabla_\theta L$. **Momentum** keeps an exponentially weighted average of past gradients and steps along it, smoothing the trajectory and accelerating through shallow directions:

$$v_t = \beta\, v_{t-1} + (1 - \beta)\, g_t, \qquad \theta \leftarrow \theta - \eta\, v_t,$$

where $\beta \in [0, 1)$ is the momentum coefficient. **Adam** (used above) additionally rescales the step per parameter using running estimates of the first and second moments of the gradient, so the effective learning rate adapts to each parameter.

6.6.9 Step 6: Train the Model

```
# TensorBoard is a visualization tool, enabling us to track metrics
# like
# loss and accuracy, visualize the model graph, view histograms.
log_dir = ".logs/fit/" +
   datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
# log_dir: the path of the directory where to save the log files
# histogram_freq: frequency (in epochs) at which to compute activation
# and weight histograms for the layers of the model
tensorboard_callback = TensorBoard(log_dir=log_dir, histogram_freq=1)

# Fit the model, i.e., train the model
# Specify training data and labels, number of epochs to train the
# model,
# validation data, i.e., data on which to evaluate the loss
# Write TensorBoard logs after every batch of training to monitor
training_history = model.fit(x_train, y_train, epochs=epochs,
                             validation_data=(x_test, y_test),
                             callbacks=[tensorboard_callback])
```

6.6.10 Step 7: Evaluate Model Accuracy

```
# Evaluate the model
# Specify testing data and labels
validation_loss, validation_accuracy = model.evaluate(x_test, y_test)

# Print loss and accuracy
print('Validation loss: ', validation_loss)
print('Validation accuracy: ', validation_accuracy)
```

Output:

```
Validation loss: 0.2004156953573227
Validation accuracy: 0.9646
```

6.6.11 Step 8: Save the Model

Save the entire model to an HDFS (Hadoop Distributed File System) file. The `.h5` extension of the file indicates that the model should be saved in Keras format as an HDFS file.

```
model_name = 'digits_recognition_mlp.h5'
model.save(model_name, save_format='h5')
loaded_model = load_model(model_name)
```

6.6.12 Step 9: Use the Model

To use the model, we call the `predict()` function:

```
# Use the model to do prediction by specifying the image(s).
# Get back a NumPy array of prediction
predictions = loaded_model.predict([x_test])
print('predictions:', predictions.shape)
```

Output:

```
predictions: (10000, 10)
```

Each prediction consists of 10 probabilities (one for each number from 0 to 9). The digit with the **highest probability** is chosen, as that would be the digit that our model is most confident with.

```
# Predictions in form of one-hot vectors (arrays of probabilities).
pd.DataFrame(predictions)
```

```
# Let's extract predictions with highest probabilities
# and detect what digits have been actually recognized.
prediction_results = np.argmax(predictions, axis=1)
pd.DataFrame(prediction_results)
```

Visualize the predictions (correct in green, wrong in red):

```
numbers_to_display = 196 # Display 196 images
# Compute number of images per row
num_cells = math.ceil(math.sqrt(numbers_to_display))
plt.figure(figsize=(10, 10)) # Each image is in size 10x10 inches
# Show all the images
for i in range(numbers_to_display):
    # Number of rows, number of columns, index (start from 1)
    plt.subplot(num_cells, num_cells, i + 1)
    plt.xticks([])  # Remove all xticks
    plt.yticks([])  # Remove all yticks
    plt.grid(False) # No grid lines
    # Check if the prediction is correct. If so, display in green.
    # Otherwise in red.
    color_map = 'Greens' if prediction_results[i] == y_test[i] else
   'Reds'
    # Display data as a color image
    plt.imshow(x_test[i], cmap=color_map)
    # Show predicted image labels
    plt.xlabel(prediction_results[i])
# Adjust the height of the padding between subplots to 1
# Adjust the width of the padding between subplots to 0.5
plt.subplots_adjust(hspace=1, wspace=0.5)
plt.show() # Show the figure
```

6.6.13 Step 10: Plotting a Confusion Matrix

The **confusion matrix** shows what numbers are recognized well by the model and what numbers the model usually confuses.

```
# Compute confusion matrix to evaluate the accuracy of a
# classification
# by creating a confusion_matrix object.
# Specify true labels and prediction results
cm = confusion_matrix(y_test, prediction_results)
```

```
# Each image is in size 9x9 inches
f, ax = plt.subplots(figsize=(9, 9))
```

```
# Draw heat map to show the magnitude in color
sn.heatmap(
    cm,             # data
    annot=True,     # Write the data in each cell
    linewidths=.5,  # Width of line
    fmt="d",        # Format of the data, decimal
    square=True,    # Make cell as square-shaped
    ax=ax           # Draw it on ax
)
plt.show() # Show the figure
```

6.7 Batch Size

6.7.1 Batch, Stochastic Gradient Descent, and Mini-batch

Batch size is a hyper-parameter defining the number of samples processed before the model's internal parameters are updated. There are three extremes:

1. **Batch Gradient Descent (Full Batch).**
   - **Batch size:** N (the entire dataset).
   - **How it works:** The model looks at every single training example, calculates the average error, and then updates the weights once per "epoch" (full pass).
2. **Stochastic Gradient Descent (SGD).**
   - **Batch size:** 1.
   - **How it works:** The model looks at one example, calculates the error, and updates the weights immediately. It repeats this for every sample.
3. **Mini-batch Gradient Descent.**
   - **Batch size:** $16, 32, 64, \ldots$ (e.g., a small, fixed number).
   - **How it works:** The model looks at a small batch of examples (e.g., 32 pictures of cats/dogs), calculates the average error for the batch, and updates the weights.

Formally, let $\ell_j(\theta) = L(\mathbf{y}^{(j)}, \hat{\mathbf{y}}^{(j)})$ be the per-sample loss and let the full-dataset loss be

$$J(\theta) = \frac{1}{N} \sum_{j=1}^{N} \ell_j(\theta).$$

The three variants differ only in the gradient estimate used at each update:

$$\begin{aligned}
\text{Full batch:} \quad & \theta \leftarrow \theta - \eta \nabla_\theta J(\theta) = \theta - \frac{\eta}{N} \sum_{j=1}^{N} \nabla_\theta \ell_j(\theta), \\
\text{SGD:} \quad & \theta \leftarrow \theta - \eta \nabla_\theta \ell_j(\theta) \quad \text{(for one randomly drawn } j\text{)}, \\
\text{Mini-batch } \mathcal{B}: \quad & \theta \leftarrow \theta - \frac{\eta}{|\mathcal{B}|} \sum_{j \in \mathcal{B}} \nabla_\theta \ell_j(\theta).
\end{aligned}$$

Here $\mathcal{B} \subseteq \{1, \ldots, N\}$ is the mini-batch of indices and $|\mathcal{B}| = B$ is the batch size. An **epoch** is one complete pass over the training set; with batch size B it therefore comprises $\lceil N/B \rceil$ parameter updates.

Why Powers of Two?

Batch sizes are set as powers of two to ensure data aligns perfectly with the way computer memory and GPU cores are designed to work, allowing for the fastest and most efficient parallel processing.

6.7.2 Why Batch Size Matters

The choice of batch size forces you to compromise between how fast you can train and how well your model learns.

Feature	Small Batch Size (e.g., 1–32)	Large Batch Size (e.g., 256+)
Speed	Slower per epoch. Lots of updates, but each update is computationally efficient (fits easily in GPU memory).	Faster per epoch. Fewer updates, but requires more memory and computation per update.
Memory	Low memory footprint. Fits easily on a single GPU.	High memory footprint. May cause "Out of Memory" (OOM) errors if the batch doesn't fit on the hardware.
Gradient Quality	Noisy / high variance. The gradient is an inaccurate estimate of the true "best" direction. This noise can help escape "shallow" local minima.	Stable / low variance. The gradient is a very accurate estimate of the true gradient. This leads to smoother convergence.
Generalization	Often better generalization. The noise helps the model find wider, flatter minima that perform better on unseen data.	Often worse generalization. Tends to converge to sharp, narrow minima. Requires careful tuning of the learning rate.

Principle

Not too big, not too small.

6.7.3 How to Set Your Batch Size

1. The Hardware Constraint (The Upper Bound)

- **Start big, then reduce:** If you are using a GPU, set a high batch size (e.g., 512 or 1024) and run your code. If you get an "Out of Memory" error, cut the batch size in half. Repeat until the model fits.
- **Rule of thumb:** Your batch size should be the largest power of two (32, 64, 128, 256) that comfortably fits in your GPU memory. This allows the GPU's parallel architecture to run optimally.

2. A Good Place to Start (The Lower Bound)

- If you have no memory issues, start with common values used in research and industry: **32, 64, or 128**.
- **32** is often a great starting point. It's small enough to provide regularization benefits but large enough for efficient computation.

3. Tuning for Performance (The Experiment)

- Once you have a working model, experiment by multiplying or dividing your batch by 2. Try 16, 32, 64, 128.
- **Monitor validation accuracy:** Does a batch size of 64 converge faster than 32? Does 128 lead to overfitting or lower accuracy?
- **The learning rate connection:** If you increase the batch size, you can often increase the learning rate. (Rule of thumb: if you double the batch size, you can try doubling the learning rate, as the gradient estimate is less noisy.)

Recommendation for Beginners

Set batch size to **32**. Ensure the code runs without memory errors. Train for one epoch. If the loss decreases steadily, you're good. If you want to optimize for speed, try increasing to 64 or 128 and slightly increase the learning rate.

6.8 MLP Analysis

6.8.1 Vanishing/Exploding Gradient

One of the problems of training a neural network (especially with many hidden layers) is the **vanishing** and **exploding gradient**.

- When we train a neural network, the gradient (or the slope) can get very big or very small (or exponentially small), which makes training difficult.
- As a consequence, the weights are not updated anymore, and **learning stalls**.

Formally, for a deep network the gradient of the loss with respect to an early-layer parameter factorises through the chain rule as a product of per-layer Jacobians,

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \mathbf{a}^{(L)}} \left(\prod_{l=2}^{L} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{a}^{(l-1)}} \right) \frac{\partial \mathbf{a}^{(1)}}{\partial W^{(1)}}, \qquad \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{a}^{(l-1)}} = \operatorname{diag}\big(f'(\mathbf{z}^{(l)})\big)\, W^{(l)}.$$

If the singular values of these Jacobians are all < 1, the product shrinks towards zero (**vanishing gradient**); if they are all > 1, it grows without bound (**exploding gradient**). With the sigmoid, $f'(z) = \sigma(z)\big(1 - \sigma(z)\big) \leq \frac{1}{4}$ at every layer, so the factors are systematically < 1 and the gradient vanishes as depth grows.

6.8.2 Detecting the Problem

Symptoms of Vanishing Gradient

- The parameters of the higher layers vary dramatically, whereas the parameters of the lower levels do not change significantly (or not at all).
- During training, the model weights may become **zero**.
- The model **learns slowly**, and after a few cycles, the training may become **stagnant**.

Symptoms of Exploding Gradient

- The model parameters are **growing exponentially**.
- During training, the model weights may become **NaN**.
- The model goes through an **avalanche learning process**.

6.8.3 Overfitting and Underfitting

Overfitting

It refers to a model that models the training data too well. It happens when a model learns the **detail and noise** in the training data to the extent that it **negatively impacts** the performance of the model on the new data.

Underfitting

It refers to a model that can neither model the training data nor generalize to new data.

6.8.4 Choosing Depth and Width

The input layer:

- Number of layers = 1.
- Number of neurons = Number of **features** (i.e., columns) in our data (e.g., for XOR, the number of neurons in the input layer is 2).

The output layer:

- Number of layers = 1.
- Number of neurons = mostly 1, unless softmax is used (just like the handwritten digits example, which uses 10).

The hidden layers — number of layers:

- If our data is **linearly separable**, no hidden layer at all.
- If data is less complex and has few dimensions or features, neural networks with **1 to 2** hidden layers would work.
- If data has large dimensions or features, **3 to 5** hidden layers can be used to get an optimum solution.

The hidden layers — number of neurons:

- The number of hidden neurons should be between the size of the input layer and the output layer.
- The most appropriate number of hidden neurons is

$$\sqrt{\text{input layer nodes} \times \text{output layer nodes}}.$$

- The number of hidden neurons should keep decreasing in subsequent layers to get closer to pattern and feature extraction and identify the target class.

Rules of Thumb Only

The above algorithms are only a general use case, and they can be moulded according to the use case. Sometimes the number of nodes in hidden layers can also increase in subsequent layers, and the number of hidden layers can also be more than the ideal case. This depends on the use case and problem statement that we are dealing with.

6.8.5 Example: Hidden Neurons

Consider a classification problem where each sample has two inputs and one output presenting the class label.

- One possible decision boundary separates the data correctly; another possible decision boundary also separates the data correctly.
- **Two lines are required** to represent the decision boundary, which tells us the **first hidden layer will have two hidden neurons**.
- The two lines are to be **connected by another neuron**, which is in the **output layer**.

Geometry Intuition

Each hidden neuron contributes one linear decision boundary (a line in 2D); the output neuron combines these boundaries into a non-linear region. Count the number of straight lines needed to separate the classes ⇒ that is the number of hidden neurons.

6.8.6 The Role of Weights

Weights are the real values associated with each feature which tell the **importance of that feature** in predicting the final value.

The effect of changing weights. Suppose we have a single-input perceptron. Let's get the output functions by setting w to $0.5, 1, 2, 3$, θ to 0, and using the sigmoid activation function:

$$y = f(0.5x + 0), \qquad y = f(1x + 0), \qquad y = f(2x + 0), \qquad y = f(3x + 0).$$

Plotting these output functions, we can see that **weights control the steepness of the activation function**.

6.8.7 Role of Biases

Now let's get another set of output functions by setting w to 1, θ to $0, 1, 2, 3$, and using the sigmoid activation function:

$$y = f(1x + 0), \qquad y = f(1x + 1), \qquad y = f(1x + 2), \qquad y = f(1x + 3).$$

According to the example, we can see that **bias is used for shifting the activation function towards the left or right**.

Weights vs. Biases

Weights control the steepness (slope) of the activation function; **biases** shift the activation function left or right.

6.8.8 Activation Functions

The choice of activation function in the hidden layer has a **large impact** on the capability and performance of the neural network.

An **activation function** in a neural network defines how the weighted sum of the input is transformed into an output from a node or nodes in a layer of the network.

- **All hidden layers** typically use the same activation function.
- The **output layer** will typically use a different activation function from the hidden layers, dependent upon the type of prediction required by the model.
- Typically, a **differentiable non-linear** activation function is used in the hidden layers of a neural network. This allows the model to learn more complex functions.

Three most commonly used activation functions in **hidden layers**:

1. Rectified Linear Activation (ReLU)
2. Logistic (Sigmoid)
3. Hyperbolic Tangent (Tanh)

ReLU Hidden Layer Activation Function ReLU function is defined as

$$f(x) = \max(0, x).$$

Its derivative, used in backpropagation, is

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0, \\ 0, & x < 0, \end{cases}$$

which is cheap to compute and is 1 (not vanishing) wherever the unit is active.

- It is the **most common** function used for hidden layers.

- It is **simple to implement** and effectively overcomes the limitations of other previously popular activation functions, such as Sigmoid and Tanh.
- Specifically, it is **less susceptible to vanishing gradients**.
- When using the ReLU function for hidden layers, it is good to use "**He Normal**" or "**He Uniform**" weight initialization and scale input data to the range 0–1 before training.

```
initializer = tf.keras.initializers.HeNormal()
# initializer = tf.keras.initializers.HeUniform()
layer = tf.keras.layers.Dense(3, kernel_initializer=initializer)
```

Sigmoid Hidden Layer Activation Function Sigmoid function is defined as

$$f(x) = \frac{1}{1 + e^{-x}}.$$

Limitation: The output of the sigmoid function is not symmetric around zero. So the output of all the neurons will be of the same sign. This makes the training of the neural network more difficult and unstable.

When using the sigmoid function for hidden layers, it is good to use a "**Glorot Normal**" or "**Glorot Uniform**" weight initialization and scale input data to the range 0–1 before training. (Note: Glorot initializer is also called **Xavier initializer**.)

```
initializer = tf.keras.initializers.GlorotNormal()
# initializer = tf.keras.initializers.GlorotUniform()
layer = tf.keras.layers.Dense(3, kernel_initializer=initializer)
```

Tanh Hidden Layer Activation Function Tanh (Hyperbolic Tangent) function is defined as

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}.$$

Its derivative is conveniently expressed in terms of its own output,

$$f'(x) = 1 - f(x)^2 = 1 - \tanh^2(x),$$

which is largest ($= 1$) at $x = 0$ and decays to 0 in the flat tails.

- Very similar to the sigmoid activation function, and even has the same S-shape, with the difference in **output range of** -1 **to** 1.
- The output of the tanh activation function is **zero-centered**; hence we can easily map the output values as strongly negative, neutral, or strongly positive.
- When used in hidden layers, the mean for the hidden layer comes out to be 0 (or very close to it). It helps in **centering the data** and makes learning for the next layer much easier.

Why are Sigmoid and Tanh More Susceptible to Vanishing Gradients?

Flat Tails of Sigmoid/Tanh

The gradient values are only significant for the range -3 to 3, and the graph gets much flatter in other regions. For values greater than 3 or less than -3, the function will have **very small gradients**. As the gradient value approaches zero, the network **ceases to learn** and suffers from the **vanishing gradients** problem.

6.8.9 Hidden-Layer Activation

Both the Sigmoid and Tanh functions can make the model more susceptible to problems during training via the so-called **vanishing gradients** problem.

The activation function used in hidden layers is typically chosen **based on the type of neural network architecture**:

- Modern neural network models with common architectures, such as **MLP** and **CNN** (covered later), will use the **ReLU** activation function or extensions.
- **Recurrent networks** (a type of neural network that has at least one loop) still commonly use **Tanh** or **sigmoid** activation functions, or even both.

Summary: Hidden-Layer Activation by Architecture

Neural Network	Commonly Used Activation Function
Multi-layer Perceptron (MLP)	ReLU activation function
Convolutional Neural Network (CNN)	ReLU activation function
Recurrent Neural Network (RNN)	Tanh and/or Sigmoid activation function

6.8.10 Output-Layer Activation

The **output layer** is the layer in a neural network model that directly outputs a prediction. There are three commonly used activation functions for the output layer:

1. Linear
2. Sigmoid/Logistic
3. Softmax

Linear Output Activation Function

$$f(x) = x.$$

- The linear activation function is also called "**identity**" (multiplied by 1.0) or "**no activation**".
- Target values used to train a model with a linear activation function in the output layer are typically **scaled before modeling** using normalization or standardization transforms.

Sigmoid Output Activation Function Recall, the Sigmoid activation function is defined as

$$f(x) = \frac{1}{1 + e^{-x}}.$$

Target labels used to train a model with a Sigmoid activation function in the output layer will have values **0 to 1**.

Softmax Output Activation Function Softmax is a mathematical function that converts an array (or vector) of numbers into an array (or vector) of **probabilities**. The softmax function is defined as follows: for $\mathbf{x} = [x_0, x_1, \ldots, x_{n-1}]$,

$$f(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{n-1} e^{x_j}}.$$

The softmax function is used as the activation function for **multi-class classification** problems where class membership is required on more than two class labels.

6.8.11 Choosing the Output Activation

We choose the activation function for the output layer **based on the prediction problem** we are solving:

- If a problem is a **regression problem**, we should use a **linear** activation function.

- If a problem is a **classification problem**, then there are three main types of classification problems, and each may use a different activation function:
  1. **Binary classification:** One node, **sigmoid** activation.
  2. **Multiclass classification:** One node per class, **softmax** activation.
  3. **Multilabel classification:** One node per class, **sigmoid** activation.

  In each case the output activation and its training loss are:

$$\begin{aligned}
\text{Regression:} \quad & \hat{y} = \mathbf{w}^\top \mathbf{x} + b, \\
& L = \|y - \hat{y}\|^2; \\
\text{Binary:} \quad & \hat{y} = \text{sigmoid}(z) = \frac{1}{1+e^{-z}} \in (0,1), \\
& L = -\big[y \log \hat{y} + (1-y)\log(1-\hat{y})\big]; \\
\text{Multiclass:} \quad & \hat{y}_i = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad \sum_i \hat{y}_i = 1, \\
& L = -\sum_c y_c \log \hat{y}_c; \\
\text{Multilabel:} \quad & \hat{y}_i = \text{sigmoid}(z_i) \text{ (one per class)}, \\
& L = -\sum_i \big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\big].
\end{aligned}$$

Multiclass vs. Multilabel

Multiclass classification makes the assumption that each sample is assigned to one and only one label. **Multilabel** classification assigns to each sample a set of target labels.
Take news article classification as an example:
- Multiclass label set: {computer, science, society}
- Multilabel label set: {computer, science, society, operating system, cryptography, electronics, religion}

6.8.12 When to Use MLP

- Multi-layer perceptrons are suitable for **classification** prediction problems where inputs are assigned a class or label.
- They are also suitable for **regression** prediction problems where a real-valued quantity is predicted given a set of inputs. Data is often provided in a **tabular format**, such as we would see in a CSV file or a spreadsheet.

6.9 Practice

One Epoch of Backpropagation by Hand

Given a multilayer perceptron with two inputs x_1, x_2, **one hidden unit** and **one output unit**. Both the hidden unit and output use the sigmoid activation function. Altogether, the network has 3 weights, w_1, w_2, w_3, and 2 biases, θ_1, θ_2.
All weights are initialized with 0.1, and all the biases are initialized with -0.1. Use sigmoid as the activation function for all units, i.e.,

$$\sigma(x) = \frac{1}{1+e^{-x}}.$$

Let the training set be as follows:

x_1	x_2	T
1	0	1
0	1	0

Determine the weights after the **first epoch** (two iterations) of the backpropagation algorithm, given a learning rate of $\eta = 0.3$.

6.9.1 Solution: Round 1 Forward

Inputs: $x_1 = 1$, $x_2 = 0$; Actual Output: $T = 1$.
Weights: $w_1 = 0.1$, $w_2 = 0.1$, $w_3 = 0.1$. Biases: $\theta_1 = -0.1$, $\theta_2 = -0.1$.

Calculations:

$$\sum_1 = x_1 \cdot w_1 + x_2 \cdot w_2 = 1 \cdot (0.1) + 0 \cdot (0.1) = 0.1$$
$$\text{Output } (O_j): \ f(\textstyle\sum_1 + \theta_1) = f(0.1 - 0.1) = 0.5$$
$$\sum_2 = O_j \cdot w_3 = 0.5 \cdot (0.1) = 0.05$$
$$\text{Output } (O_k): \ f(\textstyle\sum_2 + \theta_2) = f(0.05 - 0.1) = 0.487503$$

6.9.2 Solution: Round 1 Backward

$$\delta_k = (O_k - T_k)\, O_k(1 - O_k) = (0.487503 - 1)(0.487503)(1 - 0.487503) = -0.128044$$

Update hidden-to-output weight and output bias:

$$\text{New } w_3 = \text{Old } w_3 - \eta\, \delta_k O_j = 0.1 - 0.3(-0.128044)(0.5) = 0.119207$$
$$\text{New } \theta_2 = \text{Old } \theta_2 - \eta\, \delta_k = -0.1 - (0.3)(-0.128044) = -0.061587$$

Hidden node delta:

$$\delta_j = O_j(1 - O_j)\, \delta_k w_{jk} = 0.5(1 - 0.5)(-0.128044)(0.1) = -0.003201$$

Update input-to-hidden weights and hidden bias:

$$\text{New } w_1 = \text{Old } w_1 - \eta\, \delta_j x_1 = 0.1 - (0.3)(-0.003201)(1) = 0.100960$$
$$\text{New } w_2 = \text{Old } w_2 - \eta\, \delta_j x_2 = 0.1 - (0.3)(-0.003201)(0) = 0.1$$
$$\text{New } \theta_1 = \text{Old } \theta_1 - \eta\, \delta_j = -0.1 - (0.3)(-0.003201) = -0.099040$$

6.9.3 Solution — Round 1, Step 2: Forward Propagation

Inputs: $x_1 = 0$, $x_2 = 1$; Actual Output: $T = 0$.
Weights: $w_1 = 0.100960$, $w_2 = 0.1$, $w_3 = 0.119207$. Biases: $\theta_1 = -0.099040$, $\theta_2 = -0.061587$.

Calculations:

$$\sum_1 = x_1 \cdot w_1 + x_2 \cdot w_2 = 0 \cdot (0.100960) + 1 \cdot (0.1) = 0.1$$
$$\text{Output } (O_j): \ f(\textstyle\sum_1 + \theta_1) = f(0.1 - 0.099040) = 0.50024$$
$$\sum_2 = O_j \cdot w_3 = 0.50024 \cdot (0.119207) = 0.059632$$
$$\text{Output } (O_k): \ f(\textstyle\sum_2 + \theta_2) = f(0.059632 - 0.061587) = 0.499511$$

6.9.4 Solution — Round 1, Step 2: Backward Propagation

$$\delta_k = (O_k - T_k)\, O_k(1 - O_k) = (0.499511 - 0)(0.499511)(1 - 0.499511) = 0.124878$$

Update hidden-to-output weight and output bias:

$$\text{New } w_3 = \text{Old } w_3 - \eta\,\delta_k O_j = 0.119207 - 0.3(0.124878)(0.50024) = 0.100466$$

$$\text{New } \theta_2 = \text{Old } \theta_2 - \eta\,\delta_k = -0.061587 - (0.3)(0.124878) = -0.09905$$

Hidden node delta:

$$\delta_j = O_j(1 - O_j)\,\delta_k w_{jk} = 0.50024\,(1 - 0.50024)(0.124878)(0.119207) = 0.003722$$

Update input-to-hidden weights and hidden bias:

$$\text{New } w_1 = \text{Old } w_1 - \eta\,\delta_j x_1 = 0.100960 - (0.3)(0.003722)(0) = 0.100960$$

$$\text{New } w_2 = \text{Old } w_2 - \eta\,\delta_j x_2 = 0.1 - (0.3)(0.003722)(1) = 0.098883$$

$$\text{New } \theta_1 = \text{Old } \theta_1 - \eta\,\delta_j = -0.099040 - (0.3)(0.003722) = -0.100157$$

Answer After the First Epoch

$$w_1 = 0.100960, \quad w_2 = 0.098883, \quad w_3 = 0.100466,$$
$$\theta_1 = -0.100157, \quad \theta_2 = -0.09905.$$

Note that in Step 2, w_1 does not change because $x_1 = 0$ (the update $\eta\,\delta_j x_1$ vanishes) — the same "zero-input freezes weight" effect seen in the XOR example.

7 Image Processing

7.1 Introduction

7.1.1 Convolutional Neural Network

A **Convolutional Neural Network (CNN or ConvNet)** is a type of **Artificial Neural Network** applied to analyze visual imagery.

Research has demonstrated that **preprocessing images before feeding them to a CNN significantly improves the classification/recognition accuracy**. To learn how to preprocess images and use this powerful tool to tackle **Computer Vision** tasks, we first introduce **digital image processing fundamentals**.

7.2 Image Preprocessing

7.2.1 Image Preprocessing

Image Preprocessing

Image preprocessing refers to processing an image so that the resulting image is more suitable than the original for a specific application. A preprocessing method that works well for one application may not be the best method for another application.

7.2.2 Common Preprocessing Tasks

- **Shading correction**: to adjust image inconsistency, generally seen on the edges, due to illumination and background inhomogeneity.
- **De-blurring**: to remove the blurring artifacts from images to restore the original, sharp content.
- **De-noising**: to remove or reduce noise from an image and to recover the original image, which is considered to be noise-free, from a noisy observation.

- **Contrast enhancement**: to touch up the contrast of the image, or the difference in color and light between parts of it, to improve its perception by the human eye.

7.2.3 Digital Image Processing

Digital Image Processing (DIP)

Digital image processing (DIP) is the method to manipulate a digital image to either enhance the quality or extract relevant information. These methods provide the foundation to preprocess images that are more suitable for specific applications.

7.3 Digital Images

7.3.1 Digital Image

A **digital image** is a **two-dimensional grid of intensity values**. Formally, a grayscale image with height H and width W is a discrete function on the pixel lattice

$$I : \{0, 1, \ldots, H-1\} \times \{0, 1, \ldots, W-1\} \to \mathcal{V}, \qquad (x, y) \mapsto I(x, y),$$

where the column index $x \in \{0, \ldots, W-1\}$ and the row index $y \in \{0, \ldots, H-1\}$, and $\mathcal{V}$ is the set of admissible intensity values; the value $I(x, y) \in \mathcal{V}$ is called the **intensity** at (x, y). Equivalently, a grayscale image is a matrix $I \in \mathcal{V}^{H \times W}$, and a colour image carrying C channels per pixel is an array $I \in \mathcal{V}^{H \times W \times C}$.

- **Pixels**: Short for Picture Element. A pixel is a single point (dot) in an image.
- **Dimensions**: Specified by the width and height of the image.
  - **Image width** is the number of columns in the image.
  - **Image height** is the number of rows in the image.

For example, an image of dimensions 32×21 has image width $= 32$ pixels and image height $= 21$ pixels.

7.3.2 Image Coordinate System

A specific pixel is specified by its coordinates (x, y), where x increases from left to right, and y increases from top to bottom. **The origin $(0, 0)$ is in the top-left corner.**

The coordinate system of digital images is:

$$\begin{array}{ccccc} (0,0) & (1,0) & (2,0) & \cdots & (\text{width}-1, 0) \\ (0,1) & (1,1) & (2,1) & \cdots & (\text{width}-1, 1) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (0, \text{height}-1) & (1, \text{height}-1) & (2, \text{height}-1) & \cdots & (\text{width}-1, \text{height}-1) \end{array}$$

where width and height are the image width and image height, respectively.

Legal Coordinate Ranges and the (x, y) Convention

- The legal range of the x-coordinate is 0 to width $-$ 1.
- The legal range of the y-coordinate is 0 to height $-$ 1.

Note that this is not the usual mathematical convention: the y-axis points **downwards**, and x indexes columns while y indexes rows. When using NumPy arrays, `img.shape` returns `(height, width)` (rows first!), which is the reverse order of (x, y).

7.3.3 Grayscale Images

Grayscale is a range of gray shades from black to white. Grayscale images are most commonly used in image processing because the data are smaller and allow us to process the images in a short time.

A **grayscale digital image** is an image in which the value of each pixel carries only intensity information. A grayscale image contains **8-bit/pixel** data, which has $2^8 = 256$ different gray levels (0 for black, 127 for gray, and 255 for white). Mathematically the intensity set is

$$\mathcal{V} = \{0, 1, \ldots, 2^8 - 1\} = \{0, 1, \ldots, 255\}, \qquad I \in \{0, \ldots, 255\}^{H \times W},$$

and it is often convenient to normalise to the unit interval, $I \in [0, 1]^{H \times W}$, via the linear rescaling $I \mapsto I/255$.

Color	**Gray-level**
Black	0
Dark gray	64
Medium gray	127
Light gray	190
White	255

7.3.4 Color Images

Color images have intensity from the darkest and lightest of 3 different colors, **Red, Green, and Blue (RGB)**. The mixtures of these color intensities produce a color image. Since RGB images contain 3×8-bit intensities, they are also referred to as **24-bit color images**. An 8-bit intensity range has 256 possible values, 0 to 255. Formally, an RGB image is the three-channel array

$$I : \{0, \ldots, H-1\} \times \{0, \ldots, W-1\} \to \{0, \ldots, 255\}^3,$$
$$I(x, y) = \big(I_R(x, y),\ I_G(x, y),\ I_B(x, y)\big),$$

i.e., $I \in \{0, \ldots, 255\}^{H \times W \times 3}$.

Common colors represented in RGB:

Color	**RGB value**
Black	$(0, 0, 0)$
White	$(255, 255, 255)$
Red	$(255, 0, 0)$
Green	$(0, 255, 0)$
Blue	$(0, 0, 255)$
Yellow	$(255, 255, 0)$
Gray	$(128, 128, 128)$
Dark Gray	$(50, 50, 50)$

Grayscale Images as a Subset of RGB Images

24-bit grayscale images are a subset of RGB images where the RGB intensities are all equal (e.g., Gray $(128, 128, 128)$ and Dark Gray $(50, 50, 50)$ shown above).

7.3.5 Access Images (Colab)

To access files/images in Google Drive, you need to **mount** your Google Drive to Colab using the following code:

```
# Import drive from google.colab package
from google.colab import drive
# Import os and sys modules
import os, sys
# Mount Google Drive
drive.mount('/content/drive')
# Assume a folder "images" has been created, go to the folder "images"
os.chdir('/content/drive/My Drive/images')
# Add the path for interpreter to search
sys.path.append('/content/drive/My Drive/images')
```

7.3.6 Read Images in Colab

To read an image, you need to first import `matplotlib.image`:

```
import matplotlib.image as mpimg
```

Then use the `imread()` method of the `matplotlib.image` module.

Syntax: `mpimg.imread(fname, format=None)`

Parameters:

- `fname`: The image file to read: a filename or a file-like object opened in read-binary mode.
- `format`: The image file format assumed for reading the data. If `format` is not given, the format is deduced from the filename. If nothing can be deduced, PNG is tried.

Return value: `numpy.array`.

- (M, N) for grayscale images.
- $(M, N, 3)$ for RGB images.
- $(M, N, 4)$ for RGBA images: red, green, blue, alpha. A three-channel RGB color model supplemented with a fourth **alpha channel**. Alpha indicates how opaque each pixel is, i.e., transparency.

Value Range Depends on File Format

PNG images are returned as **float arrays (**0–1**)**. All other formats are returned as **int arrays**, with a bit depth determined by the file's contents. Keep this in mind when displaying or processing images (e.g., use `vmin=0, vmax=1` for floats, `vmin=0, vmax=255` for 8-bit integers).

7.3.7 Show Images in Colab

To show an image, you need to first import `matplotlib.pyplot`:

```
import matplotlib.pyplot as plt
```

Then use the `imshow()` method of the `matplotlib.pyplot` module.

Syntax: `plt.imshow(X, cmap, vmin, vmax)`

Parameters:

- `X`: The image data. Supported array shapes are
  - (M, N): an image with scalar data. The values are mapped to colors using normalization and a colormap.
  - $(M, N, 3)$: an image with RGB values (0–1 float or 0–255 int).
  - $(M, N, 4)$: an image with RGBA values (0–1 float or 0–255 int), i.e., including transparency.

  The first two dimensions (M, N) define the rows and columns of the image.
- `cmap`: `str` (e.g., `'gray'`) or `Colormap`. The Colormap instance or registered colormap name used to map scalar data to colors. This parameter is ignored for RGB(A) data.

- `vmin`, `vmax`: By default, `imshow` scales elements of the numpy array so that the smallest element becomes 0, the largest becomes 1, and intermediate values are mapped to the interval $[0, 1]$ by a linear function. Optionally, `imshow` can be called with arguments `vmin` and `vmax`. In such case all elements of the array smaller or equal to `vmin` are mapped to 0, all elements greater or equal to `vmax` are sent to 1, and the elements between `vmin` and `vmax` are mapped in a linear fashion into the interval $[0, 1]$.

  Returns: `AxesImage` (an image attached to an `Axes`).

7.3.8 Save Images in Colab

To save an image, first import `matplotlib.pyplot`, then use the `imsave()` method.

Syntax: `plt.imsave(fname, arr)`

Parameters:

- `fname`: a path or a file-like object to store the image in.
- `arr`: The image data. The shape can be one of $M \times N$ (luminance), $M \times N \times 3$ (RGB) or $M \times N \times 4$ (RGBA). The first two dimensions (M, N) define the rows and columns of the image.

7.3.9 Complete Example

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import numpy as np

# Read and show the image
img = mpimg.imread('snorlax.png')
plt.imshow(img)

# Find the shape of input image
[height, width, layers] = np.shape(img)

# Print all the required information
print('Dimensions: {}x{}x{}'.format(height, width, layers))
print('Total number of pixels:', width*height)

plt.imsave('snorlax2.png', img)  # Save the image
```

7.4 OpenCV Basics

7.4.1 OpenCV

OpenCV

OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. It was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in commercial products.

- The library has **more than 2500 optimized algorithms**, including a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms.
- OpenCV supports a wide variety of programming languages such as **Python, C++, Java**, etc.

7.4.2 Convert Color Images to Grayscale

In certain problems, you will find it useful to lose unnecessary information from your images to reduce space and computational complexity. Converting colored images to grayscale images is an example, since color is not necessary to recognize and interpret an image. Grayscale can be good enough for recognizing certain objects; color images contain more information than black and white images, so they can add unnecessary complexity and take up more space in memory.

RGB to Grayscale Formula

Grayscale conversion is a **point operation**: the same map is applied to every pixel independently. One way to convert color images to grayscale is the human-vision-weighted (luma) linear combination

$$I_{\text{gray}}(x, y) = w_R\, I_R(x, y) + w_G\, I_G(x, y) + w_B\, I_B(x, y),$$

with weights $w_R = 0.299$, $w_G = 0.587$, $w_B = 0.114$. Since the weights are non-negative and sum to

$$w_R + w_G + w_B = 0.299 + 0.587 + 0.114 = 1,$$

the result stays within the valid intensity range. Note that **green contributes the most** (weight 0.587) and blue the least (weight 0.114), matching the sensitivity of the human eye. The value $I_{\text{gray}}(x, y)$ is rounded to the nearest integer when stored as an 8-bit image.

To perform the above using OpenCV, first import `cv2`, then use the `cvtColor()` method.

Syntax: `cv2.cvtColor(image, code)`

Parameters:

- `image`: Image to be processed in n-dimensional array.
- `code`: Conversion code for color space. For converting RGB to grayscale, we use `cv2.COLOR_RGB2GRAY`.

Return value: Converted image.

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import cv2
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

# Read and show the image
img = mpimg.imread('snorlax.png')
plt.figure()
plt.imshow(img)

# Convert color image to gray
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

# Show the image
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

# Save the image
plt.imsave('snorlax-gray.png', grayImg)
```

7.4.3 Image Affine Transformation

Affine Transformation

An **affine transformation** is any transformation that preserves collinearity, parallelism, as well as the ratio of distances between the points (e.g., the midpoint of a line remains the midpoint after transformation). It does not necessarily preserve distances and angles. Geometric transformations such as **translation, rotation, scaling, shearing**, etc., are all affine transformations.

In general, an affine transformation maps a point $\mathbf{x} = (x, y)^\top$ to

$$\mathbf{x}' = A\mathbf{x} + \mathbf{t}, \qquad \mathbf{x}' = \begin{bmatrix} x' \\ y' \end{bmatrix}, \ \mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}, \ A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}, \ \mathbf{t} = \begin{bmatrix} b_{00} \\ b_{10} \end{bmatrix},$$

that is, a **linear transformation followed by a vector addition**:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_{00} \\ b_{10} \end{bmatrix} = \begin{bmatrix} a_{00}x + a_{01}y + b_{00} \\ a_{10}x + a_{11}y + b_{10} \end{bmatrix}$$

Using **homogeneous coordinates** (appending a 1), this can be written as a single matrix multiplication:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & b_{00} \\ a_{10} & a_{11} & b_{10} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$M = \begin{bmatrix} a_{00} & a_{01} & b_{00} \\ a_{10} & a_{11} & b_{10} \end{bmatrix} \text{ is a } 2 \times 3 \textbf{ transformation matrix}.$$

To define what transformation you want to do, you need to define M. Its first two columns form the linear part A (rotation, scaling, shearing) and its third column is the translation vector $\mathbf{t}$.

7.4.4 Applying Affine Transformations

To perform the image affine transformation using OpenCV, use the `warpAffine()` method of the `cv2` module.

Syntax: `cv2.warpAffine(src, M, dsize, flags, borderMode, borderValue)`

Parameters:

- `src`: input image.
- `M`: 2×3 transformation matrix.
- `dsize`: size of the output image.
- `flags`: combination of interpolation methods.
- `borderMode`: pixel extrapolation method.
- `borderValue`: value used in case of a constant border; by default, it is 0.

Return value: output image that has the size `dsize` and the same type as `src`.

7.4.5 Image Translation

Translation is simply the shifting of object location. Suppose we have a point $P(x, y)$ which is translated by (t_x, t_y); then the coordinates after translation, denoted by $P'(x', y')$, are given by

$$x' = x + t_x, \qquad y' = y + t_y$$

Equivalently, in vector form with $\mathbf{t} = (t_x, t_y)^\top$,

$$\mathbf{x}' = \mathbf{x} + \mathbf{t}.$$

In matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import cv2
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import numpy as np

# Read the image
img = mpimg.imread('snorlax.png')

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

# Find the shape of the gray image
rows, cols = grayImg.shape

# Form the transformation matrix of translation
M = np.float32([[1, 0, 100], [0, 1, 50]])

# Perform the transformation
translatedImg = cv2.warpAffine(grayImg, M, (cols, rows))

# Show the image
plt.figure()
plt.imshow(translatedImg, cmap='gray', vmin=0, vmax=1)
```

The code above shifts the image **100 pixels to the right** ($t_x = 100$) and **50 pixels down** ($t_y = 50$).

7.4.6 Image Reflection

Reflection refers to mirroring (or flipping) images along the x-axis or y-axis. To make sure the resulting image fits the image coordinates, after flipping the image along the x-axis, we need to translate it by the number of rows along the y-axis. Similarly, after flipping the image along the y-axis, we need to translate it by the number of columns along the x-axis.

Reflection along the x-axis. Suppose we have a point $P(x, y)$ which is reflected along the x-axis; then the coordinates after reflection, denoted by $P'(x', y')$, are given by

$$x' = x, \qquad y' = -y + (\text{rows} - 1)$$

In matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & \text{rows} - 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Reflection along the y-axis. Similarly, a point $P(x, y)$ reflected along the y-axis gives

$$x' = -x + (\text{cols} - 1), \qquad y' = y$$

In matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 & \text{cols} - 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Why the Extra Translation Term?

Flipping alone maps pixel (x, y) to negative coordinates (e.g., $y' = -y$), which fall outside the legal range $[0, \text{height} - 1]$. The translation by $\text{rows} - 1$ (or $\text{cols} - 1$) brings the flipped image back into the valid coordinate frame. Forgetting it produces an (almost entirely) blank output image.

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import cv2
import numpy as np
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

img = mpimg.imread('snorlax.png')  # Read the image

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

rows, cols = grayImg.shape  # Find the shape of the gray image

# Form the transformation matrix of x-axis reflection
M = np.float32([[1, 0, 0], [0, -1, rows-1]])
# Perform the transformation
xaxisreflection = cv2.warpAffine(grayImg, M, (cols, rows))
plt.figure()
plt.imshow(xaxisreflection, cmap='gray', vmin=0, vmax=1)

# Form the transformation matrix of y-axis reflection
M = np.float32([[-1, 0, cols-1], [0, 1, 0]])
# Perform the transformation
yaxisreflection = cv2.warpAffine(grayImg, M, (cols, rows))
plt.figure()
plt.imshow(yaxisreflection, cmap='gray', vmin=0, vmax=1)
```

7.4.7 Image Rotation

Image rotation refers to rotating an image θ degrees along (x_0, y_0). Suppose we have a point $P(x, y)$ which is rotated along the center of the image; then the coordinates after rotation, denoted $P'(x', y')$, are given by

$$x' = (x - x_0)\cos\theta + (y - y_0)\sin\theta + x_0$$
$$y' = -(x - x_0)\sin\theta + (y - y_0)\cos\theta + y_0$$

The linear part is the rotation matrix

$$R(\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix},$$

and, writing the centre as $\mathbf{c} = (x_0, y_0)^\top$ and $\mathbf{x} = (x, y)^\top$, the full map is the affine transformation

$$\mathbf{x}' = R(\theta)\,(\mathbf{x} - \mathbf{c}) + \mathbf{c}.$$

In matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & -x_0\cos\theta - y_0\sin\theta + x_0 \\ -\sin\theta & \cos\theta & x_0\sin\theta - y_0\cos\theta + y_0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
import cv2, math
import numpy as np  # Import all the required libraries
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

img = mpimg.imread('snorlax.png')  # Read the image

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

rows, cols = grayImg.shape  # Find the shape of image

# Form the transformation matrix of rotation
# The angle for math.sin and math.cos should be in radian.
# 45 degree = pi/4 radian
angle = math.pi/4
M = np.float32([[math.cos(angle), math.sin(angle),
                 -(cols//2)*math.cos(angle)
                 -(rows//2)*math.sin(angle) + (cols//2)],
                [-math.sin(angle), math.cos(angle),
                 (cols//2)*math.sin(angle)
                 -(rows//2)*math.cos(angle) + (rows//2)]])

# Another way to generate the required transformation matrix
# The angle for getRotationMatrix2D should be in degree
# M = cv2.getRotationMatrix2D((cols//2, rows//2), 45, 1.0)

# Perform the transformation
rotate45 = cv2.warpAffine(grayImg, M, (cols, rows))
plt.figure()
plt.imshow(rotate45, cmap='gray', vmin=0, vmax=1)
```

Two Ways to Get the Rotation Matrix

- Build M manually with `math.sin`/`math.cos` — the angle must be in **radians**.
- Use `cv2.getRotationMatrix2D((cols//2, rows//2), 45, 1.0)` — the angle is in **degrees**. The arguments are (center, angle, scale).

7.4.8 Image Resizing/Scaling

Resizing/Scaling images is a critical preprocessing step in computer vision.

- Machine learning models train faster on smaller images.
- Many deep learning model architectures require that our images are the same size, and our raw collected images may vary in size.

Resizing is a common approach to make the input images the same size, and it works well unless you have a very different aspect ratio from the expected input shape.

To perform image resizing/scaling using OpenCV, use the `resize()` method.

Syntax:

```
cv2.resize(src, dsize, dst, fx=0, fy=0, interpolation=INTER_LINEAR)
```

Parameters:

- `src`: Input image.
- `dsize`: The size for the output image.
- `dst` (optional): The output image with size `dsize`.
- `fx` (optional): The scale factor along the horizontal axis.
- `fy` (optional): The scale factor along the vertical axis.
- `interpolation`: The algorithm used to reconstruct the new pixels:
  - `cv2.INTER_NEAREST` (nearest neighbor interpolation)
  - `cv2.INTER_LINEAR` (bilinear interpolation)
  - `cv2.INTER_CUBIC` (bicubic interpolation)

Interpolation. To resample an image to a new grid, each output pixel is mapped back to a (generally non-integer) input coordinate (u, v). Nearest-neighbour interpolation takes the value of the closest integer lattice point,

$$I_{\text{new}}(x, y) = I\big(\text{round}(u),\ \text{round}(v)\big).$$

Bilinear interpolation instead takes the weighted average of the four surrounding pixels,

$$I_{\text{new}}(x, y) = (1-a)(1-b)\, I(i, j) + a(1-b)\, I(i+1, j) + (1-a)b\, I(i, j+1) + ab\, I(i+1, j+1),$$

where $i = \lfloor u \rfloor$, $a = u - i$, $j = \lfloor v \rfloor$, $b = v - j$ are the integer and fractional parts of the target coordinate. Scaling by factors (f_x, f_y) maps output (x, y) to input $(u, v) = (x/f_x,\ y/f_y)$. Nearest-neighbour is the fastest, while bilinear (and higher-order bicubic) interpolation gives smoother results.

```
# Assume Google Drive has been mounted, the path
# has been added for interpreter to search
# Import all the required libraries
import cv2
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

img = mpimg.imread('snorlax.png')  # Read the image

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

# Perform the transformation
resizedImg = cv2.resize(grayImg, (300, 300),
                        interpolation=cv2.INTER_LINEAR)

# Show the image
plt.figure()
plt.imshow(resizedImg, cmap='gray', vmin=0, vmax=1)
```

7.5 Operation Types

7.5.1 Point/Local/Global Operations

There are many ways to classify image operations. One way of doing so is based on the **"region"** used to process the pixels.

Three Types of Image Operations

1. **Point**: The output value at a specific coordinate is dependent only on the input value at the same coordinate.
2. **Local**: The output value at a specific coordinate is dependent on the input values in the neighborhood of that same coordinate.

3. **Global**: The output value at a specific coordinate is dependent on all the values in the input image.

In symbols, with O the output image and f the operation, the three classes are

- **Point**: $O(x,y) = f\big(I(x,y)\big)$ — only the value at the same coordinate is used.
- **Local**: $O(x,y) = f\big(\{I(u,v) : (u,v) \in \mathcal{N}(x,y)\}\big)$ — the value at (x,y) is computed from a neighbourhood $\mathcal{N}(x,y)$.
- **Global**: $O(x,y) = f(I)$ — the operation may depend on the whole image I.

Question: What Type of Operation is Color-to-Grayscale Conversion?

Question: What type of image operation is color to grayscale conversion?
Answer: Point operation! Since the output value at a specific coordinate of the grayscale image is dependent only on the input value at the same coordinate of the color image.

7.5.2 Point-based Operations (Examples)

- **Brightness adjustment**: Make images brighter or dimmer. (Optional)
- **Contrast stretching**: Adjust the contrast of images.
- **Gamma correction**: Grayscale non-linear transformation. (Optional)
- **Grayscale threshold**: Convert a grayscale image into a black and white binary image.
- **Histogram equalization**: Transformation where an output image has approximately the same number of pixels at each gray level. (Optional)

For reference, the **histogram** of an 8-bit image counts the occurrences of each intensity,

$$h(v) = \#\{(x,y) : I(x,y) = v\}, \qquad v = 0, 1, \ldots, L-1 \quad (L = 256),$$

and the normalised histogram $p(v) = h(v)/(HW)$ estimates the probability of intensity v. **Histogram equalisation** is the point transform that maps each input intensity r to

$$s = T(r) = (L-1)\sum_{v \le r} p(v) = (L-1)\,\mathrm{CDF}(r),$$

where $\mathrm{CDF}(r) = \sum_{v \le r} p(v)$ is the cumulative distribution function of the intensities; this spreads the intensities so that the output histogram is (approximately) uniform.

7.5.3 Contrast Stretching

Contrast Stretching

Contrast stretching is an image enhancement method which attempts to improve an image by stretching the range of intensity values. One way to perform contrast stretching is to stretch the minimum and maximum intensity values present to the possible minimum and maximum intensity values.

Example: Assume 0–255 is taken as the standard minimum and maximum intensity for 8-bit images.

- If the minimum intensity value ($I_{\min}$) present in the image is 100, then it is stretched to the possible minimum value 0.
- Likewise, if the maximum intensity value ($I_{\max}$) is less than the possible maximum intensity value 255, then it is stretched out to 255.

General formula for contrast stretching. For an 8-bit image with $L = 2^8 = 256$ intensity levels, stretching the observed range $[I_{\min}, I_{\max}]$ to the full range $[0, L-1]$ is the

affine map

$$I_{\text{new}} = \frac{I - I_{\min}}{I_{\max} - I_{\min}} \times (L-1) = \frac{I - I_{\min}}{I_{\max} - I_{\min}} \times 255,$$

where the factor $(I - I_{\min})/(I_{\max} - I_{\min}) \in [0, 1]$ is the normalised intensity. Here

- I is the current pixel intensity value,
- $I_{\min} = \min_{x,y} I(x, y)$ is the minimum intensity value present in the whole image,
- $I_{\max} = \max_{x,y} I(x, y)$ is the maximum intensity value present in the whole image,
- I_{new} is the output intensity value rounded up to the nearest integer value.

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import cv2
import numpy as np
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

# Read the image
img = mpimg.imread('snorlax-low-contrast.png')

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

# Convert pixel values from [0,1] to [0,255]
grayImgUint = grayImg*255
grayImgUint = grayImgUint.astype(np.uint8)

# Find min and max pixel values and perform normalization
min = np.min(grayImgUint)
max = np.max(grayImgUint)
imageContrastEnhance = ((grayImgUint-min)/(max-min))*255

# Show the image
plt.figure()
plt.imshow(imageContrastEnhance.astype(np.uint8),
           cmap='gray', vmin=0, vmax=255)
```

Contrast Stretching Requires Global Information

Although the formula is applied pixel by pixel, $I_{\min}$ and $I_{\max}$ are computed from the **whole image**, so contrast stretching depends on global statistics of the input image. Also remember to convert the float PNG values in $[0, 1]$ to $[0, 255]$ before applying the formula, and cast back to `np.uint8` for display.

7.5.4 Grayscale Thresholding

Grayscale Thresholding

Grayscale thresholding is a simple form of **image segmentation**. It is a way to create a binary image from a grayscale image or full-color image. This is typically done in order to separate "object" or foreground pixels from background pixels to aid in image processing.

The formula of grayscale thresholding with threshold T is defined as

$$I_{\text{new}} = \begin{cases} 0 & I < T \\ 255 & \text{otherwise} \end{cases}$$

This is a **point operation** whose output is a binary image, $I_{\text{new}} \in \{0, 255\}$ (or $\{0, 1\}$ after normalisation).

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import cv2
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

# Read the image
img = mpimg.imread('snorlax.png')

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

# Convert pixel values from [0,1] to [0,255]
grayImgUint = grayImg*255
grayImgUint = grayImgUint.astype(np.uint8)

# Perform thresholding
processedImg = grayImgUint > 128

# Show the image
plt.figure()
plt.imshow(processedImg, cmap='gray')
```

7.5.5 Otsu's Method

We need a way to automatically determine the threshold value T so that the result of thresholding is **reproducible**. A well-known approach is **Otsu's method**:

1. Select an initial estimate of the threshold T. A good initial value is the average intensity of the image.
2. Partition the image into two groups, R_1, R_2, using the threshold T.
3. Calculate the mean gray values μ_1 and μ_2 of the partitions R_1, R_2.
4. Compute a new threshold
$$T = \frac{1}{2}(\mu_1 + \mu_2)$$
5. Repeat steps 2–4 until the mean values μ_1 and μ_2 in successive iterations do not change.

Formally, Otsu's method selects the threshold that maximises the between-class variance of the two classes. Let $p(i) = h(i)/(HW)$ be the normalised histogram (the fraction of pixels with intensity i). For a threshold T the class weights and means are

$$\omega_0(T) = \sum_{i \le T} p(i), \qquad \omega_1(T) = \sum_{i > T} p(i) = 1 - \omega_0(T),$$

$$\mu_0(T) = \frac{\sum_{i \le T} i\, p(i)}{\omega_0(T)}, \qquad \mu_1(T) = \frac{\sum_{i > T} i\, p(i)}{\omega_1(T)},$$

and the optimal threshold maximises

$$\sigma_B^2(T) = \omega_0(T)\,\omega_1(T)\,\big(\mu_0(T) - \mu_1(T)\big)^2, \qquad T^\star = \arg\max_{0 \le T \le L-1} \sigma_B^2(T).$$

The iterative scheme above is one simple way to approach this optimum.

To perform Otsu's thresholding in OpenCV, use the `threshold()` method.

Syntax:

```
cv2.threshold(source, thresholdValue, maxVal, thresholdingTechnique)
```

Parameters:

- `source`: input image array (must be grayscale).
- `thresholdValue`: value of threshold below and above which pixel values will change accordingly.
- `maxVal`: maximum value that can be assigned to a pixel.
- `thresholdingTechnique`: the type of thresholding to be applied. (For Otsu's, we put `cv2.THRESH_BINARY + cv2.THRESH_OTSU`.)

Return values:

1. The threshold that was used.
2. The thresholded image (i.e., the binary image).

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import cv2
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

# Read the image
img = mpimg.imread('snorlax.png')

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

# Convert pixel values from [0,1] to [0,255]
grayImgUint = grayImg*255
grayImgUint = grayImgUint.astype(np.uint8)

# Perform thresholding using Otsu's method
thresh, processedImg = cv2.threshold(grayImgUint, 120, 255,
                                     cv2.THRESH_BINARY +
   cv2.THRESH_OTSU)
print('Optimal threshold:', thresh)

# Show the image
plt.figure()
plt.imshow(processedImg, cmap='gray')
```

Otsu Overrides Your Threshold Value

When `cv2.THRESH_OTSU` is used, the `thresholdValue` argument (e.g., `120`) is ignored; Otsu's algorithm finds the optimal threshold automatically and returns it as the first return value.

7.5.6 Local Operations

Recall that **local operations** refer to those where the output value at a specific coordinate is dependent on the input values in the neighborhood of that same coordinate. Some of the most common neighborhoods are the **4-connected neighborhood** and the **8-connected neighborhood**. Writing the image domain as $\Omega = \{0, \dots, W-1\} \times \{0, \dots, H-1\}$, these are

$$\mathcal{N}_4(x, y) = \{(x-1, y),\ (x+1, y),\ (x, y-1),\ (x, y+1)\} \cap \Omega,$$
$$\mathcal{N}_8(x, y) = \big\{(x+dx,\ y+dy) : dx, dy \in \{-1, 0, 1\}\big\} \setminus \{(x, y)\},$$

each restricted to the valid coordinates inside Ω.

Examples of local operations:

- **Image smoothing**: It removes noise and softens edges and corners of the image. It is also called blurring.

- **Image edge detection**: It detects the boundaries (edges) of objects, or regions within an image.
- **Image sharpening**: It removes blur, enhances details, and dehazes.

7.6 Convolution

7.6.1 Convolution

Convolution

A **convolution** is an integral that expresses the amount of overlap of one function g as it is shifted over another function f. It therefore "blends" one function with another. In mathematics, convolution is an operation performed on two functions (f and g) to produce a third function.

Convolution is one of the **most important operations in signal and image processing**. It could operate in:

- **1D** (e.g., speech processing),
- **2D** (e.g., image processing),
- **3D** (e.g., video processing).

7.6.2 Definition

Image convolution is defined as

$$O(x,y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} K(m,n)\, I(x-m,\, y-n)$$

where I is the input image and K is the **image kernel**.

- Assume the origin (i.e., $(0,0)$) of I is the top-left corner, while the origin (i.e., $(0,0)$) of K is the center of the kernel.
- If the image kernel is 3×3, then

$$O(x,y) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} K(m,n)\, I(x-m,\, y-n)$$

- The image kernel is also called the **image filter** or **image mask**.

For a $(2a+1) \times (2b+1)$ kernel K with support $\{-a,\dots,a\} \times \{-b,\dots,b\}$ this is the finite sum

$$O(x,y) = (I * K)(x,y) = \sum_{m=-a}^{a} \sum_{n=-b}^{b} K(m,n)\, I(x-m,\, y-n).$$

The closely related **correlation** replaces the flipped argument $I(x-m, y-n)$ by $I(x+m, y+n)$:

$$(I \star K)(x,y) = \sum_{m=-a}^{a} \sum_{n=-b}^{b} K(m,n)\, I(x+m,\, y+n),$$

so convolution is correlation with the 180°-rotated kernel $\tilde{K}(m,n) = K(-m,-n)$.

Worked Example: Computing One Convolution Output by Definition

Given the input image I and the 3×3 image kernel K:

$$I = \begin{matrix} 10 & 1 & 3 & 2 & 6 \\ 4 & 3 & 5 & 8 & 0 \\ 8 & 7 & 9 & 6 & 5 \end{matrix} \qquad K = \begin{matrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{matrix}$$

Find $O(1,1)$ using $O(x,y) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} K(m,n)\, I(x-m, y-n)$.

Solution. Expanding the double sum:

$$O(1,1) = \sum_{m=-1}^{1} \Big(K(m,-1)\, I(1-m,\, 1-(-1)) + K(m,0)\, I(1-m,\, 1-0) + K(m,1)\, I(1-m,\, 1-1) \Big)$$

$$\begin{aligned} O(1,1) = {} & K(-1,-1)\, I(2,2) + K(-1,0)\, I(2,1) + K(-1,1)\, I(2,0) + \\ & K(0,-1)\, I(1,2) + K(0,0)\, I(1,1) + K(0,1)\, I(1,0) + \\ & K(1,-1)\, I(0,2) + K(1,0)\, I(0,1) + K(1,1)\, I(0,0) \end{aligned}$$

Substituting the values:

$$\begin{aligned} O(1,1) &= (-1)(9) + (-1)(5) + (-1)(3) + (0)(7) + (0)(3) \\ &\quad + (0)(1) + (1)(8) + (1)(4) + (1)(10) \\ &= -9 - 5 - 3 + 8 + 4 + 10 \\ &= \boxed{5} \end{aligned}$$

Computing the remaining output image values (at $(1,2)$ and $(1,3)$, the only other positions where the 3×3 kernel fits entirely inside the image) gives the full output row:

$$O = \begin{matrix} 5 & -5 & 6 \end{matrix}$$

Convolution Flips the Kernel Indices

Note how $I(x-m, y-n)$ causes the kernel to be applied in a reversed (flipped) manner: $K(-1,-1)$ pairs with the bottom-right pixel $I(2,2)$ of the neighborhood, while $K(1,1)$ pairs with the top-left pixel $I(0,0)$. This flipping is what distinguishes **convolution** from **correlation**.

7.6.3 Intuitive View

Computing convolution by the raw formula is very tedious. There is an intuitive procedure:

1. **Inverse the kernel**, i.e., flip the kernel in both the horizontal and vertical directions about the center of the kernel.
2. **Slide** the inversed kernel over the image, centered at the interested point.
3. **Multiply** the inversed kernel data with the overlapped area.
4. **Sum and accumulate** the output.

For the kernel above:

$$\underbrace{\begin{matrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{matrix}}_{\text{Original kernel}} \xrightarrow{\text{flip horizontally}} \underbrace{\begin{matrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{matrix}}_{\text{Flipped horizontally}} \xrightarrow{\text{flip vertically}} \underbrace{\begin{matrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{matrix}}_{\text{Flipped vertically}}$$

Then overlay the flipped kernel on the neighborhood of the target pixel, multiply element-wise, and sum.

7.6.4 Boundary Problem

Problem: How to Handle Pixels at the Boundary?

When computing an output pixel at the **boundary** of an image, a portion of the convolution kernel is usually off the edge of the image. How to deal with this?

Solutions:

1. **Just ignore** those boundary pixels.
2. **Zero padding**: add a border of pixels all with value zero around the edges of the input image.

$$\begin{matrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 1 & 3 & 2 & 6 & 0 \\ 0 & 4 & 3 & 5 & 8 & 0 & 0 \\ 0 & 8 & 7 & 9 & 6 & 5 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{matrix}$$

3. **Replicating** boundary pixels (copy the nearest edge pixel outward).

$$\begin{matrix} 10 & 10 & 1 & 3 & 2 & 6 & 6 \\ 10 & 10 & 1 & 3 & 2 & 6 & 6 \\ 4 & 4 & 3 & 5 & 8 & 0 & 0 \\ 8 & 8 & 7 & 9 & 6 & 5 & 5 \\ 8 & 8 & 7 & 9 & 6 & 5 & 5 \end{matrix}$$

4. **Reflecting** boundary pixels.
5. **Mirroring** boundary pixels.

7.6.5 Convolution in OpenCV

To do image convolution, use the `filter2D()` method of the `cv2` module.

Syntax:

```
cv2.filter2D(src, ddepth, kernel, dst, anchor, delta,
borderType=cv2.BORDER_DEFAULT)
```

Parameters:

- `src`: input image that you want to convolve.
- `ddepth`: desired depth of the destination image. If `ddepth = -1`, the output image will have the same depth as `src`.
- `kernel`: convolution kernel, a single-channel floating point matrix; if you want to apply different kernels to different channels, split the image into separate color planes using `split` and process them individually.
- `dst`: output image of the same size and the same number of channels as `src`.
- `anchor`: anchor of the kernel that indicates the relative position of a filtered point within the kernel; the anchor should lie within the kernel; default value $(-1, -1)$ means that the anchor is at the kernel center.
- `delta`: optional value added to the filtered pixels before storing them in `dst`.
- `borderType`: pixel extrapolation method:
  - `cv2.BORDER_CONSTANT`: `iiiiii|abcdefgh|iiiiiii` with some specified `i`
  - `cv2.BORDER_REPLICATE`: `aaaaaa|abcdefgh|hhhhhh`
  - `cv2.BORDER_REFLECT`: `fedcba|abcdefgh|hgfedcb`
  - `cv2.BORDER_REFLECT_101`: `gfedcb|abcdefgh|gfedcba`
  - `cv2.BORDER_DEFAULT`: same as `BORDER_REFLECT_101`

Return value: filtered image.

filter2D Does NOT Mirror the Kernel

`filter2D` does not mirror the kernel for you (it actually computes a **correlation**). You will need to **flip the kernel yourself** before applying `cv2.filter2D` if you want

a true convolution. For symmetric kernels (e.g., the averaging kernel) this makes no difference, but for asymmetric kernels (e.g., Prewitt/Sobel) the sign of the result changes.

```
# Assume Google Drive has been mounted, the path has been added for
# interpreter to search
# Import all the required libraries
import cv2
import numpy as np
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

# Read the image
img = mpimg.imread('snorlax-sleep.png')

# Convert the color image to gray and show it
grayImg = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
plt.figure()
plt.imshow(grayImg, cmap='gray', vmin=0, vmax=1)

# Prepare a kernel (a sharpening kernel here)
kernel_3x3 = np.array([[0, -1, 0],
                       [-1, 5, -1],
                       [0, -1, 0]])

for i in range(5):  # Perform filtering 5 times
    grayImg = cv2.filter2D(grayImg, -1, kernel_3x3)

# Show the resulting image
plt.figure()
plt.imshow(grayImg, cmap="gray", vmin=0, vmax=1)
```

7.6.6 Smoothing Kernel

$$K_{\text{avg}} = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

- **Blurring an image can be done by averaging pixels.** Each output pixel becomes the average of the 3×3 neighborhood.
- Analogous to integration; related to the sum of pixel intensity values.
- **Effect of kernel size**: using 3×3, 5×5, 15×15 and 25×25 masks produces progressively stronger blurring — the larger the mask, the more each pixel is averaged with a wider neighborhood, so the more detail is lost.

For a general $(2a+1) \times (2b+1)$ **box (mean) filter** the output is the local average

$$O(x,y) = \frac{1}{(2a+1)(2b+1)} \sum_{m=-a}^{a} \sum_{n=-b}^{b} I(x-m,\, y-n).$$

A better-behaved smoother is the **Gaussian filter**, whose weights follow the 2D Gaussian

$$G(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right), \qquad \sum_x \sum_y G(x,y) = 1,$$

where σ controls the strength of the blur (larger $\sigma \Rightarrow$ smoother). The Gaussian kernel is the sampled (and renormalised) Gaussian, $K(m,n) \propto G(m,n)$; unlike the box filter it is isotropic and separable, $G(x,y) = G(x)\,G(y)$.

7.6.7 Sharpening Kernel

$$\underbrace{\begin{bmatrix}0 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 0\end{bmatrix}}_{\text{Original image}} + \underbrace{\begin{bmatrix}-1 & -1 & -1\\ -1 & 8 & -1\\ -1 & -1 & -1\end{bmatrix}}_{\text{Detail (Edge)}} = \underbrace{\begin{bmatrix}-1 & -1 & -1\\ -1 & 9 & -1\\ -1 & -1 & -1\end{bmatrix}}_{\text{Sharpened image}}$$

- **Sharpening has the opposite effect of blurring.** Conceptually: sharpened = original + edge detail (the high-frequency component).
- Analogous to differentiation; related to the difference of pixel intensity values.
- A commonly used sharpening variant (used in the code example above) is

$$\begin{bmatrix}0 & -1 & 0\\ -1 & 5 & -1\\ 0 & -1 & 0\end{bmatrix}.$$

Formally, sharpening can be written as unsharp masking

$$I_{\text{sharp}} = I + \alpha\,(I - I_{\text{blur}}),$$

where $I_{\text{blur}} = I * K_{\text{avg}}$ is a blurred copy, the residual $I - I_{\text{blur}}$ is the high-frequency detail, and $\alpha \geq 0$ sets the strength. The detail term is often taken as a discrete **Laplacian**,

$$\nabla^2 I \approx \begin{bmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{bmatrix} * I \qquad \text{or} \qquad \begin{bmatrix}1 & 1 & 1\\ 1 & -8 & 1\\ 1 & 1 & 1\end{bmatrix} * I,$$

so the sharpening kernel is the identity kernel plus α times such a Laplacian.

7.6.8 Prewitt Kernel

$$K_{\text{vertical}} = \begin{bmatrix}-1 & 0 & 1\\ -1 & 0 & 1\\ -1 & 0 & 1\end{bmatrix} \qquad K_{\text{horizontal}} = \begin{bmatrix}-1 & -1 & -1\\ 0 & 0 & 0\\ 1 & 1 & 1\end{bmatrix}$$

The left kernel detects **vertical edges** and the right kernel detects **horizontal edges**. Combining the two responses gives the **edge magnitude**:

$$|G| = \sqrt{G_x^2 + G_y^2}$$

where G_x and G_y are the outputs of the vertical-edge and horizontal-edge kernels, respectively. These are finite-difference approximations to the image gradient $\nabla I = (\partial I/\partial x,\ \partial I/\partial y)^\top$, with $G_x \approx \partial I/\partial x$ and $G_y \approx \partial I/\partial y$; the edge magnitude is $\|\nabla I\| = \sqrt{G_x^2 + G_y^2}$ and the edge orientation is $\theta = \text{atan2}(G_y, G_x)$. (Pixels of the processed images are usually inverted, i.e., black to white, white to black, for making them more visible.)

7.6.9 Sobel Kernel

$$K_{\text{vertical}} = \begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix} \qquad K_{\text{horizontal}} = \begin{bmatrix}-1 & -2 & -1\\ 0 & 0 & 0\\ 1 & 2 & 1\end{bmatrix}$$

- The Sobel kernels are like the Prewitt kernels but give **more weight (2) to the center row/column**, which makes them slightly more robust to noise.
- The **edge magnitude** is computed the same way: $|G| = \sqrt{G_x^2 + G_y^2}$, where $|G_x|$ and $|G_y|$ are the responses of the vertical-edge and horizontal-edge kernels.
- Equivalently, the Sobel kernels combine a 3×3 box smoothing with a central-difference derivative, $K_{\text{Sobel}} = K_{\text{smooth}} * K_{\text{diff}}$, which explains their better robustness to noise.

Blurring vs. Sharpening vs. Edge Detection

- **Smoothing kernel**: all weights positive and sum to 1 (weighted averaging) — analogous to integration.
- **Sharpening kernel**: a strong positive center with negative surroundings (weights sum to 1, e.g., $9 - 8 = 1$) — original plus detail.
- **Edge kernels (Prewitt/Sobel)**: weights sum to 0 — flat regions produce zero output, only intensity changes (edges) respond — analogous to differentiation.

7.6.10 Towards CNNs

Clearly, image convolution is powerful in finding the features of an image if we already know the right kernel to use. Kernel design is an art and has been refined over the last few decades to do some pretty amazing things with images.

The Key Question Leading to CNNs

But the important question is: what if we don't know the features we are looking for? Or what if we do know, but we don't know what the kernel should look like? — This is exactly the motivation for **Convolutional Neural Networks**: let the network learn the kernels (filters) from data instead of hand-designing them.

7.7 Practice: Worked Example

Practice Problem: Averaging Kernel and Sobel Edge Magnitude

A 3-bit/pixel image (i.e., pixel value is in range 0 to $2^3 = 8$) of size 3×3 is given below:

$$I = \begin{matrix} 3 & 7 & 6 \\ 2 & 4 & 6 \\ 4 & 7 & 2 \end{matrix}$$

1. Find the output of a 3×3 **averaging kernel** at $(1, 1)$.
2. Find the **edge magnitude** at $(1, 1)$ using the Sobel masks shown below.

$$K_{\text{horizontal}} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \qquad K_{\text{vertical}} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

(Sobel kernel for extracting horizontal edges on the left; Sobel kernel for extracting vertical edges on the right.)

Note: Edge magnitude $= \sqrt{\text{horizontal edge value}^2 + \text{vertical edge value}^2}$.

Solution.
Part 1: Averaging kernel at $(1, 1)$.
The output value of a 3×3 averaging kernel at $(1, 1)$ is the mean of all 9 pixels:

$$O(1, 1) = \frac{3 + 7 + 6 + 2 + 4 + 6 + 4 + 7 + 2}{9} = \frac{41}{9} \approx 4.56$$

Part 2: Sobel edge magnitude at $(1, 1)$.
Step 1 — Horizontal edge value. Flip the horizontal-edge kernel both horizontally and vertically:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \xrightarrow{\text{flip}} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Overlay on the image, multiply element-wise, and sum:

$$\begin{aligned} G_y &= (1)(3) + (2)(7) + (1)(6) + (0)(2) + (0)(4) \\ &\quad + (0)(6) + (-1)(4) + (-2)(7) + (-1)(2) \\ &= 3 + 14 + 6 - 4 - 14 - 2 = 3 \end{aligned}$$

Step 2 — Vertical edge value. Flip the vertical-edge kernel both horizontally and vertically:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \xrightarrow{\text{flip}} \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Overlay on the image, multiply element-wise, and sum:

$$\begin{aligned} G_x &= (1)(3) + (0)(7) + (-1)(6) + (2)(2) + (0)(4) \\ &\quad + (-2)(6) + (1)(4) + (0)(7) + (-1)(2) \\ &= 3 - 6 + 4 - 12 + 4 - 2 = -9 \end{aligned}$$

Step 3 — Edge magnitude.

$$|G| = \sqrt{G_x^2 + G_y^2} = \sqrt{(3)^2 + (-9)^2} = \sqrt{90} \approx 9.49$$

8 Convolutional Neural Networks

8.1 Introduction

8.1.1 Overview

In this topic, we continue to discuss how to implement an AI for image recognition and classification. This time, we use **Convolutional Neural Networks (CNNs)** to train the AI agent to know how to recognize/classify images.

CNNs can also be used to train AI agents to solve other problems, such as:

- natural language processing,
- speech recognition,
- recommendation systems,
- image segmentation,
- medical image analysis,
- financial time series, etc.

8.1.2 A Brief History

Convolutional neural networks were developed in the **late 1980s** and then forgotten about due to the **lack of processing power**. With the powerful **Graphical Processing Units (GPUs)**, the research on CNNs and deep learning was given new life.

CNNs are used in many applications:

- **Object recognition in images and videos** (e.g., image search in Google, tagging friends' faces on Facebook, adding filters in Snapchat and tracking movement in Kinect).
- **Natural language processing** (speech recognition in Google Assistant or Amazon's Alexa).
- **Playing games** (the defeat of the World "Go" champion by DeepMind at Google).
- **Medical innovation** (from drug discovery to prediction of disease).

8.2 What is a CNN?

A **Convolutional Neural Network (CNN or ConvNet)** is very much related to the standard **Artificial Neural Network (ANN)** that we have previously encountered.

Definition: Convolutional Neural Network

A **convolutional neural network** is a neural network with a **convolution operation** in at least one of the layers.

8.2.1 From Convolution to CNN

- Every **pixel** in an image is a **feature**, and that means it represents an **input node**.
- The result from each convolution is placed into the next layer in a **hidden node**. Each feature or pixel of the convolved image is a node in the hidden layer.
- So, what are the numbers in the kernel? They are **weights** connecting the features of the input image and the nodes of the hidden layer. These weights that connect to the nodes need to be **learned**, exactly as in a regular neural network.

The intuitive idea is that a CNN learns a **hierarchy of features**: early layers pick up simple patterns such as edges and curves, while deeper layers combine them into larger-scale structures used for recognition.

8.3 Layers of a CNN

The main types of layers used to build a Convolutional Neural Network (CNN/ConvNet) are:

1. **Input layer**
2. **Convolutional layer**
3. **Pooling layer**
4. **Fully-connected layer**
5. **Output layer**

8.3.1 Input Layer

The input image is placed into this layer. It can be:

- a single-layer 2D image (**grayscale**),
- 3-channel 2D images (**RGB color**),
- or 3D.

Running Example

In the running example of the slides, the input images are **3-channel 2D images** of 28 pixels by 28 pixels (RGB color), i.e., $28 \times 28 \times 3$. The **kernels need to be learned** and have the **same depth as the input**: the depth of the kernels applied on the input layer is 3, with spatial size 5×5.

8.3.2 Convolutional Layer

The **convolutional layer** convolves the input volume with a set of learnable kernels and produces an output volume of **feature maps** (one per kernel).

Formally, let the input be an array $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ and let the kernels be $\mathbf{K} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}$. For output channel c' and spatial location (i, j), the **cross-correlation** form of the convolution used in deep learning is

$$\mathbf{Z}[c', i, j] = b[c'] + \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} \mathbf{X}[c,\ i\,s + m,\ j\,s + n]\ \mathbf{K}[c', c, m, n]\,,$$

where s is the **stride** and $b[c']$ is the bias of channel c'. The input is implicitly **zero-padded** by p pixels on every side, and the same kernel weights are reused at every spatial

position (**weight sharing**). The array $\mathbf{Z}[c', \cdot, \cdot]$ is the **feature map** (output channel) produced by kernel c'.

The **receptive field** of an output unit is the region of the input that influences it: after a stack of L convolutional layers of size $K \times K$ with stride 1 and no pooling,

$$\mathrm{RF} = 1 + L\,(K - 1).$$

Example: Input/Output Dimensions of a Convolutional Layer

Input: an image of size $28 \times 28 \times 3$; 10 kernels, i.e., $10 \times 5 \times 5 \times 3$.
Assumption: ignore those pixels that do not have 24 neighboring pixels (a 5×5 neighborhood has 24 neighbors around the center pixel; equivalently, no padding is used, $p = 0$).
Output: $24 \times 24 \times 10$, because per spatial dimension

$$H_{\text{out}} = \left\lfloor \frac{28 - 5 + 2 \cdot 0}{1} \right\rfloor + 1 = 24,$$

and each of the 10 kernels produces one output channel ($C_{\text{out}} = 10$).

8.3.3 Stride

Stride is the amount of movement between applications of the kernel to the input image.

- The default stride (or strides in 2D) is $(1, 1)$ for the height and the width movement.
- The kernel is moved across the image left to right, top to bottom, with a one-pixel column change on the horizontal movements, then a one-pixel row change on the vertical movements.
- The stride can be changed, which has an effect both on how the kernel is applied to the image and, in turn, on the **size of the resulting output**.
- Formally, with stride s the top-left corner of the kernel is placed at input positions $(i\,s,\, j\,s)$ for $i, j \in \mathbb{N}$, jumping s pixels at a time along each axis.
- In Keras, the stride can be specified on the `Conv2D` layer via the `stride` argument, specified as a tuple with height and width.

Why Set the Stride to a Larger Number?

- Larger strides lead to **less overlap**, which means a **lower output volume**.
- **Less memory** is needed for the output.
- It helps **avoid overfitting**, especially in the case of image processing with a large number of attributes.

8.3.4 Spatial Dimensions

Suppose an input image is of size 7×7 and convolution is performed using a 3×3 kernel. The number of valid top-left positions along one dimension is $\lfloor (n - K)/s \rfloor + 1$, where n is the input size, K the kernel size and s the stride.

- With **stride** 1, the 3×3 kernel can be placed with its top-left corner at 5 horizontal and 5 vertical positions, $\lfloor (7 - 3)/1 \rfloor + 1 = 5$ — that is why the output is 5×5.
- With **stride** 2, we move the kernel 2 pixels at a time, $\lfloor (7 - 3)/2 \rfloor + 1 = 3$ — that is why the output is 3×3.

A Stride That Does Not Fit

How about a 7×7 input image and a 3×3 kernel applied with **stride** 3? We cannot apply the 3×3 kernel with stride 3 since it **does not fit** — it does not reach the input image boundary exactly: $(7 - 3)/3 + 1 = 2.33$ is not an integer.

8.3.5 Output Size Formula

Output Size of a Convolutional Layer

For an input of spatial height H_{in} (and analogously width W_{in}), a kernel of size K, padding p and stride s, the output spatial size is

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2p}{s} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2p}{s} \right\rfloor + 1.$$

The floor $\lfloor \cdot \rfloor$ drops the last partial window when the kernel cannot reach the boundary exactly.

Example: 7×7 Image with a 3×3 Kernel

Here $H_{\text{in}} = W_{\text{in}} = 7$, $K = 3$ and $p = 0$, so $H_{\text{out}} = \left\lfloor \frac{7-3}{s} \right\rfloor + 1$.

- Stride 1 $\Rightarrow$ output size $= \left\lfloor \frac{7-3}{1} \right\rfloor + 1 = 5$.
- Stride 2 $\Rightarrow$ output size $= \left\lfloor \frac{7-3}{2} \right\rfloor + 1 = 3$.
- Stride 3 $\Rightarrow$ output size $= \left\lfloor \frac{7-3}{3} \right\rfloor + 1 = 2$; the raw value 2.33 is not an integer, so the kernel **does not fit**.

8.3.6 Dilated Convolution

Dilated Convolution (a.k.a. Atrous Convolution)

Dilated convolution is a variation of the standard convolution that **skips input elements** according to a dilation rate d: the kernel touches one input pixel every d pixels, so it "sees" a **larger area of the input without increasing the number of parameters**. ($d = 1$ recovers the standard convolution.) Its output is

$$\mathbf{Z}[c', i, j] = b[c'] + \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{K_h - 1} \sum_{n=0}^{K_w - 1} \mathbf{X}[c,\ i\,s + m\,d,\ j\,s + n\,d]\ \mathbf{K}[c', c, m, n]\,,$$

i.e., consecutive kernel taps sample the input every d pixels.

- **Effective kernel size:** a $k \times k$ kernel with dilation rate d spans
$$k_{\text{eff}} = k + (k-1)(d-1)$$
pixels, e.g., a 3×3 kernel with $d = 2$ covers a 5×5 region but still has only 9 weights.
- **Output size:** plug k_{eff} into the usual formula:
$$\text{Output size} = \left\lfloor \frac{n - k_{\text{eff}} + 2p}{s} \right\rfloor + 1$$
- **'Same' padding:** choose $p = \dfrac{(k-1)\,d}{2}$ so that the output keeps the input dimensions.
- Stacking dilated layers with $d = 1, 2, 4, \ldots$ grows the **receptive field exponentially**, which is why dilated convolutions are popular for dense prediction (e.g., segmentation) and audio modeling.

Exam-Style Practice: Implementing Dilated Convolution (Final 2024S Problem 6; Final 2025F Question 8.2)

Complete the NumPy implementation of `dilated_convolution(input_array, kernel, dilation_rate=1, stride=1, padding='valid')`:

```
# 'same' padding width
pad = ((kernel.shape[0] - 1) * dilation_rate) // 2
# output dimensions
output_rows = (input_array.shape[0] - kernel.shape[0] *
   dilation_rate + 2 * pad) // stride + 1
# input rows touched by kernel row i (dilation spaces the taps)
input_row_indices = np.arange(0, output_rows * stride, stride) + i
   * dilation_rate
# accumulate the contribution of kernel entry (i, j)
output_array += kernel[i, j] *
   padded_input[np.ix_(input_row_indices, input_col_indices)]
```

Key idea: kernel entry (i, j) multiplies input pixels at rows $i \cdot d + \{0, s, 2s, \dots\}$ — the dilation rate spaces out both the kernel taps and the sampled input indices, while `np.ix_` builds the open mesh needed to slice the 2D region.

8.3.7 Zero Padding

When a kernel convolves a given input image, the dimensions of the output image are **reduced**. For example:

- The input image size is 28×28.
- Convolving the image using a 3×3 kernel, the output image size is 26×26, since we ignore those pixels without 8 neighbors.

How to preserve the size of the output image? **Zero padding!**

Zero Padding

Zero padding occurs when we add a **border of pixels, all with value zero**, around the boundaries of the input image. A border of p pixels maps an input of size H_{in} to a padded size

$$H_{\text{pad}} = H_{\text{in}} + 2p.$$

For implementations of CNNs using Keras, there are two categories of padding:

- `valid` — i.e., **no padding**;
- `same` — **pad** the original input with zeros before we convolve so that the output size is the same size as the input size.

Example: 7×7 Image, 3×3 Kernel, 1-Pixel Zero Border, Stride 1

Suppose we do padding with a 1-pixel-width border and use stride 1. What is the output image size?

- Size of image dimension = 7; size of kernel dimension = 3; pad with a 1-pixel border.
- The padded image has dimension $7 + 2 \times 1 = 9$.
- Output image size $= \dfrac{(7+2)-3}{1} + 1 = 7$ — the spatial size is preserved.

Common Choice That Preserves the Spatial Size

In general, it is common to see convolutional layers with **stride** 1, kernels of size $K \times K$, and **zero-padding with** $(K-1)/2$ pixels. It will preserve the size spatially. Padded border size:

- If $K = 3$, zero-pad with $(3-1)/2 = 1$ (1-pixel border).
- If $K = 5$, zero-pad with $(5-1)/2 = 2$ (2-pixel border).
- If $K = 7$, zero-pad with $(7-1)/2 = 3$ (3-pixel border).

All-in-One Example

Given:
- Input image size $= 32 \times 32 \times 3$.
- 10 kernels, each of size $5 \times 5 \times 3$.
- Stride $= 1$; pad with a 2-pixel border.

Output size:

$$\text{Output size} = \frac{(32+4)-5}{1} + 1 = 32 \quad \Longrightarrow \quad \text{output volume size} = 32 \times 32 \times 10.$$

Number of parameters in this layer. With weight sharing, the layer has

$$\#\text{weights} = (K_h\, K_w\, C_{\text{in}})\, C_{\text{out}}, \qquad \#\text{biases} = C_{\text{out}}.$$

Here $K_h = K_w = 5$, $C_{\text{in}} = 3$ and $C_{\text{out}} = 10$, so each kernel has $K_h K_w C_{\text{in}} = 5{\cdot}5{\cdot}3 = 75$ weights plus 1 bias $= 76$ parameters, and

$$\#\text{param} = (5 \cdot 5 \cdot 3 + 1) \cdot 10 = 76 \times 10 = 760.$$

8.3.8 Non-linearity

Convolution is a **linear** operation. We need **non-linearity**; otherwise, 2 convolution layers would be no more powerful than 1.

- The non-linearity is not its own distinct layer of the CNN, but comes as **part of the convolution layer**, as it is done in the neurons (just like a normal artificial neural network).
- In a neural network, we use different activation functions, typically the **sigmoid**. But the **rectified linear unit (ReLU)** is more popular for CNNs, as it does not require any expensive computation. It has been shown to **speed up the convergence of stochastic gradient descent** algorithms.

$$\text{sigmoid}(z) = \frac{1}{1+e^{-z}} \qquad \text{ReLU}(z) = \max(0, z).$$

8.3.9 Pooling Layer

The **pooling layer** is key to ensuring that the subsequent layers of the CNN can pick up **larger-scale detail** than just edges and curves.

- It does this by **merging pixel regions** in the convolved image together (shrinking the image) before attempting to learn kernels on it.

Two popular ones. For a pooling window of size $k \times k$ with stride s applied to a feature map $\mathbf{X}$:

- **Max pooling** applies a max filter to (typically non-overlapping) subregions,

$$\text{maxpool}[i,j] = \max_{0 \le m,\, n < k} \mathbf{X}[i\,s+m,\ j\,s+n].$$

- **Average pooling** applies an average filter to the same subregions,

$$\text{avgpool}[i,j] = \frac{1}{k^2} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \mathbf{X}[i\,s+m,\ j\,s+n].$$

Example: 2×2 Max Pooling and Average Pooling

Consider the following 6×4 representation:

$$\begin{bmatrix} 12 & 20 & 30 & 0 \\ 8 & 12 & 2 & 0 \\ 20 & 30 & 13 & 8 \\ 34 & 70 & 37 & 4 \\ 112 & 37 & 79 & 20 \\ 112 & 100 & 25 & 12 \end{bmatrix}$$

Apply 2×2 pooling over the non-overlapping subregions. For example, for the top-left subregion:

$$\text{Max: } \max(12, 20, 8, 12) = 20, \qquad \text{Average: } \frac{12 + 20 + 8 + 12}{4} = 13.$$

Repeating for all six subregions:

$$\textbf{Max pooling: } \begin{bmatrix} 20 & 30 \\ 70 & 37 \\ 112 & 79 \end{bmatrix} \qquad \textbf{Average pooling: } \begin{bmatrix} 13 & 8 \\ 38.5 & 15.5 \\ 90.25 & 34 \end{bmatrix}$$

Benefits of Pooling

Pooling can also **tolerate local distortion and translation** of the patterns of interest, and **reduce the computational load, memory usage, and the number of parameters**.

8.3.10 3D Convolution and Pooling

3D Convolution

A **3D convolution** slides a 3D kernel over a volume — e.g., video data (frames × width × height), where time is treated as the third dimension. Unlike 2D convolution, which only analyzes spatial patterns within a single frame, 3D convolution captures **temporal dynamics as well as spatial features**, which is essential for action recognition, video classification, and anomaly detection in video streams.

- **Kernel shape:** (k_D, k_H, k_W), e.g., $(3, 3, 3)$ = (keyframes, width, height).
- **Output size:** apply the usual rule independently per dimension: $\left\lfloor \frac{n-k+2p}{s} \right\rfloor + 1$.
- **#weights** $= k_D \cdot k_H \cdot k_W \cdot C_{\text{in}} \cdot \#\text{kernels}$; **#biases** $= \#\text{kernels}$.
- **3D pooling** works analogously: a $(2, 2, 2)$ max pool halves all three dimensions, e.g., $(32, 10, 64, 64) \to (32, 5, 32, 32)$, and has 0 parameters.

Exam-Style Practice: 3D CNN — Shapes, Parameters and FLOPs (Final 2024S Problem 7(c); Final 2025F Question 8.1)

(a) Shapes and parameters. Video data of shape $(32, 400, 300, 3)$ = (#keyframes, width, height, #channels) is convolved with 100 kernels of shape $(3, 3, 3)$, padding $(0, 2, 2)$, stride $(1, 2, 2)$.

- Output shape: $\left(\frac{32 - 3 + 0}{1} + 1,\ \left\lfloor \frac{400 - 3 + 4}{2} \right\rfloor + 1,\ \left\lfloor \frac{300 - 3 + 4}{2} \right\rfloor + 1,\ 100\right) = (30, 201, 151, 100)$.
- $\#\text{weights} = 3 \cdot 3 \cdot 3 \cdot 3 \cdot 100 = 8100$; $\#\text{biases} = 100$.

(b) Counting FLOPs. A `CONV3D` layer maps $(16, 10, 64, 64) \to (32, 10, 64, 64)$ with $3 \times 3 \times 3$ kernels.

- Per output element: the dot product over the kernel volume costs $3 \cdot 3 \cdot 3 \cdot 16 = 432$ multiplications + 431 additions + 1 bias = 864 operations.

- Number of output elements: $32 \cdot 10 \cdot 64 \cdot 64 = 1{,}310{,}720$.
- Total FLOPs $= 864 \times 1{,}310{,}720 = 1{,}132{,}462{,}080$.

8.3.11 Fully-Connected Layer

- The **fully-connected layer** is a traditional **multi-layer perceptron** that uses a **softmax** activation function in the output layer.
- The term "fully-connected" implies that every neuron in the previous layer is connected to every neuron in the next layer.
- The output from the convolutional and pooling layers represents **high-level features** of the input image.
- The purpose of the fully-connected layer is to use these features for **classifying** the input image into various classes based on the training dataset.

After the convolutional and pooling stages, the output volume $\mathbf{Z}$ is **flattened** into a feature vector and fed to a standard MLP with a softmax output. For a flattened feature vector $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \qquad \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}},$$

and the predicted class is

$$\hat{y} = \underset{1 \le k \le C}{\arg\max}\ \text{softmax}(\mathbf{z})_k.$$

Summary: Feature Extractors vs. Classifier

Convolutional + Pooling layers act as feature extractors from the input image, while the **fully-connected layer** acts as a classifier.

8.4 CNN Training Process

1. We initialize all kernels and parameters/weights with **random values**.
2. The network takes a training image as input, goes through the **forward propagation** step (convolution, ReLU and pooling operations along with forward propagation in the fully-connected layer) and finds the **output probabilities** for each class.
3. Calculate the **total error** at the output layer.
4. Use **backpropagation** to update all kernel values/weights and parameter values to **minimize the output error**.
5. Repeat steps 2–4 with all images in the training set.

8.5 Dropout

8.5.1 Overfitting Review

Overfitting refers to a model that was trained too much on the particulars of the training data (when the model learns the **noise** in the dataset). A model that is overfitted will **not perform well on new, unseen data**.

Underfitting typically refers to a model that has not been trained sufficiently. This could be due to insufficient training time or a model that was simply not trained properly. A model that is underfitted will perform **poorly on the training data and on new, unseen data alike**.

8.5.2 Dropout Layer

- A large neural network trained on a relatively small dataset can **overfit** the training data. This has the effect of the model learning the **statistical noise** in the training data, which results in poor performance when the model is evaluated on new data, e.g., a test dataset. The **generalization error increases** due to overfitting.
- **Dropout** is a simple way to prevent neural networks from overfitting.
- Dropout is a **regularization method**. During training, some number of layer outputs are **randomly ignored** or "dropped out".
- Dropout has the effect of forcing nodes within a layer to **probabilistically take on more or less responsibility** for the inputs.
- Because the outputs of a layer under dropout are **randomly subsampled**, it has the effect of **reducing the capacity** or **thinning the network** during training. As such, a **wider network**, e.g., more nodes, may be required when using dropout.

8.5.3 How to Dropout?

- Dropout is implemented **per-layer** in a neural network.
- It can be used with most types of layers, such as **dense fully-connected layers**, **convolutional layers**, etc.
- Dropout may be implemented on any or all hidden layers in the network and the **input layer**.
- It is **not used on the output layer**.
- A parameter is introduced that specifies the **probability** at which the outputs of the layer are dropped out (or, inversely, the probability at which the outputs of the layer are retained).
- A common value is a probability of 0.5 for retaining the output of each node in a hidden layer.

Where Dropout Is Not Applied

Dropout may be used on the input layer and any or all hidden layers, but it is **never used on the output layer**.

Exam-Style Practice: Implementing (Inverted) Dropout in NumPy (Final 2024S Problem 6(b); Final 2025F Question 8.3)

Given an input array `X` of shape $(N, 1, 1, C)$ and a keep probability `keep_prob`, implement dropout:

```
# binary mask: True with prob keep_prob
mask = np.random.rand(N, 1, 1, C) < keep_prob
X = X * mask                                    # drop random
   elements
# rescale so the expected sum is unchanged
X = X / keep_prob
```

Key idea: dividing by `keep_prob` (inverted dropout) keeps the expected activation magnitude the same as without dropout, so **no rescaling is needed at test time**. This mirrors Keras' behavior: "inputs not set to 0 are scaled up by $1/(1 - \text{rate})$".

8.6 Example: Handwritten Digits

We will build a CNN model to recognize/classify handwritten digits.

8.6.1 Dataset

We use the **Modified National Institute of Standards and Technology (MNIST)** dataset. This dataset contains two sets of samples:

- **Training data:** 60000 images of handwritten digits from 0 to 9, each of size 28 pixel × 28 pixel.
- **Testing data:** 10000 images of size 28 pixel × 28 pixel.

8.6.2 Procedures

1. Import required libraries and define global variables.
2. Load and prepare the data.
3. Build the model.
4. Compile the model.
5. Train the model.
6. Evaluate the model accuracy.
7. Save the model.
8. Use the model.

Please refer to the lecture on MLP for data exploration.

8.6.3 Step 1: Setup

```
import numpy as np # Import numpy library
import matplotlib.pyplot as plt # Import mathplot library
import datetime # Import datetime library
from keras.datasets import mnist # Import MNIST dataset
from keras.models import Sequential # Import Sequential class
# Import Conv2D, MaxPooling2D class
from keras.layers import Conv2D, MaxPooling2D
# Import Dense, Dropout, Flatten class
from keras.layers import Dense, Dropout, Flatten
# Import numpy-related utilities
from keras.utils import to_categorical
from keras.callbacks import TensorBoard # Import TensorBoard class
from keras.models import load_model # Import load_model method
# Import plot_model method
from tensorflow.keras.utils import plot_model
# Import confusion matrix method
from tensorflow.math import confusion_matrix
# Import categorical crossentropy loss
from tensorflow.keras.metrics import  categorical_crossentropy

batch_size = 128 # Number of samples per gradient update
num_classes = 10 # Number of classes in the dataset
epochs = 1 # Number of epochs to train the model
img_rows, img_cols = 28, 28 # Image dimensions
```

8.6.4 Step 2: Load Data

```
# Load data
# x_train is a NumPy array of grayscale image data with shapes (60000,
# 28, 28)
# y_train is a NumPy array of digit labels (in range 0-9) with shape
# (60000,)
# x_test is a NumPy array of grayscale image data with shapes (10000,
# 28, 28)
# y_test is a NumPy array of digit labels (in range 0-9) with shape
# (10000,)
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# Reshape the data to 4-dimension
x_train = x_train.reshape(60000,28,28,1)
# Reshape the data to 4-dimension
x_test = x_test.reshape(10000,28,28,1)

# There are 10 classes and classes are represented as unique integers
# To do so, transforming the integer into a 10 element binary vector
# with
# a 1 for the index of the class value, and 0 values for all other
# classes
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
```

8.6.5 One-Hot Encoding

`to_categorical()` converts a **class vector** (integers from 0 to the number of classes) to a **binary class matrix**, for use of **categorical crossentropy**.

Example: assuming the labeled dataset has a total of six classes (0 to 5), and `y_train` is the true label array:

```
y_train = [1, 0, 3, 4, 5, 0, 2, 1]
to_categorical(y_train, num_classes=6)
# Output
# array([[ 0., 1., 0., 0., 0., 0.],
#        [ 1., 0., 0., 0., 0., 0.],
#        [ 0., 0., 0., 1., 0., 0.],
#        [ 0., 0., 0., 0., 1., 0.],
#        [ 0., 0., 0., 0., 0., 1.],
#        [ 1., 0., 0., 0., 0., 0.],
#        [ 0., 0., 1., 0., 0., 0.],
#        [ 0., 1., 0., 0., 0., 0.]])
```

When to use one-hot encoding? In a situation where **data has no relation to each other**.

- The order of integers would be treated as a significant characteristic by a machine learning algorithm. In other words, a large number would be interpreted as better or more significant than a smaller number.
- While this is useful in some **ordinal** scenarios, certain input data lacks a ranking for categorical values, which can cause problems with predictions and performance.
- That is why we use one-hot encoding.

Benefits of One-Hot Encoding

Training data is **more usable and expressive** as a result of one-hot encoding, and it can be **re-scaled easily**.

Question: Why is One-Hot Encoding Applied to Labels?

Because the class labels are **categorical**, not ordinal: the digit label 9 is not "larger" or "better" than the digit label 3. One-hot encoding stops the model from treating the integer labels as an ordered quantity and matches the **categorical crossentropy** loss used for multi-class classification.

8.6.6 Step 3: Build the Model

Instead of building the model from scratch, we will use the software library **Keras**.

Layers:

1. **Layer 1:** Convolutional layer with 32 kernels of size 3×3 and ReLU activation (input size: $28 \times 28 \times 1$).
2. **Layer 2:** Convolutional layer with 64 kernels of size 3×3 and ReLU activation.
3. **Layer 3:** Max Pooling layer with a 2×2 kernel.
4. **Layer 4:** Dropout layer with a rate 0.25. It randomly sets input units to 0 with a frequency of 0.25 at each step during training time, which helps prevent overfitting. Inputs not set to 0 are scaled up by $1/(1 - \text{rate})$ such that the sum over all inputs is unchanged.
5. **Layer 5:** Flatten layer that will flatten a 2D image into 1D.
6. **Layer 6:** Dense layer with 128 neurons and ReLU activation.
7. **Layer 7:** Dropout layer with a rate 0.5.
8. **Layer 8:** Dense layer with 10 neurons (1 per class) and Softmax activation.

```
model = Sequential() # Create a Sequential object
# Add a convolutional layer with 32 kernels, each of size 3x3
# Use ReLU activation function, padding="valid", strides=(1,1)
# Specify the input size to this convolutional layer: (28,28,1)
# Note: Input size needs to be specified for the first layer only
model.add(Conv2D(filters=32, kernel_size=(3, 3), activation='relu',
   input_shape=(28,28,1)))
# Add another convolutional layer with 64 kernels, each of size 3x3
# Use ReLU activation function, padding="valid", strides=(1,1)
model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
# Add a max pooling layer of size 2 x 2
model.add(MaxPooling2D(pool_size=(2, 2)))
# Add a dropout layer to prevent a model from overfitting
model.add(Dropout(0.25))
# Add a flatten layer to convert the pooled data to a single column
# that is passed to the fully-connected layer
model.add(Flatten())
# Add a dense layer (fully-connected layer) and use ReLU activation
# function
model.add(Dense(units=128, activation='relu'))
# Add a dropout layer to prevent a model from overfitting
model.add(Dropout(0.5))
# Add a dense layer (fully-connected layer) and use Softmax activation
# function
model.add(Dense(units=num_classes, activation='softmax'))
```

8.6.7 Model Summary

Print the model summary with `model.summary()`:

```
 _________________________________________________________________
Layer (type)                 Output Shape              Param #
 =================================================================
conv2d (Conv2D)              (None, 26, 26, 32)        320
conv2d_1 (Conv2D)            (None, 24, 24, 64)        18496
max_pooling2d (MaxPooling2D) (None, 12, 12, 64)        0
dropout (Dropout)            (None, 12, 12, 64)        0
flatten (Flatten)            (None, 9216)              0
dense (Dense)                (None, 128)               1179776
dropout_1 (Dropout)          (None, 128)               0
dense_1 (Dense)              (None, 10)                1290
 =================================================================
Total params: 1,199,882
Trainable params: 1,199,882
Non-trainable params: 0
 _________________________________________________________________
```

Question: How to calculate the number of parameters in a conv layer?

Number of Parameters in a Convolutional Layer

With weight sharing, a convolutional layer with C_{out} kernels of size $K_h \times K_w$ operating on C_{in} input channels has

$$\#\text{weights} = (K_h\, K_w\, C_{\text{in}})\, C_{\text{out}}, \qquad \#\text{biases} = C_{\text{out}},$$

so that the total number of trainable parameters is

$$\#\text{param} = (K_h\, K_w\, C_{\text{in}} + 1)\, C_{\text{out}}.$$

(There is one bias per kernel; the same kernel is reused at every spatial position.)

Verifying every layer of the model summary:

- `conv2d`: output spatial size $(28-3)/1+1 = 26$, so $(26, 26, 32)$; parameters $(3 \cdot 3 \cdot 1 + 1) \cdot 32 = 320$.
- `conv2d_1`: output spatial size $(26-3)/1+1 = 24$, so $(24, 24, 64)$; parameters $(3 \cdot 3 \cdot 32 + 1) \cdot 64 = 18496$.
- `max_pooling2d`: 2×2 pooling halves each spatial dimension, so $(12, 12, 64)$; 0 parameters.
- `dropout`: same shape $(12, 12, 64)$; 0 parameters.
- `flatten`: $12 \cdot 12 \cdot 64 = 9216$; 0 parameters.
- `dense`: $9216 \cdot 128 + 128 = 1179776$ parameters.
- `dense_1`: $128 \cdot 10 + 10 = 1290$ parameters.
- **Total:** $320 + 18496 + 1179776 + 1290 = 1{,}199{,}882$ trainable parameters.

Plot the model:

```
plot_model(model, show_shapes=True, show_layer_names=True)
```

8.6.8 Step 4: Compile the Model

```
# Compile the model, i.e., configures the model for training
# Use crossentropy loss function since there are two or more label
# classes.
# Use adam algorithm (a stochastic gradient descent method)
# Use accuracy as metric, i.e., report on accuracy
model.compile(optimizer='adam', # Default learning rate is 0.001
              loss= categorical_crossentropy ,
              metrics=['accuracy'])
```

8.6.9 Step 5: Train the Model

```
# Create TensorBoard object to track experiment metrics like loss and
# accuracy, visualizing the model graph, etc.
log_dir=".logs/fit/" +
   datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = TensorBoard(log_dir=log_dir, histogram_freq=1)

# Fit the model, i.e., train the model
# Specify training data and labels
# Specify batch size, i.e., number of samples per gradient update
# Specify validation data, i.e., data on which to evaluate the loss
# Write TensorBoard logs after every batch of training to monitor our
# metrics
training_history = model.fit(x_train, y_train,
                             batch_size=batch_size,
                             epochs=epochs,
                             validation_data=(x_test, y_test),
                             callbacks=[tensorboard_callback])
```

8.6.10 Step 6: Evaluate the Model Accuracy

```
# Evaluate the model
# Specify test data and labels
# Set verbose to 0, i.e., silent (no progress bar)
validation_loss, validation_accuracy = model.evaluate(x_test, y_test,
   verbose=0)
# Print loss and accuracy
print('Validation loss:', validation_loss)
print('Validation accuracy:', validation_accuracy)
```

Output:

```
Validation loss: 0.07229845970869064
Validation accuracy: 0.9776999950408936
```

8.6.11 Step 7: Save the Model

Save the entire model to an **HDFS (Hadoop Distributed File System)** file. The `.h5` extension of the file indicates that the model should be saved in Keras format as an HDFS file.

```
model_name = 'digits_recognition_cnn.h5'
model.save(model_name, save_format='h5')
loaded_model = load_model(model_name)
```

8.6.12 Step 8: Use the Model

To use the model, we call the `predict()` function.

```
# Get image index from user
image_index = (int)(input("Enter an image index: "))
# Show the image in greyscale
plt.imshow(x_test[image_index].reshape(28,28),cmap='Greys')
# Use the model to do prediction by specifying the image.
# Get back a numpy array of prediction
prediction =
   loaded_model.predict(x_test[image_index].reshape(1,28,28,1))
# Print the predicted result, i.e., the one with maximum value
print('Predicted result:', prediction.argmax())
```

Output:

```
Enter an image index: 10          Enter an image index: 20
   Enter an image index: 30
Predicted result: 0               Predicted result: 9
   Predicted result: 3
```

8.7 Typical Architecture

Typical CNN models for classification tasks stack different types of layers as follows:

1. A few **convolutional layers** (usually with ReLU activation).
2. Then, a **pooling layer**.
3. Then, a few **convolutional layers** (usually with ReLU activation) and a **pooling layer**, and so on.
4. Then, one or more **fully-connected layers** (usually with ReLU activation).
5. Then, a final fully-connected **softmax** layer.

- Generally, the convolutional and pooling layers are for **feature learning**, and the fully-connected layers are for **classification**.
- Going through the convolutional layers, the feature maps typically get **smaller but deeper** (i.e., more feature maps per layer).
- The fully-connected layers form an **ordinary feedforward neural network** to which the transformed inputs are fed.

Prefer Stacked Small Kernels over One Large Kernel

Instead of using a convolutional layer with a **large kernel size** (e.g., 9×9 with 81 parameters), it is better to **stack two convolutional layers with smaller kernels** (e.g., 3×3), with $2 \times 9 = 18$ parameters for the two layers together.

8.8 Practice

8.8.1 Problem 1: Convolving an Image by Hand

Convolution with Zero Padding

Given a 5×5 input image

$$\begin{bmatrix} 5 & 1 & 3 & 7 & 8 \\ 2 & 4 & 6 & 1 & 7 \\ 3 & 4 & 9 & 7 & 2 \\ 4 & 9 & 4 & 5 & 9 \\ 7 & 9 & 8 & 7 & 9 \end{bmatrix} \quad \text{and the } 3 \times 3 \text{ kernel} \quad \begin{bmatrix} 3 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -3 \end{bmatrix},$$

estimate the output of **convolving** the input image with the kernel. Assume **zero-padding with a 1-pixel border** is performed.
Remark: Don't forget to **flip the kernel**.

Solution.

Step 1: Flip the kernel. Convolution flips the kernel (both horizontally and vertically, i.e., a 180° rotation) before sliding it over the image:

$$K = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -3 \end{bmatrix} \xrightarrow{\text{flip}} \tilde{K} = \begin{bmatrix} -3 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

Step 2: Pad the image. With a 1-pixel zero border, the 5×5 image becomes 7×7:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 5 & 1 & 3 & 7 & 8 & 0 \\ 0 & 2 & 4 & 6 & 1 & 7 & 0 \\ 0 & 3 & 4 & 9 & 7 & 2 & 0 \\ 0 & 4 & 9 & 4 & 5 & 9 & 0 \\ 0 & 7 & 9 & 8 & 7 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Step 3: Convolve (stride 1). The output size is $(5 + 2 - 3)/1 + 1 = 5$, i.e., 5×5. Since the flipped kernel has only two non-zero entries, each output pixel depends only on the top-left entry x_{TL} and the bottom-right entry x_{BR} of the corresponding 3×3 window of the padded image:

$$\text{output} = (-3) \cdot x_{\text{TL}} + 3 \cdot x_{\text{BR}}.$$

Sample computations:

- Position $(1, 1)$: $x_{\text{TL}} = 0$ (padding), $x_{\text{BR}} = 4$: $(-3)(0) + (3)(4) = 12$.
- Position $(1, 2)$: $x_{\text{TL}} = 0$, $x_{\text{BR}} = 6$: $(-3)(0) + (3)(6) = 18$.
- Position $(2, 1)$: $x_{\text{TL}} = 0$, $x_{\text{BR}} = 4$: $(-3)(0) + (3)(4) = 12$.
- Position $(2, 4)$: $x_{\text{TL}} = 3$, $x_{\text{BR}} = 2$: $(-3)(3) + (3)(2) = -3$.
- Position $(4, 1)$: $x_{\text{TL}} = 0$, $x_{\text{BR}} = 9$: $(-3)(0) + (3)(9) = 27$.

- Position $(5,5)$: $x_{\text{TL}} = 5$, $x_{\text{BR}} = 0$: $(-3)(5) + (3)(0) = -15$.

Answer:

$$\begin{bmatrix} 12 & 18 & 3 & 21 & 0 \\ 12 & 12 & 18 & -3 & -21 \\ 27 & 6 & 3 & 9 & -3 \\ 27 & 15 & 9 & 0 & -21 \\ 0 & -12 & -27 & -12 & -15 \end{bmatrix}$$

8.8.2 Problem 2: Output Dimensions and Number of Weights

Analyzing a CNN Architecture

Suppose we have a Convolutional Neural Network (CNN):

- **Input layer:** an input color image of size $128 \times 128 \times 3$.
- **First convolutional layer:** 5 different kernels, each of size $3 \times 3 \times 3$ (no zero padding and stride of size 5).
- **First max-pooling layer:** size 2×2.
- **Second convolutional layer:** 3 different kernels, each of size $5 \times 5 \times 5$ (zero padding of size 2 and stride of size 2).
- ...

1. What is the dimension of the output of the first convolutional layer, and how many weights must be learned for this layer?
2. What is the main purpose of doing pooling during training in a deep neural network? Also, what is the dimension of the output of the first pooling layer?
3. What is the dimension of the output of the second convolutional layer, and how many weights must be learned for this layer?

Solution.

Part 1: First convolutional layer. Input $128 \times 128 \times 3$, kernel $3 \times 3 \times 3$, no padding, stride 5:

$$\text{Output size} = \frac{128 - 3}{5} + 1 = 25 + 1 = 26 \quad \Longrightarrow \quad \text{output dimension} = 26 \times 26 \times 5.$$

Each unit in the convolutional layer is connected to $5 \times 3 \times 3 \times 3 = 135$ units (5 kernels, each contributing $3 \times 3 \times 3$ weights), and these weights are **shared** across all units in this layer. In formula form, with $K_h = K_w = 3$, $C_{\text{in}} = 3$ and $C_{\text{out}} = 5$,

$$\#\text{weights} = (K_h K_w C_{\text{in}})\, C_{\text{out}} = (3 \cdot 3 \cdot 3) \cdot 5 = \mathbf{135}.$$

Part 2: First max-pooling layer. The main purpose of doing pooling during training is to make sure that the **subsequent layers of the CNN can pick up larger-scale detail**. A 2×2 pooling halves each spatial dimension and keeps the depth:

$$\frac{26}{2} = 13 \quad \Longrightarrow \quad \text{output dimension} = 13 \times 13 \times 5.$$

Part 3: Second convolutional layer. Input $13 \times 13 \times 5$, kernel $5 \times 5 \times 5$, zero padding of size 2 (so each spatial dimension grows by $2 \times 2 = 4$), stride 2:

$$\text{Output size} = \frac{(13 + 4) - 5}{2} + 1 = \frac{12}{2} + 1 = 7 \quad \Longrightarrow \quad \text{output dimension} = 7 \times 7 \times 3.$$

Each unit in the convolutional layer is connected to $3 \times 5 \times 5 \times 5 = 375$ units (3 kernels, each contributing $5 \times 5 \times 5$ weights), and these weights are **shared** across all units in this layer. In formula form, with $K_h = K_w = 5$, $C_{\text{in}} = 5$ and $C_{\text{out}} = 3$,

$$\#\text{weights} = (K_h K_w C_{\text{in}})\, C_{\text{out}} = (5 \cdot 5 \cdot 5) \cdot 3 = \mathbf{375}.$$

9 PyTorch

9.1 PyTorch and CUDA

9.1.1 Introduction

PyTorch is an open-source machine learning library developed in Python, C++, and CUDA. It offers two key features:

1. **Tensor computing** (similar to NumPy) with enhanced performance using **Graphics Processing Units (GPUs)**.
2. A **deep learning framework** built on a **dynamic automatic differentiation** system.

PyTorch is ideal for both **research** and **production**.

What is PyTorch?

In summary, PyTorch is an advanced version of **NumPy** that leverages **GPU acceleration** and includes extensive **machine learning capabilities**.

9.1.2 Common Use Cases

- **Image and video processing** (e.g., computer vision tasks).
- **Natural language processing** (e.g., text classification, translation).
- **Reinforcement learning** (e.g., training agents in simulated environments).
- **Scientific computing and research** (e.g., simulations, data analysis).

9.1.3 Advantages of PyTorch

- Easy to learn and use with a **Pythonic interface**.
- Strong **community support** and extensive documentation.
- Seamless integration with Python libraries (e.g., NumPy, SciPy).
- **Dynamic computation graphs** allow for flexible model building. In machine learning or neural networks, a **graph** is made up of **nodes** (which are like tasks or values) and **edges** (which are the connections between these tasks or values).
- Extensive **pre-trained models** available through libraries like `torchvision` and **Hugging Face** (a company and online platform that focuses on advancing NLP and AI technologies).

9.1.4 CUDA

CUDA

CUDA stands for **Compute Unified Device Architecture**, a parallel computing platform and API (Application Programming Interface) developed by **Nvidia**.

- It enables **parallel computation**, significantly improving processing speed.
- With CUDA, we can leverage the power of **Nvidia GPUs** to perform common computing tasks, such as **matrix processing** and other **linear algebra operations** — for instance the matrix product $C = AB$, whose entries $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$ are all evaluated in parallel across thousands of GPU cores.

9.2 Tensors

9.2.1 Tensors

Tensor

A **tensor** can be thought of as a **multidimensional array**. A rank-k tensor is written $\mathcal{T} \in \mathbb{R}^{d_1 \times \cdots \times d_k}$ with entries $T[i_1, \ldots, i_k]$, where $0 \leq i_m < d_m$ for each mode m.

- A **0-D tensor** is a **scalar** $x \in \mathbb{R}$.
- A **1-D tensor** is a **vector** $\mathbf{x} \in \mathbb{R}^n$.
- A **2-D tensor** is a **matrix** $X \in \mathbb{R}^{m \times n}$ with entries x_{ij}.
- Higher dimensions (rank $k \geq 3$) can represent more complex data structures, e.g., a batch of RGB images $\mathcal{T} \in \mathbb{R}^{b \times 3 \times d_1 \times d_2}$.

A PyTorch tensor is similar to a NumPy `ndarray` but includes additional functionalities for **GPU acceleration**.

9.2.2 Key Attributes of Tensors

A PyTorch tensor has three key attributes:

1. **Shape** (the size of the tensor)
2. **Data type** (the type of data contained in the tensor)
3. **Device** (the location where the tensor is stored)

```
import torch

tensor = torch.tensor([1, 3, 5])  # 1-D tensor
print(f"Shape of tensor: {tensor.shape}")
print(f"Data type of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")

# Output:
# Shape of tensor: torch.Size([3])
# Data type of tensor: torch.int64
# Device tensor is stored on: cpu
```

The reported shape `torch.Size([3])` means the tensor is a vector $\mathbf{x} \in \mathbb{R}^3$: it has rank 1 and 3 entries. In general, a tensor whose shape is $(d_1, \ldots, d_k)$ is an element of $\mathbb{R}^{d_1 \times \cdots \times d_k}$.

The **data type** (dtype) fixes the numeric domain of each entry. Integer types store exact values in an interval: `torch.int8` covers $[-2^7, 2^7-1]$, `torch.uint8` covers $[0, 2^8-1]$, and `torch.int64` covers $[-2^{63}, 2^{63}-1]$. Floating-point types such as `torch.float32` and `torch.float64` instead approximate $\mathbb{R}$ with 32 or 64 bits of precision.

9.2.3 Initializing a Tensor

Tensors can be initialized in various ways:

Created directly from data, with the data type automatically inferred:

```
data = [[1, 2], [3, 4]]
tensor_data = torch.tensor(data)
```

Here `data` represents the matrix $X \in \mathbb{R}^{2 \times 2}$ with entries $x_{11} = 1$, $x_{12} = 2$, $x_{21} = 3$, and $x_{22} = 4$.

Created from NumPy arrays (and vice versa):

```
import numpy as np

numpy_array = np.array(data)
tensor_fr_np = torch.from_numpy(numpy_array)
```

Created from another tensor. The new tensor retains the properties (shape, data type) of the original tensor, unless explicitly overridden:

```
# Retains the properties of tensor_data
tensor_ones = torch.ones_like(tensor_data)
print(f"Ones Tensor: \n {tensor_ones} \n")

# Overrides the data type of tensor_data
tensor_rand = torch.rand_like(tensor_data, dtype=torch.float)
print(f"Random Tensor: \n {tensor_rand} \n")
```

9.2.4 Random and Constant Initialization

Random initialization of tensors can be controlled to produce the same random outcome each time:

```
import torch

torch.manual_seed(seed=42)  # Set the seed for reproducibility
tensor_shape = (2, 3)
rand_tensor = torch.rand(tensor_shape)    # Random tensor
ones_tensor = torch.ones(tensor_shape)    # Tensor of ones
zeros_tensor = torch.zeros(tensor_shape)  # Tensor of zeros
scalar_tensor = torch.tensor(22)          # Scalar tensor

print(f"Random Tensor: \n {rand_tensor} \n")
print(f"Ones Tensor: \n {ones_tensor} \n")
print(f"Zeros Tensor: \n {zeros_tensor} \n")
print(f"Scalar Tensor: \n {scalar_tensor.item()} \n")
```

Here `tensor_shape` $= (2, 3)$ means `rand_tensor`, `ones_tensor`, and `zeros_tensor` are all matrices in $\mathbb{R}^{2\times 3}$ whose entries are respectively sampled from $[0, 1)$, set to 1, and set to 0. `scalar_tensor` is the scalar $x = 22 \in \mathbb{R}$.

Reproducibility

Use `torch.manual_seed(seed=42)` to set the seed so that random tensor initialization produces the same random outcome each time — essential for reproducible experiments.

9.2.5 Operations on Tensors

PyTorch offers over **1200 tensor operations**, including arithmetic, linear algebra, matrix manipulation (such as transposing, indexing, and slicing), sampling, and more. For a complete list, visit: `https://docs.pytorch.org/docs/stable/torch.html`

These operations can be performed on both the **CPU** and **accelerators** like CUDA.

Using GPUs in Google Colab

In Google Colab, you can allocate an accelerator by navigating to: **Runtime** > **Change runtime type** > **GPU**. Check whether a tensor `my_tensor` is on the GPU or CPU by printing its device:

```
print(my_tensor.device)
```

9.2.6 Moving Tensors to the Accelerator

By default, tensors are created on the **CPU**. To utilize an accelerator, you must explicitly move tensors using the `.to` method (after confirming accelerator availability).

```
import torch

tensor = torch.ones(4, 4)
# Move our tensor to the current accelerator if available
if torch.cuda.is_available():
    tensor = tensor.to(torch.cuda.current_device())

print(f"First row: {tensor[0]}")
print(f"First column: {tensor[:, 0]}")
print(f"Last column: {tensor[..., -1]}")
tensor[:, 1] = 0
print(tensor)

# torch.cat is used to concatenate a sequence of tensors
# along a specified dimension
t1 = torch.cat([tensor, tensor, tensor], dim=1)
```

Here `tensor` $= X \in \mathbb{R}^{4\times4}$, so `tensor[0]` is its first row, `tensor[:, 0]` is its first column, and `tensor[..., -1]` its last column. The assignment `tensor[:, 1] = 0` zeroes the second column. Concatenating three copies along `dim=1` stacks their columns, giving the augmented matrix $t_1 = [\,X \mid X \mid X\,] \in \mathbb{R}^{4\times12}$.

Cost of Moving Tensors Between Devices

Copying large tensors between devices (CPU $\leftrightarrow$ GPU) can be **costly** in terms of time and memory! Minimize device transfers in performance-critical code.

9.2.7 Matrix Multiplication

The following example demonstrates matrix multiplication between two tensors. The variables `y1`, `y2`, and `y3` will have the same value. The expression `tensor.T` returns the **transpose** of a tensor.

```
import torch

tensor = torch.ones(4, 4)
# Move our tensor to the current accelerator if available
if torch.cuda.is_available():
    tensor = tensor.to(torch.cuda.current_device())
tensor[:, 1] = 0

# Matrix multiplication using the @ operator
y1 = tensor @ tensor.T
# Matrix multiplication using the matmul method
y2 = tensor.matmul(tensor.T)
y3 = torch.rand_like(y1)
# Matrix multiplication with output argument
torch.matmul(tensor, tensor.T, out=y3)

# This computes the element-wise product.
# z1, z2, and z3 will have the same value.
# Element-wise multiplication using the * operator
z1 = tensor * tensor
# Element-wise multiplication using the mul method
z2 = tensor.mul(tensor)
z3 = torch.rand_like(tensor)
# Element-wise multiplication with output argument
torch.mul(tensor, tensor, out=z3)
```

Writing the 4×4 tensor as X, the transpose satisfies $(X^\top)_{ij} = x_{ji}$, and the matrix

product is

$$(XX^\top)_{ij} = \sum_{k=1}^{4} x_{ik}\,(X^\top)_{kj} = \sum_{k=1}^{4} x_{ik}\,x_{jk}.$$

In contrast, element-wise multiplication gives $Z = X \odot X$ with entries $z_{ij} = x_{ij}^2$.

Matrix Multiplication vs. Element-wise Multiplication

`@` (or `matmul`) performs **matrix multiplication**, while `*` (or `mul`) performs **element-wise multiplication**. They are completely different operations — do not confuse them!

9.2.8 Single-element Tensors

If we have a one-element tensor, for example, by aggregating all values of a tensor into a single value, we can convert it to a Python numerical value using `item()`.

```
import torch

tensor = torch.ones(4, 4)
# Move our tensor to the current accelerator if available
if torch.cuda.is_available():
    tensor = tensor.to(torch.cuda.current_device())
tensor[:, 1] = 0

agg = tensor.sum()         # Aggregate the tensor values
agg_item = agg.item()      # Convert to a Python numerical value
print(agg_item, type(agg_item))  # Print the value and its type
```

For $X \in \mathbb{R}^{4\times4}$, the aggregation `tensor.sum()` computes the scalar $\sum_{i=1}^{4}\sum_{j=1}^{4} x_{ij}$, which `item()` then returns as a Python float.

9.2.9 In-place Operations

Operations that store the result directly into the operand are called **in-place operations**. They are denoted by a `_` suffix. For example: `x.copy_(y)` and `x.t_()` will modify `x`.

```
import torch

tensor = torch.ones(4, 4)
# Move our tensor to the current accelerator if available
if torch.cuda.is_available():
    tensor = tensor.to(torch.cuda.current_device())
tensor[:, 1] = 0
print(f"{tensor} \n")

tensor.add_(5)  # In-place addition of 5 to each element
print(tensor)
```

For $X \in \mathbb{R}^{4\times4}$, the in-place call `tensor.add_(5)` updates every entry via $x_{ij} \leftarrow x_{ij} + 5$, overwriting the original tensor.

9.2.10 Bridge with NumPy

Tensors on the CPU and NumPy arrays can **share the same underlying memory locations**, meaning that changes to one will affect the other.

```
import torch

my_tensor = torch.ones(5)
```

```
print(f"Tensor my_tensor: {my_tensor}")

np_array = my_tensor.numpy()  # Convert tensor to NumPy array
print(f"NumPy array np_array: {np_array}")

# In-place addition of 1 to each element of the tensor
my_tensor.add_(1)
print(f"Updated Tensor my_tensor: {my_tensor}")
# Reflects changes in tensor
print(f"Updated NumPy array np_array: {np_array}")
```

9.2.11 NumPy Bridge

- NumPy arrays can be converted to tensors using the `.from_numpy()` method.
- PyTorch tensors can be converted to NumPy arrays using the `tensor.numpy()` method.

GPU Tensors Cannot Be Converted Directly to NumPy

The PyTorch tensor must be on the **CPU** for the conversion to NumPy. If the tensor is on a GPU, transfer it to the CPU first using `.cpu()`.

```
import torch
import numpy as np

my_array = np.ones(5)                    # Create a NumPy array of ones
my_tensor = torch.from_numpy(my_array) # Convert NumPy array to tensor

# Changes in the NumPy array are reflected in the tensor
# In-place addition of 1 to the NumPy array
np.add(my_array, 1, out=my_array)
print(f"Tensor t: {my_tensor}")
print(f"NumPy array my_array: {my_array}")  # Shows updated values

another_array = my_tensor.numpy()      # Convert tensor to NumPy array
```

9.3 Example: Handwritten Digits

We will again build a **CNN model** to recognize and classify handwritten digits, but this time using **PyTorch**.

9.3.1 Dataset

We use the **Modified National Institute of Standards and Technology (MNIST)** dataset. This dataset contains two sets of samples:

- **Training data:** 60000 images of 28 pixel × 28 pixel handwritten digits from 0 to 9.
- **Testing data:** 10000 images of 28 pixel × 28 pixel.

Each grayscale image is a matrix $X \in \mathbb{R}^{28\times 28}$ with entries normalized to $[-1, 1]$, and a mini-batch of b such images forms the rank-4 tensor $\mathcal{T} \in \mathbb{R}^{b\times 1\times 28\times 28}$ (the extra dimension being the single color channel).

9.3.2 Procedures

1. Import required libraries and define global variables.
2. Load and prepare the dataset.
3. Build the model by defining a class.
4. Compile the model.
5. Train the model.

6. Evaluate the model accuracy.
7. Save the model.
8. Use the model.

9.3.3 Step 1: Setup

```
# PyTorch library for tensor operations and NNs
import torch
# Neural network modules (layers, loss functions)
import torch.nn as nn
# Functional interface for activations
import torch.nn.functional as F
# Utility for batch loading and shuffling data
from torch.utils.data import DataLoader
# Contains popular datasets and image transformations
import torchvision
# Image preprocessing transformations
from torchvision.transforms import transforms
# Generates evaluation metrics
from sklearn.metrics import classification_report
# Creates confusion matrix
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt  # For plotting images and charts
# For numerical operations and array handling
import numpy as np

BATCH_SIZE = 64        # Number of samples per gradient update
# Step size for optimizer during gradient descent
LEARNING_RATE = 0.001
# Number of complete passes through the entire training data
NUM_EPOCHS = 1

# Use GPU if available, otherwise CPU
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")
```

9.3.4 Step 2: Load and Prepare the Data

```
def load_prepare_data(batch_size):
    # Define preprocessing pipeline
    transform = transforms.Compose([
        # Convert image to tensor and scale to [0,1]
        transforms.ToTensor(),
        # Normalize: (x - 0.5) / 0.5 -> range [-1, 1]
        transforms.Normalize((0.5,), (0.5,))
    ])

    # Download and load MNIST training and test dataset
    # root: Storage directory, train: Training set flag,
    # download: Download if not present
    trainset = torchvision.datasets.MNIST(
        root='./data', train=True, download=True, transform=transform
    )
    testset = torchvision.datasets.MNIST(
        root='./data', train=False, download=True, transform=transform
    )

    # Create DataLoaders for batch processing
    # shuffle=True for training to prevent learning order patterns
    # shuffle=False for testing to maintain consistent evaluation
    trainloader = DataLoader(trainset, batch_size=batch_size,
   shuffle=True)
```

```
    testloader = DataLoader(testset, batch_size=batch_size,
   shuffle=False)
    return trainloader, testloader, trainset, testset

trainloader, testloader, trainset, testset =
   load_prepare_data(BATCH_SIZE)
```

Why Normalize to $[-1, 1]$?

`transforms.Normalize((0.5,), (0.5,))` applies $(x-0.5)/0.5$, mapping pixel values from $[0, 1]$ to $[-1, 1]$. Centered inputs typically help optimization converge faster and more stably.

9.3.5 Step 3: Define the Model

```
class MNIST_CNN(nn.Module):
    def __init__(self):
        """Initialize all neural network layers"""
        super(MNIST_CNN, self).__init__()

        # ========== Convolutional Layers (Feature Extraction)
        # ==========
        # Layer 1: Convolutional layer with 32 3x3 kernels and ReLU
        # activation
        # Input size: 28x28x1 (height, width, channels)
        # Output size: 28x28x32 (padding='same' preserves spatial
        # dimensions)
        # in_channels=1: Grayscale input (1 color channel)
        # out_channels=32: Number of filters/feature maps
        # kernel_size=3: 3x3 filter size
        # stride=1: Move filter 1 pixel at a time
        # padding=1: Add zeros around border to maintain spatial size
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32,
   kernel_size=3,
                               stride=1, padding=1)

        # Layer 2: Convolutional layer with 64 3x3 kernels and ReLU
        # activation
        # Input size: 28x28x32 (from previous layer)
        # Output size: 28x28x64
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,
   kernel_size=3,
                               stride=1, padding=1)

        # ========== Pooling Layer ==========
        # Layer 3: Max Pooling layer with 2x2 kernel
        # Reduces spatial dimensions by taking maximum value in each
        # 2x2 window
        # Input size: 28x28x64 -> Output size: 14x14x64 (halves height
        # and width)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # ========== Regularization Layers ==========
        # Layer 4: Dropout layer with rate 0.25
        self.dropout1 = nn.Dropout(0.25)

        # ========== Fully Connected Layers (Classification)
        # ==========
        # After convolution + pooling: 14x14x64 = 12,544 features
        # This will be flattened to 1D vector before the first dense
        # layer
        # Layer 6: Dense layer with 128 neurons and ReLU activation
```

```
        # Input: 12,544 flattened features -> Output: 128 features
        self.fc1 = nn.Linear(64 * 14 * 14, 128)   # 64*14*14 = 12,544

        # Layer 7: Dropout layer with rate 0.5
        self.dropout2 = nn.Dropout(0.5)

        # Layer 8: Output dense layer with 10 neurons (1 per digit
        # class)
        # Input: 128 features -> Output: 10 logits (raw scores for
        # each class)
        # Softmax activation will be applied during inference to
        # convert
        # to probabilities
        self.fc2 = nn.Linear(128, 10)
```

The `forward` method defines how data flows through the network:

```
    def forward(self, x):
        # Layer 1: First convolutional layer with ReLU activation
        # Output shape: (batch, 32, 28, 28)
        x = F.relu(self.conv1(x))

        # Layer 2: Second convolutional layer with ReLU activation
        # Output shape: (batch, 64, 28, 28)
        x = F.relu(self.conv2(x))

        # Layer 3: Max Pooling to reduce spatial dimensions
        # Output shape: (batch, 64, 14, 14)
        x = self.pool(x)

        # Layer 4: Dropout for regularization
        # Output shape: (batch, 64, 14, 14)
        x = self.dropout1(x)

        # Layer 5: Flatten operation
        # view() reshapes tensor: (batch, 64, 14, 14) -> (batch,
        # 64*14*14)
        x = x.view(x.size(0), -1)   # Output shape: (batch, 12544)

        # Layer 6: First fully connected layer with ReLU activation
        x = F.relu(self.fc1(x))     # Output shape: (batch, 128)

        # Layer 7: Dropout for regularization (higher rate for dense
        # layers)
        x = self.dropout2(x)        # Output shape: (batch, 128)

        # Layer 8: Output layer (no activation - raw logits)
        # Produces class scores before softmax
        x = self.fc2(x)             # Output shape: (batch, 10)

        # Note: Softmax is NOT applied here
        return x

# Initialize the model and move to appropriate device (GPU/CPU)
model = MNIST_CNN().to(DEVICE)
```

Model Architecture Summary

The CNN processes a $1 \times 28 \times 28$ grayscale image as follows:

$$1 \times 28 \times 28 \xrightarrow{\text{conv1}} 32 \times 28 \times 28 \xrightarrow{\text{conv2}} 64 \times 28 \times 28 \xrightarrow{\text{pool}} 64 \times 14 \times 14 \xrightarrow{\text{flatten}} 12544 \xrightarrow{\text{fc1}} 128 \xrightarrow{\text{fc2}} 10$$

Dropout (rates 0.25 and 0.5) is applied for regularization, and ReLU is used as the

activation function throughout.

No Softmax in `forward`

Softmax is NOT applied in the `forward` method. `nn.CrossEntropyLoss()` already combines `LogSoftmax` and negative log-likelihood internally, so applying softmax in the model would be redundant and incorrect. Softmax is only applied explicitly during inference to obtain interpretable probabilities.

9.3.6 Step 4: Compile the Model

```
# Loss Function: Cross Entropy Loss
# Suitable for multi-class classification problems
lossfunc = nn.CrossEntropyLoss()

# Optimizer: Adam (Adaptive Moment Estimation)
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

9.3.7 Step 5: Train the Model

```
def train_model(model, trainloader, lossfunc, optimizer, device,
   num_epochs):
    model.train()   # Set model to training mode (dropout active)
    for epoch in range(num_epochs):
        # Inner loop: iterate through batches
        for i, (inputs, labels) in enumerate(trainloader):
            # Move data to GPU/CPU (GPU significantly speeds up
            # training)
            inputs, labels = inputs.to(device), labels.to(device)

            # Reset gradients from previous iteration
            # PyTorch accumulates gradients by default
            optimizer.zero_grad()

            # Forward pass: compute predictions
            outputs = model(inputs)     # Raw logits output

            # Calculate loss between predictions and true labels
            loss = lossfunc(outputs, labels)

            # Backward pass: compute gradients
            # Uses automatic differentiation
            loss.backward()

            # Optimizer step: update model parameters
            optimizer.step()            # Gradient descent step

# Execute training
train_model(model, trainloader, lossfunc, optimizer, DEVICE,
   NUM_EPOCHS)
```

The PyTorch Training Loop

Each training iteration consists of five canonical steps:
1. `optimizer.zero_grad()` — reset accumulated gradients.
2. `outputs = model(inputs)` — forward pass.
3. `loss = lossfunc(outputs, labels)` — compute the loss.
4. `loss.backward()` — backward pass (automatic differentiation).

5. `optimizer.step()` — update the parameters.

In symbols, writing the model parameters as $\mathbf{w}$ and the loss as L, the backward pass evaluates the gradient $\nabla_{\mathbf{w}} L$, and `optimizer.step()` applies the gradient-descent update

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \, \nabla_{\mathbf{w}} L,$$

where η is the learning rate.

Forgetting `zero_grad()`

PyTorch **accumulates** gradients by default. If you forget `optimizer.zero_grad()`, gradients from previous iterations will be added to the current ones, corrupting the update.

9.3.8 Step 6: Evaluate the Model Accuracy

```
def evaluate_model(model, testloader, device):
    model.eval()    # Set to evaluation mode (dropout disabled)
    all_targets = []            # Store all true labels
    # Predicted classes and probabilities
    all_predictions, all_probabilities = [], []
    # Count correct predictions, Total predictions
    correct, total = 0, 0

    # Disable gradient computation for evaluation (memory efficient)
    with torch.no_grad():
        for inputs, labels in testloader:
            inputs, labels = inputs.to(device), labels.to(device)
            # Forward pass to get predictions
            outputs = model(inputs)

            # Apply softmax to convert logits to probabilities
            # dim=1 applies softmax across class dimension
            probabilities = F.softmax(outputs, dim=1)
            _, predicted = torch.max(outputs, 1)  # Predicted class

            # Update accuracy counters
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

            # Store results for metrics calculation
            # Move to CPU for numpy
            all_targets.extend(labels.cpu().numpy())
            all_predictions.extend(predicted.cpu().numpy())
            all_probabilities.extend(probabilities.cpu().numpy())

    # Calculate overall accuracy percentage
    accuracy = 100 * correct / total
    print(f"Test Accuracy: {accuracy:.2f}%")
    print("-" * 50)

    # Print confusion matrix: rows=true labels, cols=predicted labels
    # Diagonal elements are correct predictions
    print("\nConfusion Matrix:")
    print(confusion_matrix(all_targets, all_predictions))

    # Print classification report with per-class metrics
    # Includes: precision, recall, f1-score, support
    print("\nClassification Report:")
    print(classification_report(all_targets, all_predictions))

    return accuracy, all_probabilities
```

```
# Execute evaluation
test_accuracy, test_probabilities = evaluate_model(model, testloader,
    DEVICE)
```

For each image the softmax layer converts the ten logits $z_0, \ldots, z_9$ into probabilities $p_c = \dfrac{e^{z_c}}{\sum_{c'=0}^{9} e^{z_{c'}}}$ (computed along `dim=1`), and the predicted class is $\arg\max_c p_c$. The reported accuracy is $\text{acc} = 100 \cdot \frac{\text{correct}}{\text{total}}\%$. The confusion matrix $C \in \mathbb{N}^{10\times 10}$ has entry C_{ij} equal to the number of test images whose true class is i and predicted class is j, so its diagonal sums to the number of correct predictions, $\sum_{i=0}^{9} C_{ii} = \text{correct}$.

`model.train()` vs. `model.eval()`

Always call `model.train()` before training and `model.eval()` before evaluation. Layers like **dropout** behave differently in the two modes: dropout is active in training mode but disabled in evaluation mode. Wrap evaluation code in `torch.no_grad()` to save memory.

Example Output

Running the code produces output similar to the following (test accuracy 98.55%):

```
Using device: cpu
Test Accuracy: 98.55%
 ----------------------------------------------------

Confusion Matrix:
[[ 970    1    3    0    0    0    2    1    1    2]
 [   0 1132    3    0    0    0    0    0    0    0]
 [   1    5 1020    0    1    0    0    4    1    0]
 [   0    0    5  989    0    3    0    6    6    1]
 [   1    0    1    0  972    0    2    0    2    4]
 [   2    0    0    5    0  880    2    1    2    0]
 [   8    3    0    1    5    2  937    0    2    0]
 [   0    1   10    1    1    0    0 1014    1    0]
 [   5    1    3    1    2    0    0    1  959    2]
 [   4    0    0    1    5    3    0    9    5  982]]

Classification Report:
              precision    recall  f1-score   support
           0       0.98      0.99      0.98       980
           1       0.99      1.00      0.99      1135
           2       0.98      0.99      0.98      1032
           3       0.99      0.98      0.99      1010
           4       0.99      0.99      0.99       982
           5       0.99      0.99      0.99       892
           6       0.99      0.98      0.99       958
           7       0.98      0.99      0.98      1028
           8       0.98      0.98      0.98       974
           9       0.99      0.97      0.98      1009
    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000

Model saved to mnist_cnn_model.pth
```

9.3.9 Step 7: Save the Model

```
def save_model(model, path, test_accuracy):
    # Create checkpoint dictionary with model and metadata
```

```
    torch.save({
        # Learnable parameters
        'model_state_dict': model.state_dict(),
        # String representation of architecture
        'model_architecture': str(model),
        'test_accuracy': test_accuracy,           # Performance metric
        # Human-readable configuration
        'model_config': {
            'conv1': '32 filters, 3x3 kernel, ReLU',
            'conv2': '64 filters, 3x3 kernel, ReLU',
            'pool': 'MaxPooling 2x2',
            'dropout1': '0.25',
            'flatten': 'Yes',
            'fc1': '128 neurons, ReLU',
            'dropout2': '0.5',
            'fc2': '10 neurons, Softmax'
        }
    }, path)
    print(f"\nModel saved to {path}")

# Save the trained model to file
save_model(model, 'mnist_cnn_model.pth', test_accuracy)
```

9.3.10 Step 8: Use the Model

```
def use_model_for_prediction(model_path, image_tensor, device):
    # Load model
    model = MNIST_CNN().to(device)
    model.load_state_dict(torch.load(model_path,
   map_location=device)['model_state_dict'])
    model.eval()

    # Make prediction
    with torch.no_grad():
        image_tensor = image_tensor.to(device)
        probabilities = F.softmax(model(image_tensor), dim=1)
        predicted_class = torch.argmax(probabilities, 1).item()
        confidence = probabilities[0][predicted_class].item()
        all_probs = probabilities[0].cpu().numpy()

    return predicted_class, confidence, all_probs

# Get a sample image from the test set (first image)
sample_image, true_label = testset[0]
# Add batch dimension: (1, 28, 28) -> (1, 1, 28, 28);
# PyTorch expects batch dimension for input
sample_image_batch = sample_image.unsqueeze(0)

# Make prediction using loaded model
predicted_class, confidence, all_probs = use_model_for_prediction(
    'mnist_cnn_model.pth', sample_image_batch, DEVICE
)
```

Adding a Batch Dimension

PyTorch models expect a **batch dimension**. A single MNIST image is a tensor of shape $(1, 28, 28)$; use `unsqueeze(0)` to turn it into the batched tensor $\mathcal{T} \in \mathbb{R}^{1\times1\times28\times28}$ before feeding it to the model.

9.4 Automatic Differentiation

9.4.1 Automatic Differentiation

When training neural networks, the most frequently used algorithm is **back propagation**. Parameters (model weights) are adjusted according to the **gradient of the loss function** with respect to the given parameter.

torch.autograd

PyTorch has a built-in differentiation engine called `torch.autograd`, which supports **automatic computation of gradients** for any computational graph.

The code below defines a small computational graph. `w` and `b` are parameters which we need to optimize, so we must be able to compute the gradients of the loss function with respect to those variables. To do so, we set the `requires_grad` property of those tensors.

```
import torch

x = torch.ones(5)                # input tensor
y = torch.zeros(3)               # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w) + b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```

Concretely, $\mathbf{x} \in \mathbb{R}^5$ is the input, $W \in \mathbb{R}^{5\times 3}$ and $\mathbf{b} \in \mathbb{R}^3$ are the learnable parameters, and $\mathbf{y} \in \mathbb{R}^3$ is the target. The graph computes

$$\mathbf{z} = \mathbf{x}^\top W + \mathbf{b} \in \mathbb{R}^3, \qquad L = \text{bce_with_logits}(\mathbf{z}, \mathbf{y}).$$

9.4.2 Computing Gradients

To optimize the weights of parameters in the neural network, we need to compute the derivatives of our loss function with respect to the parameters. To compute those derivatives, we call `loss.backward()`, and then retrieve the values from `w.grad` and `b.grad`. For the graph above these are the partial derivatives $\frac{\partial L}{\partial W}$ (a 5×3 matrix) and $\frac{\partial L}{\partial \mathbf{b}}$ (a 3-vector).

```
loss.backward()
print(w.grad)
print(b.grad)
```

```
# Output:
# tensor([[0.2529, 0.3264, 0.3269],
#         [0.2529, 0.3264, 0.3269],
#         [0.2529, 0.3264, 0.3269],
#         [0.2529, 0.3264, 0.3269],
#         [0.2529, 0.3264, 0.3269]])
# tensor([0.2529, 0.3264, 0.3269])
```

9.4.3 Disabling Gradient Tracking

By default, all tensors with `requires_grad=True` are tracking their computational history and support gradient computation. However, there are some cases when we do not need that — for example, when we have trained the model and just want to apply it to some input data, i.e., we only want to do **forward propagation**.

We can stop tracking computations by surrounding our computation code with a `torch.no_grad()` block:

```
z = torch.matmul(x, w) + b
print(z.requires_grad)  # Output: True

with torch.no_grad():
    z = torch.matmul(x, w) + b
    print(z.requires_grad)  # Output: False
```

Why Disable Gradient Tracking?

Disabling gradient tracking during inference/evaluation reduces **memory consumption** and **speeds up** computation, since PyTorch no longer records the operations needed for backpropagation.

9.4.4 Best Practices

- Use **GPU** for large-scale models to speed up training.
- Regularly **monitor memory usage** to avoid out-of-memory errors.
- Utilize **built-in functions** for tensor operations to optimize performance.
- Keep your code **modular and reusable** by defining functions and classes.
- Regularly **save and validate** your model during training.

9.4.5 Further Resources

- **Official Documentation:** `https://pytorch.org/docs/stable/index.html`
- **Tutorials:** `https://pytorch.org/tutorials/`
- **Books:**
  - Deep Learning with PyTorch by Eli Stevens, Luca Antiga, and Thomas Viehmann.
  - Programming PyTorch for Deep Learning by Ian Pointer.
- **Online Courses:** Coursera, Udacity, and edX offer courses on PyTorch.

10 Minimax and Alpha-Beta Pruning

10.1 Games and Tic-Tac-Toe

10.1.1 Overview

In this topic, we learn how to implement an AI for a 2-player game, **Tic-Tac-Toe**. The AI agent makes use of searching strategies to optimally play the game:

- **Brute-force: Minimax** — the plain minimax algorithm that enumerates the whole game tree.
- **Minimax with Alpha-Beta Pruning** — minimax plus a pruning technique that avoids exploring branches that cannot affect the final decision.

The same AI strategies can also be applied to other 2-player games, such as **Connect-Four**, **Chess**, **Checkers**, and **Backgammon**.

What does "brute force" mean?

Brute force refers to straightforward methods of solving a problem — enumerate everything and pick the best, without clever shortcuts.

10.1.2 Why Games?

Games are the oldest, most well-studied domain in artificial intelligence. Why?

- They are **fun**.
- They are **easy to represent**, and the rules are clear.

- The possible combinations of moves can be huge, e.g., there are $\sim 10^{154}$ possible moves in chess.
- They are like the "real world" in that decisions have to be made and time is important.
- It is **easy to determine when a program is doing well**.

10.1.3 Game Types

Three Ways to Classify Games

- **Perfect vs. imperfect information game**
  - Perfect: the player knows all the possible moves of himself and the opponent and their results.
  - Imperfect: the player does not know all the possible moves of the opponent.
- **Zero-sum vs. non-zero-sum game**
  - Zero-sum: a two-person game where one player's gain is equal to the other player's loss on any given play of the game. Writing the players' utilities for a joint move $a = (a_1, a_2)$ as $u_1(a)$ and $u_2(a)$, this means
$$u_1(a) + u_2(a) = 0 \qquad \text{for every joint move } a.$$
  - Non-zero-sum: a two-person game where a player's gain is not necessarily equal to the other player's loss on any given play of the game, i.e. there exists at least one a with $u_1(a) + u_2(a) \neq 0$.
- **Deterministic vs. non-deterministic game**
  - Deterministic: a game that does not involve random choices. The successor of a state s under move a is a function, $s' = \mathrm{succ}(s, a)$.
  - Non-deterministic: a game that involves some random choices, so that the successor is governed by a distribution
$$s' \sim P\big(s' \mid s, a\big), \qquad \sum_{s'} P\big(s' \mid s, a\big) = 1.$$

10.1.4 Game Examples

Game	Perfect Information?	Zero-sum?	Deterministic?
Tic-Tac-Toe	Yes	Yes	Yes
Chess	Yes	Yes	Yes
Monopoly	Yes	No	No
Poker	No	Yes	No

Tic-Tac-Toe as an example:

- Has **perfect information**: all players know the moves previously made by all other players.
- Is **zero-sum**: it involves two players; if player 1 wins, she scores 1 and player 2 who loses scores -1; if the game ends in a draw, both players score 0. Formally, at every terminal state s_T the utilities satisfy $u_1(s_T) + u_2(s_T) = 0$ with
$$u_1(s_T) = \begin{cases} +1, & \text{player 1 wins at } s_T, \\ 0, & \text{the game is drawn}, \\ -1, & \text{player 2 wins at } s_T. \end{cases}$$
- Is **deterministic**: it is a game that does not involve random choices.

Exercise: Classify the Other Games

Question: Could you explain the others? For example, why does chess have perfect information, is zero-sum, and deterministic?

Answer sketch:
- **Chess** — Perfect information: the whole board and all previous moves are visible to both players. Zero-sum: one player's win is the other player's loss (win $+1$, loss -1, draw 0). Deterministic: every move is chosen by a player; no dice, cards, or randomness is involved.
- **Monopoly** — Perfect information: money, properties, and positions are all visible. Not zero-sum: the bank can inject money into the game (e.g., passing GO), so one player's gain is not necessarily another's loss. Not deterministic: dice rolls introduce random choices.
- **Poker** — Imperfect information: opponents' cards are hidden. Zero-sum: the chips one player wins are exactly the chips the others lose. Not deterministic: cards are dealt randomly.

10.1.5 Tic-Tac-Toe

Tic-Tac-Toe (also called Noughts and Crosses) is a paper-and-pencil game for two players, who take turns marking the spaces in a 3×3 grid. The player who succeeds in placing three of their marks in a **diagonal**, **horizontal**, or **vertical** line is the winner.

10.1.6 Naive Approach

A simple strategy for playing Tic-Tac-Toe:

1. First, if there is a move that will **win the game**, play it.
2. If not, check if there is a move that will **block a win** from the opposing player. If so, place your marker in the blocking spot.
3. Otherwise, just place your marker in a **random empty cell**.

Writing s for the current state and $A(s)$ for the set of legal moves, this hand-crafted policy π selects

$$a^\star = \pi(s) = \begin{cases} a_w, & \text{if } \exists\, a_w \in A(s) \text{ that wins immediately,} \\ a_b, & \text{else if } \exists\, a_b \in A(s) \text{ that blocks the opponent's win,} \\ a \sim \mathrm{Unif}\big(A(s)\big), & \text{otherwise,} \end{cases}$$

i.e. it is a purely one-step greedy rule with no look-ahead.

Naïve Approach $\neq$ Optimal Play

With the naïve approach, the AI may **not** play optimally, i.e., the AI may not achieve the best possible results. A fixed set of local rules (win, block, random) does not look ahead into future moves, so it can miss forced wins and fall into traps. To guarantee optimal play we need to search the game tree — this is what minimax does.

10.2 Minimax

10.2.1 Brute Force

The brute-force approach **enumerates all the possible states of the game**. At the very beginning, a player can choose any of the 9 available empty spaces on the game board. We can represent these 9 available moves using a **tree**.

Counting the states level by level:

- **1st action:** 9 possible choices at the very beginning of the game.
- **2nd action:** 9×8 (for each of the 9 possible first choices, we have 8 possible remaining choices).

- **3rd action:** $9 \times 8 \times 7$ (for each of the $9 \times 8 = 72$ previous choices we have 7 possible remaining choices).
- $\cdots$
- **Last action:** $9! = 9 \times 8 \times 7 \times \cdots \times 1 = 362880$ choices.

So, with k denoting the number of marks already placed, the total number of states is

$$\begin{aligned} N_{\text{states}} &= \sum_{k=1}^{9} \prod_{j=0}^{k-1} (9-j) \;=\; \sum_{k=1}^{9} \frac{9!}{(9-k)!} \\ &= \frac{9!}{0!} + \frac{9!}{1!} + \frac{9!}{2!} + \cdots + \frac{9!}{8!} \\ &= (9 \times 8 \times \cdots \times 1) + (9 \times 8 \times \cdots \times 2) + \cdots + (9 \times 8) + (9) \\ &= 986409. \end{aligned}$$

Questions on the Brute-Force Approach

1. **Is it really true that there are** 986409 **possible states in the game?**
   Answer: No. The game can end before there is no more empty space on the board (someone wins early), so the actual number of reachable states is smaller.
2. **How to determine which move to take based on the current game state?**
   Answer: Use **minimax**.

10.2.2 Algorithm

Minimax

Minimax is a recursive algorithm which is used to choose an **optimal move** for a player, assuming that the other player is also playing optimally.
In minimax, the two players are called the **maximizer** (denoted as **MAX**) and the **minimizer** (denoted as **MIN**), and we define a scoring method from the standpoint of the MAX player:

- The **maximizer** (the AI of the Tic-Tac-Toe game) tries to maximize the score.
- The **minimizer** (the human player of the Tic-Tac-Toe game) tries to minimize the score of the AI.

Formally, let s be a state, $A(s)$ the set of legal moves in s, and $s' = \text{succ}(s, a)$ the successor state reached by playing a. With terminal utility $u(s_T)$ (see below), the **minimax value** $V(s)$ is defined recursively by

$$V(s) = \begin{cases} u(s), & s \text{ is a terminal state}, \\ \max\limits_{a \in A(s)} V\big(\text{succ}(s,a)\big), & s \text{ is a MAX-node}, \\ \min\limits_{a \in A(s)} V\big(\text{succ}(s,a)\big), & s \text{ is a MIN-node}. \end{cases}$$

Equivalently, expanding one full MAX–MIN ply,

$$V(s) = \max_{a \in A(s)} \min_{a' \in A(s')} u\big(s, a, a'\big), \qquad s' = \text{succ}(s, a).$$

The move finally played by MAX is the **minimax decision**

$$a^\star = \arg\max_{a \in A(s)} V\big(\text{succ}(s,a)\big).$$

Why the name "minimax"?

It is called minimax because it helps in **mini**mizing the **max**imum loss that the other

player can force us to receive. Equivalently, MAX selects

$$a^\star = \arg\max_{a \in A(s)} \min_{a' \in A(s')} u\big(s, a, a'\big),$$

i.e. it maximizes the worst-case payoff that the opponent cannot take away.

Utility Function

A **utility function** $u : \mathcal{S}_{\text{terminal}} \to \mathbb{R}$ gives numeric values to terminal states s_T. For example, in Tic-Tac-Toe, from the standpoint of MAX,

$$u(s_T) = \begin{cases} +1, & \text{MAX wins at } s_T, \\ 0, & \text{the game is drawn,} \\ -1, & \text{MIN wins at } s_T. \end{cases}$$

10.2.3 Steps

To evaluate a node n in a game tree, write $C(n)$ for the set of children of n and $V(n)$ for its value:

1. Construct the **complete game tree** below n.
2. Apply the **utility function** to each terminal state, i.e. set $V(c) = u(c)$ for every terminal leaf c.
3. Select a node that has not been evaluated yet, and all of its children have been evaluated:
   - Node is a **MAX-node**: $V(n) = \max_{c \in C(n)} V(c)$ (the maximum of the children nodes' values).
   - Node is a **MIN-node**: $V(n) = \min_{c \in C(n)} V(c)$ (the minimum of the children nodes' values).
4. Repeat Step 3 until all nodes are evaluated.
5. For the node n, choose the **minimax decision**:
   - n is a **MAX-node**: choose the move that leads to the highest value,

$$a^\star = \arg\max_{a \in A(n)} V\big(\text{succ}(n, a)\big).$$

   - n is a **MIN-node**: choose the move that leads to the lowest value,

$$a^\star = \arg\min_{a \in A(n)} V\big(\text{succ}(n, a)\big).$$

Search Order: Depth-First Search

We search the game tree in a **Depth-First-Search (DFS)** manner to find the value of the root. Depth-first search is an algorithm for traversing or searching tree data structures: the algorithm starts at the root node and explores as far as possible along each branch before backtracking.

10.2.4 Worked Example

Minimax: Worked Example

Setup. Consider a game tree of 4 levels: the root at level 0 is a MAX-node (it is X's turn), level 1 contains MIN-nodes, level 2 contains MAX-nodes, and level 3 contains terminal states. We write $V(n)$ for the value of node n and $C(n)$ for its children, and

evaluate bottom-up with

$$V(n) = \begin{cases} u(n), & n \text{ terminal}, \\ \max\limits_{c \in C(n)} V(c), & n \text{ a MAX-node}, \\ \min\limits_{c \in C(n)} V(c), & n \text{ a MIN-node}. \end{cases}$$

Terminal states' utility:

$$u(s_T) = \begin{cases} +1, & \text{MAX (X) wins}, \\ 0, & \text{draw}, \\ -1, & \text{MAX (X) loses}. \end{cases}$$

Initialization:

- For the **maximizer** (MAX-nodes), initialize each node to negative infinity, $V(n) \leftarrow -\infty$ (the identity element of max).
- For the **minimizer** (MIN-nodes), initialize each node to positive infinity, $V(n) \leftarrow +\infty$ (the identity element of min).

Step-by-step evaluation: each node accumulates its value incrementally as its children are processed, applying

$$V(n) \leftarrow \max\big(V(n), V(c)\big) \qquad \text{at a MAX-node},$$
$$V(n) \leftarrow \min\big(V(n), V(c)\big) \qquad \text{at a MIN-node}.$$

1. **Leftmost state in level 2 (MAX-node):** the maximizer updates its value with the maximum of $-\infty$ and 0, where the value 0 is obtained from its child in level 3:
$$\max(-\infty, 0) = 0.$$
Its value is changed from $-\infty$ to 0.
2. **Leftmost state in level 1 (MIN-node):** the minimizer updates its value with the minimum of ∞ and 0, where the value 0 is obtained from its left child in level 2:
$$\min(\infty, 0) = 0.$$
Its value is changed from ∞ to 0.
3. **Same leftmost state in level 1 (MIN-node):** the minimizer updates its value with the minimum of 0 and -1, where the value -1 is obtained from its right child in level 2:
$$\min(0, -1) = -1.$$
Its value is changed from 0 to -1.
4. **State in level 0 (root, MAX-node):** the maximizer updates its value with the maximum of $-\infty$ and -1, where the value -1 is obtained from its left child in level 1:
$$\max(-\infty, -1) = -1.$$
Its value is changed from $-\infty$ to -1.
5. **State in level 0 (root, MAX-node):** the maximizer updates its value with the maximum of -1 and $+1$, where the value $+1$ is obtained from its middle child in level 1:
$$\max(-1, +1) = +1.$$
Its value is changed from -1 to $+1$.
6. **Level-2 state in the right subtree (MAX-node):** the maximizer updates its value with the maximum of $-\infty$ and $+1$, where the value $+1$ is obtained from its child in level 3:
$$\max(-\infty, +1) = +1.$$
Its value is changed from $-\infty$ to $+1$.
7. **Rightmost state in level 1 (MIN-node):** the minimizer updates its value with the minimum of ∞ and $+1$, where the value $+1$ is obtained from its left child in

level 2:

$$\min(\infty, +1) = +1.$$

Its value is changed from ∞ to $+1$.

8. **Another level-2 state in the right subtree (MAX-node):** the maximizer updates its value with the maximum of $-\infty$ and 0, where the value 0 is obtained from its child in level 3:

$$\max(-\infty, 0) = 0.$$

Its value is changed from $-\infty$ to 0.

9. **Rightmost state in level 1 (MIN-node):** the minimizer updates its value with the minimum of $+1$ and 0, where the value 0 is obtained from its right child in level 2:

$$\min(+1, 0) = 0.$$

Its value is changed from $+1$ to 0.

10. **State in level 0 (root, MAX-node):** the maximizer updates its value with the maximum of $+1$ and 0, where the value 0 is obtained from its right child in level 1:

$$\max(+1, 0) = +1.$$

Its value remains unchanged at $+1$.

Conclusion. In the top position, it is X's turn:

- If X plays in the **top-center**, then O can guarantee a win (value -1).
- If X plays in the **left-center**, X will win (value $+1$).
- If X plays in the **right-center**, then O can force a draw (value 0).

Therefore X will choose to play in the **left-center**, and the minimax value of the root is $+1$.

10.2.5 Implementation

We now implement the complete Tic-Tac-Toe AI in Python. The board is represented as a single-dimensional vector of 9 cells, $\mathbf{b} = (b_1, \ldots, b_9) \in \{-1, 0, +1\}^9$, where 0 = empty, 1 = AI's mark (X), -1 = human's mark (O).

Drawing the board. The function `drawboard()` prints the current state of the game:

```
# Draw the current state of the game
def drawboard(board):
    print("Current state: \n\n")
    for i in range(0, 9):
        if board[i] == 0:   # If value is 0, it's an empty cell
            print("- ", end=" ")
        if board[i] == 1:   # If value is 1, it's the mark of AI
            print("X ", end=" ")
        if board[i] == -1:  # If value is -1, it's the mark of human
            print("O ", end=" ")
        # Move to the next line after printing 3 elements
        if (i+1) % 3 == 0:
            print("\n")
    print("\n")  # Move the next line after printing the game board
```

Checking the game state. The function `check_game()` returns -1 if the human player wins, 1 if the AI player wins, 0 for a draw, and 2 if the game is not finished:

```
# Check if there is a player wins. If human player wins, return -1.
# If AI player wins, return 1. If draw, return 0. If the game is not
# finished, return 2.
def check_game(board):
    # There are 8 possible winning patterns.
    # 0, 1, 2 (1st row), 3, 4, 5 (2nd row), 6, 7, 8 (3rd row)
    # 0, 3, 6 (1st col), 1, 4, 7 (2nd col), 2, 5, 8 (3rd col)
```

```
    # 0, 4, 8 (Diagonal - top-left to bottom-right)
    # 2, 4, 6 (Diagonal - top-right to bottom-left)
    winning_pos = [[0,1,2], [3,4,5], [6,7,8], [0,3,6],
                   [1,4,7], [2,5,8], [0,4,8], [2,4,6]]
    # Check all possible winning patterns for each player.
    for i in range(0, 8):
        # Check whether the cell is non-empty AND the winning
        # positions with the same mark
        if board[winning_pos[i][0]] != 0 and \
           board[winning_pos[i][0]] == board[winning_pos[i][1]] and \
           board[winning_pos[i][0]] == board[winning_pos[i][2]]:
            # Fulfilled the winning condition, return the mark of
            # player
            return board[winning_pos[i][2]]
    # When reaching here, it means no player wins yet
    # Check whether there is an empty cell (i.e. with 0)
    # If so, the game is not finished. Return 2
    if any(element == 0 for element in board):
        return 2
    return 0  # Reach here, it means the game draws
```

Handling the human's move. The function `human_turn()` obtains the user's input and updates the board:

```
# Obtain the user's input and update the board
def human_turn(board):
    pos = input("Enter O's position [1 to 9]: ")
    pos = int(pos)  # Obtain user input, i.e., where to put the mark
    # If the cell picked by the human player is non-empty (i.e.,
    # non-zero), illegal
    if board[pos-1] != 0:
        print("Illegal Move!")
        exit(0)  # Terminate the program
    # When reaching here, the move is legal and we put -1 (put the
    # mark of human player)
    board[pos-1] = -1
```

The minimax search. The recursive function `minimax()` evaluates a board position for the given player. Writing $p \in \{+1, -1\}$ for the player to move ($+1$ = AI/MAX, -1 = human/MIN), terminal(s) for the predicate "the game has ended", and succ(s, i) for the position obtained by playing in cell i, it returns

$$\text{minimax}(s,p) = \begin{cases} u(s), & \text{terminal}(s), \\ \max\limits_{i:\, s_i=0} \text{minimax}\big(\text{succ}(s,i),\, -p\big), & p = +1, \\ \min\limits_{i:\, s_i=0} \text{minimax}\big(\text{succ}(s,i),\, -p\big), & p = -1, \end{cases}$$

where the recursion alternates the player sign $p \leftarrow -p$ at each ply:

```
# Perform minimax search
def minimax(board, player):
    global count  # Refer to the global variable count in main
    count += 1    # Make 1 call of minimax already
    result = check_game(board)  # Check if any player wins
    if result != 2:  # If the result is not 2, the game is finished,
        return result  # return 1 if the winner is AI,
                       # otherwise return -1 (i.e., human is the
                       # winner)
    scores = []  # Create an empty list
    for i in range(0, 9):  # Check all board locations
        if board[i] == 0:      # If the cell is empty
            # Try to put the player's mark at cell i+1
```

```
            board[i] = player
            # Perform minimax for the next player, and append its
            # score to the list
            scores.append(minimax(board, player*-1))
            board[i] = 0  # Undo the trial
    # Return the max score if the player is AI or
    # return the min score if the player is human
    return max(scores) if player == 1 else min(scores)
```

Making the AI's move. The function `ai_turn()` tries every empty cell, runs `minimax()` for each, and picks the move with the highest score. With the board $\mathbf{b}$ and the human (MIN, $p = -1$) to move after the AI, the chosen cell is

$$i^\star = \operatorname*{arg\,max}_{i \in \{1,\dots,9\},\, b_i = 0} \operatorname{minimax}\big(\operatorname{succ}(\mathbf{b}, i),\, -1\big).$$

```
# This function makes computer's move based on minimax.
def ai_turn(board):
    pos = -1        # Initialize pos to illegal value, -1, here
    max_value = -2  # Initialize max value so far to -2,
                    # which is a value smaller than the min value
                    # possible
    for i in range(0, 9):
        if board[i] == 0:      # If the cell is empty
            board[i] = 1       # Try to put X at cell i+1
            # Calculate minimax score for the human player
            score = minimax(board, -1)
            board[i] = 0       # Undo the trial
            # If we can a better score in the next level,
            if score > max_value:
                max_value = score  # update the score and pos
                pos = i
    board[pos] = 1  # Put X at pos, which is the best move
```

The main loop. The main function initializes the board, asks who plays first, and alternates turns (9 turns in total):

```
# Main function
# The board is represented in a single dimensional list
# Initialize the board to all zeros (i.e., all empty cells)
board = [0, 0, 0, 0, 0, 0, 0, 0, 0]
print("Computer: X VS. You: O")
first = input("Play first (Y/N) :")
# If first is 'Y'/'y', human wants to play first
if first == 'Y' or first == 'y':
    play_first = 1
else:
    play_first = 0
for i in range(0, 9):  # Play the game. 9 turns in total
    # If the value returned by check_game is not 2,
    if check_game(board) != 2:
        break                    # the game is finished
    if (i+play_first) % 2 == 0:  # Take turns to play the game
        count = 0  # Count the number of minimax calls
        ai_turn(board)
        print("Count:", count)  # Print the count
    else:
        drawboard(board)
        human_turn(board)
```

Announcing the result. After the loop, check the final state of the game:

```
result = check_game(board)  # Check the game again
```

```
if result == 0:   # If the result is 0, draw
    drawboard(board)
    print("Draw!!!")
if result == 1:   # If the result is 1, AI wins
    drawboard(board)
    print("AI(X) Wins! Human(O) Loses!")
if result == -1:  # If the result is -1, human wins
    drawboard(board)
    print("Human(O) Wins! AI(X) Loses!")
```

The `count` Variable

The global variable `count` records how many times `minimax()` is called for a single AI move. Run the same program with plain minimax and with alpha-beta pruning (below) and compare the counts — this shows concretely how much search effort pruning saves.

10.2.6 Non-Optimal MIN

Minimax Against a Sub-Optimal Opponent

The definition of optimal play for MAX assumes MIN plays optimally, i.e., minimax maximizes the worst-case outcome for MAX. If MIN does **not** play optimally, MAX will do **even better** — never worse.
Formally, let σ be an arbitrary (possibly sub-optimal) strategy for MIN, so that its reply in state s' is $a' = \sigma(s') \in A(s')$. Comparing the two values,

$$V(s) = \max_{a \in A(s)} \min_{a' \in A(s')} u\big(s, a, a'\big) \;\le\; \max_{a \in A(s)} u\big(s, a, \sigma(s')\big) = V_\sigma(s).$$

The inequality holds because $\sigma(s') \in A(s')$ is one of the moves over which the inner min is taken. Hence a sub-optimal MIN can only increase MAX's guaranteed payoff.

10.2.7 Drawbacks

- It has a **huge branching factor** b, i.e. the number of children at each node, $b = |A(s)|$, which makes the process of reaching the goal state slow.
- It can take a **long time** to evaluate an entire game tree, especially for games such as Go or Chess.
- **Search and evaluation of unnecessary nodes or branches** of the game tree degrades the overall performance and efficiency of the engine.
- Both MIN and MAX players have lots of choices to decide from.
- Exploring the entire tree is **not possible** as there is a restriction of time and space.

The cost is governed by the branching factor $b = |A(s)|$ and the search depth d (number of plies): a uniform game tree of depth d contains

$$N_{\text{nodes}} = \sum_{k=0}^{d} b^k = \frac{b^{d+1} - 1}{b - 1} = O\big(b^d\big)$$

nodes, so plain minimax needs $O(b^d)$ time (and $O(bd)$ space under depth-first search). This exponential blow-up is exactly what motivates pruning and depth limits.

Solution

Alpha-Beta Pruning — prune away the branches that cannot possibly influence the final decision.

10.3 Alpha-Beta Pruning

10.3.1 Motivation

The minimax algorithm provides us with the optimal path as expected. But did we really need to do all that work? **No!** Let us demonstrate it using an example.

Motivating Observation

Suppose we are at the middle level of a game tree, at a MIN-node whose three terminal children have values 6, 5, and 4, and we are finding the minimum value among them.

- We have already calculated the value of node 6. So we are **at least guaranteed** that the minimum value between these 3 nodes is ≤ 6.
- On the level above (the top level), we are trying to maximize the values. We have already calculated a value of 7 to the left. Something that is guaranteed to be 6 or less will **never be greater than** 7.
- So we **do not need to go any further** with the calculations of 5 and 4.

The same reasoning applies to the MIN-node with terminal values 3, 2, 1: once we see 3, its value is guaranteed to be $\leq 3 < 7$, so 2 and 1 can be skipped as well.
In the language of the running bounds introduced below (with α the best value found for MAX so far and β the best value found for MIN so far), the two cut-offs are

$$\beta = \min(\infty, 6) = 6, \quad \alpha = 7 \implies \alpha = 7 \geq \beta = 6 \Rightarrow \text{ prune children 5, 4,}$$

$$\beta = \min(\infty, 3) = 3, \quad \alpha = 7 \implies \alpha = 7 \geq \beta = 3 \Rightarrow \text{ prune children 2, 1.}$$

10.3.2 Idea

Idea: prevent ourselves from exploring branches of the game tree that are not worth exploring (because they will have no effect on the final outcome).

Alpha and Beta

We define two values α and β to store information about the potential of a branch for us or the opponent:

- α is the **best (highest) value found so far along the path for MAX**.
- β is the **best (lowest) value found so far along the path for MIN**.

Formally, after a set $\mathcal{E}$ of nodes has been evaluated, the running bounds are

$$\alpha = \max\Big\{\alpha_0, \max_{c \in \mathcal{E},\ c \text{ a MAX-node}} V(c)\Big\}, \qquad \beta = \min\Big\{\beta_0, \min_{c \in \mathcal{E},\ c \text{ a MIN-node}} V(c)\Big\},$$

initialized with $\alpha_0 = -\infty$ and $\beta_0 = +\infty$. Along the search path they maintain the **window invariant**

$$\alpha \ \leq \ V(s) \ \leq \ \beta,$$

i.e. $V(s)$ is always bracketed by a lower bound α (what MAX can already guarantee) and an upper bound β (what MIN can already guarantee).
Two kinds of cutoffs:

- **α-cutoff:** terminate a branch because MAX already has a better opportunity elsewhere, i.e. $V(s) \leq \alpha$.
- **β-cutoff:** terminate a branch because MIN already has a better opportunity elsewhere, i.e. $V(s) \geq \beta$.

Pruning condition: the window is empty and we can prune a branch precisely when

$$\alpha \geq \beta,$$

since then no value of the pruned subtree can satisfy the invariant, and the parent's decision is already fixed.

Do Not Mix Up α and β

- α belongs to **MAX** (a lower bound on what MAX can guarantee); it is updated only at **MAX-nodes**: $\alpha \leftarrow \max(\alpha, \text{value})$.
- β belongs to **MIN** (an upper bound on what MIN can guarantee); it is updated only at **MIN-nodes**: $\beta \leftarrow \min(\beta, \text{value})$.
- The pruning test is $\alpha \geq \beta$ (greater-or-equal), not $\alpha > \beta$.

10.3.3 Procedure

To evaluate a root node in a game tree, we define two values α and β to keep track of the current lower bound (the best score found so far for MAX) and upper bound (the best score found so far for MIN). Initially, the root node has values $(\alpha, \beta) = (-\infty, \infty)$, so the window invariant $\alpha \leq V(s) \leq \beta$ holds trivially.

Writing $\mathrm{ab}(s, \alpha, \beta)$ for the pruned search value, the recursion is

$$\mathrm{ab}(s, \alpha, \beta) = \begin{cases} u(s), & \text{terminal}(s), \\ \max\limits_{a \in A(s)} \ \mathrm{ab}\big(\mathrm{succ}(s, a), \alpha, \beta\big), & s \text{ a MAX-node}, \\ \min\limits_{a \in A(s)} \ \mathrm{ab}\big(\mathrm{succ}(s, a), \alpha, \beta\big), & s \text{ a MIN-node}, \end{cases}$$

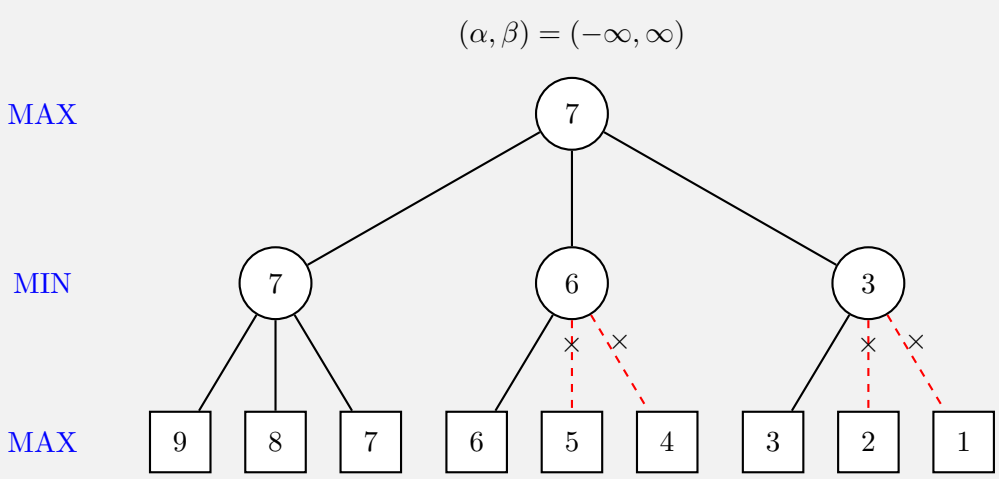
where the bound is tightened during accumulation — at a MAX-node $\alpha \leftarrow \max(\alpha, v)$ and at a MIN-node $\beta \leftarrow \min(\beta, v)$, and the loop over $A(s)$ stops as soon as $\alpha \geq \beta$.

1. Expand the node **depth-first** until a node that satisfies the **cutoff test** $\alpha \geq \beta$ is reached. During expansion, pass the current values of α and β of the parent down to child nodes.
2. Evaluate the cutoff node.
3. Update the values of all the nodes that have so far been expanded according to the **minimax algorithm**.
4. Update the (α, β) values of all the nodes that have so far been expanded according to:
   - For a **MAX-node**: $\alpha \leftarrow \max(\alpha, v)$, where v is the current node value (a lower bound for MAX).
   - For a **MIN-node**: $\beta \leftarrow \min(\beta, v)$, where v is the current node value (an upper bound for MIN).
5. **Prune** all children of any node whose $\alpha \geq \beta$.
6. **Backtrack** to a node that has not been pruned, and go back to Step 1. If there is no such node to backtrack to, then **return** the value assigned to the root node.

10.3.4 Worked Example

Alpha-Beta Pruning: Worked Example

Setup. A game tree with a MAX-node as the root, three MIN-nodes at the middle level, and terminal states below. The terminal values are $9, 8, 7$ (under the left MIN-node), $6, 5, 4$ (under the middle MIN-node), and $3, 2, 1$ (under the right MIN-node).



Dashed red edges are the branches pruned by α-cuts: once the middle MIN-node sees 6, the remaining values 5 and 4 cannot change the outcome, and the same happens for 2 and 1 under the right MIN-node.

1. **Initialization at the root (MAX-node):** set $\alpha = -\infty$ and $\beta = \infty$. Check whether $\alpha \geq \beta$: it is not $(-\infty < \infty)$, therefore no pruning is done.
2. **Descend to the left MIN-node**, passing down $\alpha = -\infty$ and $\beta = \infty$. At the left node of the middle level, process the first terminal value 9:
   - Node value $= \min(\text{node value}, 9) = \min(\infty, 9) = 9$.
   - $\beta = \min(\beta, 9) = \min(\infty, 9) = 9$.
   - Check whether $\alpha \geq \beta$: it is not, therefore no pruning is done.
3. **Still at the left MIN-node**, process the second terminal value 8:
   - Node value $= \min(\text{node value}, 8) = \min(9, 8) = 8$.
   - $\beta = \min(\beta, 8) = \min(9, 8) = 8$.
   - Check whether $\alpha \geq \beta$: it is not, therefore no pruning is done.
4. **Still at the left MIN-node**, process the third terminal value 7:
   - Node value $= \min(\text{node value}, 7) = \min(8, 7) = 7$.
   - $\beta = \min(\beta, 7) = \min(8, 7) = 7$.
   - Check whether $\alpha \geq \beta$: it is not, therefore no pruning is done.

   The left MIN-node is fully evaluated with value 7.
5. **Backtrack to the root**, where the value of the middle-left node, i.e. 7, is passed to the parent. At the root node, process the returned value 7:
   - Node value $= \max(\text{node value}, 7) = \max(-\infty, 7) = 7$.
   - $\alpha = \max(\alpha, 7) = \max(-\infty, 7) = 7$.
   - Check whether $\alpha \geq \beta$: it is not $(7 < \infty)$, therefore no pruning is done.
6. **Descend to the middle MIN-node**, passing down $\alpha = 7$ and $\beta = \infty$. At the middle node of the middle level, process the first terminal value 6:
   - Node value $= \min(\text{node value}, 6) = \min(\infty, 6) = 6$.
   - $\beta = \min(\beta, 6) = \min(\infty, 6) = 6$.
   - Check whether $\alpha \geq \beta$: we notice that $\alpha = 7$ is greater than $\beta = 6$, and therefore **pruning is done** — the remaining terminal children 5 and 4 are never evaluated.
7. **Backtrack to the root**, where the value of the center node of the middle layer, i.e. 6, is passed to the parent. At the root node, process the returned value 6:
   - Node value $= \max(\text{node value}, 6) = \max(7, 6) = 7$.
   - $\alpha = \max(\alpha, 6) = \max(7, 6) = 7$.
   - Check whether $\alpha \geq \beta$: it is not, therefore no pruning is done.
8. **Descend to the right MIN-node**, passing down $\alpha = 7$ and $\beta = \infty$. At the right node of the middle level, process the first terminal value 3:
   - Node value $= \min(\text{node value}, 3) = \min(\infty, 3) = 3$.

- $\beta = \min(\beta, 3) = \min(\infty, 3) = 3$.
- Check whether $\alpha \geq \beta$: we notice that $\alpha = 7$ is greater than $\beta = 3$, and therefore **pruning is done** — the remaining terminal children 2 and 1 are never evaluated.

9. **Backtrack to the root**, where the value of the right node of the middle layer, i.e. 3, is passed to the parent. At the root node, process the returned value 3:
   - Node value = max(node value, 3) = max(7, 3) = 7.
   - $\alpha = \max(\alpha, 3) = \max(7, 3) = 7$.
   - Check whether $\alpha \geq \beta$: it is not, therefore no pruning is done.

Conclusion. The best value for the root node is 7. The terminal nodes with values 5, 4, 2, and 1 were never computed. Hence the optimal value for the maximizer is 7 for this example — exactly the same answer as plain minimax, but with less work.

Pruning Never Changes the Answer

Alpha-beta pruning returns **exactly the same minimax value** as plain minimax. It only avoids exploring branches that provably cannot affect the final decision. If your pruned search gives a different root value than minimax, the pruning was done incorrectly.

10.3.5 Implementation

Replace the `minimax()` function defined above with the following `minimax_abp()`, which carries α and β down the recursion and breaks out of the loop as soon as $\alpha \geq \beta$:

$$\begin{aligned}
&\text{MAX-node } (p = +1): && v \leftarrow \max\big(v,\ \text{abp}(s', -p, \alpha, \beta)\big),\\
&&& \alpha \leftarrow \max(\alpha, v), \quad \text{stop if } \alpha \geq \beta,\\
&\text{MIN-node } (p = -1): && v \leftarrow \min\big(v,\ \text{abp}(s', -p, \alpha, \beta)\big),\\
&&& \beta \leftarrow \min(\beta, v), \quad \text{stop if } \alpha \geq \beta.
\end{aligned}$$

where $s' = \text{succ}(s, i)$ ranges over the empty cells i and $\text{abp}(\cdot, \cdot, \alpha, \beta)$ denotes the recursive call:

```
# Perform minimax search with alpha-beta pruning
def minimax_abp(board, player, alpha=-float('inf'),
   beta=float('inf')):
    global count  # Refer to the global variable count in main
    count += 1    # Make 1 call of minimax_abp already
    result = check_game(board)  # Check if any player wins
    if result != 2:  # If the result is not 2, the game is finished,
        return result  # return 1 if the winner is AI,
                       # otherwise return -1 (i.e., human is the
                       # winner)
    if player == 1:
        value = -float('inf')
        for i in range(0, 9):  # Check all board locations
            if board[i] == 0:      # If the cell is empty
                # Try to put the player's mark at cell i+1
                board[i] = player
                # Perform minimax with alpha-beta pruning for the next
                # player
                # and update current value if needed
                value = max(value, minimax_abp(board, player*-1,
   alpha, beta))
                # Update alpha with max of current alpha and value
                alpha = max(alpha, value)
                board[i] = 0  # Undo the trial
                if alpha >= beta:
```

```
                break
    else:
        value = float('inf')
        for i in range(0, 9):  # Check all board locations
            if board[i] == 0:      # If the cell is empty
                # Try to put the player's mark at cell i+1
                board[i] = player
                # Perform minimax with alpha-beta pruning for the next
                # player
                # and update current value if needed
                value = min(value, minimax_abp(board, player*-1,
alpha, beta))
                # Update beta with min of current beta and value
                beta = min(beta, value)
                board[i] = 0  # Undo the trial
                if alpha >= beta:
                    break
    # Return the max score if the player is AI or
    # return the min score if the player is human
    return value
```

Exercise: Alpha-Beta Pruning on Tic-Tac-Toe

Question: Can you find the optimal move for the Tic-Tac-Toe example using alpha-beta pruning?
Answer sketch: Yes — apply `minimax_abp()` to the same game tree. The optimal move is still the **left-center** cell with value +1: pruning only skips branches that cannot change the decision, so the final value and the chosen move are identical to plain minimax, but fewer nodes are explored (compare the printed `Count` values).

10.3.6 Advantages

- It plays a great role in **reducing the number of nodes** which are explored by minimax.
- When one chance or option is found at the minimum, it **stops assessing a move** (cutoff).
- This method also helps to **improve the search procedure** in an effective way.

Quantitatively, with an optimal move ordering the number of leaves examined drops from b^d to about $b^{d/2}$:

$$O\big(b^d\big) \xrightarrow{\text{optimal move ordering}} O\big(b^{d/2}\big),$$

i.e. the same time budget supports roughly twice the search depth — while the returned value is guaranteed to be identical to plain minimax.

10.3.7 Drawbacks and Remedies

Disadvantages:

- Alpha-beta prunes a large part of the search space, but it **still needs to search all the way to terminal states**.
- Note that for some games, the moves must be made in a **reasonable amount of time**.

Approaches to Remedy the Problem

- **Set a depth limit** — do not search all the way down; stop at a fixed depth. Combining a depth limit d with a heuristic evaluation function eval(s) gives **depth-**

limited minimax

$$V_{\text{cut}}(s,d) = \begin{cases} u(s), & \not\Vdash\big[\text{terminal}(s)\big] = 1, \\ \text{eval}(s), & \not\Vdash\big[\text{terminal}(s)\big] = 0 \ \text{ and } \ d = 0, \\ \max\limits_{a\in A(s)} V_{\text{cut}}\big(\text{succ}(s,a),\, d-1\big), & s \text{ a MAX-node and } d > 0, \\ \min\limits_{a\in A(s)} V_{\text{cut}}\big(\text{succ}(s,a),\, d-1\big), & s \text{ a MIN-node and } d > 0, \end{cases}$$

with the decision $a^\star = \arg\max_{a\in A(s)} V_{\text{cut}}(\text{succ}(s,a), d-1)$ at a MAX-node.

- **Heuristic evaluation function** — an estimated desirability or utility $\text{eval}(s) \approx V(s)$ of a position, used at the depth limit instead of the true utility. A good evaluation function should:
  - be **efficient** to compute (cheap in time, i.e. low $O(\cdot)$ cost);
  - **agree with the utility function on terminal states**, i.e. $\text{eval}(s) = u(s)$ for every $s \in \mathcal{S}_{\text{terminal}}$;
  - **accurately reflect the chances of winning**, i.e. the induced ordering by eval should match the true probability of winning.

10.3.8 Practice

Practice Problem: Minimax Values and Alpha-Beta Pruning on a Labeled Tree

Consider a game tree filled with the values returned by the standard minimax algorithm, with edges labeled E-1, E-2, . . . , E-30.

(a) State the **best initial move** for the first player (i.e., E-1 or E-2).

(b) Apply **alpha-beta pruning** on the given tree and state the branches that need not be examined or considered by putting down the edge label(s) (e.g., E-1, E-2, E-3, . . . , E-30).

Solution:

(a) **E-1** — the subtree reached via E-1 has the larger minimax value, so the first player (MAX) should take edge E-1:

$$a^\star = \arg\max_{a\in\{\text{E-1},\,\text{E-2}\}} V\big(\text{succ}(s_{\text{root}}, a)\big) = \text{E-1}, \qquad V(\text{E-1}) > V(\text{E-2}).$$

(b) The branches that need not be examined are: **E-18, E-22, E-26, E-28, and E-30**. Each of them lies below a node at which $\alpha \geq \beta$ is already satisfied once an earlier sibling subtree has been evaluated, so they are cut off by alpha-beta pruning.

11 AI Ethics

11.1 AI Ethics

11.1.1 Introduction

AI ethics is a term that refers to the **ethical issues surrounding the use of Artificial Intelligence (AI) technology**.

- As the use of AI systems becomes more prevalent across the globe, **governments, industry groups and AI-focused executives** are grappling with how to ensure the technology is used ethically.
- AI ethics is about **technological change and its impact on individual lives**, but also about **transformations in society**.

11.1.2 Definition of AI Ethics

Definition of AI Ethics (The Alan Turing Institute, UK)

The UK's Alan Turing Institute defines **AI ethics** as a set of values, principles and techniques that employ widely accepted standards of 'right' and 'wrong' to guide the development and use of AI technologies.

In practice, this means ensuring that organizations using AI have the **right AI ethics policy and governance practices** to ensure the technology is used **for good** and does not **unintentionally harm people**.

11.1.3 Key Questions in the Field of AI Ethics

1. What are applications for AI systems ethical for a given organization?
2. How can companies ensure AI systems are built to operate in a **fair and unbiased** way?
3. What processes do companies need to ensure AI systems **continue to function ethically over time**?

11.1.4 Potential 'Harms' That AI Systems May Cause

- **Privacy invasion:** Invading people's right to privacy by processing data **without consent**, or handling it in a way that reveals personal information without an individual's consent.
- **Bias and unfairness:** Making **biased or unfair decisions** or recommendations about certain populations or demographics.
- **Lack of explainability:** Making decisions in a way that **can't be explained in plain language**, so it is unclear if their conclusions are fair and unbiased.
- **Unreliability:** Making **unreliable decisions** or delivering **poor quality outcomes** due to model implementation issues.
- **Lack of accountability:** Denying people their right to **accountability** for the decisions AI systems make about them.

Five Categories of AI Harms

Remember the five potential harms: (1) privacy invasion, (2) biased/unfair decisions, (3) unexplainable decisions, (4) unreliable decisions, (5) denied accountability. These map naturally onto the AI principles discussed later.

11.1.5 How the 'Harms' of AI May Impact the Real World

- **Centralized power structures** in tech giants.
- **Violence of data acquisition**, data labeling and cleaning.
- The use of AI tools to **harm marginalized groups**: e.g., pornographic content generation using **deepfakes**.
- **Unemployment**; making employees work harder and feel worthless.
- **Generation and dissemination of disinformation**.

11.1.6 Is AI Ethical?

AI Itself is Neither Ethical Nor Unethical

AI technology itself is **neither ethical nor unethical**. Instead, enterprises must establish **principles or frameworks** to ensure that they use AI systems ethically and responsibly and **guard against AI misuse**.

Problems:

- There is **non-consensus** about what ethical responsibilities enterprises have for different applications of AI technology.
- Different AI-focused executives can look at the **same use case** for AI and draw **different conclusions** about their moral responsibilities.

"Is AI Ethical?" is the Wrong Framing

Do not say "AI is ethical" or "AI is unethical". The technology is neutral; it is the principles, frameworks, and governance established by the enterprises using AI that determine whether it is applied ethically. Also note that there is no universal consensus on ethical responsibilities — reasonable people can disagree about the same use case.

11.1.7 AI Ethics Issues and Considerations

There are many **ethical dilemmas** associated with AI use. These range from:

- deciding **whose lives autonomous vehicles should prioritize saving** in a multi-person crash situation, to
- ensuring that **credit scoring AIs do not discriminate** against people unfairly based on factors such as gender.

11.2 AI Principles

11.2.1 Emergence

In the last few years, a number of institutions have published AI principles:

- **Principles for Algorithmic Transparency and Accountability** (ACM, 2017)
- **IEEE's General Principles of Ethical Autonomous and Intelligent Systems** (IEEE, 2017)
- **Five principles for cross-sector AI code** (UK House of Lords, 2018)
- **Ethics Guidelines for Trustworthy AI** (European Commission, 2019)
- **AI Ethics Principles** (Google, 2019, 2020, 2021)
  `https://ai.google/principles/`

11.2.2 EU Principles

1. **Human agency and oversight:** AI systems should **empower human beings**, allowing them to make informed decisions.
2. **Technical robustness and safety:** AI systems need to be **resilient and secure**. They need to be safe, ensuring a **fallback plan** if something goes wrong.
3. **Privacy and data governance:** Besides ensuring **full respect for privacy and data protection**, adequate **data governance mechanisms** must also be ensured.
4. **Transparency:** The **data, system and AI business models** should be transparent.
5. **Diversity, non-discrimination and fairness: Unfair bias must be avoided**.
6. **Societal and environmental well-being:** AI systems should **benefit all human beings**.
7. **Accountability:** Mechanisms should be implemented to ensure **responsibility and accountability** for AI systems.

11.2.3 Common Grounds

There are many different lists of principles, but it seems that they can be **synthesized into five fundamental principles**:

The Five Common Grounds of AI Ethics

1. **Autonomy:** People should be able to make their own decisions, e.g., human-in-the-loop, privacy protection.
2. **Beneficence:** Society at large should **benefit**.
3. **Non-maleficence: Harmful consequences should be avoided**, e.g., systems should be robust.
4. **Justice: Diversity, non-discrimination and fairness**.
5. **Explicability: Transparency and explainability**.

11.2.4 Limits of Principles

It is good to state principles. However, they also create problems since they are **very high-level**.

- **They can be interpreted in different ways.** For example, autonomous killer drones can be considered beneficial for the soldiers, or morally impermissible, because machines decide about life and death.
- **They can conflict with each other in concrete cases.** For example, privacy and data collection for health science can conflict.
- **They can come into conflict in practice.** For example, an excellent diagnosis might still be preferable even if its reasoning cannot be explained (beneficence vs. explicability).

High-Level Principles Are Ambiguous

The same principle can justify opposite conclusions (killer drones: beneficial to soldiers vs. morally impermissible), and principles can conflict with one another (privacy vs. health data collection; accuracy vs. explainability in medical diagnosis). Never treat principles as unambiguous decision rules.

11.2.5 Interpretive Labor

In The Utopia of Rules, **David Graeber** introduces the concept of **interpretive labor** — the differential between the energy put into explaining an idea and the energy needed to understand it.

- Everyone interprets a given action/idea differently.
- Often you need to **put work in to understand what people mean**. This can be literal or more figurative.
- The work it takes to do that is **interpretive labor**.

Interpretive Labor (David Graeber)

"... within relations of domination, it is generally the **subordinates** who are effectively relegated the work of understanding how the social relations in question really work. ... It's those who do not have the power to hire and fire who are left with the work of figuring out what actually did go wrong so as to make sure it doesn't happen again."

11.2.6 Orientation, Not Checklists

It is nevertheless good to have such principles as **orientation points** and to evaluate solutions, but only having high-level principles is **NOT enough**.

- Ethical principles for AI are **not checklists or boxes to tick once and forget**.
- They are **codes of behavior** — for AI systems but, most importantly, **for us**.

- It is we who need to be **fair, nondiscriminatory, and accountable**; to ensure **privacy** for ourselves and others; and to aim at **social and environmental well-being**.

Virginia Dignum's Reminder

"The codes of ethics are for us; AI systems will follow." — Virginia Dignum. Ethics is primarily a human responsibility; the behavior of AI systems follows from the people who design, deploy, and govern them.

11.3 Areas

The general public and business community are becoming increasingly aware of the ethical issues surrounding AI use. There are **three main areas** to ensure the AI models in production in their organizations function ethically:

1. **Data Ethics**
2. **AI Model Fairness**
3. **AI Model Monitoring and Maintenance**

11.3.1 Data Ethics

Ensuring AI models function in a way that is **fair, unbiased and in customers' best interests** starts with ensuring the **data** that feeds into them is **collected, governed and used ethically**.

- This means ensuring companies secure the **proper consent** from customers before using their data and handle it in a **secure way that respects their privacy**.
- This means taking **proactive steps to address data bias** in those datasets and ensure the **populations being analyzed are fairly represented** in the data.

11.3.2 Model Fairness

- Ensuring the data that feeds AI models is **relatively unbiased**.
- AI and ML ethics requires enterprises to ensure that those models are **built in a way that ensures they make their decisions or recommendations fairly**.

11.3.3 Monitoring and Maintenance

- Some machine learning models **adjust themselves** to improve their accuracy.
- Others may see their accuracy **change over time** due to changes in the data flowing into them, in a phenomenon known as **'AI model drift'**.
- That means enterprises must also ensure AI models are **effectively monitored and maintained** to continue functioning as intended over time.

AI Model Drift

AI model drift is the phenomenon where a model's accuracy changes (typically degrades) over time due to changes in the data flowing into it. This is why continuous monitoring and maintenance is an ethical requirement, not a one-time check.

11.4 AI Fairness

11.4.1 Overview

The topic of enforcing **fairness** has become important, in particular in AI and machine learning. Key questions:

- What is fairness in AI?
- Why care about fairness in AI and ML?

- What kind of unfairness could there be?
- What causes unfairness?

11.4.2 Fairness in AI

What is Fairness in AI?

- Developing systems that are **equitable** and avoid **perpetuating bias or discrimination** against any individual or group.
- Considering the **diverse needs and circumstances of all stakeholders** impacted by AI use.
- Fairness extends **beyond a technical concept** and embodies **broader social standards** related to equity.

Formally, let $a \in \{0, 1\}$ denote a **sensitive attribute** (e.g., race or gender), $y \in \{0, 1\}$ the true label and $\hat{y} \in \{0, 1\}$ the model's prediction. Several common — and, in general, mutually incompatible — fairness criteria can then be stated rigorously:

- **Demographic parity** (statistical parity): $|P(\hat{y} = 1 \mid a = 0) - P(\hat{y} = 1 \mid a = 1)| \leq \epsilon$.
- **Equalized odds**: $|P(\hat{y} = 1 \mid y = c, a = 0) - P(\hat{y} = 1 \mid y = c, a = 1)| \leq \epsilon$ for each $c \in \{0, 1\}$.
- **Predictive parity**: $|P(y = 1 \mid \hat{y} = 1, a = 0) - P(y = 1 \mid \hat{y} = 1, a = 1)| \leq \epsilon$.

Here $\epsilon \geq 0$ is a tolerated slack; exact fairness corresponds to $\epsilon = 0$. The COMPAS case below violates equalized odds, since the false-positive rate differs across racial groups.

11.4.3 Why It Matters

Many things become automated by machines:

- **Employers** select candidates by using ML systems.
- **LinkedIn and XING** use ML systems to rank candidates.
- **Courts in the US** use ML systems to predict **recidivism**.
- **Banks** use credit rating systems, which use ML.
- **Amazon and Netflix** use recommendation systems.

If these systems act unfairly, **groups and individuals may suffer**.

11.4.4 Example: Text-to-Image Bias

Midjourney Bias (Text-to-Image Models)

Midjourney is a popular text-to-image system that was released in 2022.

- When prompted with "influential person", it generated most of — or even all — images of **older-looking white males**.
- In a similar vein, typing "someone who is intelligent" into Midjourney leads to four images of **eyeglass-wearing, elderly white men**. The last image is particularly reminiscent of **Albert Einstein**.

This shows how generative models can reflect and amplify **stereotypes** present in their training data.

11.4.5 Example: COMPAS

Machine Bias: The COMPAS Case

Correctional Offender Management Profiling for Alternative Sanctions (COMPAS) is a case management and decision support tool developed and owned by **Northpointe (now Equivant)**, used by **U.S. courts** to assess the likelihood of a defendant becoming a **recidivist** (rearrest within 2 years).

- The algorithm's **false positives** (defendants predicted to re-offend but who actually did not) were **disproportionately black**, and the **false negatives** (defendants predicted not to re-offend but who actually re-offended) were **disproportionately white**. ⇒ **Bias against black defendants**. Formally, with $a \in \{0, 1\}$ the race indicator and $\hat{y}, y \in \{0, 1\}$ the predicted and true recidivism labels, the group-wise **false positive rate** and **false negative rate** are

$$\mathrm{FPR}_a = \frac{\#\{i : a_i = a,\ \hat{y}_i = 1,\ y_i = 0\}}{\#\{i : a_i = a,\ y_i = 0\}}, \qquad \mathrm{FNR}_a = \frac{\#\{i : a_i = a,\ \hat{y}_i = 0,\ y_i = 1\}}{\#\{i : a_i = a,\ y_i = 1\}},$$

  and COMPAS exhibits $\mathrm{FPR}_{\text{black}} > \mathrm{FPR}_{\text{white}}$ together with $\mathrm{FNR}_{\text{black}} < \mathrm{FNR}_{\text{white}}$.
- A famous example: **Bernard Parker** was rated high risk; **Dylan Fugett** was rated low risk (analysis by ProPublica, "Machine Bias": software used across the country to predict future criminals is biased against blacks).
- Another general critique of COMPAS is that since the algorithms it uses are **trade secrets**, they cannot be examined by the public and affected parties — which may be a **violation of due process** (**transparency issue**).

Two Distinct Problems in COMPAS

COMPAS illustrates two separate ethical failures: (1) **fairness** — asymmetric false positive/negative rates across racial groups; (2) **transparency** — trade-secret algorithms that cannot be publicly examined, potentially violating due process.

11.4.6 Sources of Unfairness

1. **Human Bias**
   - Data **reflects human decisions and biases**.
   - Example: Machine learning for **hiring decisions**. Data from previous hiring decisions **perpetuates existing biases**.
   - Could we reduce bias by measuring employee success? **Harder to measure**, and institutional biases can impact success.
2. **Negative Feedback Loops**
   - Data collected in a **biased fashion**.
   - **Negative feedback loop:** future observations **confirm predictions** and reduce further contradicting evidence.
   - Example: Allocation of **police attention** based on prevalence of crime.
3. **Sample Size Disparity**
   - Models for a **minority group may be less accurate** if less data is used. For a group g with n_g samples, the group-wise empirical error

$$\mathrm{err}_g = \frac{1}{n_g} \sum_{i:\, a_i = g} \mathbb{1}[\hat{y}_i \neq y_i]$$

     is estimated from fewer observations when n_g is small, so it can be both systematically larger and noisier than for the majority group.
   - Example: **Race representation in medical studies**.
4. **Unreliable Data**

- Data from minority groups is **less reliable or less informative** ⇒ models may be **less accurate for minority groups** ⇒ **beneficial interventions may be less available** to minority groups.
- Examples: Inaccurate **census** in predominantly minority neighborhoods; medical interventions with **limited diagnostic tools**.

5. **Proxies**
   - Even if **sensitive attributes** (e.g., gender or race) are not used by the model, there may be other **proxy features** that are **correlated** with sensitive attributes. Formally, even when the sensitive attribute a is excluded from the feature vector $\mathbf{x}$, a proxy z may satisfy $z \in \mathbf{x}$ while $|\text{Corr}(z, a)|$ is close to 1, so z leaks information about a into the prediction $\hat{y}$.
   - Example: In the case of race, other variables correlated with race — such as **postcode** — may be selected by the algorithm.
6. **Cultural Unawareness**
   - **Humans work in AI systems.** Choices are made by humans, and sometimes these choices can create biases.
   - Example: **Lack of diversity** in the AI developers and data science teams might negatively affect people who do not fit the group of AI developers.

Removing Sensitive Attributes Does NOT Guarantee Fairness

Simply dropping protected attributes like gender or race from the training data is insufficient: **proxy features** (e.g., postcode) correlated with the sensitive attribute can still leak bias into the model's decisions.

11.5 Moral Machines

11.5.1 Moral Decisions

Philosophers usually consider machines **not capable of making moral decisions**. However, one can try to find **properties** such that machines could act morally. Machines need to have at least:

Requirements for a Machine to be a Moral Agent

1. **Beliefs about the world**
2. **Pro-attitudes** (intentions)
3. **Moral knowledge**
4. **The possibility to compute what consequences** one's own action can have

— in which case they can be considered as **moral agents**.

11.5.2 Example

- **Self-driving cars** will come into situations where they have to **choose between bad alternatives** (e.g., killing the passenger or a pedestrian).
- How should such a car choose in such a situation?
- Note that because of its much **faster reactivity**, a car might be able to make decisions where a human cannot at all.
- Ask what **ordinary people** think a car should do in such moral dilemmas.
- **Moral Machine Website:** `https://www.moralmachine.net/`

The Trolley-Style Dilemma is a Metaphor

The self-driving car dilemma is really a **metaphor** used to highlight the **ethical challenges** that autonomous machines may encounter, rather than aiming at portraying a realistic situation.

11.5.3 Practice Problem

Matching EU Principles with Common Grounds

Match the European Union Principles with the given Common Grounds by completing the following table with numbers.

Seven European Union Principles:

(A) Human agency and oversight
(B) Technical robustness and safety
(C) Privacy and data governance
(D) Transparency
(E) Diversity, non-discrimination and fairness
(F) Societal and environmental well-being
(G) Accountability

Common Grounds:

(1) Autonomy
(2) Beneficence
(3) Non-maleficence
(4) Justice
(5) Explicability

Answer:

EU Principle	Common Ground
A	1
B	3
C	1
D	5
E	4
F	2
G	1

That is: Human agency and oversight → Autonomy; Technical robustness and safety → Non-maleficence; Privacy and data governance → Autonomy; Transparency → Explicability; Diversity, non-discrimination and fairness → Justice; Societal and environmental well-being → Beneficence; Accountability → Autonomy.

Summary

- **AI ethics** concerns the ethical issues around AI use; the Alan Turing Institute defines it as values, principles and techniques employing standards of right and wrong to guide AI development and use.
- Potential harms: privacy invasion, biased decisions, unexplainable decisions, unreliable decisions, denied accountability; real-world impacts include centralized tech power, deepfake abuse, unemployment, and disinformation.
- AI itself is **neither ethical nor unethical** — enterprises must establish principles and frameworks, and there is **no consensus** on ethical responsibilities.
- Major published principles include ACM (2017), IEEE (2017), UK House of Lords (2018), EU Ethics Guidelines (2019), and Google (2019–2021).
- The **seven EU principles** (human agency and oversight; technical robustness and safety; privacy and data governance; transparency; diversity, non-discrimination and fairness; societal and environmental well-being; accountability) synthesize into **five common grounds**: autonomy, beneficence, non-maleficence, justice, explicability.
- High-level principles are ambiguous and can conflict; Graeber's **interpretive labor** explains asymmetric burdens of understanding. Principles are orientation points and codes of behavior for us, not checklists — "The codes of ethics are for us; AI

systems will follow." (Dignum).
- Three operational areas of AI ethics: **data ethics** (consent, privacy, representation), **model fairness**, and **model monitoring and maintenance** (guarding against AI model drift).
- Fairness failures illustrated by **Midjourney** (stereotypical images) and **COMPAS** (racially asymmetric false positive/negative rates plus a trade-secret transparency problem).
- Six causes of unfairness: **human bias, negative feedback loops, sample size disparity, unreliable data, proxies, cultural unawareness**.
- Machines as moral agents require beliefs, pro-attitudes, moral knowledge, and the ability to compute consequences; the self-driving car dilemma (cf. Moral Machine) is a metaphor for the ethical challenges of autonomous systems.

12 Reinforcement Learning

12.1 Introduction

12.1.1 Goal

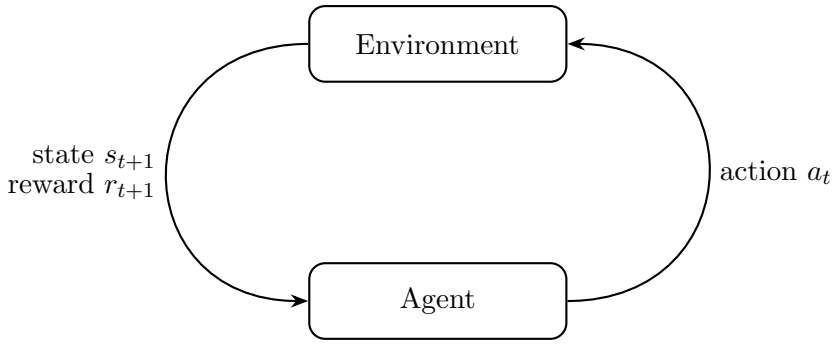
The goal of **Reinforcement Learning (RL)** is to design an autonomous/intelligent agent that learns by **interacting with an environment**.

The Standard RL Setting

In the standard RL setting, the following loop repeats at every time step:
1. The agent **perceives a state** s_t at every time step.
2. The agent **chooses an action** a_t.
3. The action is **applied to the environment**.
4. The environment returns a **reward** and a **new state** s_{t+1}.
5. The agent trains a **policy** to choose actions that maximize the sum of rewards.

Formally, the agent seeks a policy π that maximizes the **expected discounted return**

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \qquad \pi^* = \arg\max_{\pi} \mathbb{E}_{\pi}[G_t]\,.$$



12.1.2 Definitions

Two technically correct (but somewhat abstract) definitions from experts in the field:

- **NVIDIA:** "RL, a type of machine learning, in which agents take actions in an environment aimed at maximizing their cumulative rewards."

- **Gartner:** "RL is based on rewarding desired behaviors or punishing undesired ones. Instead of one input producing one output, the algorithm produces a variety of outputs and is trained to select the right one based on certain variables."

The above definitions are technically provided by experts in the field; however, for someone who is starting with RL, these definitions might feel a little bit difficult. A friendlier definition:

Beginner-Friendly Definition of RL

Through a series of **Trial and Error** methods, an agent keeps learning continuously in an interactive environment from its own actions and experiences. The only goal of it is to find a suitable action model which would increase the total cumulative reward of the agent. It learns via **interaction and feedback**.

12.1.3 Example: Training a Dog

Imagine training your dog to complete a task within an environment.

- First, the trainer **issues a command**, which the dog observes (observation); the dog then responds by **taking an action**.
- If the action is close to the desired behavior, the trainer will likely provide a **reward**, such as a food treat or a toy. Otherwise, no reward or a negative reward will be provided.
- At the beginning of training, the dog will likely take more random actions — like rolling over when the command given is "sit" — as it is trying to associate specific observations with actions and rewards.
- This **association**, or **mapping**, between observations and actions is called a **policy**.
- From the dog's perspective, the ideal case would be one in which it would respond correctly to every command, so that it gets as many treats as possible.
- So, the whole meaning of reinforcement learning training is to "tune" the dog's policy so that it learns the desired behaviors that will maximize some reward.
- After training is complete, the dog should be able to observe the owner and take the appropriate action — for example, sitting when commanded to "sit" — by using the internal policy it has developed.

Policy

A **policy** π is the association (mapping) from observations (states) to actions, written $a = \pi(s)$ (deterministic) or $\pi(a \mid s)$ (stochastic). Training in RL means tuning this policy so that the resulting behaviors maximize the cumulative reward $\mathbb{E}_\pi[G_t]$.

Example: Identifying the RL Components in Dog Training

Question: List the following in reference to the dog training example: Agent, Environment, Observations, Actions, Rewards, Policy.

Answer:
- **Agent:** Your dog.
- **Environment:** Your home, backyard, or any other place where you teach and play with your dog.
- **Observations:** What the dog observes (e.g., the trainer's command).
- **Actions:** Sit, Roll, Stand, Walk, etc.
- **Rewards:** Food treat or a toy.
- **Policy:** Generate the correct actions from the observations.

Example: Identifying the RL Components in Automated Parking

Question: Consider the task of parking a vehicle using an automated driving system. List the following in reference to this problem: Agent, Environment, Observations, Actions, Rewards, Policy.

Answer:
- **Agent:** Vehicle computer.
- **Environment:** Parking area.
- **Observations:** Readings from sensors such as cameras, GPS, and lidar (light detection and ranging).
- **Actions:** Generate steering, braking, and acceleration commands.
- **Rewards:** Reach the parking point as soon as possible.
- **Policy:** Generate the correct actions from the observations.

12.2 Basic Concepts

12.2.1 Why Discounting?

Example: Academic Life — What Is Wrong with This Argument?

An assistant professor gets paid, say, 160K per year. How much, in total, will the assistant professor earn in their life?

$$160 + 160 + 160 + 160 + 160 + \cdots = \sum_{k=0}^{\infty} 160 = +\infty.$$

Question: What's wrong with this argument?

Answer: The argument treats every future payment as equally valuable as money received today, and sums infinitely many of them. In reality, **a reward in the future is worth less than the same reward now**, so the future payments must be discounted — which makes the total finite.

12.2.2 Discounted Rewards

A reward (payment) in the future is not worth quite as much as a reward now, because of inflation. For example, being promised \$10,000 next year is worth only 90% as much as receiving \$10,000 right now.

Discounted Sum of Future Rewards

Discounting uses a parameter called the **discount factor** γ, with $0 \leq \gamma \leq 1$, and a power of it multiplies each reward. The **discounted sum of future rewards** (the **return**) starting at time step t is

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \end{aligned}$$

so the reward received now (at time $t+1$) is weighted by $\gamma^0 = 1$, the reward one time step ahead by γ^1, the reward two time steps ahead by γ^2, and so on. People in economics and probabilistic decision-making do this all the time.

Geometric Series Reminder

For $0 \leq \gamma < 1$, the infinite geometric series converges:

$$1 + \gamma + \gamma^2 + \gamma^3 + \ldots = \frac{1}{1-\gamma}$$

This is exactly why discounting turns an infinite total reward into a finite number.

Example: Discounted Sum of the Professor's Earnings

Question: Assume a payment n years in the future is worth only $(0.9)^n$ of a payment now. What is the assistant professor's future discounted sum of rewards (salary 160K per year)?

Answer:

$$\begin{aligned} &160 + 160 \times (0.9)^1 + 160 \times (0.9)^2 + 160 \times (0.9)^3 + \ldots \\ &= 160 \times \left(1 + 0.9 + (0.9)^2 + (0.9)^3 + \ldots\right) \\ &= 160 \times \left(\frac{1}{1-0.9}\right) = 1600 \end{aligned}$$

So the discounted lifetime earning is 1600K = 1.6M — a finite number, even though the undiscounted sum diverges.

Infinite Undiscounted Sums

Never sum an infinite stream of equal rewards directly — the sum diverges to infinity and carries no useful information. Always apply a discount factor $\gamma < 1$ (or a finite horizon) so that the total becomes finite and comparable across policies.

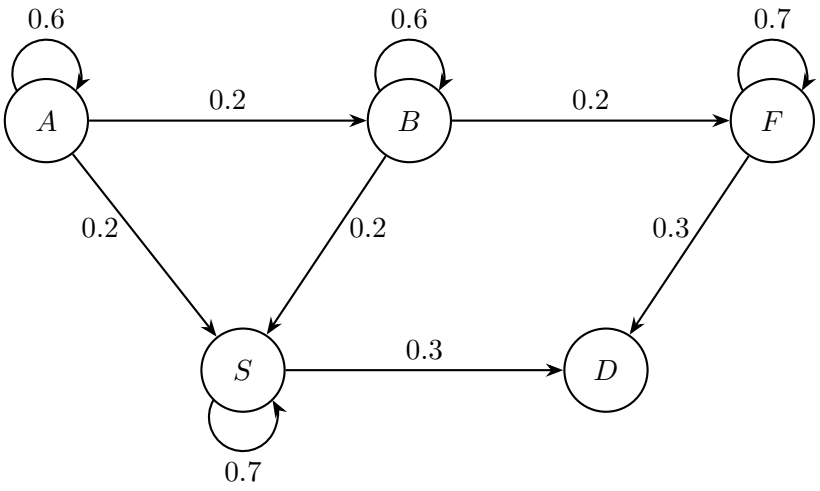
12.2.3 Example: Academic Life

Consider the following Markov system modelling an academic career, with five states:

State	Meaning	Reward
A	Assistant Professor	160
B	Associate Professor	480
F	Full Professor	3200
S	On the Street	80
D	Dead	0

The transition probabilities are:

From	To	Probability
A	A (self-loop)	0.6
A	B	0.2
A	S	0.2
B	B (self-loop)	0.6
B	F	0.2
B	S	0.2
F	F (self-loop)	0.7
F	D	0.3
S	S (self-loop)	0.7
S	D	0.3
D	—	(no outgoing transitions)



Define the following quantities:

- $V_A = \mathbb{E}[G_t \mid S_t = A]$: expected discounted sum of future rewards starting in state A
- $V_B = \mathbb{E}[G_t \mid S_t = B]$: expected discounted sum of future rewards starting in state B
- $V_F = \mathbb{E}[G_t \mid S_t = F]$: expected discounted sum of future rewards starting in state F
- $V_S = \mathbb{E}[G_t \mid S_t = S]$: expected discounted sum of future rewards starting in state S
- $V_D = \mathbb{E}[G_t \mid S_t = D]$: expected discounted sum of future rewards starting in state D

Question: Assume discount factor $\gamma = 0.9$. How do we compute V_A, V_B, V_F, V_S, V_D?

Hint: Expected Value Computation

Suppose we roll a fair 6-sided die. The expected value of our die roll is

$$\left(\frac{1}{6}\right) \times 1 + \left(\frac{1}{6}\right) \times 2 + \left(\frac{1}{6}\right) \times 3 + \left(\frac{1}{6}\right) \times 4 + \left(\frac{1}{6}\right) \times 5 + \left(\frac{1}{6}\right) \times 6 = 3.5$$

The value of a state is computed the same way: a probability-weighted average over what can happen next, plus the immediate reward.

12.2.4 Sampling Episodes

Starting from state A, there are so many different possibilities — each episode (path through the system) occurs with a different probability. Each **sample episode** contributes its rewards weighted by the probability of the path taken so far, discounted by γ^t at time step t.

Example: Sample Episodes Starting from State A ($\gamma = 0.9$)

Sample episodes, all starting from A:

Episode $A \to B \to F \to D$:

$$160 + (0.2)(0.9)^1(480) + (0.2)(0.2)(0.9)^2(3200) + (0.2)(0.2)(0.3)(0.9)^3(0)$$
$$= 160 + 86.4 + 103.68 + 0 = 350.08$$

Episode $A \to A \to S \to D$:

$$160 + (0.6)(0.9)^1(160) + (0.6)(0.2)(0.9)^2(80) + (0.6)(0.2)(0.3)(0.9)^3(0)$$
$$= 160 + 86.4 + 7.776 + 0 = 254.176$$

Episode $A \to B \to S \to D$:

$$160 + (0.2)(0.9)^1(480) + (0.2)(0.2)(0.9)^2(80) + (0.2)(0.2)(0.3)(0.9)^3(0)$$
$$= 160 + 86.4 + 2.592 + 0 = 248.992$$

And there are infinitely many more episodes ($A \to A \to A \to \ldots$). It is very difficult to compute V_A by enumerating episodes. $>.<$

12.2.5 Bootstrapping

Idea: instead of enumerating infinite episodes, compute values for finite horizons and build up.

Let $V_A^1, V_B^1, V_F^1, V_S^1, V_D^1$ be the expected discounted sum of rewards over the next 1 time step from now. These are just the immediate rewards. Then let V_A^2 be the expected discounted sum of rewards over the next 2 time steps — it can be found from the V^1 values.

Example: Computing V^1 and V_A^2 for the Academic Life ($\gamma = 0.9$)

Step 1 (horizon $k = 1$): the 1-step values are exactly the immediate rewards:

$$V_A^1 = 160, \qquad V_B^1 = 480, \qquad V_F^1 = 3200, \qquad V_S^1 = 80, \qquad V_D^1 = 0$$

Step 2 (horizon $k = 2$): use the transition probabilities out of A to average the V^1 values:

$$\begin{aligned} V_A^2 &= 160 + \gamma \left(T_{AA} V_A^1 + T_{AB} V_B^1 + T_{AF} V_F^1 + T_{AS} V_S^1 + T_{AD} V_D^1 \right) \\ &= 160 + 0.9 \times (0.6(160) + 0.2(480) + 0(3200) + 0.2(80) + 0(0)) \\ &= 160 + 0.9 \times (96 + 96 + 0 + 16 + 0) = 160 + 0.9 \times 208 = 347.2 \end{aligned}$$

Exercise: Do you know how to compute V_B^2, V_F^2, V_S^2, V_D^2 the same way? (Apply each state's own row of transition probabilities.)

This bootstrapping idea — computing k-step values from $(k-1)$-step values — is exactly **value iteration**, formalized below.

12.2.6 Markov Property

Markov Property

A state S_t is **Markov** if and only if

$$\mathbb{P}(S_{t+1} = s' \mid S_t = s) = \mathbb{P}(S_{t+1} = s' \mid S_1 = s_1, \ldots, S_t = s), \qquad \forall\, s', s, s_1, \ldots, s_t \in S.$$

Interpretation:
- The future is independent of the past given the present.
- The present state captures **all relevant information** from the history.
- Once the present state is known, the history may be **thrown away**.

12.3 Problem Formulation

12.3.1 Markov System with Rewards

A **Markov system** (with rewards) consists of the following:

1. A set of N **states** $\{s_1, s_2, \ldots, s_N\}$.

2. A **transition probability matrix** (i.e., a 2D array showing the probability of going from one state to another state):

$$T = \begin{bmatrix} T_{11} & T_{12} & \cdots & T_{1N} \\ T_{21} & T_{22} & \cdots & T_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ T_{N1} & T_{N2} & \cdots & T_{NN} \end{bmatrix} \quad \text{(rows: From; columns: To)}$$

where $T_{ij} = \mathbb{P}(S_{t+1} = s_j \mid S_t = s_i)$ is the probability of moving from state s_i to state s_j.

Note: each row of the matrix sums to 1.

3. Each state has a **reward** $\{r_1, r_2, \ldots, r_N\}$.
4. There is a **discount factor** γ, where $0 < \gamma < 1$. All future rewards are discounted by γ.

The matrix T is exactly the transition matrix P of the underlying (reward-free) **Markov chain**, with entries $p_{ij} = T_{ij} = \mathbb{P}(S_{t+1} = s_j \mid S_t = s_i)$; every row of P is a probability distribution over the next state, $\sum_{j=1}^{N} p_{ij} = 1$. A probability vector $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_N)$ with $\sum_{i=1}^{N} \pi_i = 1$ is a **stationary distribution** of the chain if it is invariant under one transition step,

$$\boldsymbol{\pi} = \boldsymbol{\pi} P \quad \Longleftrightarrow \quad \pi_j = \sum_{i=1}^{N} \pi_i \, p_{ij} \ \text{ for all } j,$$

so $\boldsymbol{\pi}$ describes the long-run fraction of time the chain spends in each state.

Example: The Academic Life as a Markov System

Question: What are the states, transition probability matrix, rewards, and discount factor for the academic life problem?

Answer:

- **States:** $\{A, B, F, S, D\}$
- **Transition probability matrix** (rows/columns in the order A, B, F, S, D):

$$T = \left[\begin{array}{c|ccccc} & A & B & F & S & D \\ \hline A & 0.6 & 0.2 & 0 & 0.2 & 0 \\ B & 0 & 0.6 & 0.2 & 0.2 & 0 \\ F & 0 & 0 & 0.7 & 0 & 0.3 \\ S & 0 & 0 & 0 & 0.7 & 0.3 \\ D & 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

- **Rewards:** $\{160, 480, 3200, 80, 0\}$
- **Discount factor:** 0.9

Row Sums of the Transition Matrix

Each row of a transition probability matrix must sum to 1 — except for a **dead-end (absorbing) state** like D above, which has no outgoing transitions and hence a row of all zeros. Do not "fix" such a row by inventing transitions; it simply models the end of every episode.

12.3.2 Value Function

Solving a Markov system means finding the value of every state. For a fixed policy, the **state-value function** is

$$V(s_i) = \mathbb{E}[G_t \mid S_t = s_i] = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s_i\right].$$

The value satisfies the **Bellman expectation equation** (for a Markov reward process):

$$\begin{aligned} V(s_i) &= r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}\, V(s_j) \\ &= r(s_i) + \gamma \left(T_{i1} V(s_1) + T_{i2} V(s_2) + \ldots + T_{iN} V(s_N) \right), \end{aligned}$$

where $r(s_i)$ is the immediate reward and $\gamma \sum_j T_{ij} V(s_j)$ is the discounted expected value of the next state.

Using vector notation, with the value vector $\mathbf{V} = (V(s_1), \ldots, V(s_N))^\top$ and reward vector $\mathbf{r} = (r_1, \ldots, r_N)^\top$, the recursion above is the compact linear system $\mathbf{V} = \mathbf{r} + \gamma\, T\, \mathbf{V}$, i.e., written out in full,

$$\begin{bmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_N \end{bmatrix} + \gamma \begin{bmatrix} T_{11} & T_{12} & \cdots & T_{1N} \\ T_{21} & T_{22} & \cdots & T_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ T_{N1} & T_{N2} & \cdots & T_{NN} \end{bmatrix} \begin{bmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{bmatrix}.$$

This is the **Bellman equation** for a Markov reward system.

12.3.3 Direct Solution

The equation above is a linear equation, which can be solved directly by rearrangement:

$$\begin{aligned} \mathbf{V} &= \mathbf{r} + \gamma\, T\, \mathbf{V} \\ (I - \gamma T)\, \mathbf{V} &= \mathbf{r} \\ \mathbf{V} &= (I - \gamma T)^{-1} \mathbf{r} \end{aligned}$$

where I is the $N \times N$ identity matrix and $(I - \gamma T)^{-1}$ is the matrix inverse.

- **The good thing** about directly solving the above equation is that you get an exact number.
- **The bad thing** is that it is slow if you have a large number of states, i.e., N is big (matrix inversion is expensive).

There are many **iterative methods** for solving the equation, e.g.,

1. **Dynamic programming** (we will do Value Iteration),
2. **Monte-Carlo evaluation**,
3. **Temporal-Difference learning**.

12.3.4 Value Iteration

Define the finite-horizon values:

- $V_i^1(s)$ = expected discounted sum of rewards over the next 1 time step from now
- $V_i^2(s)$ = expected discounted sum of rewards over the next 2 time steps from now
- $V_i^3(s)$ = expected discounted sum of rewards over the next 3 time steps from now
- $\cdots$
- $V_i^k(s)$ = expected discounted sum of rewards over the next k time steps from now

Value Iteration Recurrence (No Actions)

The formulas to compute all of them:

$$V^1(s_i) = r(s_i)$$
$$V^2(s_i) = r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}\, V^1(s_j)$$
$$V^3(s_i) = r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}\, V^2(s_j)$$
$$\vdots$$
$$V^k(s_i) = r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}\, V^{k-1}(s_j)$$

Equivalently, each sweep is the vector update $V^k = R + \gamma\, T\, V^{k-1}$. As $k \to \infty$, V^k converges (geometrically, at rate γ) to the true value $V = (I - \gamma T)^{-1} R$.

12.3.5 Example: Weather

Example: Value Iteration for the Weather System ($\gamma = 0.5$)

Consider a weather system with three states — S (Sun, reward 4), W (Wind, reward 0), H (Hail, reward -8) — and transition matrix (rows/columns in the order S, W, H):

$$T = \left[\begin{array}{c|ccc} & S & W & H \\ \hline S & 0.5 & 0.5 & 0 \\ W & 0.5 & 0 & 0.5 \\ H & 0 & 0.5 & 0.5 \end{array}\right]$$

i.e., from Sun we go to Sun or Wind with probability 0.5 each; from Wind to Sun or Hail; from Hail to Wind or stay in Hail.

Iteration $k = 1$ (immediate rewards):

$$V^1(S) = r(S) = 4, \qquad V^1(W) = r(W) = 0, \qquad V^1(H) = r(H) = -8$$

Iteration $k = 2$:

$$\begin{aligned} V^2(S) &= r(S) + \gamma \left(T_{SS}V^1(S) + T_{SW}V^1(W) + T_{SH}V^1(H)\right) \\ &= 4 + 0.5\,(0.5 \times 4 + 0.5 \times 0 + 0 \times (-8)) = 5 \\ V^2(W) &= r(W) + \gamma \left(T_{WS}V^1(S) + T_{WW}V^1(W) + T_{WH}V^1(H)\right) \\ &= 0 + 0.5\,(0.5 \times 4 + 0 \times 0 + 0.5 \times (-8)) = -1 \\ V^2(H) &= r(H) + \gamma \left(T_{HS}V^1(S) + T_{HW}V^1(W) + T_{HH}V^1(H)\right) \\ &= -8 + 0.5\,(0 \times 4 + 0.5 \times 0 + 0.5 \times (-8)) = -10 \end{aligned}$$

Iteration $k = 3$:

$$\begin{aligned} V^3(S) &= 4 + 0.5\,(0.5 \times 5 + 0.5 \times (-1) + 0 \times (-10)) = 5 \\ V^3(W) &= 0 + 0.5\,(0.5 \times 5 + 0 \times (-1) + 0.5 \times (-10)) = -1.25 \\ V^3(H) &= -8 + 0.5\,(0 \times 5 + 0.5 \times (-1) + 0.5 \times (-10)) = -10.75 \end{aligned}$$

Iteration $k = 4$:

$$V^4(S) = 4 + 0.5\,(0.5 \times 5 + 0.5 \times (-1.25) + 0 \times (-10.75)) = 4.9375$$
$$V^4(W) = 0 + 0.5\,(0.5 \times 5 + 0 \times (-1.25) + 0.5 \times (-10.75)) = -1.4375$$
$$V^4(H) = -8 + 0.5\,(0 \times 5 + 0.5 \times (-1.25) + 0.5 \times (-10.75)) = -11$$

Iteration $k = 5$:

$$V^5(S) = 4 + 0.5\,(0.5 \times 4.9375 + 0.5 \times (-1.4375) + 0 \times (-11)) = 4.875$$
$$V^5(W) = 0 + 0.5\,(0.5 \times 4.9375 + 0 \times (-1.4375) + 0.5 \times (-11)) = -1.515625$$
$$V^5(H) = -8 + 0.5\,(0 \times 4.9375 + 0.5 \times (-1.4375) + 0.5 \times (-11)) = -11.109375$$

Summary of iterations:

k	$V_k(S)$	$V_k(W)$	$V_k(H)$
1	4	0	-8
2	5	-1	-10
3	5	-1.25	-10.75
4	4.9375	-1.4375	-11
5	4.875	-1.515625	-11.109375

Observe that successive values change less and less — the iteration is converging.

12.3.6 Algorithm and Stopping

The value iteration procedure for solving the system of linear equations:

1. Compute $V^1(s_i)$ for each i in range $[1, N]$.
2. Compute $V^2(s_i)$ for each i in range $[1, N]$.
3. Compute $V^3(s_i)$ for each i in range $[1, N]$.

   ...
4. Compute $V^k(s_i)$ for each i in range $[1, N]$.

Stopping Criterion

When to stop? When the maximum absolute difference between two successive expected discounted sums of rewards (V^k and V^{k-1}) is less than a threshold ξ, i.e.,

$$\left\|V^k - V^{k-1}\right\|_\infty = \max_i \left|V^k(s_i) - V^{k-1}(s_i)\right| < \xi.$$

```
import numpy as np

# Weather example: states ordered as (S, W, H)
T = np.array([[0.5, 0.5, 0.0],
              [0.5, 0.0, 0.5],
              [0.0, 0.5, 0.5]])
R = np.array([4.0, 0.0, -8.0])
gamma = 0.5
xi = 1e-6                       # stopping threshold

V = R.copy()                    # V^1(s) = r(s)
while True:
    V_new = R + gamma * (T @ V)        # Bellman update
    if np.max(np.abs(V_new - V)) < xi: # stopping criterion
        V = V_new
        break
    V = V_new
print(V)                        # converged state values
```

12.4 Markov Decision Processes

12.4.1 Definition

A **Markov Decision Process (MDP)** is a Markov reward process with decisions. It is an environment in which **all states are Markov**.

Markov Decision Process

A Markov Decision Process is a tuple $\langle S, A, T, R, \gamma \rangle$:

- S: a finite set of **states** $\{s_1, s_2, \ldots, s_N\}$
- A: a finite set of **actions** $\{a_1, a_2, \ldots, a_M\}$
- T: a **transition probability matrix** per action, with entries

$$T_{ij}^a = p(s_j \mid s_i, a) = \mathbb{P}(S_{t+1} = s_j \mid S_t = s_i, A_t = a), \qquad \sum_{j=1}^{N} T_{ij}^a = 1 \;\; \forall\, i, a;$$

- R: each state has a **reward** $\{r_1, r_2, \ldots, r_N\}$
- γ: a **discount factor**, $0 \leq \gamma \leq 1$

Given a policy $\pi(a \mid s)$, the induced **state-value** and **action-value** functions are

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]\,,$$
$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]\,,$$

and V^π obeys the **Bellman expectation equation**

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)\,[r(s, a) + \gamma\, V^\pi(s')]\,,$$

where $p(s' \mid s, a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a) = T_{ss'}^a$ and $r(s, a)$ is the expected immediate reward.

12.4.2 Bellman Optimality Equation

How to determine actions? As before, define $V_i^1(s), V_i^2(s), \ldots, V_i^k(s)$ as the expected discounted sums of rewards over the next $1, 2, \ldots, k$ time steps. Now, at every step the agent chooses the best action:

$$\begin{aligned}
V^1(s_i) &= r(s_i) \\
V^2(s_i) &= \max_a \left(r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}^a\, V^1(s_j) \right) \\
V^3(s_i) &= \max_a \left(r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}^a\, V^2(s_j) \right) \\
&\vdots \\
V^k(s_i) &= \max_a \left(r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}^a\, V^{k-1}(s_j) \right)
\end{aligned}$$

Bellman Optimality Equation

The fixed point of the iteration above (as $k \to \infty$, $V^k \to V^*$) is the **Bellman optimality equation**

$$V^*(s) = \max_a \sum_{s'} p(s' \mid s, a) \left[r(s, a) + \gamma V^*(s') \right],$$

which, for the tabular transition matrices, reads

$$V^*(s_i) = \max_a \left(r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}^a V^*(s_j) \right).$$

The only difference from the no-action case is the $\max_a$: in each state we pick the action that yields the largest expected discounted sum of future rewards.

Markov Reward System vs. MDP

In a plain Markov system with rewards there are no decisions, so value iteration just averages over the next states using the (single) transition matrix T. In an MDP there is one transition matrix T^a per action, and the update takes a **maximum over actions** — forgetting the $\max_a$ turns an MDP back into a Markov reward system and gives wrong values.

12.4.3 Policies

Policy

A **policy** π specifies the agent's behaviour. It may be deterministic, $\pi : S \to A$ with $a = \pi(s)$, or stochastic, $\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s)$, a conditional distribution over actions with $\sum_a \pi(a \mid s) = 1$ for every state s. A deterministic policy is the special case $\pi(a \mid s) = \mathbb{1}[a = \pi(s)]$.

To illustrate, consider an MDP with four states — PU (Poor & Unknown, reward 0), PF (Poor & Famous, reward 0), RU (Rich & Unknown, reward 10), RF (Rich & Famous, reward 10) — and two actions A (Advertise) and S (Save). Two example policies:

Policy 1	
State	**Action**
PU	S
PF	A
RU	S
RF	A

Policy 2	
State	**Action**
PU	A
PF	A
RU	A
RF	A

Different policies lead to different expected discounted sums of rewards — the goal is to find the best one.

12.4.4 Extracting the Policy

Extracting the Near-Optimal Policy from Value Iteration

1. Compute $V^k(s_i)$ for all i using value iteration, until

$$\left\| V^{k+1} - V^k \right\|_\infty = \max_i \left| V^{k+1}(s_i) - V^k(s_i) \right| < \xi,$$

2. Compute the (near-)optimal policy $\pi(s_i)$ in state s_i as

$$\pi(s_i) = \arg\max_a \left(r(s_i) + \gamma \sum_{j=1}^{N} T_{ij}^a \, V^{k-1}(s_j) \right).$$

Once it is done, the near-optimal policy consists of taking the action that leads to the state(s) with maximum state value.

12.4.5 Example: Poor/Rich MDP

Example: Full Value Iteration for the PU/PF/RU/RF MDP ($\gamma = 0.9$)

Setup. States: PU (Poor & Unknown), PF (Poor & Famous), RU (Rich & Unknown), RF (Rich & Famous). Actions: A (Advertise), S (Save).
Reward vector (order PU, PF, RU, RF):

$$\mathbf{R} = [0 \quad 0 \quad 10 \quad 10]$$

Transition matrices:

$$T^A = \left[\begin{array}{c|cccc} & PU & PF & RU & RF \\ \hline PU & 0.5 & 0.5 & 0 & 0 \\ PF & 0 & 1 & 0 & 0 \\ RU & 0.5 & 0.5 & 0 & 0 \\ RF & 0 & 1 & 0 & 0 \end{array}\right] \qquad T^S = \left[\begin{array}{c|cccc} & PU & PF & RU & RF \\ \hline PU & 1 & 0 & 0 & 0 \\ PF & 0.5 & 0 & 0 & 0.5 \\ RU & 0.5 & 0 & 0.5 & 0 \\ RF & 0 & 0 & 0.5 & 0.5 \end{array}\right]$$

Iteration $k = 1$ (immediate rewards):

$$V^1(PU) = 0, \qquad V^1(PF) = 0, \qquad V^1(RU) = 10, \qquad V^1(RF) = 10$$

Iteration $k = 2$:

$$\begin{aligned} V^2(PU) &= \max_{a \in \{A,S\}} \Big(r(PU) + \gamma \textstyle\sum_{s'} T_{PU,s'}^a V^1(s') \Big) \\ &= \max\big(0 + 0.9(0.5{\cdot}0 + 0.5{\cdot}0 + 0{\cdot}10 + 0{\cdot}10), \\ &\qquad 0 + 0.9(1{\cdot}0 + 0{\cdot}0 + 0{\cdot}10 + 0{\cdot}10)\big) = \max(0, 0) = 0 \end{aligned}$$

(Tie: both A and S are optimal at PU this round.)

$$\begin{aligned} V^2(PF) &= \max\big(0 + 0.9(0{\cdot}0 + 1{\cdot}0 + 0{\cdot}10 + 0{\cdot}10), \\ &\qquad 0 + 0.9(0.5{\cdot}0 + 0{\cdot}0 + 0{\cdot}10 + 0.5{\cdot}10)\big) = \max(0, 4.5) = 4.5 \;\Rightarrow\; S \end{aligned}$$

$$\begin{aligned} V^2(RU) &= \max\big(10 + 0.9(0.5{\cdot}0 + 0.5{\cdot}0 + 0{\cdot}10 + 0{\cdot}10), \\ &\qquad 10 + 0.9(0.5{\cdot}0 + 0{\cdot}0 + 0.5{\cdot}10 + 0{\cdot}10)\big) = \max(10, 14.5) = 14.5 \;\Rightarrow\; S \end{aligned}$$

$$\begin{aligned} V^2(RF) &= \max\big(10 + 0.9(0{\cdot}0 + 1{\cdot}0 + 0{\cdot}10 + 0{\cdot}10), \\ &\qquad 10 + 0.9(0{\cdot}0 + 0{\cdot}0 + 0.5{\cdot}10 + 0.5{\cdot}10)\big) = \max(10, 19) = 19 \;\Rightarrow\; S \end{aligned}$$

Iteration $k = 3$:

$$\begin{aligned} V^3(PU) &= \max\big(0 + 0.9(0.5{\cdot}0 + 0.5{\cdot}4.5 + 0{\cdot}14.5 + 0{\cdot}19), \\ &\qquad 0 + 0.9(1{\cdot}0 + 0{\cdot}4.5 + 0{\cdot}14.5 + 0{\cdot}19)\big) = \max(2.025, 0) = 2.025 \;\Rightarrow\; A \end{aligned}$$

$$V^3(PF) = \max\big(0 + 0.9(0{\cdot}0 + 1{\cdot}4.5 + 0{\cdot}14.5 + 0{\cdot}19),\\ 0 + 0.9(0.5{\cdot}0 + 0{\cdot}4.5 + 0{\cdot}14.5 + 0.5{\cdot}19)\big)\\ = \max(4.05, 8.55) = 8.55 \;\Rightarrow\; S$$

$$V^3(RU) = \max\big(10 + 0.9(0.5{\cdot}0 + 0.5{\cdot}4.5 + 0{\cdot}14.5 + 0{\cdot}19),\\ 10 + 0.9(0.5{\cdot}0 + 0{\cdot}4.5 + 0.5{\cdot}14.5 + 0{\cdot}19)\big)\\ = \max(12.025, 16.525) = 16.525 \;\Rightarrow\; S$$

$$V^3(RF) = \max\big(10 + 0.9(0{\cdot}0 + 1{\cdot}4.5 + 0{\cdot}14.5 + 0{\cdot}19),\\ 10 + 0.9(0{\cdot}0 + 0{\cdot}4.5 + 0.5{\cdot}14.5 + 0.5{\cdot}19)\big)\\ = \max(14.05, 25.075) = 25.075 \;\Rightarrow\; S$$

Iteration $k = 4$:

$$V^4(PU) = \max\big(0 + 0.9(0.5{\cdot}2.025 + 0.5{\cdot}8.55 + 0{\cdot}16.525 + 0{\cdot}25.075),\\ 0 + 0.9(1{\cdot}2.025 + 0{\cdot}8.55 + 0{\cdot}16.525 + 0{\cdot}25.075)\big)\\ = \max(4.75875, 1.8225) = 4.75875 \;\Rightarrow\; A$$

$$V^4(PF) = \max\big(0 + 0.9(0{\cdot}2.025 + 1{\cdot}8.55 + 0{\cdot}16.525 + 0{\cdot}25.075),\\ 0 + 0.9(0.5{\cdot}2.025 + 0{\cdot}8.55 + 0{\cdot}16.525 + 0.5{\cdot}25.075)\big)\\ = \max(7.695, 12.195) = 12.195 \;\Rightarrow\; S$$

$$V^4(RU) = \max\big(10 + 0.9(0.5{\cdot}2.025 + 0.5{\cdot}8.55 + 0{\cdot}16.525 + 0{\cdot}25.075),\\ 10 + 0.9(0.5{\cdot}2.025 + 0{\cdot}8.55 + 0.5{\cdot}16.525 + 0{\cdot}25.075)\big)\\ = \max(14.75875, 18.3475) = 18.3475 \;\Rightarrow\; S$$

$$V^4(RF) = \max\big(10 + 0.9(0{\cdot}2.025 + 1{\cdot}8.55 + 0{\cdot}16.525 + 0{\cdot}25.075),\\ 10 + 0.9(0{\cdot}2.025 + 0{\cdot}8.55 + 0.5{\cdot}16.525 + 0.5{\cdot}25.075)\big)\\ = \max(17.695, 28.72) = 28.72 \;\Rightarrow\; S$$

Iterations $k = 5$ and $k = 6$: applying the same update rule gives

$$V^5:\ V^5(PU) = 7.62919,\ V^5(PF) = 15.0654,\\ V^5(RU) = 20.3978,\ V^5(RF) = 31.1804,\\ V^6:\ V^6(PU) = 10.2126,\ V^6(PF) = 17.4643,\\ V^6(RU) = 22.6121,\ V^6(RF) = 33.2102$$

with policy (A, S, S, S) in both rounds.

Summary of values and policies:

k	$V(PU)$	$V(PF)$	$V(RU)$	$V(RF)$	$\pi(PU)$	$\pi(PF)$	$\pi(RU)$	$\pi(RF)$
1	0	0	10	10	—	—	—	—
2	0	4.5	14.5	19	A/S	S	S	S
3	2.025	8.55	16.525	25.075	A	S	S	S
4	4.75875	12.195	18.3475	28.72	A	S	S	S
5	7.62919	15.0654	20.3978	31.1804	A	S	S	S
6	10.2126	17.4643	22.6121	33.2102	A	S	S	S

The near-optimal policy stabilizes at $\pi(PU) = A$ (Advertise when Poor & Unknown, to become famous) and $\pi = S$ (Save) in all other states.

Practice: Can you calculate the remaining ones (e.g., verify V^5 and V^6, and continue until $\max_i |V^{k+1}(s_i) - V^k(s_i)| < \xi$)? :)

```
import numpy as np

# States: PU, PF, RU, RF; actions: 0 = A (advertise), 1 = S (save)
TA = np.array([[0.5, 0.5, 0.0, 0.0],
               [0.0, 1.0, 0.0, 0.0],
               [0.5, 0.5, 0.0, 0.0],
               [0.0, 1.0, 0.0, 0.0]])
TS = np.array([[1.0, 0.0, 0.0, 0.0],
               [0.5, 0.0, 0.0, 0.5],
               [0.5, 0.0, 0.5, 0.0],
               [0.0, 0.0, 0.5, 0.5]])
T = np.stack([TA, TS])          # T[a][i][j] = P(j | i, a)
R = np.array([0.0, 0.0, 10.0, 10.0])
gamma = 0.9
xi = 1e-6

V = R.copy()                    # V^1(s) = r(s)
while True:
    # Q[a][i]: value of action a in state i
    Q = R + gamma * (T @ V)
    V_new = Q.max(axis=0)                # Bellman optimality update
    if np.max(np.abs(V_new - V)) < xi:   # stopping criterion
        V = V_new
        break
    V = V_new

Q = R + gamma * (T @ V)
policy = Q.argmax(axis=0)       # 0 = A, 1 = S
print(V, policy)
```

12.4.6 Pros and Cons

Pros and Cons of Value Iteration

Pros:
- Will converge towards optimal values.
- Good for a small set of states.

Cons:
- Value iteration has to touch every state in every iteration, so if we have a large number of total states, value iteration suffers.
- It is slow because we have to consider all actions at every state, and often there are many actions.

12.4.7 Temporal-Difference Learning and Q-Learning

The iterative methods listed earlier can all be written as sample-based updates that do not require the full model. Writing the **learning rate (step size)** as $\alpha \in (0, 1]$:

- **Monte-Carlo** update (targets the full sampled return G_t):
$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_t - V(S_t) \right].$$
- **Temporal-Difference (TD)** update (bootstraps from the next estimate); the **TD error** is
$$\delta_t = R_{t+1} + \gamma\, V(S_{t+1}) - V(S_t), \qquad V(S_t) \leftarrow V(S_t) + \alpha\, \delta_t.$$

Action values and Q-learning. For control we learn the **action-value** function
$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$
Off-policy **Q-learning** updates Q toward the Bellman optimality target with the greedy

bootstrap $\max_{a'} Q(s', a')$:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

At convergence $Q \to Q^*$, and the optimal greedy policy is

$$\pi^*(s) = \arg\max_a Q^*(s, a).$$

ϵ-greedy exploration. To keep exploring while exploiting, the agent follows

$$\pi(a \mid s) = \begin{cases} 1 - \epsilon + \dfrac{\epsilon}{|A|}, & a = \arg\max_{a'} Q(s, a'), \\ \dfrac{\epsilon}{|A|}, & \text{otherwise}, \end{cases}$$

so that every action is tried with probability at least $\epsilon/|A|$.